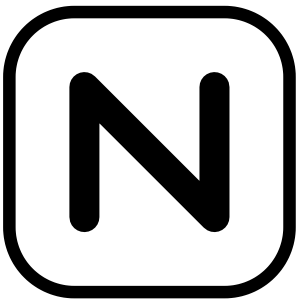
A COMPLETE GUIDE



HOW TO BUILD THE

NOVA C-IDE

From a blank folder to a working C and C++ programming environment —
editor, build tools, debugger, language server and terminal —
explained in plain English, one step at a time.

BY TEAM NOVA

First edition · 2026

HTML · CSS · JAVASCRIPT · NODE.JS · ELECTRON · C · C++ · CMAKE · JSON

How to Build the NOVA C-IDE
by Team NOVA

First edition, 2026.

NOVA IDE is an in-house tool. This book describes how it was built so that anyone on the team can understand it, rebuild it, change it and grow it.

All screenshots in this book were taken from the real NOVA IDE running on Windows 11 and were converted to black and white for printing. Every code sample marked with an **OUTPUT** box was compiled and run while this book was being made, so the output you see is the real output.

Trademarks: CLion and JetBrains are trademarks of JetBrains s.r.o. NOVA was designed from public documentation and its own ideas. It contains no JetBrains code. Visual Studio Code, Monaco, Electron, xterm.js, clangd, LLVM, GDB, CMake and other names belong to their respective owners.

Set in Times New Roman, 12 point, with 1.5 line spacing.

Contents

Preface: Read This First 1
Who this book is for 1
What you will learn 1
How the book is organised 2
The special boxes 2
What you need on your computer 3
A promise, and an honest word 4
How to study 4

PART I — UNDERSTANDING IDES 5

Chapter 1. What Is an IDE? 6
1.1 The workshop idea 6
1.2 Editor versus IDE 7
1.3 A short history 7
1.4 What CLion does, and why we built NOVA 8
1.5 A picture of NOVA 8

Chapter 2. The Parts of an IDE 11
2.1 The code editor 11
2.2 The project tree 12
2.3 Tabs 12
2.4 The build system 12
2.5 The run console 13
2.6 The problems panel and error squiggles 13
2.7 The language server (code intelligence) 13
2.8 The debugger 14
2.9 The terminal 15
2.10 Search and navigation 15
2.11 The status bar and settings 16
2.12 All the parts together 16

Chapter 3. From Code to a Running Program 18
3.1 Your computer only speaks one language 18
3.2 Compilers and interpreters 18
3.3 The four steps inside a C++ compiler 19
3.4 How does an interpreter really work? 20
3.5 So where does an IDE fit? 21

Chapter 4. How NOVA Is Designed 23
4.1 Two kinds of processes 23
4.2 How the two sides talk: IPC 24
4.3 NOVA also talks to other programs 25
4.4 The folder map 27
4.5 How big is NOVA? 27

Chapter 5. The Languages Behind NOVA 30
5.1 HTML: the skeleton 30
5.2 CSS: the skin 30
5.3 JavaScript: the brain 31

5.4 Node.js and Electron: the engine room 31
5.5 C and C++: the cargo 31
5.6 CMake and JSON: the glue 31
5.7 Helpers you will meet briefly 31
5.8 The learning order 32

PART II — THE LANGUAGES 33

Chapter 6. HTML: The Skeleton 34
6.1 What HTML is, in one minute 34
6.2 Your first page 34
6.3 The tree behind the page 35
6.4 The elements you will use every day 36
6.5 Attributes, ids and classes 37
6.6 Page layout with semantic tags 38
6.7 The hidden attribute, and a real NOVA bug 38
6.8 Scripts and style sheets 39
6.9 SVG: drawing with text 39
6.10 Accessibility in one page 40
6.11 Mini assignments 40
6.12 Big assignment: The skeleton of a mini IDE 41
6.13 Quiz 41

Chapter 7. CSS: The Skin 43
7.1 What CSS does 43
7.2 Your first rule 43
7.3 Selectors: choosing what to style 44
7.4 Units and colours 44
7.5 The box model: everything is a box 44
7.6 Variables: change the whole look in one place 45
7.7 Layout, part 1: Flexbox 46
7.8 Layout, part 2: Grid 47
7.9 Position, overflow and layers 49
7.10 Making it feel alive: transitions and animation 49
7.11 The cascade: who wins when rules clash? 49
7.12 Media: dark and light, and print 50
7.13 Mini assignments 50
7.14 Big assignment: Make MiniIDE beautiful 51
7.15 Quiz 51

Chapter 8. JavaScript: The Brain 53
8.1 What JavaScript is 53
8.2 Running JavaScript 53
8.3 Variables and types 54
8.4 Comparing and deciding 55
8.5 Loops 56
8.6 Functions 56
8.7 Arrays: lists of things 57
8.8 Objects: things with named parts 58
8.9 Collections: Map and Set 58
8.10 Strings, patterns and regular expressions 59
8.11 Scope and closures 60
8.12 Classes (a short look) 61

8.13 When things go wrong: errors 62
8.14 Time: the event loop, callbacks, promises and async/await 62
8.15 Useful tools for a real program 66
8.16 JavaScript in the browser: the DOM 67
8.17 Modules (a short look) 67
8.18 Worked project: a tiny interpreter 68
8.19 Mini assignments 69
8.20 Big assignment: Bring MiniIDE to life 70
8.21 Quiz 71

Chapter 9. Node.js and Electron: The Engine Room 73

9.1 Why we need them 73
9.2 Node basics 73
9.3 Events: EventEmitter 76
9.4 Starting other programs: child_process 76
9.5 Electron: a window made of web technology 79
9.6 The safe bridge: preload and contextBridge 81
9.7 Dialogs, menus and other native features 83
9.8 A story about two bugs 83
9.9 Packaging (a first look) 84
9.10 Mini assignments 84
9.11 Big assignment: A file explorer with a Run button 84
9.12 Quiz 85

Chapter 10. C: The Foundation 87

10.1 Why learn C? 87
10.2 Setting up 87
10.3 Hello, world 88
10.4 Variables and types 89
10.5 Deciding and repeating 90
10.6 Functions 91
10.7 Arrays and strings 92
10.8 Pointers: the big idea 93
10.9 Memory: the stack and the heap 95
10.10 Structs, enums and typedef 97
10.11 Reading and writing files 99
10.12 Function pointers and callbacks 99
10.13 The preprocessor 100
10.14 Many files: headers and linking 101
10.15 Putting it together: a growable list 102
10.16 Bits and bytes 105
10.17 Debugging C in NOVA 105
10.18 Mini assignments 105
10.19 Big assignment: Library manager 106
10.20 Quiz 107

Chapter 11. C++: C With Superpowers 109

11.1 What C++ adds 109
11.2 Hello, C++ 109
11.3 Everyday tools 110
11.4 The standard library containers 111
11.5 Classes: your own types 114
11.6 RAII: the idea that defines C++ 116

11.7 Inheritance and polymorphism 118
11.8 Operator overloading 120
11.9 Templates: code for any type 120
11.10 Errors: exceptions 121
11.11 Move semantics, briefly 122
11.12 Modern C++ goodies 123
11.13 Threads, in one example 123
11.14 Worked project: a C++ tokenizer 124
11.15 Reading C++ error messages 126
11.16 Mini assignments 127
11.17 Big assignment: A source-code outline tool 127
11.18 Quiz 128

Chapter 12. CMake: The Recipe Book 130
12.1 The problem CMake solves 130
12.2 Your first CMakeLists.txt 130
12.3 The two commands 131
12.4 Build types: Debug and Release 132
12.5 Libraries and linking 132
12.6 Variables, conditions and loops 133
12.7 Splitting a big project: add_subdirectory 134
12.8 Testing with CTest 134
12.9 Fetching other people's code 135
12.10 compile_commands.json: how clangd learns your flags 135
12.11 A real NOVA story 135
12.12 Common errors and fixes 136
12.13 Mini assignments 136
12.14 Big assignment: A multi-target project 137
12.15 Quiz 138

Chapter 13. JSON and Protocols: How Programs Talk 139
13.1 Why this chapter exists 139
13.2 JSON: the common alphabet 139
13.3 Framing: where does one message end? 140
13.4 JSON-RPC: asking questions across the pipe 142
13.5 GDB/MI: talking to the debugger 144
13.6 The terminal: bytes and escape codes 147
13.7 Designing your own protocol: lessons 147
13.8 Mini assignments 148
13.9 Big assignment: A fake language server 149
13.10 Quiz 150

PART III — BUILDING NOVA, STEP BY STEP 151

Chapter 14. Build Guide, Part 1: Setup and the First Window 152
14.1 The plan 152
14.2 Step 1: Install the tools 153
14.3 Step 2: Create the project 154
14.4 Step 3: The first window 155
14.5 Step 4: The skeleton and the skin 157
14.6 Troubleshooting Steps 1 to 4 159
14.7 Mini assignments 160
14.8 Quiz 160

Chapter 15. Build Guide, Part 2: Editor, Files, Build and Problems 162
15.1 Step 5: Bring in the code editor 162
15.2 Step 6: The file tree, tabs and saving 165
15.3 Step 7: Build and run 169
15.4 Step 8: Problems, squiggles and links 174
15.5 Mini assignments 177
15.6 Quiz 177

Chapter 16. Build Guide, Part 3: Search, Terminal, Intelligence and Debugger 179
16.1 Step 9: Search, Search Everywhere and the command palette 179
16.2 Step 10: A real terminal 182
16.3 Step 11: Code intelligence with clangd 184
16.4 Step 12: The debugger 189
16.5 Finishing touches 192
16.6 The file-by-file reference 196
16.7 Mini assignments 198
16.8 Quiz 198

PART IV — GOING FURTHER 200

Chapter 17. Beyond NOVA: Bigger Projects to Build Next 201
17.1 You are further along than you think 201
17.2 The menu of projects 201
17.3 Project 1: A Git panel 202
17.4 Project 2: An expandable variable tree 203
17.5 Project 3: A test runner 203
17.6 Project 4: Package NOVA as an installer 204
17.7 Project 5: A plug-in system 204
17.8 Project 6: A sanitizer viewer 205
17.9 Project 7: Profiler and coverage 205
17.10 Project 8: Your own language server 205
17.11 Project 9: A toy programming language 206
17.12 Project 10: A real parser for colouring 206
17.13 Project 11: Remote development 207
17.14 Project 12: Grow up: TypeScript and a bundler 207
17.15 How to learn from here 207

APPENDICES 209

Appendix A. Nomenclature (Words and Functions) 210
A.1 How to read this list 210
A.2 General IDE and programming words 210
A.3 Web words: HTML, CSS and the browser 212
A.4 JavaScript words 213
A.5 Node.js and Electron words 213
A.6 C and C++ words 214
A.7 Build and tool words 215
A.8 Most-used functions and commands 216

Appendix B. Cheat Sheets and Troubleshooting 221
B.1 NOVA keyboard shortcuts 221
B.2 Command-line cheat sheet 222
B.3 Troubleshooting table 222

Appendix C. Roadmaps for the Big Assignments **224**
C.1 MiniIDE: HTML (Chapter 6) 224
C.2 MiniIDE: CSS (Chapter 7) 224
C.3 MiniIDE: JavaScript (Chapter 8) 224
C.4 Folder Peek: Node and Electron (Chapter 9) 225
C.5 Library manager: C (Chapter 10) 225
C.6 Outline: C++ (Chapter 11) 225
C.7 GeoKit: CMake (Chapter 12) 226
C.8 EchoLS: Protocols (Chapter 13) 226

Preface: Read This First

Who this book is for

This book is for you if you have ever looked at a program like CLion, Visual Studio or VS Code and thought: *"How does this thing actually work? Could I build one?"*

You do not need to be an expert. You do not need a degree. If you can use a computer, type, and stay curious, you can follow this book. Every hard word is explained the first time it shows up. Every big idea gets a picture. And every important line of code gets a **why**: why it was written that way, and what would go wrong if it were not.

By the end you will have built, understood and be able to change a real programming tool called **NOVA IDE**. NOVA is a free, in-house program for writing, building, running and debugging **C** and **C++** code. It has a code editor, a file tree, a build button, a debugger, a terminal and a "smart helper" that suggests code and points out mistakes.

What you will learn

You will learn two things at the same time:

1. **How an IDE is put together.** What parts it has, what each part does, and why it matters.
2. **The languages used to build it.** HTML, CSS, JavaScript, Node.js, Electron, C, C++ and CMake. Each language gets its own chapter that starts from zero and ends with a big project.

Body part	Language	Files	Job
Skeleton	HTML	`index.html`	*What is on the screen: buttons, panels, boxes*
Skin	CSS	`styles.css`	*How it looks: colours, sizes, spacing, dark/light*
Brain	JavaScript	`app.js lsp.js debug.js ...`	*What happens when you click, type or press a key*
Engine room	Node.js	`main.js preload.js`	*Talks to the computer: files, programs, windows*
The cargo	C and C++	`your code`	*The programs NOVA edits, builds and debugs*
The glue	CMake, JSON	`CMakeLists.txt, messages`	*Tells compilers what to build; carries messages*

Figure 0.1. *The six "body parts" of NOVA and the language that builds each one.*

How the book is organised

Part	What is inside	Chapters
I. Understanding IDEs	What an IDE is, its parts, how code becomes a running program, how NOVA is designed	1 to 5
II. The Languages	One chapter per language, beginner to advanced, with exercises	6 to 13
III. Building NOVA	A guided, step-by-step build of the whole IDE, file by file	14 to 16
IV. Going Further	Bigger projects you can build next	17
Appendices	A word list (nomenclature), shortcuts, and troubleshooting	A to C

You can read straight through. Or, if you already know a language, skip its chapter. But do not skip Part I: it gives you the map for everything else.

The special boxes

Look out for these boxes. They are the heart of the book.

> **WHY?**
> A **WHY?** box explains the reason behind something. Programmers who only know *what* to type get stuck. Programmers who know *why* can fix problems and invent new things. Read every one.

WHAT IF IT WERE DIFFERENT?

A **WHAT IF** box shows what would break if we did it another way. Seeing the failure makes the correct way stick in your memory.

KEY WORD

A **KEY WORD** box defines a piece of jargon in plain English. All of them are collected again in the Nomenclature (Appendix A).

TRY IT

A **TRY IT** box asks you to type something and watch what happens. Typing it yourself teaches you ten times more than reading it.

MINI ASSIGNMENT

A **MINI ASSIGNMENT** is a small exercise (5 to 20 minutes) that uses what you just learned.

BIG ASSIGNMENT

A **BIG ASSIGNMENT** is a larger project at the end of each language chapter. It combines everything in the chapter. It is meant to take a few hours and to make you proud.

QUIZ

A **QUIZ** ends each chapter. Answer on paper first, then check the answers that follow it.

CHECKPOINT

A **CHECKPOINT** in Part III tells you what you should see on screen at that moment. If your screen looks different, stop and fix it before moving on.

What you need on your computer

To follow along you need a Windows, Mac or Linux computer and these free tools. Chapter 14 shows how to install each one.

Tool	What it is for
Node.js	Runs JavaScript outside the browser. NOVA's engine runs on it.
A C/C++ compiler (g++ and gcc)	Turns your C and C++ code into programs.
CMake	Describes how to build bigger C and C++ projects.
gdb	The debugger: lets you pause a program and look inside it.
clangd	The "smart helper" that powers suggestions and error hints.

Tool	What it is for
Any text editor	To type the code. VS Code is a fine choice for the first steps.

A promise, and an honest word

Here is the promise: if you read each chapter, type every example, do every assignment and take every quiz, you will be able to read, change and extend NOVA, and you will have a solid base in every language it uses.

And here is the honest word: nobody masters a language by reading alone. You become good by **writing code, making mistakes and fixing them**. This book gives you the path, the maps and the exercises. The walking is up to you.

How to study

- **Read a little, type a lot.** After each new idea, type the example. Change a number. Break it on purpose.
- **Do the quiz before you look at the answers.** Struggling for an answer is how the memory forms.
- **Do not rush the "why" boxes.** They are the part you will still remember in a year.
- **Keep a notebook.** Write down words you do not know. Then look them up in Appendix A.
- **When stuck, read the error message slowly.** Most error messages tell you the file, the line and the problem. NOVA shows them in the Problems panel for exactly this reason.

Welcome to Team NOVA. Let's build something.

Part I — Understanding IDEs

CHAPTER 1

What Is an IDE?

1.1 The workshop idea

Imagine you want to build a wooden chair. You could do it with a hand saw, a hammer, a ruler and a pencil, each kept in a different shed across the garden. Or you could do it in a workshop where the saw, hammer, ruler and pencil are on one bench, the lights are good, and a helper hands you the right tool at the right moment.

An **IDE** is that workshop, but for writing programs.

> **KEY WORD** ***IDE***
> **IDE** stands for **Integrated Development Environment**. *Integrated* means "joined together into one". *Development* means "making software". *Environment* means "the place where you work". So an IDE is one place where all the tools for making software are joined together.

You can write a program in the simplest text editor, such as Notepad. Programs have been written that way for decades. But then you must save the file, open a separate window, type a long command to turn your code into a program, run it, and read the errors in yet another window. Each step is small. Together they are slow and tiring.

An IDE puts it all in one window. You type your code, press one green button, and the program runs.

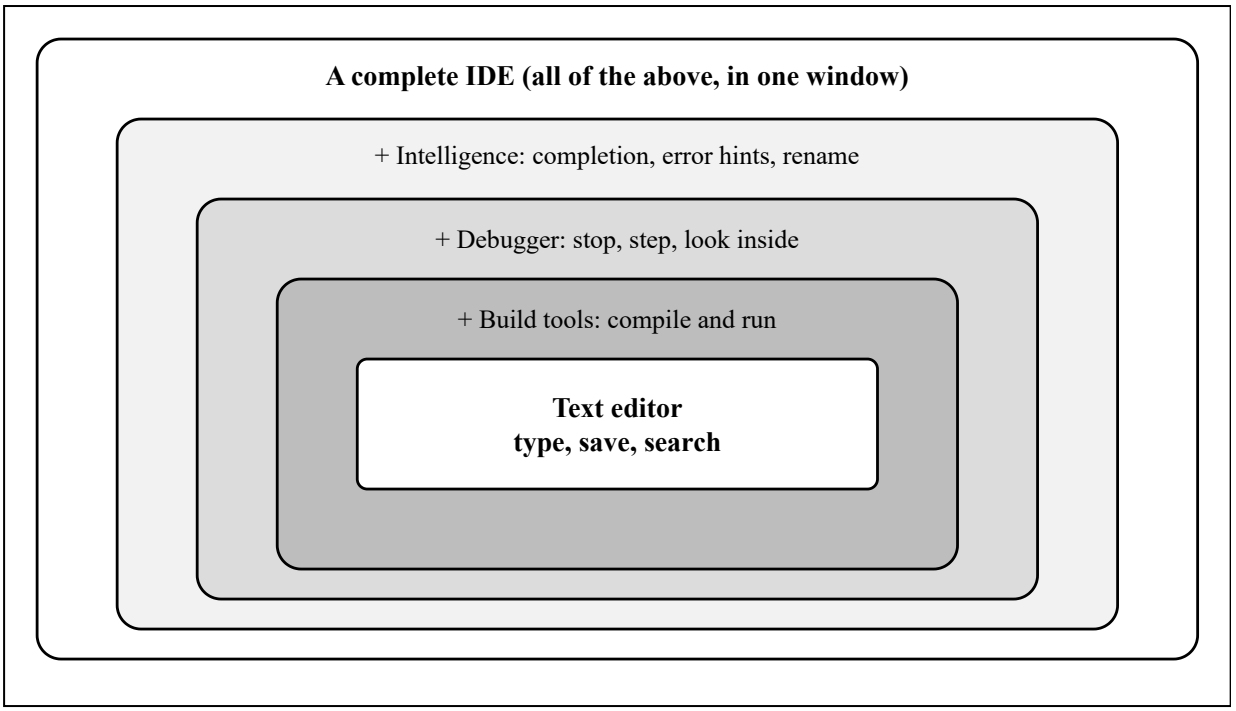


Figure 1.1. *A text editor is the centre of the onion. An IDE wraps more and more helpful layers around it.*

1.2 Editor versus IDE

Many people mix these two up, so let's be clear.

	Text editor	IDE
Typing and saving code	Yes	Yes
Colouring words (syntax highlighting)	Often	Yes
Press one button to build and run	No	Yes
Understands your code (suggests, finds errors)	No	Yes
Debugger built in	No	Yes
Terminal built in	Rarely	Yes
Starts fast, small	Yes	Heavier

> IN PLAIN WORDS *In plain words*
>
> A text editor is a **pencil**. An IDE is a **pencil, a ruler, an eraser, a calculator and a teacher looking over your shoulder**, all in one.

1.3 A short history

- **1950s to 1970s.** Programmers wrote code on punch cards. There was no screen. You handed the cards in and came back hours later to see if it worked.

- **1980s.** Programs like Turbo Pascal (1983) put an editor and a compiler into one program. You could write, run and fix code in seconds. This was the first popular IDE.
- **1990s.** Visual Studio and Delphi added drag-and-drop windows, debuggers and project management.
- **2000s.** Eclipse and IntelliJ IDEA added *code understanding*: the IDE learned to suggest the next word and rename things safely.
- **2010s to today.** JetBrains made tools for each language. **CLion** is their IDE for C and C++. Microsoft's VS Code popularised the idea of a lightweight editor extended with plug-ins and a **language server** (you will meet this in Chapter 2).

1.4 What CLion does, and why we built NOVA

CLion is a professional IDE for C and C++. It has a project tree, a smart editor, CMake support, a debugger, a terminal and a dark, modern look. It is excellent. But it is paid software, and it is closed: nobody outside JetBrains can see how it is built.

Team NOVA decided to build **our own** IDE with the same *workflow*: the same layout, the same shortcuts, the same ideas. The goals were:

1. **Free to use** inside the team, forever.
2. **Understandable.** Every file small enough to read in one sitting.
3. **A teaching tool.** If you build an IDE, you learn how all your other tools work.

> **WHY?** ***Why build your own IDE at all?***
> Because an IDE touches almost every big idea in programming: files, processes, text, windows, networks of messages, compilers and debuggers. Building one is like building an engine to learn how cars work. You will not beat a professional car factory. But you will never again be confused by what is under the bonnet.

> **WATCH OUT** ***Watch out***
> NOVA is **not** a copy of CLion's code. CLion's program files are private and protected by law. NOVA was designed from public documentation, common knowledge and the team's own ideas, and it uses open-source building blocks (Monaco, Electron, xterm.js, clangd, gdb). This is both legal and a better way to learn.

1.5 A picture of NOVA

Here is NOVA running. Do not worry about the details yet. By the end of the book you will know what every pixel is for.



Figure 1.2. *NOVA's welcome screen: start a new project, open a folder, or pick a recent project.*

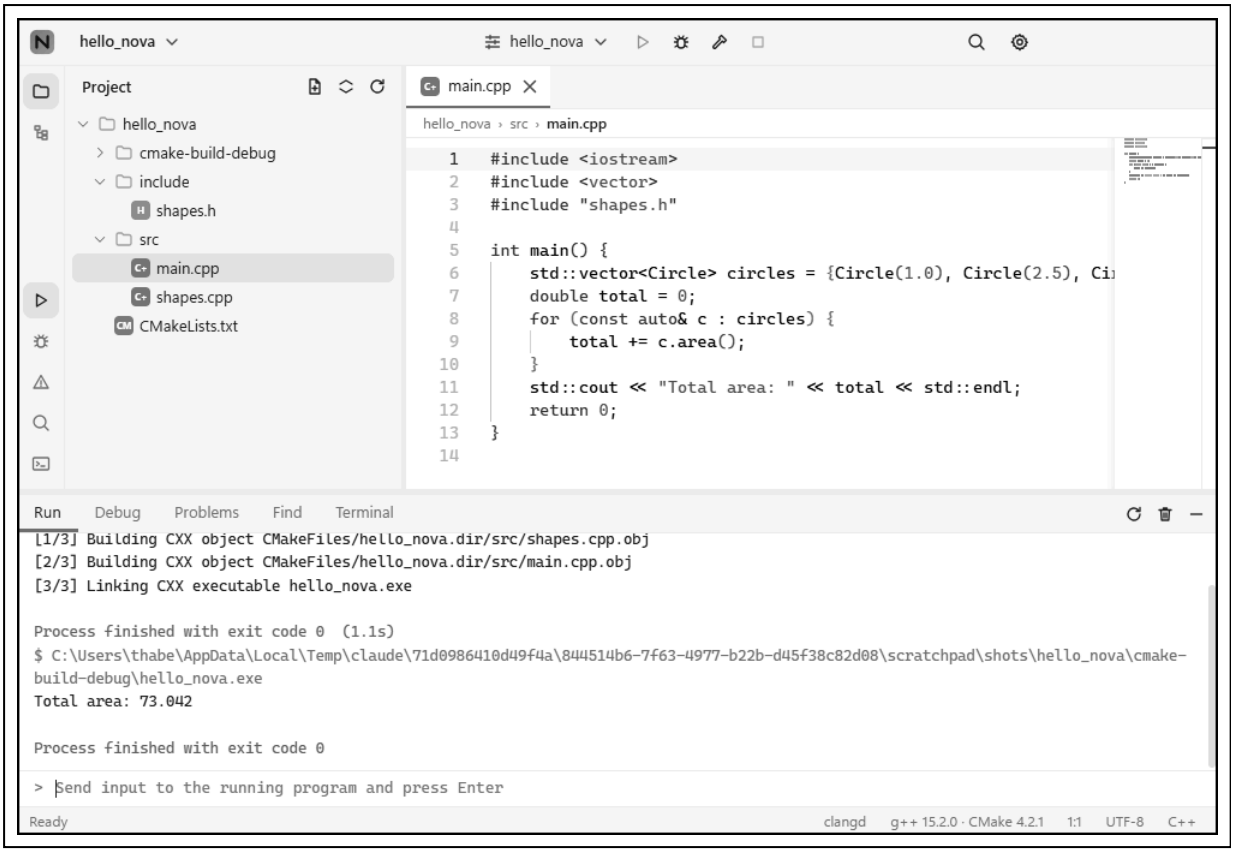


Figure 1.3. *NOVA with a project open. The file tree is on the left, the code in the middle, and the Run panel at the bottom shows the program's output.*

QUIZ

1. What does the word "integrated" mean in IDE?
2. Name three things an IDE does that a plain text editor usually does not.
3. Which famous IDE from JetBrains is for C and C++?
4. Give two reasons Team NOVA built its own IDE.
5. Is NOVA built from CLion's source code? Explain.

ANSWERS

1. Joined together into one. All tools live in one program.
2. Any three of: build and run with one button, suggest code as you type, find mistakes while you type, debugger, built-in terminal, project tree.
3. CLion.
4. Free to use, easy to understand, and a great way to learn how developer tools work.
5. No. CLion's code is private. NOVA was designed from public ideas and open-source parts.

CHAPTER 2

The Parts of an IDE

An IDE is a team of small helpers. Each helper does one job well. When you understand the helpers, no IDE will ever look mysterious again. Let's meet them one by one. For each we answer three questions: **What is it? Why does it matter? What happens if it is missing?**

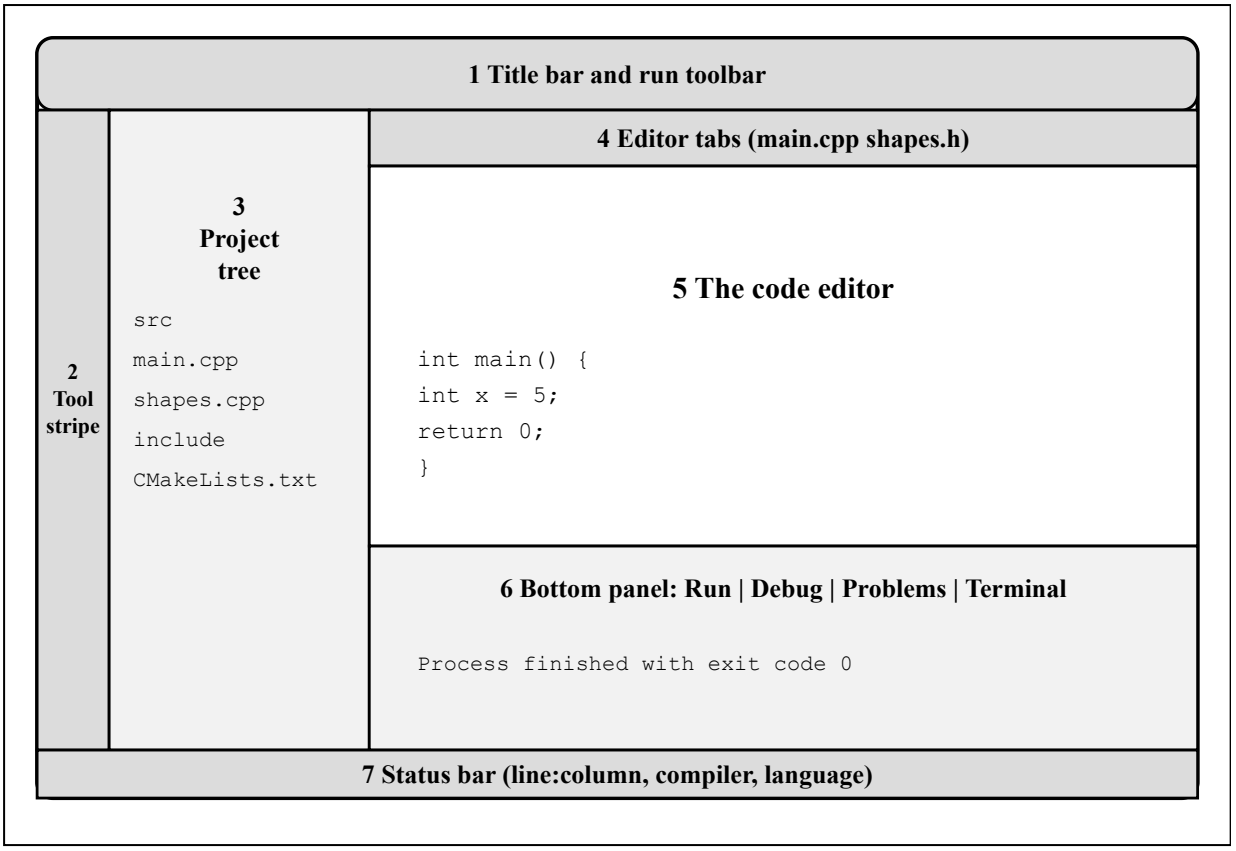


Figure 2.1. *The seven regions of an IDE window. The numbers match the sections below.*

2.1 The code editor

What it is. The big area where you type code. A good code editor does more than store letters. It colours keywords, matches brackets, lets you fold blocks, finds and replaces text, and handles huge files without slowing down.

Why it matters. You will look at this area for 95 percent of your working day. If it is slow, ugly or clumsy, everything feels bad.

If it were missing. You would have nothing to type into. There is no IDE without an editor.

> **KEY WORD** ***Syntax highlighting***
> Colouring different kinds of words differently (for example keywords orange, strings green, numbers blue). It lets your eyes see the *structure* of the code instantly. A missing quote mark shows up because everything after it turns green.

NOVA does not build an editor from nothing. That would take years. It uses **Monaco**, the open-source editor that powers VS Code. We add our own colours, snippets and features on top.

2.2 The project tree

What it is. The panel on the left that shows your folders and files as a tree you can open and close.

Why it matters. Real programs have tens or thousands of files. You need a map. The tree is that map. It also lets you create, rename and delete files without leaving the IDE.

If it were missing. You would have to open files by typing their names, and you would lose track of how your project is organised.

2.3 Tabs

What they are. Each open file gets a tab at the top, like pages in a web browser.

Why they matter. You constantly jump between a few files. Tabs make that one click. A small dot on a tab warns you that the file has unsaved changes.

2.4 The build system

What it is. The part that turns your code into a program you can run. It starts the **compiler** and the **linker** for you, with the right options, and reports problems.

Why it matters. Building a big C++ program by hand needs a very long command. Get one option wrong and it fails. The build system remembers the recipe so that you press one button.

If it were missing. You would copy and paste long commands into a terminal every time you changed one line.

NOVA's build system uses **CMake** to describe the recipe, **Ninja** to follow it quickly, and **g++** (the compiler) to do the real translation. Chapter 3 explains each.

2.5 The run console

What it is. A panel at the bottom that shows what your program prints and lets you type input back to it.

Why it matters. Programs talk to you through text. If you cannot see it, you cannot test your work.

2.6 The problems panel and error squiggles

What it is. A list of mistakes and warnings, plus red or yellow wavy lines under the exact words that are wrong.

Why it matters. Mistakes are normal. What matters is how fast you find them. A wavy line under the word is far faster than reading line 4,287 of a log.

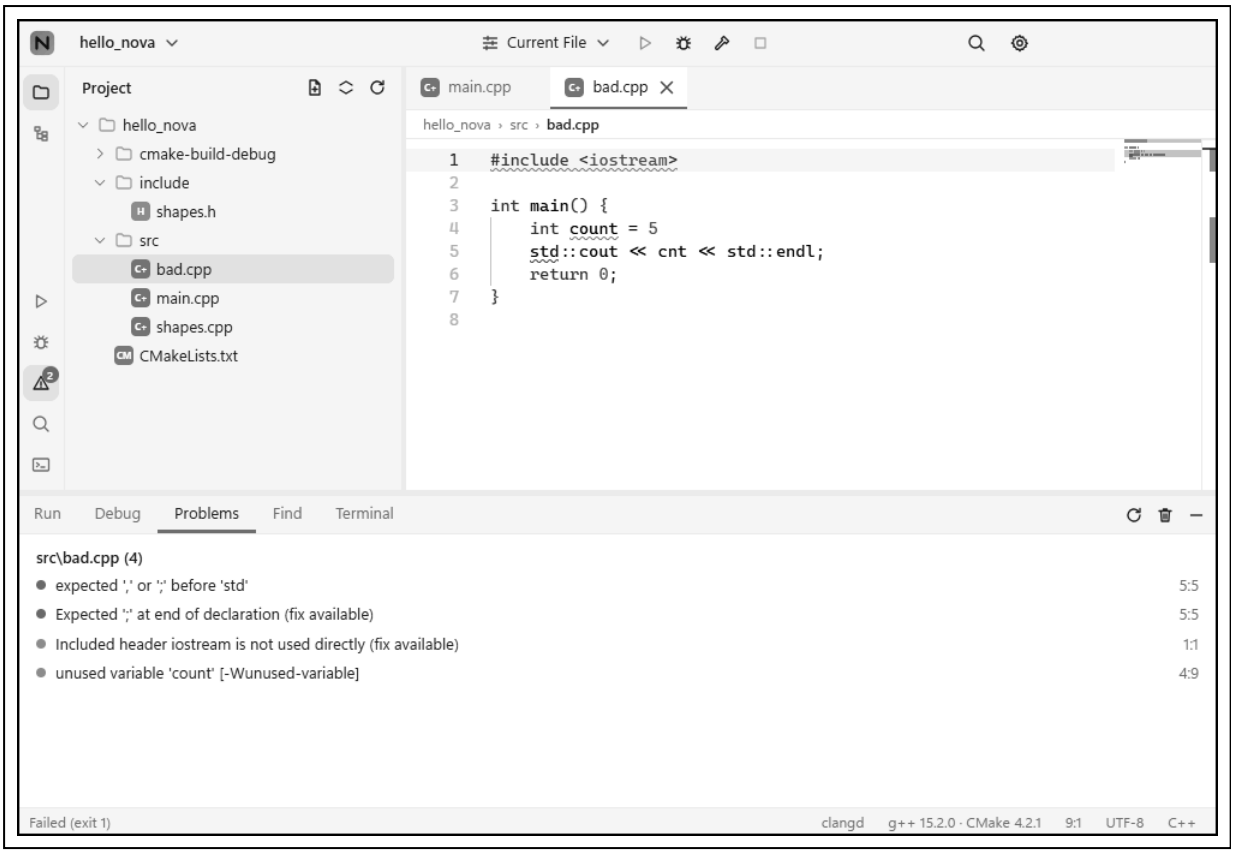


Figure 2.2. *Errors and warnings in NOVA. The wavy lines in the editor and the entries in the Problems panel come from two different sources: the compiler and the language server.*

2.7 The language server (code intelligence)

What it is. A separate program that actually *understands* your C++. When you type `std::vec` it knows you might want `vector`. When you hover over a function it shows its documentation. When you press a key on a name it can jump to where it was defined.

Why it matters. This is the biggest difference between a plain editor and an IDE. It makes you faster and catches errors before you even build.

If it were missing. You could still write code, but you would type everything yourself and find every mistake only after building.

> KEY WORD ***Language server and LSP***
> A **language server** is a program that understands one programming language. **LSP** (the **Language Server Protocol**) is the agreed set of messages an editor and a language server use to talk. Because the messages are standard, one server (like **clangd** for C and C++) works in many editors. NOVA uses clangd.

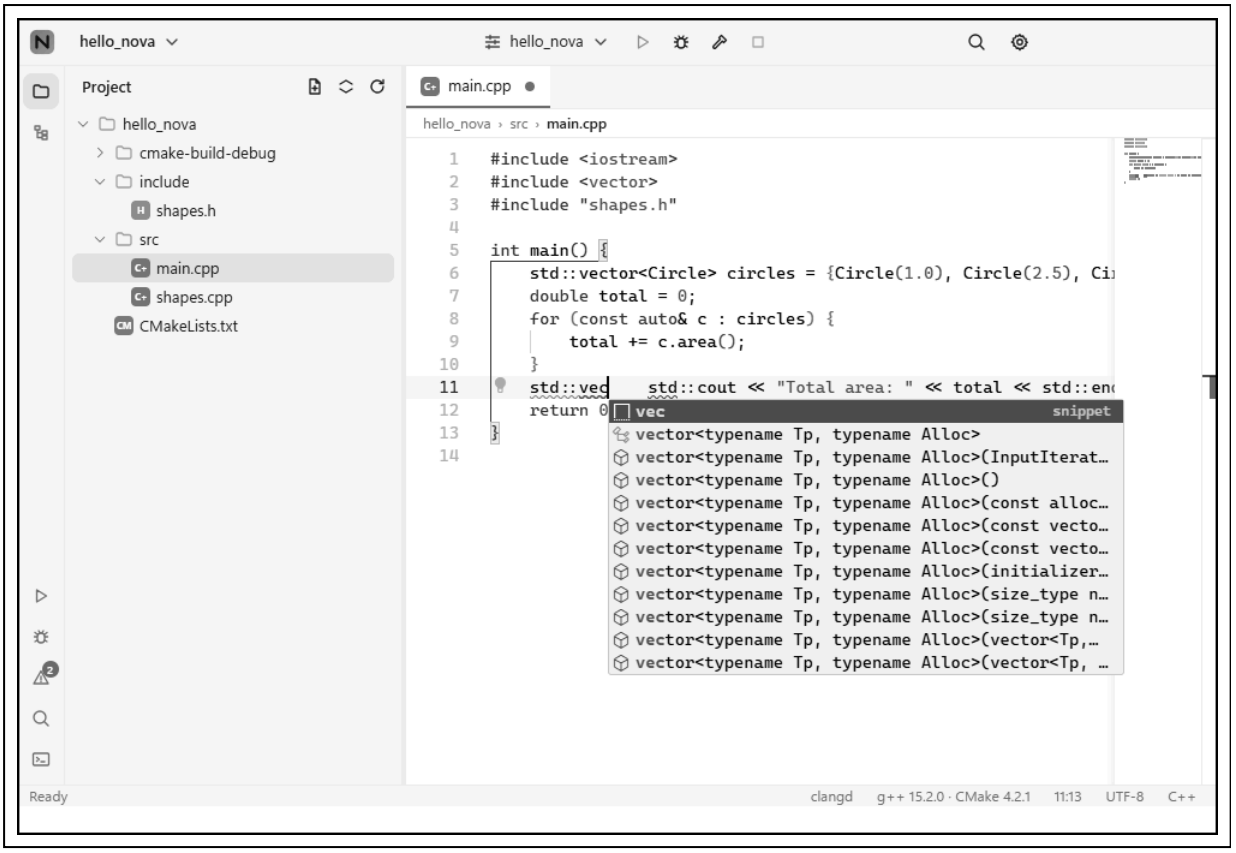


Figure 2.3. *Auto-completion in action. NOVA asked clangd what could follow "std::vec" and clangd answered with a list.*

2.8 The debugger

What it is. A tool that lets you **pause** a running program at a chosen line, look at the values of its variables, and move forward one step at a time.

Why it matters. Printing messages to find a bug is like searching a dark room with a match. A debugger turns on the light.

If it were missing. You would guess. Programmers have lost days of their lives to guessing.

KEY WORD ***Breakpoint***

A **breakpoint** is a mark on a line that says "stop here when you get to it". It shows as a red dot in the margin.

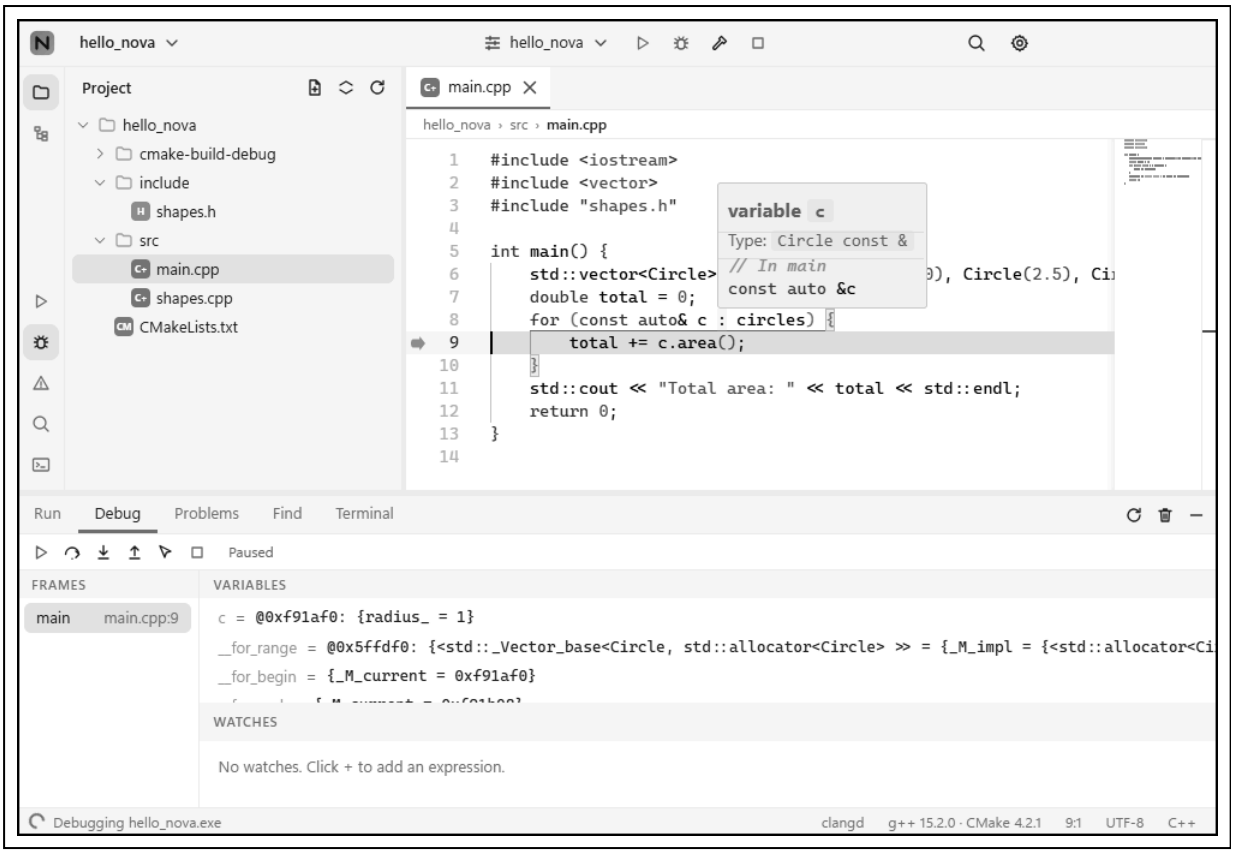


Figure 2.4. *NOVA paused at a breakpoint on line 9. The arrow marks the current line; below are the call stack, the variables and the watch list.*

2.9 The terminal

What it is. A real command line inside the IDE.

Why it matters. Some jobs are easiest from the command line: running git, listing files, trying a command. Having it inside the IDE means you never switch windows.

2.10 Search and navigation

What it is. Tools to find things fast: find in the current file, find across the whole project, jump to a file by typing a few letters, jump to a function by name.

Why it matters. In a big project, *finding* code takes more time than *writing* it. NOVA copies JetBrains' famous **double-Shift** "Search Everywhere".

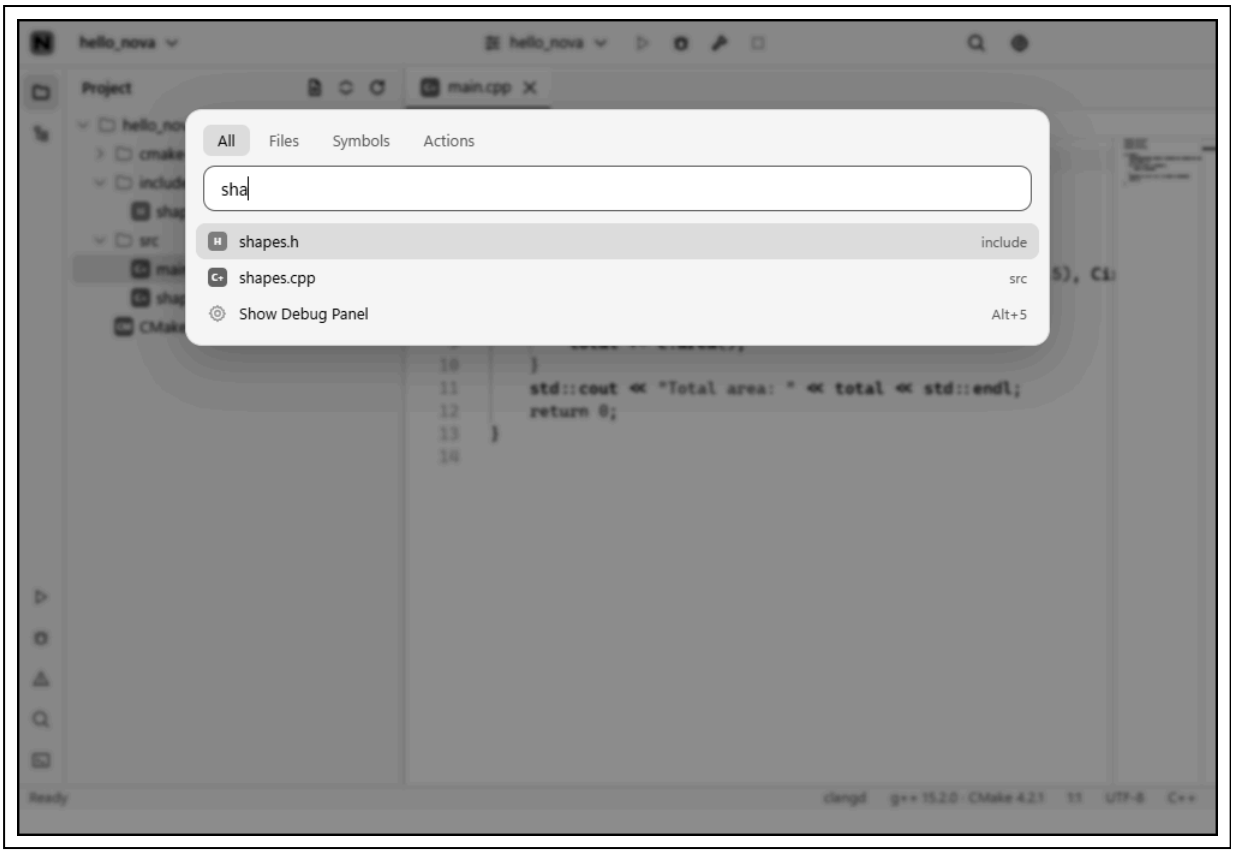


Figure 2.5. *Search Everywhere. Typing "sha" finds files, symbols and actions whose names contain those letters in order.*

2.11 The status bar and settings

What they are. A strip at the bottom with small facts (line and column, which compiler, which language), and a settings window for fonts, themes and options.

Why they matter. Small facts prevent big confusion. You always know which compiler is in use and where your cursor is.

2.12 All the parts together

The magic of an IDE is not any single part. It is that all the parts **share information**. The compiler's error appears as a wavy line in the editor. The debugger's current line is highlighted in the editor. The tree opens files in the editor. Everything is connected.

QUIZ

1. Which part of an IDE shows a map of your files?
2. What is a breakpoint?
3. What does the language server do that the compiler does not?
4. Why does the status bar matter, even though it is small?

5. Name the open-source editor NOVA is built on.
6. Why is "integrated" the most important word in IDE?

ANSWERS

1. The project tree.
2. A marker on a line telling the debugger to pause the program when it reaches that line.
3. It understands your code while you type: it suggests code, shows documentation, finds errors early and jumps to definitions. The compiler only runs when you build.
4. It tells you facts (cursor position, compiler, language) at a glance, preventing confusion.
5. Monaco (the editor of VS Code).
6. Because the value comes from the parts sharing information, for example an error found by the compiler appearing as a wavy line in the editor.

CHAPTER 3

From Code to a Running Program

To build an IDE for C and C++ you must understand what happens between "I typed some code" and "the program runs". This chapter explains it in simple steps. It also answers a common question: *what is an interpreter, and is NOVA one?*

3.1 Your computer only speaks one language

A computer's brain, the **CPU**, understands only **machine code**: long lists of numbers. Nobody wants to write that. So humans invented **programming languages** like C++, which are readable. A special program must translate our readable code into machine code. There are two ways to do the translation.

3.2 Compilers and interpreters

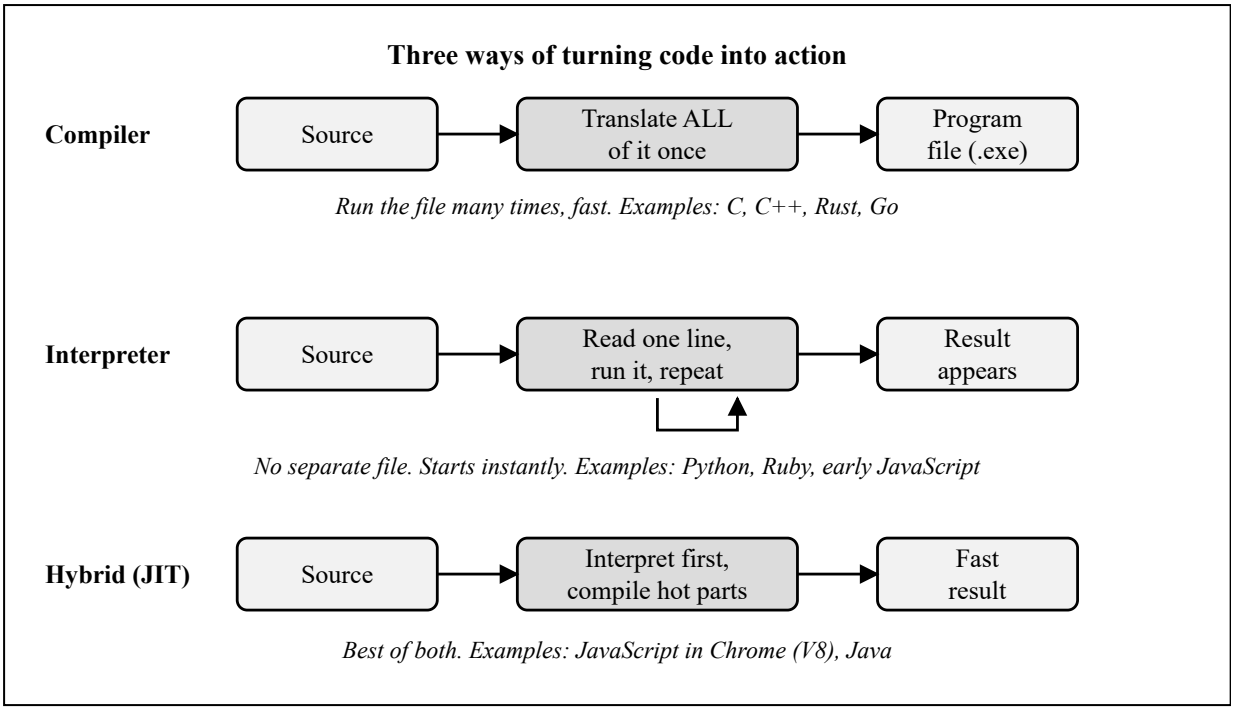


Figure 3.1. *Three ways of turning code into action.*

KEY WORD ***Compiler***

A **compiler** translates your *whole* program into a separate machine-code file **once**, before it runs. Afterwards you run the file as many times as you like, very fast. **C and C++ use compilers.**

KEY WORD ***Interpreter***

An **interpreter** reads your program and carries out the instructions **as it goes**, one by one, with no separate file. Starting is instant, but running is usually slower. **Python and (originally) JavaScript use interpreters.**

Think of a speech. A **compiler** is a translator who takes your whole written speech home, translates it carefully, and gives you a finished translated copy. An **interpreter** is a translator who stands beside you and translates every sentence live as you speak.

WHY? ***Why does this matter for an IDE?***

The folder NOVA lives in is called "C Interpreter" but NOVA is not an interpreter. It is an IDE that **drives a compiler**. This matters because it decides what the IDE must do: when you press Run, it must start the compiler, wait, check for errors, then start the new program. An interpreter-based IDE would simply feed your code to the interpreter. (JavaScript, which NOVA itself is written in, is actually a *hybrid*: modern browsers interpret first and compile the busy parts. See the figure.)

3.3 The four steps inside a C++ compiler

When you type `g++ main.cpp -o hello.exe`, four stages happen quietly. You must know them because every error you will ever see comes from one of them.

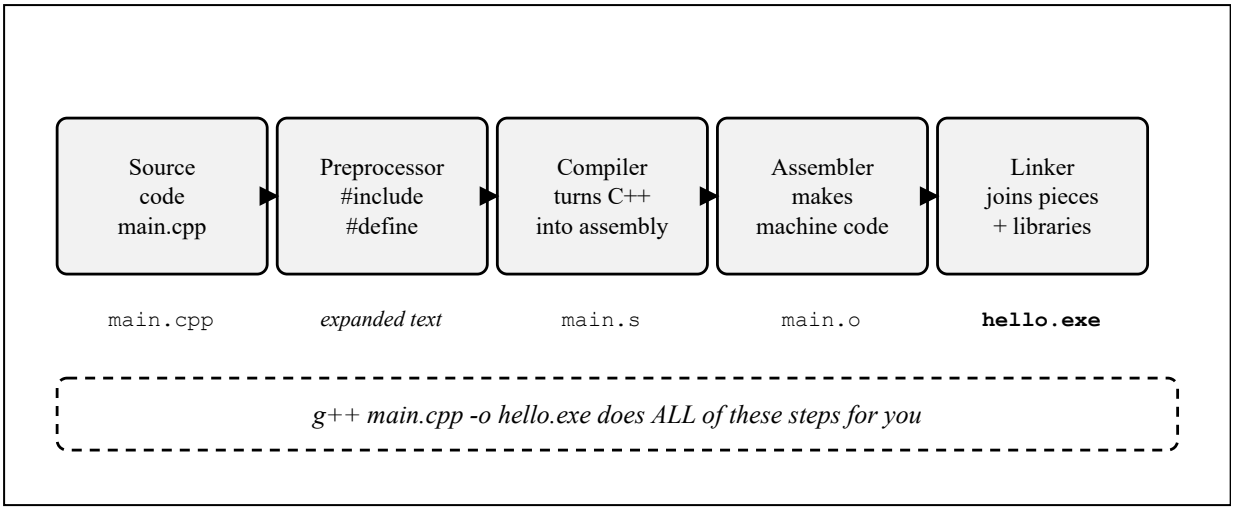


Figure 3.2. *The four stages of turning source code into a program.*

1. **Preprocessing.** Lines that begin with `#`, like `#include <iostream>`, are handled first. `#include` simply pastes the contents of another file in. After this step, your file is one very long piece of text.
2. **Compiling.** The compiler checks the grammar of your code and translates it into **assembly**, a low-level readable form of machine code. *Syntax errors* ("missing semicolon") come from here.
3. **Assembling.** The assembler turns assembly into **object code** (`main.o`): real machine code, but with gaps where it needs to find other pieces.
4. **Linking.** The **linker** joins your object files and libraries into one finished program, filling in the gaps. *"Undefined reference"* errors come from here.

> **WHAT IF IT WERE DIFFERENT?** ***What if there were no linker?***
>
> Large programs are written in many files. Each file is compiled on its own, so each has gaps where it calls functions defined elsewhere. Without a linker, nothing would join the files. You would be forced to write all your code in a single file. The linker is what makes big projects possible.

3.4 How does an interpreter really work?

Even though NOVA is not an interpreter, you should know how one works. It is a famous exercise and you will build a tiny one in the JavaScript chapter. An interpreter has three stages.

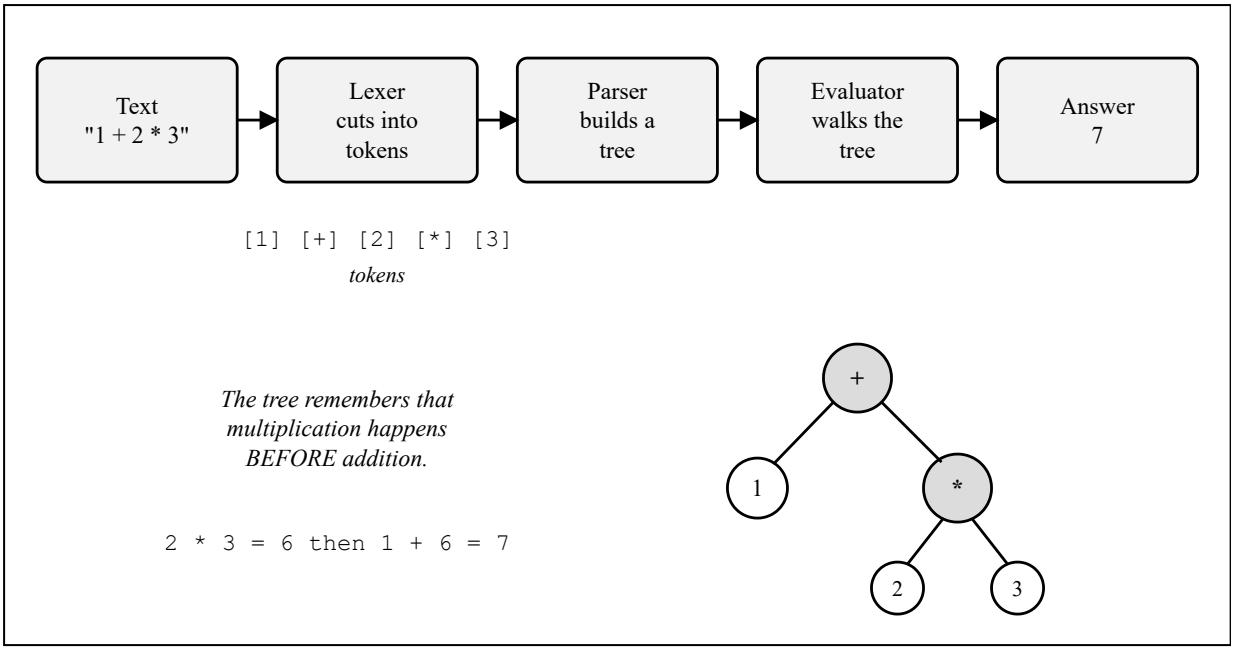


Figure 3.3. *The stages of a tiny interpreter, shown on the sum "1 + 2 * 3".*

1. **Lexer** (or tokenizer). It cuts the text into small meaningful pieces called **tokens**: numbers, plus signs, names.

2. **Parser.** It arranges tokens into a **tree** that shows the structure, called an **AST** (Abstract Syntax Tree). The tree is how the machine knows that `2 * 3` happens *before* adding 1.
3. **Evaluator.** It walks the tree from the bottom up, calculating as it goes.

> **WHY?** ***Why a tree and not just a list?***
> Because meaning depends on **grouping**. In `1 + 2 * 3` the answer is 7, not 9, because multiplication binds tighter. A flat list of tokens cannot show grouping. A tree can. Compilers, language servers and IDE features like "rename" all use trees like this. When clangd highlights a word it has already built the tree of your file.

3.5 So where does an IDE fit?

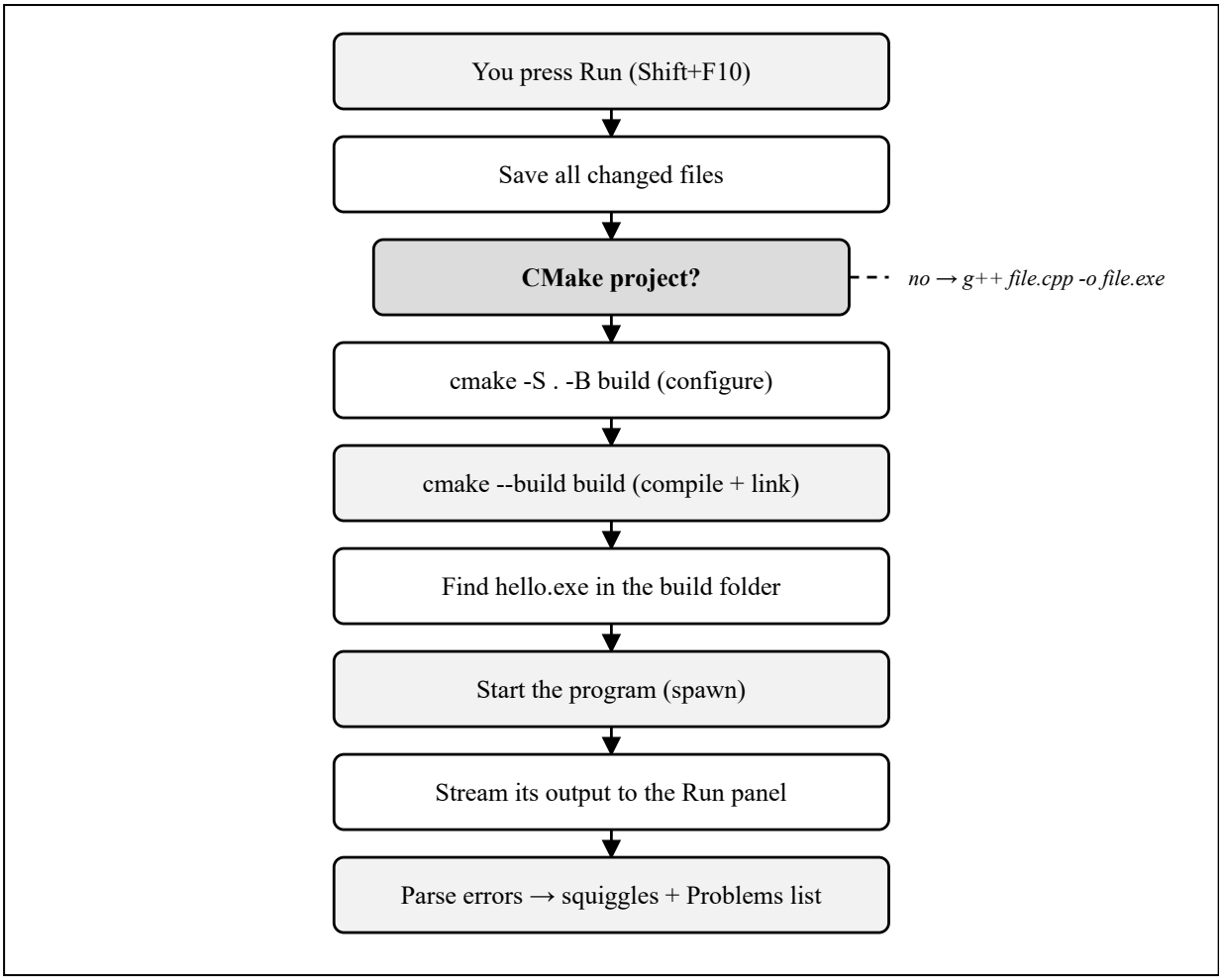


Figure 3.4. *What NOVA does when you press the Run button.*

An IDE is the **manager** that calls all these workers for you. The flowchart shows what NOVA does when you press Run. Notice that NOVA never translates any code itself. It **starts other programs** (cmake, g++, your program), **listens** to what they print, and **shows** the result nicely. That is the central idea of this whole book:

> ***An IDE is a friendly control panel for other programs.***

QUIZ

1. What is the difference between a compiler and an interpreter?
2. Is NOVA an interpreter? What is it?
3. Name the four stages inside a C++ compiler run.
4. Which stage produces "undefined reference" errors?
5. What does a lexer produce?
6. Why do interpreters and compilers build a tree?
7. In one sentence, what does an IDE do when you press Run?

ANSWERS

1. A compiler translates the whole program first into a separate file; an interpreter carries out the code as it reads it.
2. No. NOVA is an IDE that drives a compiler (g++) and other tools.
3. Preprocessing, compiling, assembling, linking.
4. Linking.
5. Tokens (small meaningful pieces such as numbers, names and operators).
6. A tree shows grouping and order of operations, which a flat list cannot.
7. It saves files, starts the build tools, starts the program, shows its output and shows any errors in the editor.

CHAPTER 4

How NOVA Is Designed

Now we look at the **blueprint** of NOVA. Do not worry if some words are new; each is explained here and again in the language chapters.

4.1 Two kinds of processes

NOVA is built with **Electron**. Electron is a toolkit that lets you build desktop programs out of web technology (HTML, CSS and JavaScript). Chrome and VS Code are built this way.

An Electron program is split into two kinds of **processes**. A **process** is a running program with its own private memory.

- The **main process** (our file `main.js`) is the *engine room*. It runs on **Node.js**, so it is allowed to touch files, start other programs and open windows.
- The **renderer process** (our files in the `renderer` folder) is the *window you see*. It is a web page, written in HTML, CSS and JavaScript. For safety it is *not* allowed to touch your disk or start programs.

Between them sits a thin **bridge** (`preload.js`) that decides exactly which doors are open.

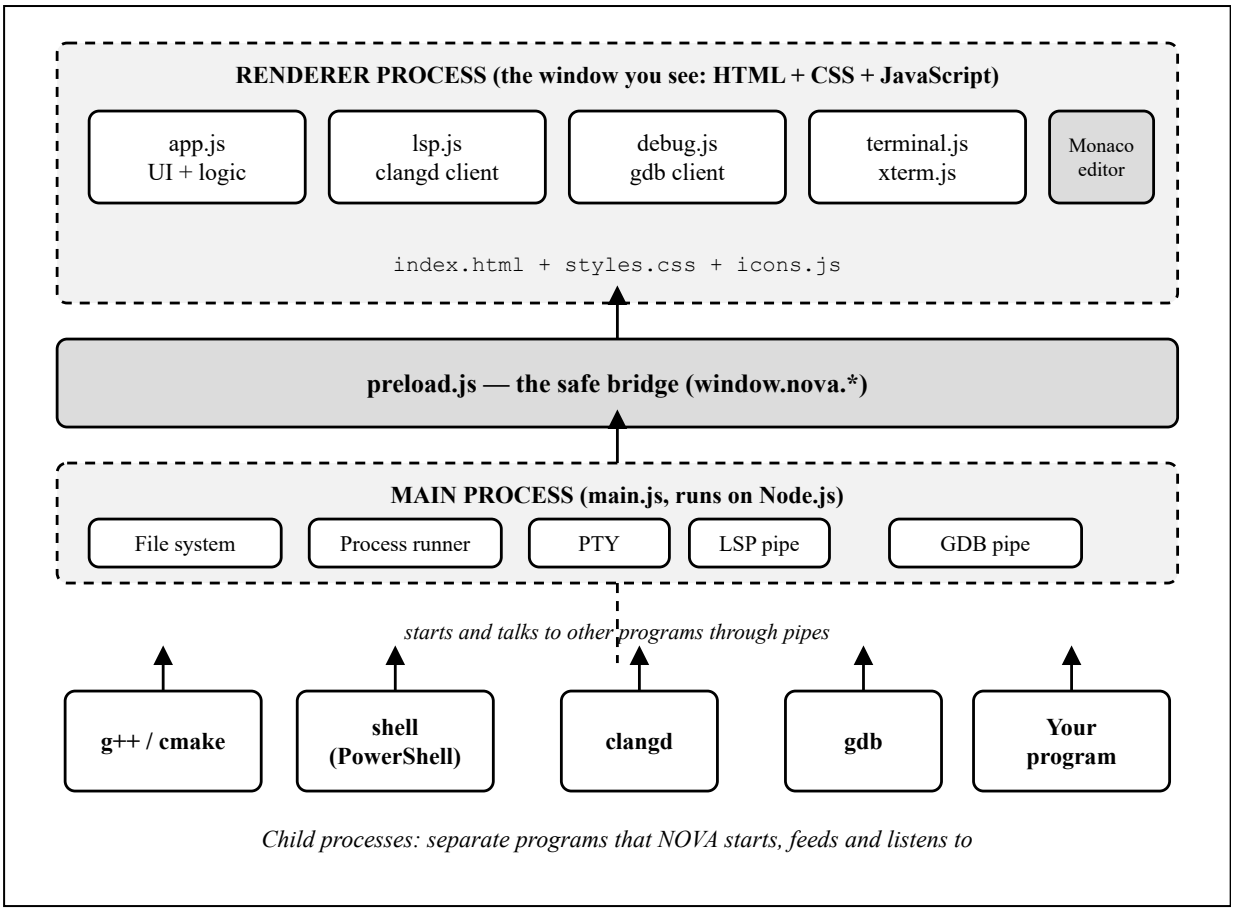


Figure 4.1. *NOVA's architecture. The window at the top never touches the computer directly. Everything passes through the bridge to the main process, which starts the other programs.*

> **WHY?** ***Why split it in two?***
>
> **Safety and stability.** A web page can show content from anywhere. If it had full power over your computer, one bad page could delete your files. Splitting the powers means the window can only do what the bridge allows. It also means that if the window freezes, the engine room keeps running.

> **WHAT IF IT WERE DIFFERENT?** ***What if the window could touch the files directly?***
>
> It would work, and it would be simpler to write. But any script that sneaked into the window (for example from a malicious file you opened) could read, change or delete anything on your computer. The two-process design closes that door. Electron's own guidelines recommend it.

4.2 How the two sides talk: IPC

The window and the engine room are separate programs, so they talk by sending messages. This is **IPC**, **Inter-Process Communication**.

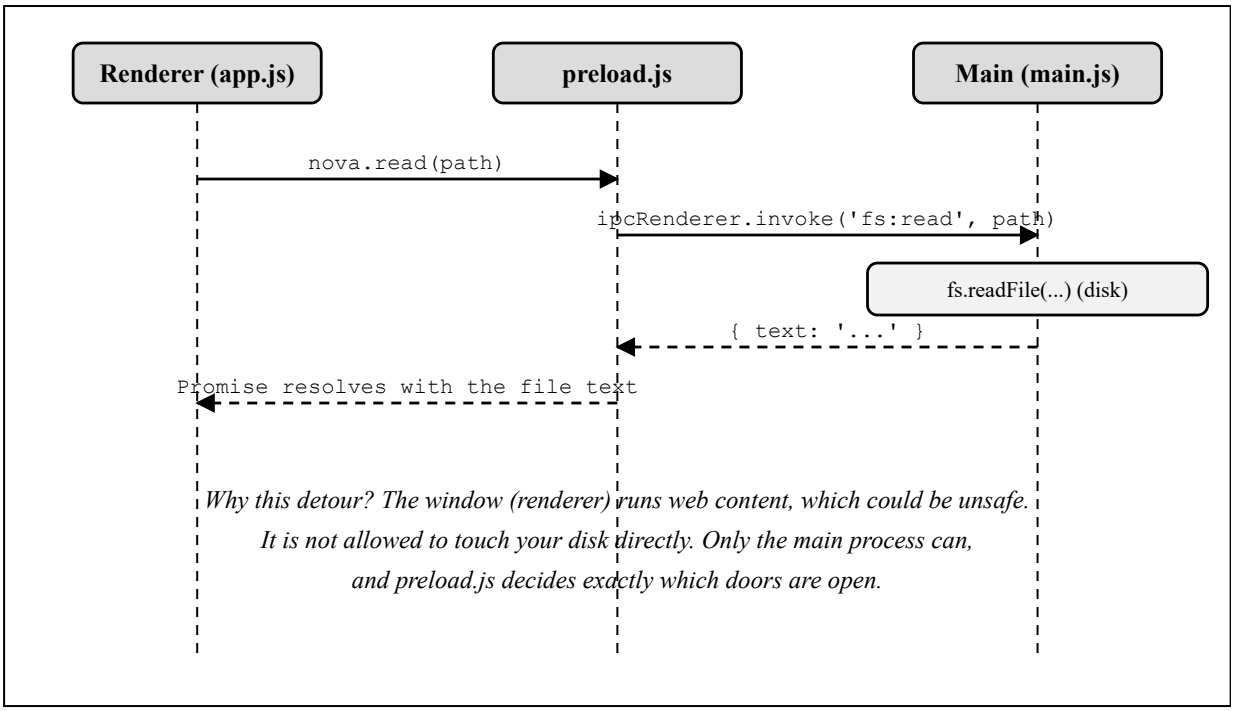


Figure 4.2. *One IPC round trip: the window asks for a file and gets its text back.*

In NOVA you will see lines like `await nova.read(path)`. That one line travels through the bridge to the main process, which reads the file from disk, and the answer travels back. The word `await` means "wait here until the answer arrives, but keep the window alive".

4.3 NOVA also talks to other programs

The main process does not only read files. It starts helper programs and **pipes** text to and from them. A **pipe** is a one-way channel for text between two programs, like a tube.

Helper program	What NOVA uses it for	How NOVA talks to it
g++ / gcc	Compile code	Command-line arguments; reads its output
cmake and ninja	Describe and perform builds	Command-line arguments; reads output
your program	Run what you built	Reads its output, can send input
clangd	Code intelligence	JSON messages (LSP) over pipes
gdb	Debugging	Text commands (GDB/MI) over pipes
PowerShell (via a PTY)	The terminal	A pseudo-terminal

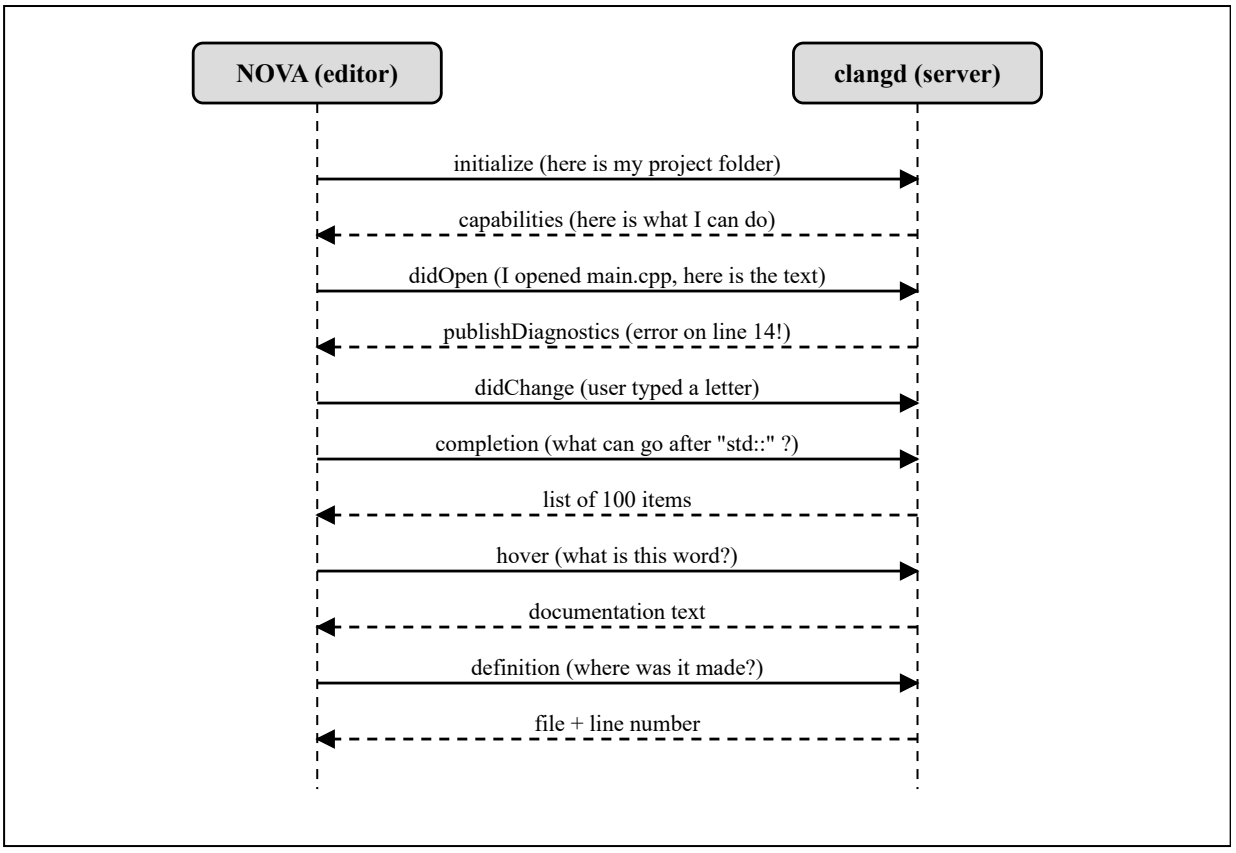


Figure 4.3. *A conversation between NOVA and the language server clangd.*

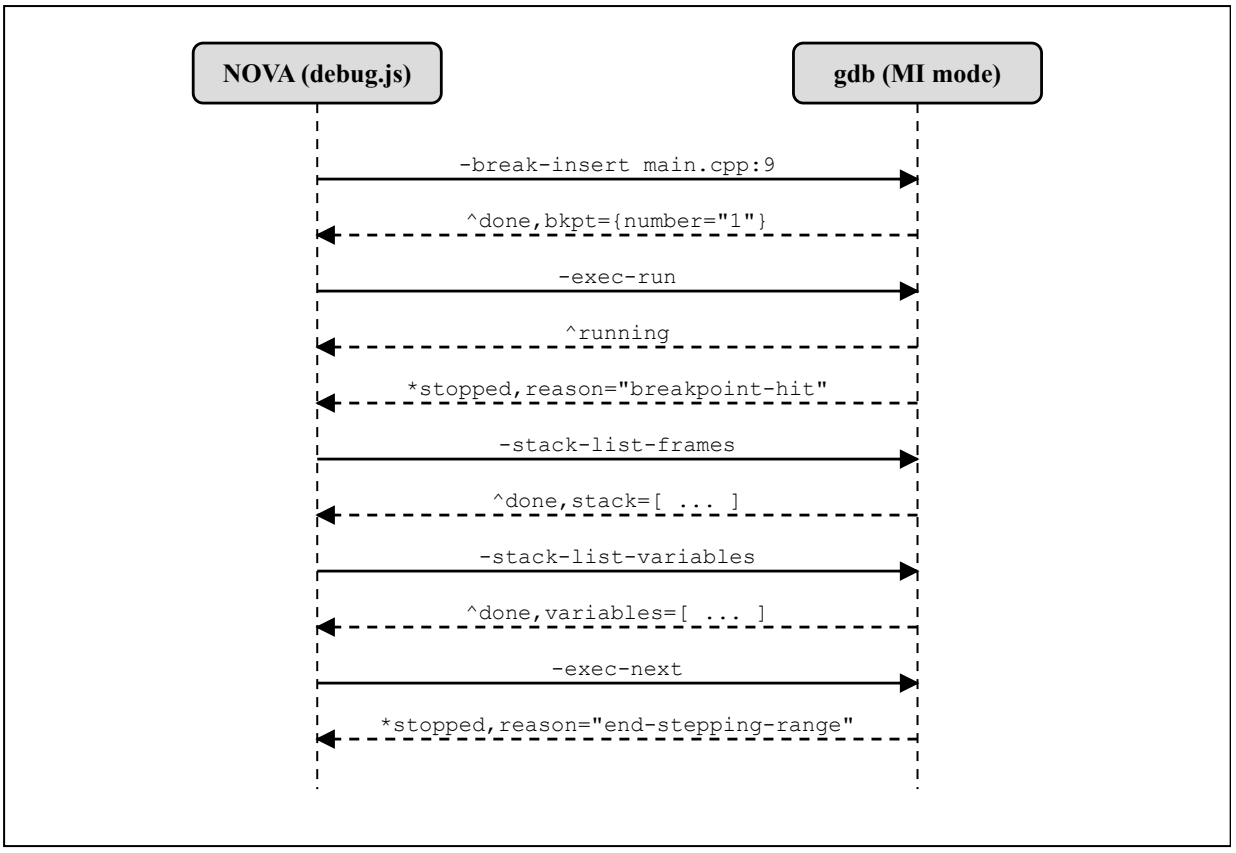


Figure 4.4. *A conversation between NOVA and the debugger gdb.*

4.4 The folder map

```
[+] NOVA - C Interpreter/
    package.json project name, scripts, dependencies
    main.js MAIN process: window, files, processes, PTY, clangd, gdb
    preload.js the safe bridge (window.nova)
    launch.js starts Electron without the Node-mode flag
    Launch NOVA.bat double-click launcher
    [+] renderer/
        index.html the skeleton of the window
        styles.css all colours and layout
        icons.js tiny SVG icon set
        app.js UI, tabs, tree, build/run, search
        lsp.js clangd (language server) client
        debug.js gdb (debugger) client
        terminal.js xterm.js terminal
    node_modules/ downloaded libraries (Electron, Monaco, xterm...)
    docs/ this book
```

Figure 4.5. *Every file in NOVA with a one-line description.*

4.5 How big is NOVA?

NOVA is small enough for one person to read in a weekend. This chart shows how many lines each file has.

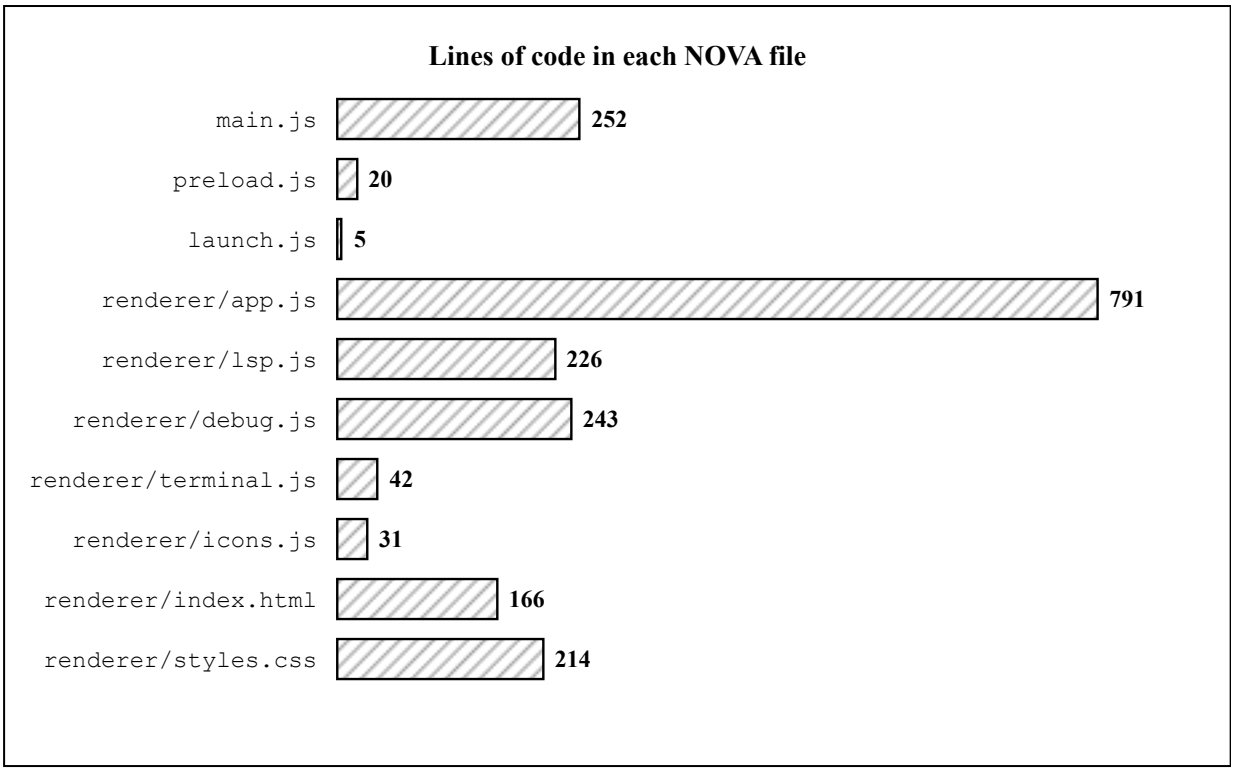


Figure 4.6. *Lines of code per file. app.js is the largest because it holds most of the window's logic.*

WHY? ***Why keep files small and separate?***

Each file does **one job**. `lsp.js` knows only how to talk to clangd. `debug.js` knows only how to talk to gdb. If the debugger breaks, you open one file. If you want to replace clangd with another language server, you change one file. This idea is called **separation of concerns**, and it is the most important habit of professional programmers.

QUIZ

1. What is a process?
2. Which process may touch the disk: main or renderer?
3. What is the job of preload.js?
4. What does IPC stand for?
5. Why does NOVA talk to clangd with JSON messages?
6. What does "separation of concerns" mean, and why is it useful?

ANSWERS

1. A running program with its own private memory.
2. The main process.
3. It is the safe bridge: it exposes only chosen functions (window.nova.*) from the main process to the window.
4. Inter-Process Communication.

5. LSP, the standard language-server protocol, is made of JSON messages, so one server works with many editors.
6. Each file or part does one job. It makes code easier to read, fix and replace.

CHAPTER 5

The Languages Behind NOVA

NOVA is a team of languages, each with its own job. Using the right language for each job is what makes the program small.

Body part	Language	Files	Job
Skeleton	HTML	`index.html`	*What is on the screen: buttons, panels, boxes*
Skin	CSS	`styles.css`	*How it looks: colours, sizes, spacing, dark/light*
Brain	JavaScript	`app.js lsp.js debug.js ...`	*What happens when you click, type or press a key*
Engine room	Node.js	`main.js preload.js`	*Talks to the computer: files, programs, windows*
The cargo	C and C++	`your code`	*The programs NOVA edits, builds and debugs*
The glue	CMake, JSON	`CMakeLists.txt, messages`	*Tells compilers what to build; carries messages*

Figure 5.1. *Who does what. HTML is the skeleton, CSS the skin, JavaScript the brain, Node.js the engine room, C and C++ the cargo, and CMake plus JSON the glue.*

5.1 HTML: the skeleton

HTML (HyperText Markup Language) describes *what is on the screen*: a toolbar, a sidebar, a button. It is not a programming language, because it makes no decisions. It is a **markup** language: it labels things. In NOVA, `index.html` lists every panel.

5.2 CSS: the skin

CSS (Cascading Style Sheets) says *how things look*: colours, sizes, spacing, fonts. NOVA's dark and light themes are just two sets of colours in `styles.css`. Change a colour in one place and the whole IDE changes.

5.3 JavaScript: the brain

JavaScript is a real programming language. It makes decisions and reacts to clicks, key presses and messages. Most of NOVA's logic is JavaScript: opening tabs, running builds, talking to clangd and gdb.

> **WHY?** ***Why JavaScript and not C++ for the IDE itself?***
> You could write an IDE in C++. Many do. But JavaScript with Electron gives us a ready-made window system, text editor (Monaco), terminal (xterm.js) and fast screen drawing. We write far less code, and JavaScript lets us change something and see the result in seconds. The price is more memory use. For a team tool, that is a good trade.

5.4 Node.js and Electron: the engine room

Node.js is JavaScript that runs *outside* a browser, so it can read files, start programs and use the network. **Electron** bundles Node.js with a browser engine to create desktop applications. NOVA's `main.js` and `preload.js` use them.

5.5 C and C++: the cargo

C and C++ are the languages NOVA is built **for**. They are not used to build NOVA's window, but you need to know them well to test, use and extend the IDE: you will write C and C++ programs to try every feature. They are also the language of the tools NOVA drives (gcc, g++, clangd and gdb are themselves written in C and C++).

5.6 CMake and JSON: the glue

CMake is a small language for describing how to build C and C++ projects. NOVA writes CMake files for new projects and runs CMake for you. **JSON** is a simple text format for data. NOVA uses it for settings, for the compile database and for every message exchanged with clangd.

5.7 Helpers you will meet briefly

- **GDB/MI** and **LSP**: "mini languages" of messages. We study them in Chapter 13.
- **Batch** (`Launch NOVA.bat`) and **PowerShell**: small scripting languages used to start the program.
- **Python**: not part of NOVA itself. The book's own build script uses it to turn this text into the PDF you are reading.

5.8 The learning order

The language chapters are in the order that makes learning easiest:

1. HTML (easiest, you see results at once)
2. CSS (makes it pretty)
3. JavaScript (your first real programming)
4. Node.js and Electron (power tools)
5. C (the foundation of low-level computing)
6. C++ (C with superpowers)
7. CMake (builds)
8. JSON and protocols (how programs talk)

QUIZ

1. Which language describes what is on the screen?
2. Which describes how it looks?
3. Why is HTML called a markup language and not a programming language?
4. Why was JavaScript chosen to build the IDE window?
5. Is NOVA's window written in C++? What is C++ used for in this book?
6. What does Node.js allow JavaScript to do that a browser does not?

ANSWERS

1. HTML.
2. CSS.
3. It only labels and structures things; it does not make decisions or calculate.
4. With Electron it gives a ready-made window, editor and terminal, needs far less code, and changes can be seen in seconds.
5. No. C and C++ are the languages that NOVA edits, builds and debugs, and the languages the helper tools are written in.
6. Read and write files, start other programs and use the network.

Part II — The Languages

CHAPTER 6

HTML: The Skeleton

6.1 What HTML is, in one minute

Every web page you have ever seen, and the window of every Electron program, starts as an **HTML** file. HTML tells the computer **what things are on the page**: "this is a heading", "this is a button", "this is a list".

> **KEY WORD** ***HTML***
> **HyperText Markup Language.** *Markup* means you wrap pieces of text in **tags** that label them. HTML does not calculate or decide. It only describes.

Think of a school report. The title is big and centred. Below are paragraphs. There may be a table and a picture. HTML is the way you tell the computer "this part is the title, this part is a paragraph, this part is a table".

6.2 Your first page

Create a file called `hello.html`. Type this exactly:

HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, NOVA!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

Double-click the file. Your browser opens it. You just made a page.

Let's read it line by line.

Line	Meaning
`<!doctype html>`	"This page uses modern HTML." Always the first line.
`<html lang="en">`	The root. Everything lives inside it. `lang` says the page is in English.
`<head>`	Information *about* the page that is not shown in the window.

Line	Meaning
`<meta charset="utf-8" />`	Tells the browser how to read letters, so that symbols and other languages show correctly.
`<title>`	The name shown on the browser tab.
`<body>`	Everything that you actually see.
`<h1>`	A top-level heading.
`<p>`	A paragraph.

WHY? ***Why do tags come in pairs?***

A tag like `<p>` is an **opening tag** and `</p>` is the **closing tag**. The pair says where the paragraph starts and where it ends. Without the closing tag the browser would have to guess, and it would often guess wrong, making the whole page strange. The text between is the **content**. The whole thing together is an **element**.

WHAT IF IT WERE DIFFERENT? ***What if you forget `<!doctype html>`?***

The browser switches to **quirks mode**, an old compatibility mode from the 1990s. Sizes and spacing behave differently, and your page looks subtly wrong. This is a good example of a tiny line that prevents a big headache. NOVA's `index.html` starts with it.

6.3 The tree behind the page

When the browser reads your HTML it builds a **tree** in memory called the **DOM** (Document Object Model). Every element is a branch, every piece of text a leaf.

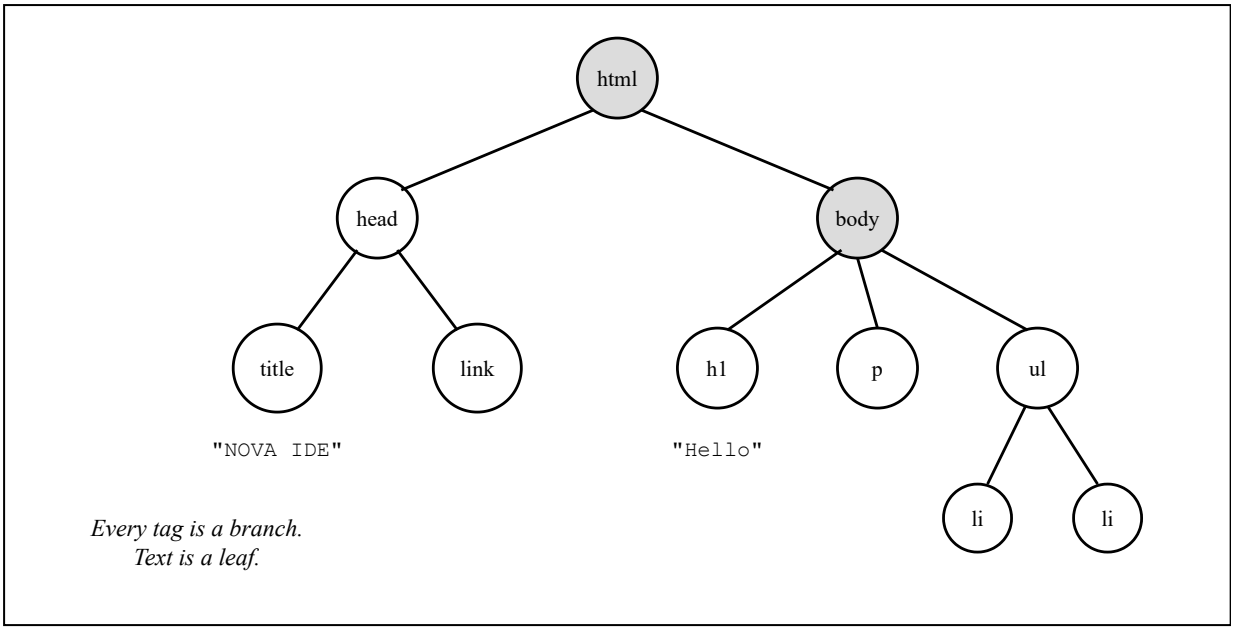


Figure 6.1. *The DOM tree. JavaScript can walk this tree and change any branch, which is how NOVA updates the screen.*

WHY? ***Why does the tree matter?***

Because JavaScript does not edit the text of your HTML file. It edits the **tree**. When NOVA opens a tab, JavaScript adds a new branch to the tree, and the browser redraws. Understanding the tree is the key to understanding every web program.

6.4 The elements you will use every day

Headings and text

HTML

```
<h1>Biggest heading</h1>
<h2>Second level</h2>
<h3>Third level</h3>
<p>A paragraph of text with <b>bold</b>, <i>italic</i> and <code>code</code>.
</p>
<br />   <!-- a line break -->
<hr />   <!-- a horizontal line -->
```

Headings go from `h1` (largest, use once) to `h6`. Use them for structure, not just for size. People who use screen readers jump from heading to heading to find their way.

Lists

HTML

```
<ul>            <!-- unordered: bullets -->
  <li>Editor</li>
  <li>Compiler</li>
  <li>Debugger</li>
</ul>

<ol>            <!-- ordered: numbers -->
  <li>Write</li>
  <li>Build</li>
  <li>Run</li>
</ol>
```

Links and images

HTML

```
<a href="https://example.com">Click me</a>
<img src="logo.png" alt="The NOVA logo" width="120" />
```

The **attribute** `href` says where a link goes. The attribute `alt` is text shown (or read aloud) if the picture cannot be seen. Never leave `alt` out.

Buttons and inputs

HTML

```
<button id="btnRun" title="Run (Shift+F10)">Run</button>
<input type="text" id="findQ" placeholder="Find in files…" />
<label><input type="checkbox" id="findCase" /> Match case</label>
<select id="theme">
  <option value="dark">Dark</option>
  <option value="light">Light</option>
</select>
```

This is very close to what NOVA's toolbar and search box are made of.

Tables

HTML

```
<table>
  <tr><th>Key</th><th>Action</th></tr>
  <tr><td>Shift+F10</td><td>Run</td></tr>
  <tr><td>Ctrl+F9</td><td>Build</td></tr>
</table>
```

Use tables for **data**, never for page layout. (CSS does layout, Chapter 7.)

> **TRY IT** ***Try it***
> Make a page called `about-me.html` with a heading, two paragraphs, a list of three hobbies and a link to a favourite website. Open it in the browser. Then change one tag to a different one (for example `ul` to `ol`) and refresh.

6.5 Attributes, ids and classes

An **attribute** gives extra information to a tag: `<a href="...">`. Two attributes are the most important in all of web programming:

- `id` gives an element a **unique name**: `<div id="sidebar">`. Only one element on the page may have a given id. JavaScript uses ids to find elements: `document.querySelector('#sidebar')`.
- `class` gives an element a **group name**: `<button class="ib sm">`. Many elements may share a class. CSS uses classes to style whole groups at once.

> **WHY?** ***Why both id and class?***
> An `id` is like a **passport number**: unique to one person. A `class` is like a **school uniform**: many people wear it. When you need to reach exactly one element (the Run button) use an id. When you want to style many (all small icon buttons) use a class. Mixing them up is a very common beginner error.

Data attributes

You can invent your own attributes starting with `data-`. NOVA uses them to remember which panel a button opens:

```
HTML
<button class="sb" data-panel="project" title="Project (Alt+1)">…</button>
<button class="sb" data-bottom="problems" title="Problems (Alt+6)">…</button>
```

JavaScript reads them back with `button.dataset.panel`. This lets one small piece of code serve many buttons, instead of writing a separate function for each.

6.6 Page layout with semantic tags

Old pages used `<div>` for everything. Modern HTML has tags that **say what they are**:

```
HTML
<header>  top bar or title area  </header>
<nav>     a group of navigation buttons  </nav>
<aside>   a sidebar  </aside>
<main>    the main content  </main>
<section> a themed group of content  </section>
<footer>  bottom strip  </footer>
```

NOVA's skeleton uses exactly these: a `<header id="titlebar">`, a `<nav class="stripe">`, an `<aside id="sidebar">`, a `<main id="center">`, a `<section id="bottom">` and a `<footer id="status">`. Look back at the anatomy figure in Chapter 2: those seven regions are these tags.

> **WHY?** ***Why use semantic tags instead of div?***
> For **meaning**. A screen reader can say "navigation" or "main content" and let the user jump there. Search engines understand the page better. And *you* will understand your own file better when you return to it in six months. Meaning costs nothing and helps everyone.

6.7 The hidden attribute, and a real NOVA bug

HTML has a built-in attribute called `hidden`. Any element with `hidden` is not shown:

```
HTML
<div id="bottom" hidden>…the bottom panel…</div>
```

NOVA turns panels on and off by setting `element.hidden = true` or `false` from JavaScript. But there is a trap that cost the team an hour.

WHAT IF IT WERE DIFFERENT? ***What if you also give that element a CSS `display` rule?***

If your CSS says `#bottom { display: flex; }`, that rule **beats** the browser's built-in "hidden means display none" rule, and the panel stays visible even though it has the `hidden` attribute! The fix is one line of CSS that makes `hidden` always win:

```
[hidden] { display: none !important; }
```

This line is the very first line of NOVA's `styles.css`. Now you know why.

6.8 Scripts and style sheets

An HTML file loads other files with two tags:

HTML

```
<link rel="stylesheet" href="styles.css" />   <!-- the CSS (looks) -->
<script src="app.js"></script>                <!-- the JavaScript (behaviour) --
>
```

Where you put the `<script>` matters. NOVA's scripts sit at the **bottom** of the body. Why? Because the browser reads the page top to bottom. If a script runs before the elements it needs exist, `document.querySelector` finds nothing. At the bottom, everything above already exists.

WHY? ***Why does the order of NOVA's scripts matter?***

In NOVA's `index.html`, the xterm terminal library is loaded **before** Monaco's loader. The reason is subtle: Monaco's loader creates a global function called `define`. The xterm library checks for `define` and, if it finds one, registers itself in a different way that does not create the global `Terminal` that NOVA needs. Loading xterm first avoids the clash. This was a real bug found during development, and it is a good lesson: **order of scripts can change behaviour**.

6.9 SVG: drawing with text

SVG (Scalable Vector Graphics) lets you draw shapes with HTML-like tags. Every icon in NOVA and every diagram in this book is SVG. It is just text, it stays sharp at any size, and you can colour it from CSS.

HTML

```
<svg viewBox="0 0 120 60" width="240">
  <rect x="5" y="5" width="110" height="50" rx="8" fill="white" stroke="black"
stroke-width="3"/>
  <circle cx="30" cy="30" r="12" fill="black"/>
  <text x="50" y="36" font-size="16">NOVA</text>
  <path d="M95 20 l10 10 l-10 10" fill="none" stroke="black" stroke-width="3"/>
</svg>
```

NOVA's icons are described by a single `path` string such as `M4.5 3l8 5-8 5z` (a triangle: *move to* a point, *line to* the next two, *close*). The whole icon set in `icons.js` is thirty lines.

> **WHY?** ***Why SVG icons and not picture files?***
> A picture file (PNG) has a fixed number of pixels: it gets blurry when enlarged and cannot change colour. SVG is maths: it stays crisp at any size, and by writing `stroke="currentColor"` the icon automatically takes the colour of the text around it, so a single icon works in both dark and light themes.

6.10 Accessibility in one page

About one person in six has some disability. Good HTML helps them for free:

- Use real `<button>` tags for buttons (not a styled `<div>`). Keyboard users can reach and press them.
- Always give images `alt` text and buttons a `title` or label.
- Use headings in order.
- Make sure text has strong contrast against its background.
- Make everything usable with the keyboard alone. (A programming tool should! NOVA has shortcuts for almost everything.)

6.11 Mini assignments

MINI ASSIGNMENT ***Assignment 6.1: Business card***

Build `card.html` for yourself: name as `h1`, job title as `h2`, a photo with `alt` text, a list of skills, and a link to an email address (`href="mailto:you@example.com"`).

MINI ASSIGNMENT ***Assignment 6.2: A table of shortcuts***

Make a table of five keyboard shortcuts from Appendix B with columns *Shortcut*, *Action*, *Where it works*. Use `<th>` for the header row.

MINI ASSIGNMENT ***Assignment 6.3: Your own icon***

Draw a simple icon (a house, a gear or a folder) in SVG inside a 16 by 16 `viewBox`. Use only `path`, `rect` and `circle`. Colour it with `stroke="currentColor"` and then wrap it in a `<button>`.

6.12 Big assignment: The skeleton of a mini IDE

BIG ASSIGNMENT ***Build "MiniIDE" in pure HTML***

Goal. Create `miniide.html`, a page that has the *structure* of an IDE but no colours or behaviour yet. You will style it in Chapter 7 and bring it to life in Chapter 8.

Requirements.

1. A `<header>` with a logo (inline SVG), a project name and three `<button>` elements: Run, Build and Stop.
2. A `<nav>` with four icon buttons (Project, Search, Problems, Terminal). Each carries a `data-panel` attribute.
3. An `<aside>` containing a nested list (`ul` inside `ul`) that looks like a project tree: a `src` folder with `main.cpp` and `util.cpp`, an `include` folder with `util.h`, and a `CMakeLists.txt`.
4. A `<main>` with a row of three tab elements and a `<pre><code>` block holding a small C++ program.
5. A `<section>` at the bottom with four tab buttons and four panels (Run, Problems, Find, Terminal). Only the Run panel has no `hidden` attribute.
6. A `<footer>` showing line, column, and the word "UTF-8".
7. Every interactive element has an `id` or a `data-` attribute that JavaScript will be able to find later.

Stretch goals. Add a `<dialog>` element for a Settings window that contains a `<select>` for theme and a `<input type="number">` for font size. Add `aria-label` attributes to every icon-only button.

How to check yourself. Open the file with no CSS. It will look plain, like a 1995 web page, but a person should be able to read it from top to bottom and understand the layout. Compare it with `renderer/index.html` in the NOVA project.

6.13 Quiz

QUIZ

1. What is the difference between an opening tag and a closing tag?

2. What is the DOM?
3. When would you use `id` and when `class`?
4. Why is `<!doctype html>` the first line?
5. Why should you use `<button>` rather than a styled `<div>`?
6. What is a `data-` attribute used for?
7. Give two benefits of SVG icons over PNG files.
8. A panel has the `hidden` attribute but is still visible. What is the most likely cause?
9. Why does NOVA load xterm.js before Monaco's loader?
10. What does `alt` do on an image?

ANSWERS

1. The opening tag (`<p>`) starts an element and the closing tag (`</p>`) ends it.
2. The Document Object Model: the tree the browser builds in memory from your HTML. JavaScript changes the tree.
3. Use `id` for one unique element; use `class` for a group of elements that share a style or behaviour.
4. It makes the browser use modern mode instead of the old "quirks mode", which changes sizes and spacing.
5. A real button works with the keyboard and screen readers automatically.
6. To store your own small pieces of information on an element so JavaScript can read them (`element.dataset`).
7. They stay sharp at any size and can take on the text colour (`currentColor`), so one icon works in all themes. They are also tiny.
8. A CSS `display` rule on that element overrides the browser's default for `hidden`. Fix with `[hidden] { display: none !important; }`.
9. Monaco's loader defines a global `define`, which makes xterm register as a module instead of creating the global `Terminal`.
10. It gives replacement text for people who cannot see the image (or if the image fails to load).

CHAPTER 7

CSS: The Skin

7.1 What CSS does

If HTML is the skeleton, **CSS** is the skin, clothes and make-up. It decides colours, sizes, spacing, fonts and where things sit on the screen.

> **KEY WORD** ***CSS***
> **Cascading Style Sheets.** A list of **rules**. Each rule says: *for elements that match this selector, set these properties*. "Cascading" means that when rules clash, there is a fair order for deciding which one wins.

7.2 Your first rule

CSS

```
h1 {
  color: black;
  font-size: 28pt;
}
```

Parts of a rule:

- `h1` is the **selector**: which elements to style.
- Between the braces `{ }` is the **declaration block**.
- `color: black;` is one **declaration**: a **property** (`color`), a colon, a **value** (`black`) and a semicolon.

Three ways to add CSS to HTML:

1. **External file** (best): `<link rel="stylesheet" href="styles.css" />`. NOVA does this.
2. **A `<style>` block** inside the page head.
3. **Inline** on one tag: `<p style="color:red">`. Quick but messy.

> **WHY?** ***Why keep CSS in its own file?***
> Because one file can style many pages, the browser remembers (caches) it, and you change the look of the whole program in one place. Mixing CSS into HTML is like writing instructions on every single brick of a house.

7.3 Selectors: choosing what to style

Selector	Picks	Example
`p`	every `<p>`	`p { margin: 0; }`
`.tab`	every element with class `tab`	`.tab { padding: 0 12px; }`
`#sidebar`	the one element with id `sidebar`	`#sidebar { width: 280px; }`
`.tab.active`	has *both* classes	`.tab.active { color: black; }`
`.node:hover`	when the mouse is over it	`.node:hover { background: grey; }`
`#tabs .tab`	a `.tab` *inside* `#tabs`	
`button:disabled`	disabled buttons	`opacity: .35;`
`li:nth-child(even)`	every second item	zebra stripes

> **TRY IT** ***Try it***
> Make a page with six `<li>` items. Write a rule that gives every even item a grey background using `:nth-child(even)`, and a rule that makes an item bold when the mouse is over it.

7.4 Units and colours

Sizes: `px` (pixels, fixed), `%` (percentage of the parent), `em`/`rem` (relative to font size), `vh`/`vw` (percent of the window height/width), `pt` (points; used in this book for text).

Colours can be names (`black`), hex codes (`#3574f0`), or `rgb(53, 116, 240)`. A hex code is red, green and blue written in base 16: `#RRGGBB`. An 8-digit code adds transparency: `#ffffff1c` is white at about 11 percent opacity (NOVA uses it for hover highlights).

7.5 The box model: everything is a box

This is the single most important idea in CSS. **Every element is a rectangular box** made of four layers.

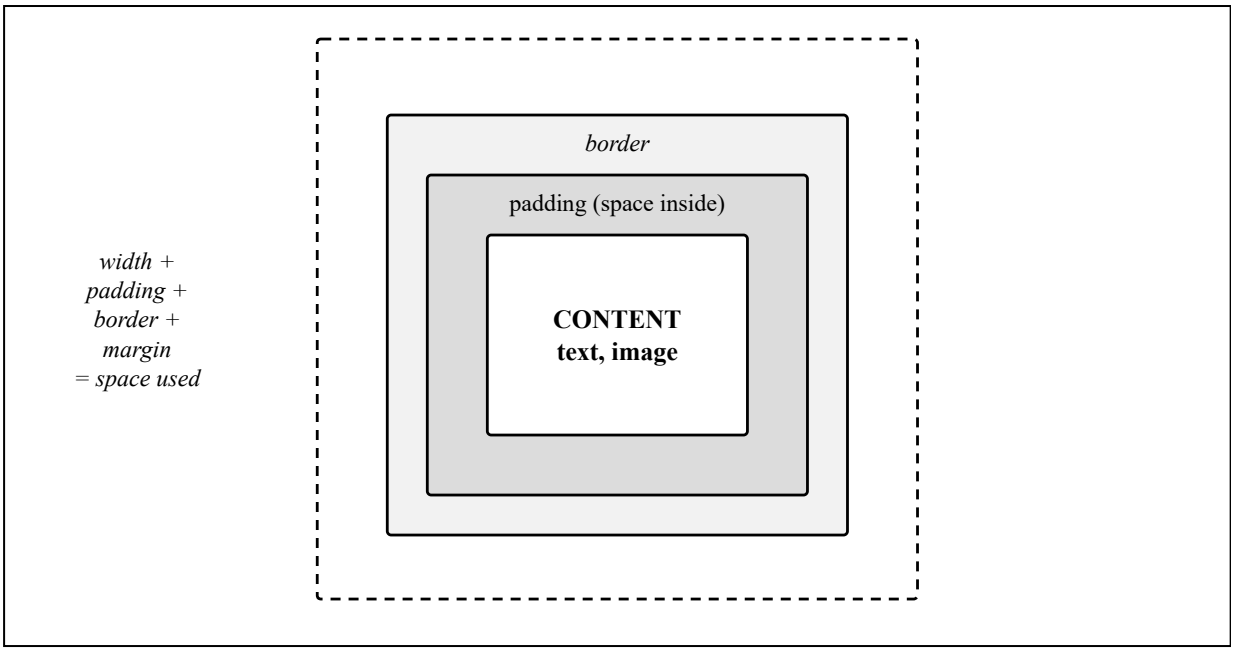


Figure 7.1. *The CSS box model. Total width = content + padding + border + margin.*

CSS

```
.node {
  padding: 4px 8px;        /* space INSIDE the border */
  border: 1px solid grey;  /* the line */
  margin: 2px 0;           /* space OUTSIDE the border */
}
```

WHY? ***Why does NOVA say box-sizing: border-box?***

By default, `width: 100px` sets only the **content** width, and padding and border are added on top, so the box ends up wider than 100px. That surprises everyone. One of the first rules in NOVA's CSS is `* { box-sizing: border-box; }`, which makes `width` include padding and border. A 100px box stays 100px. Almost every modern site starts with this line.

WHAT IF IT WERE DIFFERENT? ***What if you skip it?***

Imagine a panel set to `width: 100%` with `padding: 10px`. Without `border-box` it is 100% **plus** 20px wide, so it sticks out past the window and a horizontal scrollbar appears. This is one of the most common layout bugs in the world.

7.6 Variables: change the whole look in one place

CSS has **custom properties** (variables). You define them once and use them everywhere with `var()`:

CSS

```
:root {
  --bg: #1e1f22;      /* main background */
  --text: #dfe1e5;    /* text colour */
  --accent: #3574f0;  /* blue highlights */
}
body  { background: var(--bg); color: var(--text); }
.btn.primary { background: var(--accent); }
```

`:root` means "the top of the page", so the variables are available everywhere. Now watch the trick that gives NOVA its themes:

CSS

```
:root[data-theme="light"] {
  --bg: #ffffff;
  --text: #1e1f22;
}
```

When JavaScript sets `document.documentElement.dataset.theme = 'light'`, the second block takes over, every `var(--bg)` changes, and the entire IDE is light. No other code is needed.

> **WHY?** ***Why variables instead of typing the colours everywhere?***
> NOVA uses its background colour in dozens of places. With variables, changing it is a one-line edit. Without them you would hunt through the file, miss three spots, and end up with an ugly mismatch. Variables are also what make themes (dark, light, high-contrast) cheap to add.

7.7 Layout, part 1: Flexbox

Flexbox arranges items in a **row or a column** and spreads out the space for you. It is how almost every toolbar is made.

CSS

```
.toolbar {
  display: flex;                 /* turn flexbox on for the children */
  align-items: center;           /* line them up vertically */
  justify-content: space-between;/* push first and last to the edges */
  gap: 6px;                      /* space between items */
}
```

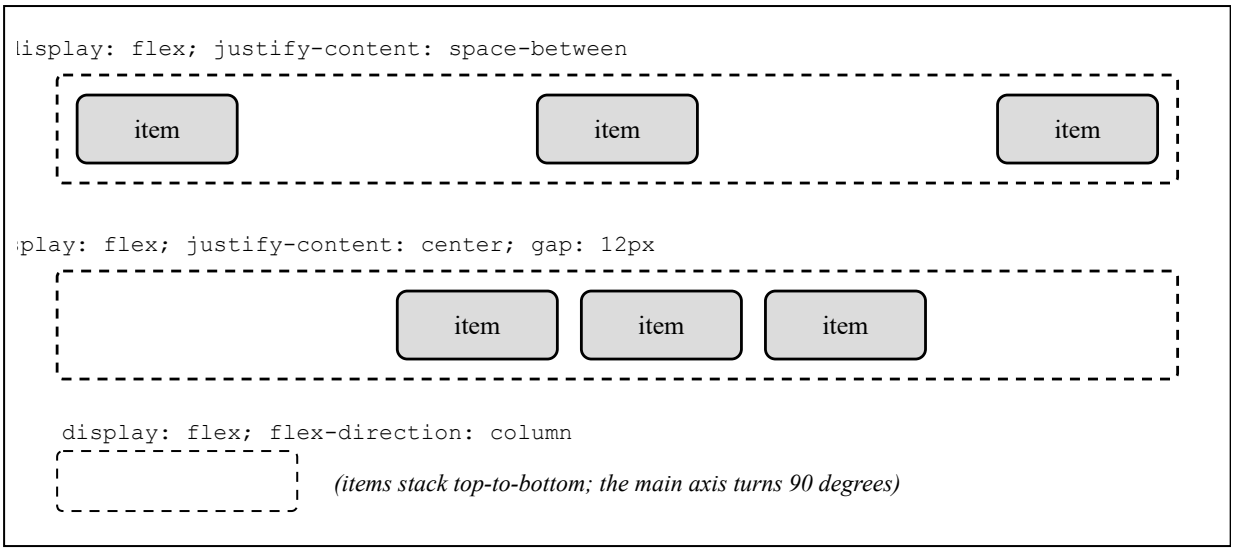


Figure 7.2. *Three flexbox settings. justify-content moves items along the main axis; flex-direction turns the axis.*

Key flexbox properties:

Property	What it controls
`display: flex`	Starts flexbox on the parent
`flex-direction`	`row` (sideways) or `column` (downwards)
`justify-content`	Spacing along the main axis
`align-items`	Alignment across the main axis
`gap`	Space between items
`flex: 1`	"Take all the leftover space"

NOVA's tab bar, status bar, toolbar, bottom-panel header and the Debug panel's button row are all flexbox. The central column (`#center`) uses `display: flex; flex-direction: column` so the tabs sit on top of the editor, which has `flex: 1` and grows to fill the rest.

7.8 Layout, part 2: Grid

Grid places things in **rows and columns at once**. It is perfect for whole-window layouts.

CSS

```
#app {
  display: grid;
  grid-template-rows: 40px 1fr auto auto 24px;  /* title, workspace, splitter,
bottom, status */
  height: 100vh;
}
#workspace {
  display: grid;
  grid-template-columns: 40px 280px 4px 1fr;    /* stripe, sidebar, splitter,
editor */
}
```

The magic unit is `fr` (a *fraction* of the free space). `1fr` means "whatever is left after the fixed sizes".

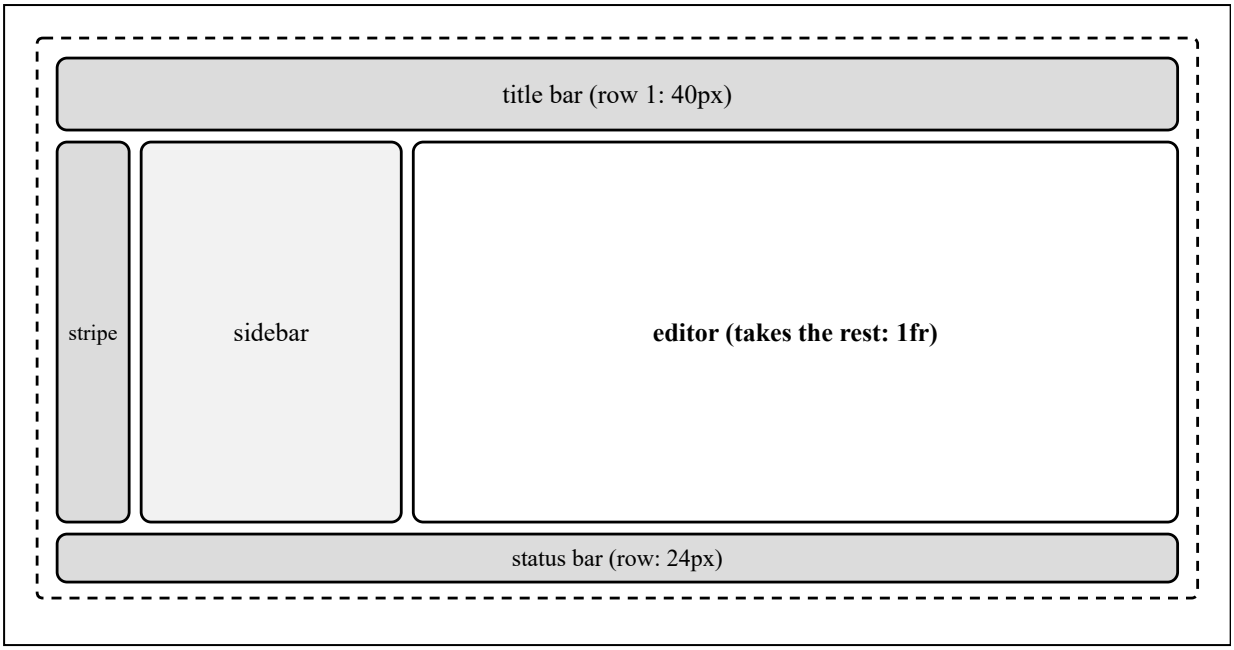


Figure 7.3. *NOVA's window is a grid. Fixed rows and columns surround one flexible area, the editor.*

> **WHY?** ***Why 1fr for the editor?***
> When you resize the window, the title bar must stay 40px tall and the stripe 40px wide, while the editor absorbs all the change. `1fr` expresses exactly that. Doing this with percentages would need maths and would break at odd sizes.

CSS variables in the grid: resizable panels

NOVA lets you drag the sidebar edge. The grid width is written as `var(--side-w, 280px)`. While you drag, JavaScript simply changes the variable:

JAVASCRIPT

```
document.documentElement.style.setProperty('--side-w', newWidth + 'px');
```

The grid recalculates by itself. No loops, no manual resizing of children.

7.9 Position, overflow and layers

- `position: absolute; inset: 0;` stretches an element to fill its nearest positioned parent. NOVA's editor and welcome screen both use it so they can sit on top of each other.
- `position: fixed` pins an element to the window (used for popup menus and toasts).
- `overflow: auto` adds scrollbars only when needed.
- `z-index` decides which overlapping element is on top. NOVA's modal overlay uses a high number so it covers the app.

> **WHAT IF IT WERE DIFFERENT?** ***What if the sidebar had no min-height: 0?***
> This is a famous flexbox and grid trap. Flex and grid children refuse to shrink below their content size unless you say `min-height: 0`. Without it, a long file tree would make the sidebar taller than the window and the whole page would scroll instead of the tree. NOVA's CSS sets `min-height: 0` on the workspace and panels for this reason.

7.10 Making it feel alive: transitions and animation

CSS

```
.chip { transition: background .12s; }
.chip:hover { background: var(--hover); }

@keyframes spin { to { transform: rotate(360deg); } }
#stBuild.busy::before { animation: spin .7s linear infinite; }
```

- `transition` smooths a change over time (0.12 seconds is quick but visible).
- `@keyframes` describes an animation. NOVA's little spinner in the status bar is a circle with one transparent side, rotated forever.
- `::before` and `::after` are **pseudo-elements**: extra decoration that CSS can add without touching HTML.

> **WHY?** ***Why are NOVA's animations so short?***
> Tools are used hundreds of times a day. A half-second fade is charming the first time and annoying the hundredth. Professional tools keep motion under about 150 milliseconds: enough to feel smooth, too short to wait for.

7.11 The cascade: who wins when rules clash?

If two rules set the same property on the same element, the browser picks the winner in this order:

1. `!important` beats everything (use very rarely).

2. **More specific** selectors beat less specific. An id (`#sidebar`) beats a class (`.tab`), which beats a tag (`p`).
3. If still tied, the rule that comes **later** in the file wins.

> **WATCH OUT** ***Watch out***
> Do not fight specificity with `!important` everywhere. NOVA uses it in only a handful of places, each for a good reason: forcing `[hidden]` to hide, letting the debugger line highlight and breakpoint dots beat Monaco's own styles, and keeping the terminal background transparent. If you need it twice in a row, rethink the selectors.

7.12 Media: dark and light, and print

CSS can read the user's preferences. For example, a page can follow the system theme:

CSS

```
@media (prefers-color-scheme: dark) {
  :root { --bg: #1e1f22; }
}
```

And it can prepare a page for printing. This very book is made from HTML and CSS: the black-and-white boxes, the 1.5 line spacing and the Times New Roman font come from a style sheet. For example:

CSS

```
body { font-family: 'Times New Roman', serif; font-size: 12pt; line-height: 1.5;
}
.co-why { border-left: 7px solid #000; background: #f7f7f7; }
```

7.13 Mini assignments

> **MINI ASSIGNMENT** ***Assignment 7.1: Style your card***
> Take your `card.html` from Chapter 6. Add a style sheet that gives the card a 1px border, 20px padding, 12px rounded corners, a maximum width of 400px, and centres it on the page (`margin: 40px auto`).

> **MINI ASSIGNMENT** ***Assignment 7.2: Two themes***
> Define `--bg`, `--text` and `--accent` variables. Add a button that toggles `data-theme` between `dark` and `light` (you will need one line of JavaScript, shown in Chapter 8). Check that every colour on the page comes from a variable.

MINI ASSIGNMENT ***Assignment 7.3: A toolbar***

Build a row of five buttons using flexbox. The first button sits at the far left, the last at the far right, and the middle three are centred together. Hint: `margin-left: auto` on the last item.

7.14 Big assignment: Make MiniIDE beautiful

BIG ASSIGNMENT ***Style the MiniIDE from Chapter 6***

Goal. Turn your plain `miniide.html` into something that looks like a real IDE, using only CSS.

Requirements.

1. Use CSS variables for **at least eight** colours. Provide a dark theme (default) and a light theme (`:root[data-theme="light"]`).
2. Make the whole page a **grid**: title bar (40px), middle area (1fr), bottom panel (200px), status bar (24px). The page must exactly fill the window with no page scrollbar.
3. The middle area is a second **grid** with columns: icon stripe (40px), sidebar (260px), editor (1fr).
4. The project tree shows indentation for nested folders and a hover highlight. The selected file has a blue background.
5. The editor tabs are a flex row. The active tab has a coloured bottom border and brighter text.
6. The code block has a monospace font, line height 1.5, and a subtle border.
7. Buttons have rounded corners, hover and pressed states, and a disabled look. The Run button is green, Stop is red.
8. The status bar has a small spinning circle (CSS animation) next to "Building…".
9. Add `[hidden] { display: none !important; }` and prove why you need it by temporarily removing it.
10. Everything must still work when the window is narrowed to 700px (no overlapping text).

Stretch goals. Add a draggable splitter by changing a CSS variable from 5 lines of JavaScript. Add a `@media print` rule that hides the toolbar and stripe. Recreate NOVA's rounded "chip" buttons exactly.

How to check yourself. Put your page next to the screenshot of NOVA in Chapter 1. The layout and proportions should look related.

7.15 Quiz

QUIZ

1. What are the three parts of a CSS declaration?
2. Which wins: `#sidebar` or `.tab`? Why?
3. Write the box model layers from inside to outside.
4. What does `box-sizing: border-box` change?
5. What does `1fr` mean in a grid?
6. How does NOVA switch between dark and light themes?
7. What does `[hidden] { display: none !important; }` fix?
8. Why do flex and grid children sometimes need `min-height: 0`?
9. What is a pseudo-element? Give one example.
10. Why are NOVA's transitions about 0.12 seconds long?

ANSWERS

1. A property, a colon with a value, and a semicolon (e.g. `color: black;`).
2. `#sidebar`: an id is more specific than a class.
3. Content, padding, border, margin.
4. `width` and `height` then include padding and border, so boxes do not grow unexpectedly.
5. One fraction of the free space left after fixed sizes are removed.
6. JavaScript sets the `data-theme` attribute on the root element; a second rule block redefines the colour variables.
7. A CSS `display` rule overriding the browser's `hidden` behaviour: it forces hidden elements to stay hidden.
8. They refuse to shrink below their content size by default; `min-height: 0` lets them shrink so the content can scroll.
9. Extra generated content or styling attached to an element, such as `::before` (NOVA's spinner uses it).
10. Short motion feels smooth but never makes the user wait, which matters for tools used hundreds of times a day.

CHAPTER 8

JavaScript: The Brain

8.1 What JavaScript is

JavaScript (JS) is the language that makes things *happen*. HTML says what is on the page and CSS says how it looks, but only JavaScript can say *"when the user clicks Run, do this"*. Most of NOVA is JavaScript: about two thousand lines.

JavaScript began in 1995 as a quick scripting tool for web pages. Today it runs in browsers, on servers (Node.js), in desktop apps (Electron), on phones and even on tiny devices. It is one of the most used languages in the world.

> **KEY WORD** ***Script***
> A **script** is a program that is run directly from its text by an interpreter or engine, without you producing a separate program file first.

8.2 Running JavaScript

There are two easy places to run it.

1. **The browser console.** Press F12 in any browser, click *Console*, and type. Great for quick tests.
2. **Node.js.** Save code in a file such as `test.js` and run `node test.js` in a terminal. All the runnable examples in this chapter work this way, and the outputs shown were produced by really running them while this book was made.

Your first program:

JAVASCRIPT

```
console.log("Hello, NOVA!");
console.log(2 + 3);
```

OUTPUT

```
Hello, NOVA!
5
```

`console.log` prints a value. It is your best friend for finding out what your program is doing.

8.3 Variables and types

A **variable** is a labelled box that holds a value. Use `const` for values that never change and `let` for values that will.

JAVASCRIPT

```
const name = "NOVA";         // text (a "string")
let version = 1;             // a number
const isFree = true;         // true or false (a "boolean")
const nothing = null;        // "on purpose, there is nothing"
let notSet;                  // undefined: "nobody gave me a value"

version = version + 1;       // allowed: let can change
console.log(name, version, isFree, nothing, notSet);
console.log(typeof name, typeof version, typeof isFree);
```

OUTPUT

```
NOVA 2 true null undefined
string number boolean
```

WHY? ***Why prefer const?***

When you read `const total = 5`, you know instantly that `total` never changes, so you can stop worrying about it. Fewer things that can change means fewer places for a bug to hide. NOVA's code uses `const` almost everywhere and `let` only for things that must change, such as a counter.

WATCH OUT ***Watch out***

Older JavaScript used `var`. It has confusing scope rules. You will see it in old code but should not write it in new code.

Strings and template strings

JAVASCRIPT

```
const lang = "C++";
const n = 3;
console.log("I like " + lang + " " + n + " times");   // the old, clumsy way
console.log(`I like ${lang} ${n} times`);              // template string: much
nicer
console.log(`Two plus two is ${2 + 2}`);              // any expression fits in
${ }
console.log("hello".toUpperCase(), "a,b,c".split(","), "  trim me ".trim());
console.log("NOVA IDE".includes("IDE"), "NOVA".length, "abc".slice(1));
```

OUTPUT

```
I like C++ 3 times
I like C++ 3 times
Two plus two is 4
HELLO [ 'a', 'b', 'c' ] trim me
true 4 bc
```

Template strings use backticks (the key left of `1`). NOVA uses them constantly to build HTML and messages: `` `${rel(f)} (${rs.length})` `` makes the file headings in the Find panel.

Numbers

JAVASCRIPT

```
console.log(10 / 3);              // 3.3333333333333335
console.log(Math.floor(10 / 3));  // 3
console.log(10 % 3);              // 1 (the remainder)
console.log(0.1 + 0.2);           // 0.30000000000000004  (!)
console.log((0.1 + 0.2).toFixed(2));
console.log(Number("42") + 1, parseInt("12px"), Math.max(3, 9, 4));
```

OUTPUT

```
3.3333333333333335
3
1
0.30000000000000004
0.30
43 12 9
```

> **WHY?** ***Why is 0.1 + 0.2 not 0.3?***
> Computers store numbers in binary. Just as one third cannot be written exactly in decimal (0.3333...), a tenth cannot be written exactly in binary. The tiny error is tolerated in most work. Never compare decimals with `===`. The same behaviour exists in C and C++ (`double`), so you will meet it again.

8.4 Comparing and deciding

JAVASCRIPT

```
console.log(5 === 5, 5 === "5", 5 == "5");   // three equals: strict. Two:
sloppy.
console.log(5 !== 6, 3 < 4, 3 >= 3);
console.log(true && false, true || false, !true);

const age = 17;
if (age >= 18) {
  console.log("adult");
} else if (age >= 13) {
  console.log("teen");
} else {
  console.log("child");
}
console.log(age >= 18 ? "can vote" : "cannot vote yet");   // the "ternary"
shortcut
```

OUTPUT

```
true false true
true true true
false true false
teen
cannot vote yet
```

WHY? ***Why always ===?***

`==` quietly converts types before comparing, so `5 == "5"` is true and `0 == ""` is true, which causes strange bugs. `===` compares both value *and* type and does what you expect. Use it always. (This is a good habit that C and C++ also reward: be exact about types.)

8.5 Loops

JAVASCRIPT

```
for (let i = 1; i <= 3; i++) {
  console.log("lap", i);
}

let n = 3;
while (n > 0) {
  console.log("countdown", n);
  n--;
}

for (const word of ["editor", "compiler", "debugger"]) {
  console.log("part:", word);
}
```

OUTPUT

```
lap 1
lap 2
lap 3
countdown 3
countdown 2
countdown 1
part: editor
part: compiler
part: debugger
```

8.6 Functions

A **function** is a named recipe you can use again and again.

JAVASCRIPT

```
function add(a, b) {
  return a + b;
}
const multiply = (a, b) => a * b;           // arrow function: shorter
const greet = (name = "friend") => `Hello, ${name}!`;   // default value

console.log(add(2, 3), multiply(4, 5));
console.log(greet(), greet("Team NOVA"));
```

OUTPUT

```
5 20
Hello, friend! Hello, Team NOVA!
```

Why functions? They stop you repeating yourself and give names to ideas. NOVA's very first helper is a function:

JAVASCRIPT

```
const $ = (s, r = document) => r.querySelector(s);
```

It is a shortcut: instead of writing `document.querySelector('#tabs')` forty times, NOVA writes `$('#tabs')`. The `r = document` is a **default value**: if you do not pass a second argument it searches the whole page.

8.7 Arrays: lists of things

JAVASCRIPT

```
const parts = ["editor", "tree", "terminal"];
parts.push("debugger");                        // add to the end
console.log(parts.length, parts[0], parts[parts.length - 1]);

const lengths = parts.map(p => p.length);              // transform every item
const longOnes = parts.filter(p => p.length > 5);      // keep some items
const total = lengths.reduce((sum, n) => sum + n, 0);  // boil down to one
value
console.log(lengths, longOnes, total);

console.log(parts.find(p => p.startsWith("t")));       // first match
console.log(parts.some(p => p === "tree"), parts.every(p => p.length > 3));
console.log([...parts].sort(), parts.includes("tree"),
parts.indexOf("terminal"));
console.log(parts.slice(1, 3), parts.join(" | "));
```

OUTPUT

```
4 editor debugger
[ 6, 4, 8, 8 ] [ 'editor', 'terminal', 'debugger' ] 26
tree
true true
[ 'debugger', 'editor', 'terminal', 'tree' ] true 2
[ 'tree', 'terminal' ] editor | tree | terminal | debugger
```

`map`, `filter` and `reduce` are the three most useful tools in JavaScript. Learn them well.

> **WHY?** ***Why map and filter instead of a for loop?***
> You *can* do the same with a loop. But `map` says *"make a new list by changing each item"* in a single readable line, and it cannot accidentally change the original. NOVA's file search is one line of this style: `files.filter(f => /\.(c|cc|cpp|cxx)$/i.test(f))` picks the C and C++ files out of a project.

8.8 Objects: things with named parts

JAVASCRIPT

```
const tab = { name: "main.cpp", dirty: false, line: 12 };
tab.dirty = true;                       // change
tab.language = "C++";                   // add
console.log(tab.name, tab["line"], tab);

const { name, line } = tab;             // destructuring: pull parts out
console.log(name, line);

const copy = { ...tab, line: 99 };      // spread: copy and override
console.log(copy.line, tab.line);

console.log(Object.keys(tab));
console.log(JSON.stringify(tab));
console.log(JSON.parse('{"a":1,"b":[2,3]}').b[1]);
```

OUTPUT

```
main.cpp 12 { name: 'main.cpp', dirty: true, line: 12, language: 'C++' }
main.cpp 12
99 12
[ 'name', 'dirty', 'line', 'language' ]
{"name":"main.cpp","dirty":true,"line":12,"language":"C++"}
3
```

NOVA's whole state is one object, `S`, with fields such as `S.root`, `S.tabs`, `S.active` and `S.settings`. Having **one place** where the state lives makes it easy to find any fact about the IDE.

> **WHY?** ***Why one big state object?***
> When the window shows the wrong thing, the first question is "what does the program *believe*?". If every fact lives in one object you can type `S` in the developer tools and read the answer. Scattering state over many variables makes bugs much harder to find. Many professional frameworks (Redux, Vuex) are built on this exact idea.

8.9 Collections: Map and Set

JAVASCRIPT

```
const seen = new Set([1, 2, 2, 3, 3, 3]);       // each value only once
console.log(seen.size, seen.has(2));

const cache = new Map();                         // key to value, any type of
key
cache.set("main.cpp", 120);
cache.set("util.cpp", 45);
console.log(cache.get("main.cpp"), cache.has("x"), [...cache.keys()]);
```

OUTPUT

```
3 true
120 false [ 'main.cpp', 'util.cpp' ]
```

NOVA uses a `Set` for the folders that are open in the tree (`S.expanded`) and a `Map` to remember which breakpoint number gdb gave each line.

8.10 Strings, patterns and regular expressions

A **regular expression** ("regex") is a pattern for matching text. It looks scary, but it is a small tool you learn in parts.

JAVASCRIPT

```
console.log(/\d+/.exec("line 42 here"));            // finds the digits
console.log("a1b22c333".match(/\d+/g));             // all matches
console.log("2026-11-12".replace(/-/g, "/"));
console.log(/^[A-Z]/.test("NOVA"), /^[A-Z]/.test("nova"));
```

OUTPUT

```
[ '42', index: 5, input: 'line 42 here', groups: undefined ]
[ '1', '22', '333' ]
2026/11/12
true false
```

The most useful pieces: `\d` a digit, `\w` a letter/digit/underscore, `\s` a space, `.` any character, `+` one or more, `*` zero or more, `?` optional, `^` start, `$` end, `( )` capture a part, `[abc]` any of these.

A real NOVA regex: reading compiler errors

When g++ finds a mistake it prints a line like `src/main.cpp:14:9: error: 'x' was not declared`. NOVA must pull that line apart to draw a wavy underline. Here is the real pattern from `app.js`:

JAVASCRIPT

```
const DIAG_RE = /^((?:[A-Za-z]:)?[^:\n]+?):(\d+):(?:(\d+):)? (fatal
error|error|warning|note): (.*)$/;

const lines = [
  "src/main.cpp:14:9: error: 'x' was not declared in this scope",
  "C:/proj/util.cpp:7:1: warning: unused variable 'n'",
  "Building CXX object main.o",
];
for (const line of lines) {
  const m = line.match(DIAG_RE);
  if (m) {
    const [, file, row, col, kind, msg] = m;
    console.log({ file, row: +row, col: +col, kind, msg });
  } else {
    console.log("(not an error line)", line);
  }
}
```

OUTPUT

```
{
  file: 'src/main.cpp',
  row: 14,
  col: 9,
  kind: 'error',
  msg: "'x' was not declared in this scope"
}
{
  file: 'C:/proj/util.cpp',
  row: 7,
  col: 1,
  kind: 'warning',
  msg: "unused variable 'n'"
}
(not an error line) Building CXX object main.o
```

WHY? ***Why the odd bit (?:[A-Za-z]:)?***

On Windows a path starts with a drive letter and a colon, like `C:/proj`. But the colon is also the separator between file, line and column! The optional group `(?:[A-Za-z]:)?` says "a drive letter and colon may come first, and that colon belongs to the file name". Without it, `C` would be treated as the file and `/proj/util.cpp` as the line number, and every error on a Windows path would be lost. This is a bug the team met on the very first test.

8.11 Scope and closures

A variable lives in the **scope** where it was created: inside `{ }`, inside a function, or global. An inner function can see the variables of every box around it. A function that **remembers** variables from where it was created is called a **closure**.

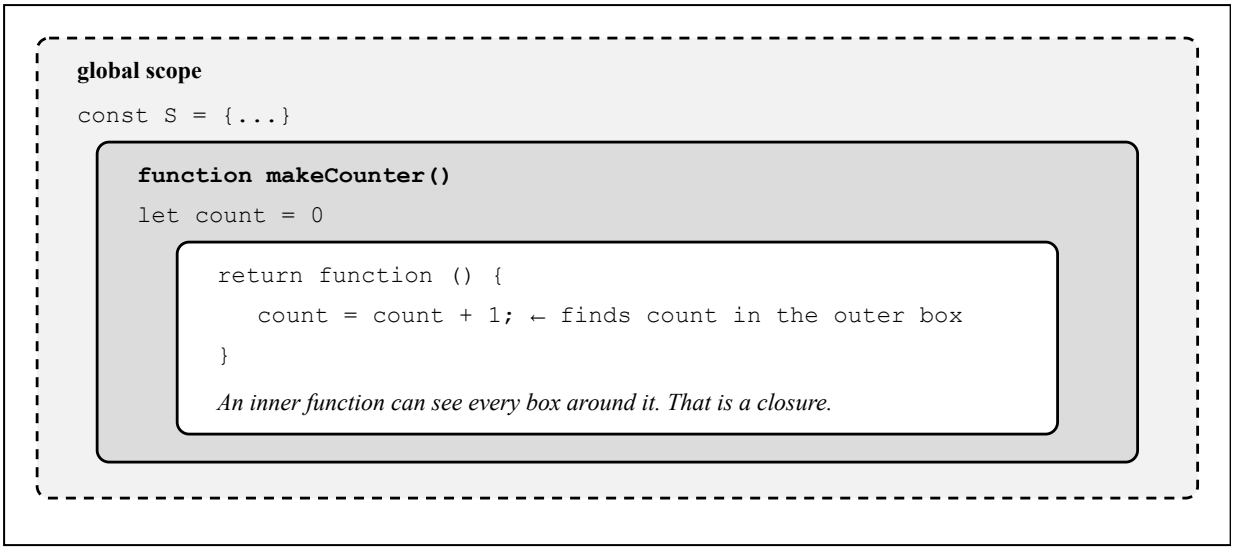


Figure 8.1. *An inner function can see outwards through every enclosing box. That is a closure.*

JAVASCRIPT

```
function makeCounter() {
  let count = 0;                  // private to makeCounter
  return () => ++count;           // the inner function remembers count
}
const a = makeCounter();
const b = makeCounter();
console.log(a(), a(), a(), b());   // each counter has its own count
```

OUTPUT

```
1 2 3 1
```

> **WHY?** ***Why do closures matter?***
>
> They let you keep private data and build tools that "remember". NOVA's `splitter` function creates one dragging behaviour for each divider, and each one remembers *which* divider it belongs to. Closures are also why event handlers work: the handler remembers the tab it was made for.

8.12 Classes (a short look)

JAVASCRIPT

```
class Tab {
  constructor(name) { this.name = name; this.dirty = false; }
  edit() { this.dirty = true; return this; }
  label() { return this.dirty ? `${this.name} ●` : this.name; }
}
const t = new Tab("main.cpp");
console.log(t.label());
console.log(t.edit().label());
```

OUTPUT

```
main.cpp
main.cpp ●
```

NOVA mostly uses plain objects and functions, which is a style many JavaScript programmers prefer for small programs. You will see classes in the C++ chapter, where they are central.

8.13 When things go wrong: errors

JAVASCRIPT

```
function parsePort(text) {
  const n = Number(text);
  if (!Number.isInteger(n) || n < 1 || n > 65535) throw new Error("Bad port: " +
text);
  return n;
}
try {
  console.log(parsePort("8080"));
  console.log(parsePort("hello"));
} catch (e) {
  console.log("Caught:", e.message);
} finally {
  console.log("done");
}
```

OUTPUT

```
8080
Caught: Bad port: hello
done
```

`throw` raises an error; `try/catch` handles it so the program does not crash. NOVA wraps file opening in `try/catch` so that opening a broken file shows a friendly message instead of freezing the IDE.

8.14 Time: the event loop, callbacks, promises and async/await

This is the hardest and most important part of JavaScript. Take it slowly.

One thing at a time

JavaScript can do **only one thing at a time**. Imagine a single waiter in a restaurant. If the waiter stood at the kitchen for ten minutes waiting for one soup, nobody else would be served. Instead the waiter drops the order at the kitchen and serves other tables. When the soup is ready, a bell rings.

In JavaScript the slow jobs (reading a file, waiting for a timer, a network request) are handed away. When they finish, their **callback** waits in a queue. The **event loop** checks over and over: *"is the waiter free? Then serve the next callback."*

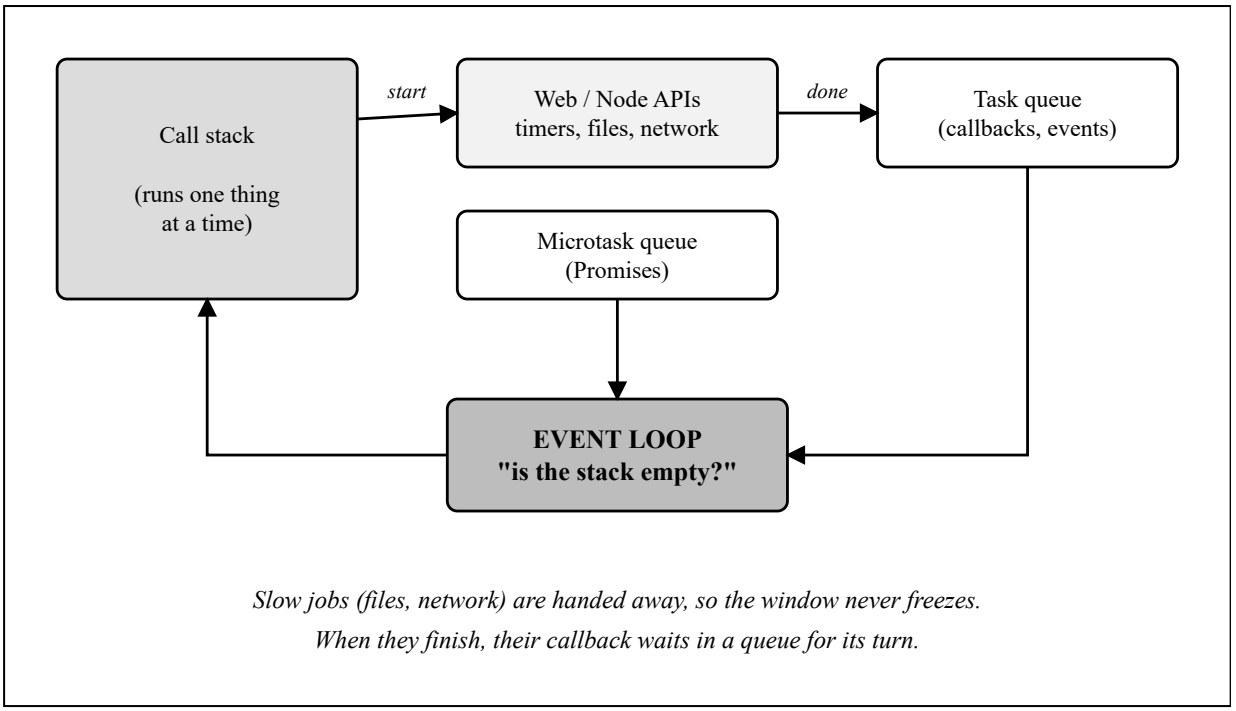


Figure 8.2. *The event loop. Slow jobs run elsewhere. Finished jobs queue up, and the loop feeds them one at a time to the call stack.*

JAVASCRIPT

```
console.log("1 start");
setTimeout(() => console.log("4 timer (even with 0 delay)"), 0);
Promise.resolve().then(() => console.log("3 promise"));
console.log("2 end");
```

OUTPUT

```
1 start
2 end
3 promise
4 timer (even with 0 delay)
```

Study the output. Even a timer with zero delay waits until the current code has finished. Promises jump the queue and run before timers.

> **WHY?** ***Why does this matter for an IDE?***
> If NOVA read a big file with a method that *blocked* the waiter, the whole window would freeze until the disk finished, and you could not click, type or scroll. Because every disk, process and message operation is **asynchronous**, the window always stays alive. This is the single biggest reason an Electron IDE feels smooth.

Callbacks

The oldest style: pass a function to be called later.

JAVASCRIPT

```
function fakeRead(file, done) {
  setTimeout(() => done(null, "contents of " + file), 100);
}
fakeRead("main.cpp", (err, text) => {
  console.log(err, text);
});
console.log("waiting...");
```

OUTPUT

```
waiting...
null contents of main.cpp
```

The trouble comes when jobs depend on each other: callback inside callback inside callback (nicknamed "callback hell").

Promises

A **promise** is an IOU: "I do not have the value yet, but I will either give you one or tell you why not."

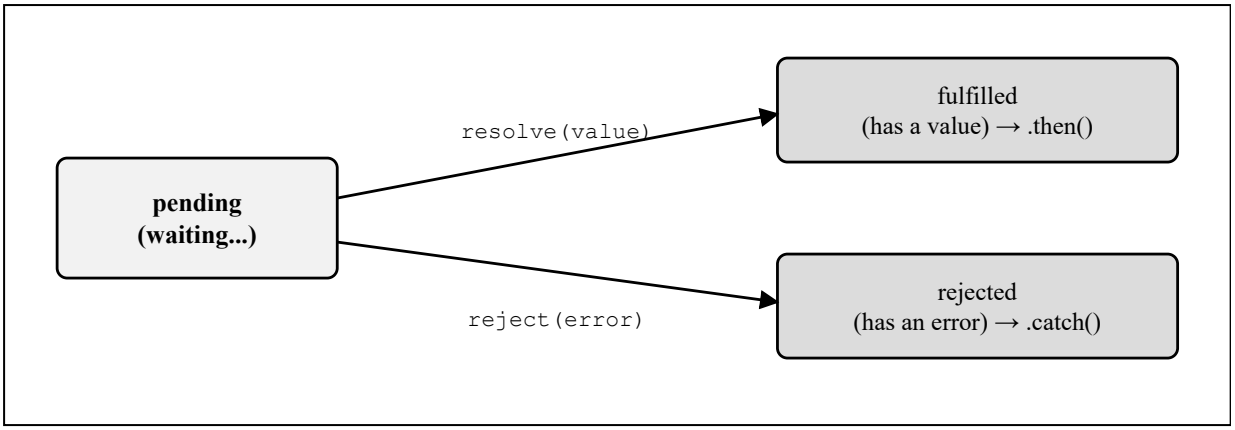


Figure 8.3. *A promise is pending, then becomes fulfilled with a value or rejected with an error.*

JAVASCRIPT

```
const wait = (ms) => new Promise(resolve => setTimeout(resolve, ms));

wait(100)
  .then(() => { console.log("step 1"); return 42; })
  .then(n => { console.log("step 2 got", n); throw new Error("oops"); })
  .catch(e => console.log("caught", e.message));
```

OUTPUT

```
step 1
step 2 got 42
caught oops
```

async and await

`async/await` is a nicer way to write promises so that asynchronous code reads like normal top-to-bottom code.

JAVASCRIPT

```
const wait = (ms, v) => new Promise(r => setTimeout(() => r(v), ms));

async function build() {
  console.log("configuring...");
  const cfg = await wait(100, "config ok");     // pause THIS function, not the
program
  console.log(cfg);
  const exe = await wait(100, "hello.exe");
  console.log("built", exe);
  return exe;
}
build().then(r => console.log("finished:", r));
console.log("(other work keeps running)");
```

OUTPUT

```
configuring...
(other work keeps running)
config ok
built hello.exe
finished: hello.exe
```

This is exactly the shape of NOVA's `runPipeline`: configure, `await` it, then build, `await` that, then run. Here is a piece of the real code:

JAVASCRIPT

```
async function openFile(path, pos) {
  let tab = S.tabs.find(t => key(t.path) === key(path));
  if (!tab) {
    let r; try { r = await nova.read(path); } catch (e) { return toast('Cannot
open: ' + e.message); }
    ...
```

> **WHAT IF IT WERE DIFFERENT?** ***What if you forget await?***
>
> You get a **promise object** instead of the value. `const r = nova.read(path)` would make `r` a promise, and `r.text` would be `undefined`. The code runs without an error and does the wrong thing. If a value looks like `Promise { <pending> }` you forgot an `await`.

Running many things at once

JAVASCRIPT

```
const wait = (ms, v) => new Promise(r => setTimeout(() => r(v), ms));
async function main() {
  const t0 = Date.now();
  const results = await Promise.all([wait(200, "A"), wait(200, "B"), wait(200,
"C")]);
  console.log(results, Date.now() - t0 < 400 ? "ran together" : "ran one by
one");
}
main();
```

OUTPUT

```
[ 'A', 'B', 'C' ] ran together
```

8.15 Useful tools for a real program

Debouncing: wait until the user stops

Every key you press changes the file, but you do not want to bother the language server after each letter. A **debounce** waits until typing pauses.

JAVASCRIPT

```
function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);                       // cancel the earlier plan
    timer = setTimeout(() => fn(...args), ms); // make a new plan
  };
}
const send = debounce((text) => console.log("sent:", text), 120);
send("s"); send("st"); send("std"); send("std::");   // four quick "keystrokes"
```

OUTPUT

```
sent: std::
```

Only one message is sent. This is the idea behind `lspDidChange` in NOVA, which waits 120 milliseconds.

> **WHY?** ***Why does debounce use a closure?***
>
> The variable `timer` must survive between calls so that the next call can cancel the previous timer. The closure keeps it alive and private. This is a perfect, tiny example of why closures exist.

Optional chaining and defaults

JAVASCRIPT

```
const config = { editor: { font: "Consolas" } };
console.log(config.editor?.font, config.terminal?.font, config.terminal?.font ??
"default");
```

OUTPUT

```
Consolas undefined default
```

`?.` stops safely if something is missing; `??` supplies a default only if the value is `null` or `undefined`. You will see both through NOVA's code.

8.16 JavaScript in the browser: the DOM

Everything above works in Node. In a browser window you also have the **DOM** (Chapter 6) to read and change the page. These examples need a page, so you cannot run them with Node. Type them into the console of any web page.

```javascript
const btn = document.querySelector('#btnRun');          // find one element
const all = document.querySelectorAll('.tab');          // find many

const div = document.createElement('div');              // make a new element
div.className = 'tab active';
div.textContent = 'main.cpp';
document.querySelector('#tabs').appendChild(div);       // put it in the page

btn.addEventListener('click', () => console.log('Run!'));   // react to a click
document.documentElement.dataset.theme = 'light';           // set an attribute
```

An **event** is something the user (or the system) does: click, key press, mouse move, resize. An **event listener** is a function that waits for it.

Event delegation

Imagine a list with a thousand tabs. Giving each its own listener is wasteful. Instead put **one** listener on the parent and check which child was clicked:

```javascript
document.querySelector('#tabs').addEventListener('click', (e) => {
  const tab = e.target.closest('.tab');
  if (tab) console.log('clicked', tab.textContent);
});
```

NOVA's Run console uses this: one listener on the whole output area catches clicks on any file link in a compiler message and opens that file at that line.

8.17 Modules (a short look)

Large programs are split into files. In Node you can write:

```javascript
const fs = require('fs');                   // load a built-in module
const { join } = require('path');           // load just one function
module.exports = { myFunction };            // share from your own file
```

NOVA's renderer files do not use modules. They are plain `<script>` tags that share one global scope, which is the simplest approach for a small tool. Chapter 9 shows Node's modules in action.

8.18 Worked project: a tiny interpreter

It is time to put the pieces together and build what Chapter 3 described: a **lexer**, a **parser** and an **evaluator** for sums such as `1 + 2 * 3`. This runs in Node.

JAVASCRIPT

```
// 1. LEXER: text -> tokens
function tokenize(src) {
  const tokens = [];
  let i = 0;
  while (i < src.length) {
    const c = src[i];
    if (/\s/.test(c)) { i++; continue; }
    if (/[0-9.]/.test(c)) {
      let n = "";
      while (i < src.length && /[0-9.]/.test(src[i])) n += src[i++];
      tokens.push({ type: "num", value: parseFloat(n) });
      continue;
    }
    if ("+-*/()".includes(c)) { tokens.push({ type: c }); i++; continue; }
    throw new Error("Unexpected character: " + c);
  }
  return tokens;
}

// 2. PARSER: tokens -> tree
function parse(tokens) {
  let pos = 0;
  const peek = () => tokens[pos];
  const next = () => tokens[pos++];
  function expression() {                       // handles + and -
    let left = term();
    while (peek() && (peek().type === "+" || peek().type === "-")) {
      const op = next().type;
      left = { op, left, right: term() };
    }
    return left;
  }
  function term() {                             // handles * and /  (binds
tighter)
    let left = factor();
    while (peek() && (peek().type === "*" || peek().type === "/")) {
      const op = next().type;
      left = { op, left, right: factor() };
    }
    return left;
  }
  function factor() {                           // numbers, (brackets),
negatives
    const t = next();
    if (!t) throw new Error("Unexpected end");
    if (t.type === "num") return { num: t.value };
    if (t.type === "-") return { op: "neg", left: factor() };
    if (t.type === "(") {
      const e = expression();
      const close = next();
      if (!close || close.type !== ")") throw new Error("Missing )");
      return e;
```

```
    }
    throw new Error("Unexpected token " + t.type);
  }
  const tree = expression();
  if (pos < tokens.length) throw new Error("Extra input");
  return tree;
}

// 3. EVALUATOR: walk the tree
function evaluate(n) {
  if ("num" in n) return n.num;
  if (n.op === "neg") return -evaluate(n.left);
  const a = evaluate(n.left), b = evaluate(n.right);
  switch (n.op) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/": return a / b;
  }
}

const run = (src) => evaluate(parse(tokenize(src)));
console.log(run("1 + 2 * 3"));
console.log(run("(1 + 2) * 3"));
console.log(run("10 / 4 - -2"));
console.log(JSON.stringify(parse(tokenize("1 + 2 * 3"))));
try { run("2 * (3 + "); } catch (e) { console.log("error:", e.message); }
```

OUTPUT

```
7
9
4.5
{"op":"+","left":{"num":1},"right":{"op":"*","left":{"num":2},"right":
{"num":3}}}
error: Unexpected end
```

WHY? ***Why are there two functions, expression and term?***

That is how the parser knows `*` happens before `+`. `expression` handles the **low-priority** operators and calls `term` for each piece. `term` handles the **high-priority** operators and calls `factor`. So `2 * 3` is gathered into one node *before* `expression` sees the plus sign. Each level of priority gets its own function. This technique is called **recursive descent**, and it is how many real compilers read code.

8.19 Mini assignments

MINI ASSIGNMENT ***Assignment 8.1: FizzBuzz***

Print the numbers 1 to 30. For multiples of 3 print `Fizz`, for multiples of 5 print `Buzz`, and for both print `FizzBuzz`.

MINI ASSIGNMENT ***Assignment 8.2: Word counter***

Write `countWords(text)` that returns an object such as `{ the: 3, cat: 1 }`. Make it ignore upper and lower case and punctuation. *Hint:* `text.toLowerCase().match(/\w+/g)` *and a loop or* `reduce`.

MINI ASSIGNMENT ***Assignment 8.3: Parse a compile error***

Extend the `DIAG_RE` example: write `summarise(lines)` that takes an array of g++ output lines and returns `{ errors: N, warnings: N, files: [..unique names..] }`.

MINI ASSIGNMENT ***Assignment 8.4: Retry with promises***

Write `retry(fn, times)` that calls an async function and, if it fails, tries again up to `times` times, waiting longer each time (100ms, 200ms, 400ms). Test it with a function that fails twice and then succeeds.

MINI ASSIGNMENT ***Assignment 8.5: Add variables to the interpreter***

Teach the interpreter to understand `x = 5` and then `x * 2`. You will need a new token type for names, a place to store values (a `Map`) and a new tree node.

8.20 Big assignment: Bring MiniIDE to life

BIG ASSIGNMENT ***MiniIDE: behaviour in JavaScript***

Goal. Take your styled `miniide.html` from Chapter 7 and make it work. No libraries. About 250 lines of JavaScript.

Requirements.

1. **A fake project.** Store a project in an object: `{ "src/main.cpp": "…code…", "src/util.cpp": "…", "include/util.h": "…", "CMakeLists.txt": "…" }`. Draw the tree from it dynamically (create elements from the object, do not type the list by hand).
2. **Tabs.** Clicking a file opens it in a tab (or switches to its tab if already open). Tabs can be closed with a button. The editor area shows the file's text in a `<textarea>`.
3. **Dirty marker.** Editing a file shows a dot on its tab. Ctrl+S "saves" it back into the project object and removes the dot.
4. **Run button.** Pressing it appends fake build output line by line using `async/await` and a `wait()` helper. Show a spinner in the status bar while it runs. A Stop button cancels the run (you will need a flag that the loop checks).

5. **Find in files.** A search box scans every file's text and lists matches as *file:line text*. Clicking a result opens that file at that line.
6. **Problems.** Scan the code for the text `TODO` or the word `error` and list those lines in a Problems panel with a count badge.
7. **Theme and settings.** A toggle switches the theme by setting `data-theme`. Save the theme and font size to `localStorage` and restore them on load.
8. **Keyboard shortcuts.** Ctrl+S (save), Shift+F10 (run), Ctrl+W (close tab).
9. **One state object** named `S` holds everything. Typing `S` in the console shows the whole IDE.
10. **Debounce** the find-in-files search so it runs 200ms after typing stops.

Stretch goals. Add a command palette that opens with Ctrl+Shift+A and fuzzy-matches actions. Add `contenteditable` syntax colouring using a regular expression for keywords. Save the project object to `localStorage` so it survives a refresh.

How to check yourself. Open the browser developer console. There should be **no red errors** during any of the actions. Compare how NOVA's `openFile`, `renderTabs` and `doFind` (in `app.js`) do the same jobs.

8.21 Quiz

QUIZ

1. What is the difference between `let` and `const`?
2. Why is `===` preferred over `==`?
3. What do `map`, `filter` and `reduce` each do?
4. What is a closure? Give one use.
5. In what order does the output appear: a `setTimeout` of 0, a `Promise.then`, and a normal `console.log`?
6. What does `await` do, and what happens if you forget it?
7. Why does NOVA never block while reading a file?
8. What is event delegation and why use it?
9. Why does the NOVA error-line regex contain `(?:[A-Za-z]:)?`
10. What does a debounce do and why does it need a closure?

ANSWERS

1. `let` can be reassigned; `const` cannot. Prefer `const` so fewer things can change.
2. `===` compares value and type with no hidden conversion; `==` converts types and gives surprising results.
3. `map` changes every item into a new list; `filter` keeps some items; `reduce` combines all items into one value.

4. A function that remembers variables from where it was made. Uses: private data (a counter), debounce, event handlers.
5. Normal `console.log` first, then the promise, then the timeout.
6. It pauses the async function until the promise finishes and gives the value. Without it you get a pending promise object instead of the value.
7. File operations are asynchronous: the work happens elsewhere, and the program is free until the result arrives, so the window stays responsive.
8. Putting one listener on a parent and checking `e.target` instead of one on each child. It is cheaper and also works for elements added later.
9. On Windows a path can start with a drive letter and a colon (`C:`), and without this optional group the colon would be mistaken for the separator between file and line number.
10. A debounce waits until calls stop for a short time and then runs once. It needs a closure so the timer variable survives between calls and can be cancelled by the next call.

CHAPTER 9

Node.js and Electron: The Engine Room

9.1 Why we need them

A normal web page is locked in a safe room called the **sandbox**. It cannot read your files, start programs or use your hard disk. That keeps you safe when you visit a website. But an IDE *must* read files, start compilers and talk to debuggers.

Two tools open the safe room, carefully:

- **Node.js** lets JavaScript run on your computer, outside the browser, with powers to read files, start programs and use the network.
- **Electron** bundles Node.js together with the Chromium browser engine so you can build a normal desktop program (with its own window and icon) out of web technology.

> **KEY WORD** ***Module***
> A **module** is a ready-made bundle of functions that you load with `require`. Node ships with many built-in modules: `fs` (files), `path` (file names), `child_process` (start programs), `os` (information about the computer), `events` (messaging).

9.2 Node basics

Install Node from nodejs.org, then in a terminal type `node --version`. Save any JavaScript file and run it with `node file.js`. Everything from Chapter 8 works, and now you also have these modules.

path: working with file names

Never build file names by gluing strings with `+ "/" +`. Windows uses backslashes, other systems use forward slashes. The `path` module hides the difference.

JAVASCRIPT

```
const path = require("path");

console.log(path.posix.join("src", "main.cpp"));
console.log(path.win32.join("src", "main.cpp"));
console.log(path.posix.basename("/home/me/project/main.cpp"));
console.log(path.posix.dirname("/home/me/project/main.cpp"));
console.log(path.posix.extname("main.cpp"));
console.log(path.posix.basename("main.cpp", ".cpp"));
console.log(path.posix.resolve("/a/b", "../c"));
```

OUTPUT

```
src/main.cpp
src\main.cpp
main.cpp
/home/me/project
.cpp
main
/a/c
```

WHY? ***Why not just use "a" + "/" + "b"?***

Because the right slash depends on the operating system, and doubled or missing slashes sneak in. `path.join` fixes all of that, even removes accidental doubles, and `path.dirname`/`basename` cut paths apart correctly. NOVA's own `join`, `base` and `dirname` helpers in `app.js` exist because the *window* (renderer) cannot use `path`, so they were written by hand to do the same job.

fs: reading and writing files

JAVASCRIPT

```
const fs = require("fs");
const os = require("os");
const path = require("path");

const dir = fs.mkdtempSync(path.join(os.tmpdir(), "nova-demo-"));   // a fresh
empty folder
const file = path.join(dir, "hello.txt");

fs.writeFileSync(file, "Hello from Node!\nSecond line.\n");
const text = fs.readFileSync(file, "utf8");
console.log(text.split("\n").length - 1, "lines:", JSON.stringify(text));

console.log(fs.existsSync(file), fs.readdirSync(dir));
fs.mkdirSync(path.join(dir, "sub"));
console.log(fs.readdirSync(dir, { withFileTypes: true }).map(e =>
(e.isDirectory() ? "[dir] " : "") + e.name));
console.log(fs.statSync(file).size, "bytes");

fs.rmSync(dir, { recursive: true, force: true });                    // tidy up
console.log("exists after delete:", fs.existsSync(dir));
```

OUTPUT

```
2 lines: "Hello from Node!\nSecond line.\n"
true [ 'hello.txt' ]
[ 'hello.txt', '[dir] sub' ]
30 bytes
exists after delete: false
```

Notice `Sync` at the end of every function name. Those functions **block** (the waiter stands at the kitchen). They are fine for short scripts. In a program with a window you use the promise versions so that the program is never frozen:

JAVASCRIPT

```
const fs = require("fs");
const os = require("os");
const path = require("path");

async function main() {
  const dir = await fs.promises.mkdtemp(path.join(os.tmpdir(), "nova-demo-"));
  const file = path.join(dir, "note.txt");
  await fs.promises.writeFile(file, "async is polite");
  console.log(await fs.promises.readFile(file, "utf8"));
  await fs.promises.rm(dir, { recursive: true, force: true });
}
main();
```

OUTPUT

```
async is polite
```

This is exactly how NOVA's main process reads files for the window:

JAVASCRIPT

```
ipcMain.handle('fs:read', async (_e, f) => {
  const buf = await fs.promises.readFile(f);
  if (buf.includes(0)) return { binary: true };
  return { text: buf.toString('utf8') };
});
```

> **WHY?** ***Why does NOVA check buf.includes(0)?***
> Text files never contain the byte `0`, but program files (`.exe`), images and other binary files are full of them. If NOVA tried to show a `.exe` as text, the editor would display garbage and might hang. A single cheap check protects the whole editor.

os and process

JAVASCRIPT

```
const os = require("os");
console.log(typeof os.platform(), typeof os.homedir(), os.EOL === "\n" || os.EOL
=== "\r\n");
console.log(process.argv.length >= 2, typeof process.env.PATH, typeof
process.pid);
console.log(process.cwd().length > 0);
```

OUTPUT

```
string string true
true string number
true
```

`process.argv` holds the command-line arguments, `process.env` the environment variables (including `PATH`, the list of folders where programs are found) and `process.cwd()` the folder you started in.

9.3 Events: EventEmitter

Much of Node is built on **events**: "tell me when X happens".

JAVASCRIPT

```
const { EventEmitter } = require("events");

const bus = new EventEmitter();
bus.on("saved", (name) => console.log("listener A: saved", name));
bus.on("saved", (name) => console.log("listener B: refresh tree for", name));
bus.once("opened", (name) => console.log("opened (only the first time):",
name));

bus.emit("saved", "main.cpp");
bus.emit("opened", "main.cpp");
bus.emit("opened", "util.cpp");   // nobody listens any more
```

OUTPUT

```
listener A: saved main.cpp
listener B: refresh tree for main.cpp
opened (only the first time): main.cpp
```

> **WHY?** ***Why events?***
> They let one part of a program announce something **without knowing who cares**. When a file is saved, the code that saves it does not need to know that the tab bar, the tree, clangd and the build system all want to know. They each subscribe. Adding a new listener never changes the saving code. This is how big programs stay flexible.

9.4 Starting other programs: child_process

This is the **most important Node module for an IDE**. It lets you start a program (the compiler, git, the debugger), feed it input and read its output.

JAVASCRIPT

```
const { spawn } = require("child_process");

// Start a child program: here we use Node itself as the "program".
const child = spawn(process.execPath, ["-e", "console.log('hello from child');
console.error('a warning'); process.exit(3)"]);

let out = "", err = "";
child.stdout.on("data", (chunk) => (out += chunk));
child.stderr.on("data", (chunk) => (err += chunk));
child.on("close", (code) => {
  console.log("stdout:", out.trim());
  console.log("stderr:", err.trim());
  console.log("exit code:", code);
});
```

OUTPUT

```
stdout: hello from child
stderr: a warning
exit code: 3
```

Every program has three standard streams:

Stream	Name	Used for
`stdin`	standard input	Text going **into** the program (keyboard or a pipe)
`stdout`	standard output	Normal text coming **out**
`stderr`	standard error	Error and warning text coming out

The **exit code** is a number the program returns when it ends: **0 means success**, anything else means failure. NOVA prints it at the end of every run: *"Process finished with exit code 0"*.

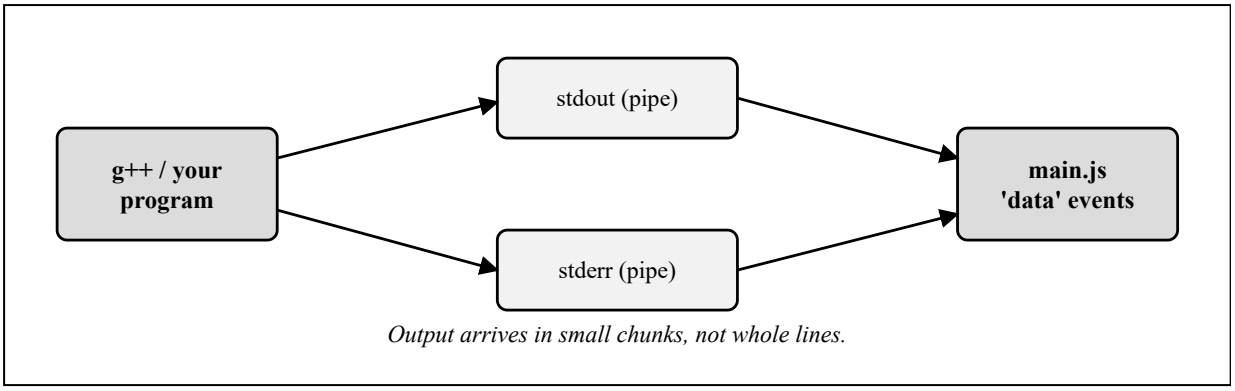


Figure 9.1. *A child program writes to two pipes. Our program listens to 'data' events as they arrive.*

> **WHY?** ***Why are stdout and stderr separate?***
>
> So you can treat them differently. NOVA draws stderr text in red. You can also save the normal output to a file and still see errors on the screen. Compilers print their error messages on stderr for exactly this reason.

Output arrives in chunks

JAVASCRIPT

```
const { spawn } = require("child_process");
const script = "let n=0; const t=setInterval(()=>{ process.stdout.write('part '+
(++n)+'\\n'); if(n==3){clearInterval(t)} }, 50)";
const child = spawn(process.execPath, ["-e", script]);
let chunks = 0;
child.stdout.on("data", (c) => { chunks++; console.log("received:",
c.toString().trim()); });
child.on("close", () => console.log("chunks:", chunks));
```

OUTPUT

```
received: part 1
received: part 2
received: part 3
chunks: 3
```

Data does not arrive as neat whole lines. A line may be split across two chunks, or two lines can arrive in one chunk. This is why NOVA keeps a **buffer** and only parses *complete* lines (see `execStep` in `app.js`).

> **WHAT IF IT WERE DIFFERENT?** ***What if you parse each chunk as if it were a line?***
> Sometimes a compiler error such as `main.cpp:14:9: error: 'x' was not declared` arrives split into `main.cpp:14:9: err` and `or: 'x' was not declared`. The pattern would match neither piece, and the error would silently vanish. Buffering until the newline is the cure.

Worked example: a tiny process runner

NOVA wraps `spawn` in a helper so the rest of the code can simply `await` the result. Here is the same idea:

JAVASCRIPT

```
const { spawn } = require("child_process");

function runCommand(cmd, args, input) {
  return new Promise((resolve) => {
    const child = spawn(cmd, args);
    let stdout = "", stderr = "";
    child.stdout.on("data", d => (stdout += d));
    child.stderr.on("data", d => (stderr += d));
    child.on("error", (e) => resolve({ code: -1, stdout, stderr: stderr +
e.message }));
    child.on("close", (code) => resolve({ code, stdout, stderr }));
    if (input !== undefined) { child.stdin.write(input); child.stdin.end(); }
  });
}

async function main() {
  const r1 = await runCommand(process.execPath, ["-e", "console.log(1 + 1)"]);
  console.log(r1.code, JSON.stringify(r1.stdout));
```

```
  const r2 = await runCommand(process.execPath, ["-e", "process.stdin.on('data',
d => console.log(d.toString().toUpperCase()))"], "shout this");
  console.log(r2.code, JSON.stringify(r2.stdout.trim()));

  const r3 = await runCommand("no-such-program-xyz", []);
  console.log(r3.code, r3.stderr.includes("ENOENT"));
}
main();
```

OUTPUT

```
0 "2\n"
0 "SHOUT THIS"
-1 true
```

Study `r3`. Starting a program that does not exist does not crash Node; it fires an `'error'` event. NOVA handles it by printing *"Failed to start"* in the Run panel. If the `'error'` handler were missing, Node would throw an uncaught exception and the whole IDE would crash.

> **WATCH OUT** ***Watch out***
> Always handle the `'error'` event on a child process. It is the most common cause of mysterious crashes in tools that start other programs.

9.5 Electron: a window made of web technology

The two process types again

We met them in Chapter 4: a **main process** (Node, full powers) and **renderer processes** (web pages, limited powers). Here is the smallest Electron program that opens a window. It needs three files in one folder.

`package.json`:

JSON

```
{
  "name": "hello-electron",
  "version": "1.0.0",
  "main": "main.js",
  "scripts": { "start": "electron ." },
  "dependencies": { "electron": "^45.0.0" }
}
```

`main.js`:

JAVASCRIPT

```
const { app, BrowserWindow } = require('electron');

function createWindow() {
  const win = new BrowserWindow({ width: 900, height: 600 });
  win.loadFile('index.html');
}
app.whenReady().then(createWindow);
app.on('window-all-closed', () => app.quit());
```

`index.html`:

HTML

```
<!doctype html>
<html><body><h1>Hello from Electron!</h1></body></html>
```

Run `npm install` once, then `npm start`. A real window appears.

Line by line:

- `app` represents the whole program: its life from start to quit.
- `app.whenReady()` waits until Electron has finished starting up. Creating a window earlier would fail.
- `new BrowserWindow({...})` makes a window. `loadFile` fills it with your HTML page.
- `window-all-closed` fires when the user closes the last window. We quit, so the program does not linger invisibly.

Making a window look modern

NOVA's real window options, from `main.js`:

JAVASCRIPT

```
win = new BrowserWindow({
  width: 1500, height: 920, minWidth: 900, minHeight: 600,
  backgroundColor: '#1e1f22',
  frame: false,
  titleBarStyle: 'hidden',
  titleBarOverlay: { color: '#2b2d30', symbolColor: '#dfe1e5', height: 40 },
  webPreferences: { preload: path.join(__dirname, 'preload.js'),
                    contextIsolation: true, nodeIntegration: false }
});
```

WHY? ***Why each option?***

- `backgroundColor` is shown for the instant before the page paints. Without it you see a white flash on a dark app, which feels cheap.
- `frame: false` and `titleBarStyle: 'hidden'` remove the standard grey title bar, so NOVA can draw its own toolbar there.
- `titleBarOverlay` keeps the native minimise, maximise and close buttons, painted in NOVA's colours. We get the modern look without rebuilding those buttons.
- `minWidth` and `minHeight` stop the user from shrinking the window until the layout breaks.

9.6 The safe bridge: preload and contextBridge

Now the most important security idea in Electron.

JAVASCRIPT

```
// main.js
webPreferences: { preload: path.join(__dirname, 'preload.js'), contextIsolation:
true, nodeIntegration: false }
```

- `nodeIntegration: false` means the web page **cannot** use `require`, `fs` or `child_process`.
- `contextIsolation: true` means the preload script runs in a **separate world** from the page, so the page cannot tamper with it.

The preload script is allowed to use Electron's APIs, and it hands the page only a small, named set of functions:

JAVASCRIPT

```
// preload.js
const { contextBridge, ipcRenderer } = require('electron');
const inv = (ch) => (...a) => ipcRenderer.invoke(ch, ...a);
contextBridge.exposeInMainWorld('nova', {
  read: inv('fs:read'),
  write: inv('fs:write'),
  readDir: inv('fs:readDir'),
  start: inv('proc:start'),
  kill: inv('proc:kill'),
  onData: (cb) => ipcRenderer.on('proc:data', (_e, m) => cb(m)),
});
```

After this, any script in the window can call `window.nova.read(path)` and **nothing else** from Node. The helper `inv` is a closure factory: it turns the channel name into a function that forwards its arguments.

WHAT IF IT WERE DIFFERENT? ***What if nodeIntegration were true?***

A script in your page could write `require('child_process').exec('del /s /q C:\\*')`. If your IDE ever displayed one malicious string as HTML (for example a hostile file name or compiler message), your computer could be wiped. With the bridge, the worst a page can do is call the few functions you chose to expose, and each can be written to check its arguments.

invoke and handle: asking and answering

The main process registers **handlers** for channel names:

JAVASCRIPT

```
// main.js
ipcMain.handle('fs:readDir', async (_e, dir) => {
  const ents = await fs.promises.readdir(dir, { withFileTypes: true });
  return ents
    .filter(d => !IGNORE.has(d.name))
    .map(d => ({ name: d.name, path: path.join(dir, d.name), dir:
d.isDirectory() }))
    .sort((a, b) => (b.dir - a.dir) || a.name.localeCompare(b.name));
});
```

The window then writes `const files = await nova.readDir(folder)`. The reply is whatever the handler returns, delivered through a promise.

WHY? ***Why sort inside the main process?***

The sort puts folders before files and then orders alphabetically, ignoring capital letters. `(b.dir - a.dir)` uses a trick: `true - false` is `1`, so directories rise to the top. Doing it once, in the engine room, means every part of the window gets a tidy list for free.

Pushing messages: send

Some information flows the *other* way, not as an answer but as news: a program has printed a line. The main process pushes it:

JAVASCRIPT

```
// main.js
child.stdout.on('data', d =>
  win.webContents.send('proc:data', { id, stream: 'out', data: d.toString() }));
```

and the bridge forwards it to a listener the window registered with `nova.onData(callback)`.

Worked example: a mini IPC bus you can run

You cannot run Electron's IPC outside Electron, but you can build the *idea* with a few lines and see why `invoke` and `handle` feel like function calls:

JAVASCRIPT

```
const handlers = new Map();
const ipcMain = { handle: (ch, fn) => handlers.set(ch, fn) };
const ipcRenderer = { invoke: async (ch, ...args) => {
  const fn = handlers.get(ch);
  if (!fn) throw new Error("No handler for " + ch);
  return fn({}, ...args);
} };

// "main process" side
ipcMain.handle("math:add", async (_e, a, b) => a + b);
ipcMain.handle("fs:fakeRead", async (_e, name) => ({ text: "contents of " + name
}));

// "window" side
(async () => {
  console.log(await ipcRenderer.invoke("math:add", 20, 22));
  console.log(await ipcRenderer.invoke("fs:fakeRead", "main.cpp"));
  try { await ipcRenderer.invoke("nope"); } catch (e) { console.log("error:",
e.message); }
})();
```

OUTPUT

```
42
{ text: 'contents of main.cpp' }
error: No handler for nope
```

The real thing also copies your arguments between processes (so the two sides never share memory), but the shape is identical.

9.7 Dialogs, menus and other native features

JAVASCRIPT

```
const { dialog } = require('electron');
const r = await dialog.showOpenDialog(win, { properties: ['openDirectory'] });
if (!r.canceled) console.log(r.filePaths[0]);
```

This opens the operating system's own folder picker. NOVA uses it for *Open Folder*. Because it is native, it looks right on every system and is familiar to the user.

9.8 A story about two bugs

> **IN PLAIN WORDS** ***The ELECTRON_RUN_AS_NODE trap***
>
> While building NOVA, the first launch printed `Cannot read properties of undefined (reading 'whenReady')`. The cause: the shell had an environment variable `ELECTRON_RUN_AS_NODE=1` set, which tells Electron to behave like plain Node, so `require('electron')` returned nothing useful. NOVA's `launch.js` and `Launch NOVA.bat` now clear that variable before starting. Lesson: *when a tool behaves strangely, check its environment.*

IN PLAIN WORDS ***The silent compiler***

Another time, g++ failed with no message at all when started from one particular terminal, yet worked in another. The two terminals had different environments. The lesson: an IDE runs programs in the environment *it* was started with. NOVA passes `env` explicitly to every child (`env: { ...process.env, ...opts.env }`) so it can be controlled and inspected.

9.9 Packaging (a first look)

To give NOVA to colleagues without asking them to install Node, you "package" it into an installer. The common tool is **electron-builder** or **electron-forge**. A basic command looks like `npx electron-builder --win`. It produces a `.exe` installer. Chapter 17 lists this as a first "next step".

9.10 Mini assignments

MINI ASSIGNMENT ***Assignment 9.1: Folder lister***

Write a Node script that takes a folder name as `process.argv[2]` and prints every file inside it (recursively), indented by depth, folders first.

MINI ASSIGNMENT ***Assignment 9.2: Run and time***

Write `timeIt(cmd, args)` that uses `spawn`, measures how long the program ran, and prints exit code, duration and the number of output lines.

MINI ASSIGNMENT ***Assignment 9.3: Line buffer***

Write a `LineBuffer` class with a method `push(chunk)` that returns only the *complete* lines found so far, and keeps the unfinished tail for next time. Test it with chunks that split lines in the middle.

MINI ASSIGNMENT ***Assignment 9.4: Hello, Electron***

Build the three-file Electron app from this chapter. Then add a button in `index.html` and a preload bridge so that clicking it shows the current time returned from the main process.

9.11 Big assignment: A file explorer with a Run button

BIG ASSIGNMENT ***"Folder Peek": your first desktop app***

Goal. Build a small Electron app that lets you browse a project and run Node scripts, using only what you have learned.

Requirements.

1. A frameless-style window with a custom title bar (`titleBarOverlay`), a dark `backgroundColor` and sensible `minWidth`/`minHeight`.
2. `contextIsolation: true` and `nodeIntegration: false`. The preload exposes exactly six functions through `contextBridge` (open folder, list directory, read file, write file, run a command, kill a command) and nothing else.
3. *Open Folder* uses `dialog.showOpenDialog`. The list-directory handler returns folders first, sorted by name, ignoring `node_modules` and `.git`.
4. The window shows the folder as an expandable tree (build it with the DOM from Chapter 8) and shows the selected text file in a `<textarea>`. Binary files show a friendly message.
5. *Save* (Ctrl+S) writes the file. A dirty dot appears while edited.
6. A *Run* button for `.js` files starts `node thatfile.js` with `spawn`, streams stdout (normal) and stderr (red) into a panel, and shows the exit code at the end. A *Stop* button kills it.
7. If the program does not exist or fails to start, show an error message in the panel instead of crashing.
8. Keep **all** paths in the main process built with `path.join`.
9. Write a short README explaining each IPC channel you created.

Stretch goals. Remember the last folder (store it as a JSON file in `app.getPath('userData')`). Add a stdin box to send input to the running script. Add find-in-files using a recursive `fs` walk in the main process. Package it with electron-builder.

How to check yourself. Open the DevTools console in your window (`win.webContents.openDevTools()`) and try typing `require('fs')`. It must say `require is not defined`. That proves your window is safe. Compare your handlers with `main.js` in NOVA.

9.12 Quiz

QUIZ

1. Why can't a normal web page read your files?
2. What do `Sync` functions do that the promise versions do not?
3. What are stdin, stdout and stderr?
4. What does an exit code of 0 mean?
5. Why must you handle the `'error'` event on a child process?
6. What do `nodeIntegration: false` and `contextIsolation: true` achieve?

7. What is `contextBridge.exposeInMainWorld` for?
8. What is the difference between `ipcRenderer.invoke` and `webContents.send`?
9. Why does NOVA buffer output before parsing it for errors?
10. Why use `path.join` instead of joining strings?

ANSWERS

1. The browser sandbox forbids it for your safety.
2. `Sync` functions block (freeze) the program until finished; promise versions let it do other things while waiting.
3. Standard input (text going in), standard output (normal text coming out), standard error (error text coming out).
4. The program succeeded.
5. If the program cannot start (for example it does not exist), Node fires `'error'`; with no handler Node throws and the whole IDE can crash.
6. The page cannot use Node's powers directly, and the preload script is protected from the page's own scripts.
7. To give the page a small, named set of functions from the preload script, nothing more.
8. `invoke` is the window asking the main process a question and getting an answer; `send` is the main process pushing news to the window.
9. Output arrives in arbitrary chunks, so lines can be split; only complete lines can be matched reliably.
10. It uses the correct separator for the operating system and removes accidental double separators.

CHAPTER 10

C: The Foundation

10.1 Why learn C?

C was created in 1972 and is still everywhere: operating systems, phones, game engines, car engines, and even the compilers and debuggers that NOVA drives. C is small. You can learn all of it. And it shows you what the computer is *really* doing, which makes you better at every other language.

C gives you great power and few safety nets. It trusts you. That is why it is fast, and why mistakes can be nasty. In this chapter you will learn both the power and the discipline.

> **IN PLAIN WORDS** ***In plain words***
> Most languages are like driving an automatic car: comfortable, but you do not see the gears. C is a manual car with no seat-belt warning. You feel everything, and you learn how engines work.

10.2 Setting up

You need a compiler. NOVA uses **gcc** (the C compiler of the GNU project). Check it is installed:

```
gcc --version
```

Save code in a file ending in `.c` (for example `hello.c`) and build it:

```
gcc hello.c -o hello.exe
hello.exe
```

Or open the file in NOVA and press the green **Run** button. NOVA runs exactly the same commands for you (with extra options `-g -Wall -Wextra`: `-g` adds debugging information, `-Wall -Wextra` turn on helpful warnings).

10.3 Hello, world

C

```
#include <stdio.h>

int main(void) {
    printf("Hello, NOVA!\n");
    return 0;
}
```

OUTPUT

```
Hello, NOVA!
```

Line by line:

- `#include <stdio.h>` pastes in the description of the standard input/output functions, including `printf`. Without it the compiler would not know what `printf` is.
- `int main(void)` is where every C program starts. `int` says that it gives back a whole number when it ends.
- `{ }` hold the body of the function.
- `printf("...")` prints text. `\n` is a **newline** character (it moves to the next line).
- `return 0;` ends the program and says "success".
- Every statement ends with a semicolon `;`.

> **WHY?** ***Why does main return 0?***
> The number is the program's **exit code**, which the operating system and tools such as NOVA read. By agreement **0 means success** and anything else means failure. NOVA prints it after every run: *"Process finished with exit code 0"*. Build scripts rely on it: if your program returns non-zero, the build stops.

> **WHAT IF IT WERE DIFFERENT?** ***What if you forget the semicolon or the #include?***
> The compiler stops with an error such as `expected ';' before 'return'` or a warning like `implicit declaration of function 'printf'`. These appear as red and yellow wavy lines in NOVA. Reading these messages carefully is a core skill: the line number is where the compiler *noticed* the problem, which is sometimes one line after where you made it.

10.4 Variables and types

C

```
#include <stdio.h>

int main(void) {
    int age = 25;               // whole number
    double price = 19.99;       // number with decimals
    char letter = 'N';          // one character (note the single quotes)
    const int days = 7;         // cannot change

    printf("age=%d price=%.2f letter=%c days=%d\n", age, price, letter, days);
    printf("sizes: int=%zu double=%zu char=%zu\n", sizeof(int), sizeof(double),
sizeof(char));
    age = age + 1;
    printf("next year: %d\n", age);
    return 0;
}
```

OUTPUT

```
age=25 price=19.99 letter=N days=7
sizes: int=4 double=8 char=1
next year: 26
```

The pieces `%d`, `%.2f`, `%c`, `%zu` are **format specifiers**: placeholders that `printf` fills from the values that follow.

Specifier	Prints	Example
`%d`	whole number (int)	`42`
`%u`	unsigned whole number	`42`
`%f` `%.2f`	decimal number (2 decimals)	`3.14`
`%c`	one character	`A`
`%s`	text (string)	`hello`
`%x`	hexadecimal	`ff`
`%p`	a pointer (address)	`0x7ffd...`
`%zu`	size values (`sizeof`)	`8`

WHY? ***Why do types exist?***

A variable is just a place in memory. The **type** tells the compiler how many bytes to set aside and how to read them. The same four bytes can mean the number 1078530011 or the decimal 3.14159. Without the type the computer would not know which. (Python and JavaScript decide types at run time. C decides at compile time, which is faster and catches mistakes earlier.)

Sizes matter

A **byte** is 8 bits. On most computers: `char` is 1 byte, `int` is 4 bytes, `long long` is 8, `double` is 8, and a pointer is 8 on a 64-bit system. An `int` holds roughly minus two billion to plus two billion. Go past it and the number **wraps around**:

C

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("INT_MAX = %d\n", INT_MAX);
    unsigned int big = 4000000000u;
    printf("unsigned big = %u\n", big);
    unsigned char small = 250;
    small = small + 10;                 // 260 does not fit in one byte
    printf("small after +10 = %d\n", small);
    return 0;
}
```

OUTPUT

```
INT_MAX = 2147483647
unsigned big = 4000000000
small after +10 = 4
```

> **WATCH OUT** ***Watch out***
>
> Overflowing a *signed* `int` is **undefined behaviour** in C: the compiler is allowed to do anything. Unsigned overflow is defined (it wraps). Many famous security bugs come from number overflow.

10.5 Deciding and repeating

C

```
#include <stdio.h>

int main(void) {
    int score = 72;
    if (score >= 90) printf("A\n");
    else if (score >= 70) printf("B\n");
    else printf("C or below\n");

    for (int i = 1; i <= 5; i++) printf("%d ", i * i);
    printf("\n");

    int n = 10, steps = 0;
    while (n != 1) {                        // the Collatz puzzle
        n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
        steps++;
    }
    printf("steps to reach 1: %d\n", steps);

    switch (score / 10) {
        case 10: case 9: printf("excellent\n"); break;
```

```
        case 7: printf("good\n"); break;
        default: printf("keep going\n");
    }
    return 0;
}
```

OUTPUT

```
B
1 4 9 16 25
steps to reach 1: 6
good
```

WHAT IF IT WERE DIFFERENT? ***What if you forget break in a switch?***

Execution "falls through" into the next case and runs its code too. Sometimes that is done on purpose (as with `case 10: case 9:` above) but forgetting `break` is a classic bug. Modern compilers warn about it with `-Wextra`, which is one reason NOVA always passes that option.

10.6 Functions

C

```
#include <stdio.h>

int square(int x);                  // prototype: promise that this function
exists

int main(void) {
    printf("%d %d\n", square(7), square(square(2)));
    return 0;
}

int square(int x) {                 // definition
    return x * x;
}
```

OUTPUT

```
49 16
```

A **prototype** tells the compiler the function's name, input types and output type before it is defined, so you can call it earlier in the file. Prototypes for a whole set of functions go in a **header file** (`.h`), as you will see in the multi-file section.

WHY? ***Why does C copy arguments?***

C passes arguments **by value**: the function receives a *copy*. Changing the copy never changes the original. That is safe and predictable. When you *do* want the function to change the caller's variable, you pass its **address** (a pointer), which is the next big topic.

10.7 Arrays and strings

An **array** is a row of items of the same type, side by side in memory.

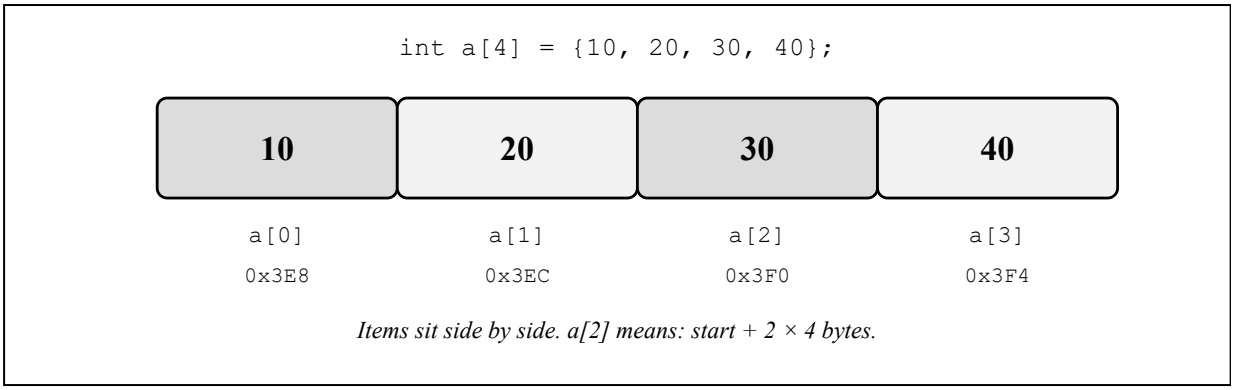


Figure 10.1. *An array is a row of boxes. The element a[2] lives at start + 2 × (size of one item).*

C

```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int sum = 0;
    for (int i = 0; i < 5; i++) sum += a[i];
    printf("sum=%d average=%.1f\n", sum, sum / 5.0);
    printf("bytes in a: %zu, items: %zu\n", sizeof a, sizeof a / sizeof a[0]);
    return 0;
}
```

OUTPUT

```
sum=150 average=30.0
bytes in a: 20, items: 5
```

> **WHAT IF IT WERE DIFFERENT?** ***What if you read a[5] or a[-1]?***
>
> C does **not check array bounds**. You will read (or overwrite) whatever happens to sit next to the array. The program may print garbage, crash, or seem fine until the worst moment. This is called **undefined behaviour** and is the root of countless security holes. Always make sure your index is between 0 and size minus 1.

Strings are character arrays

C has no string type. A **string** is an array of `char` that ends with a special zero character `'\0'`, the **null terminator**.

C

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char name[] = "NOVA";                    // 5 bytes: N O V A \0
```

```
    printf("%s has length %zu but takes %zu bytes\n", name, strlen(name), sizeof
name);

    char copy[20];
    strcpy(copy, name);
    strcat(copy, " IDE");
    printf("%s\n", copy);
    printf("%d %d\n", strcmp("apple", "banana") < 0, strcmp("same", "same"));

    for (int i = 0; copy[i] != '\0'; i++) copy[i] = (char)toupper((unsigned
char)copy[i]);
    printf("%s\n", copy);

    char text[] = "one two three";
    for (char *tok = strtok(text, " "); tok; tok = strtok(NULL, " ")) printf("
[%s]", tok);
    printf("\n");
    return 0;
}
```

OUTPUT

```
NOVA has length 4 but takes 5 bytes
NOVA IDE
1 0
NOVA IDE
[one][two][three]
```

WHY? ***Why the null terminator?***

The string functions need to know where a string ends, but an array does not carry its own length. So the convention is: the string ends at the first zero. `strlen` simply counts until it finds one. The price is danger: if the zero is missing, the functions run on into memory that is not yours.

WATCH OUT ***Watch out***

`strcpy` and `strcat` do not check that the destination is big enough. Copying a long string into a short buffer is a **buffer overflow**. Prefer `snprintf(buf, sizeof buf, ...)`, which cuts the output to fit.

10.8 Pointers: the big idea

Every variable lives at an **address** in memory. A **pointer** is a variable that holds an address.

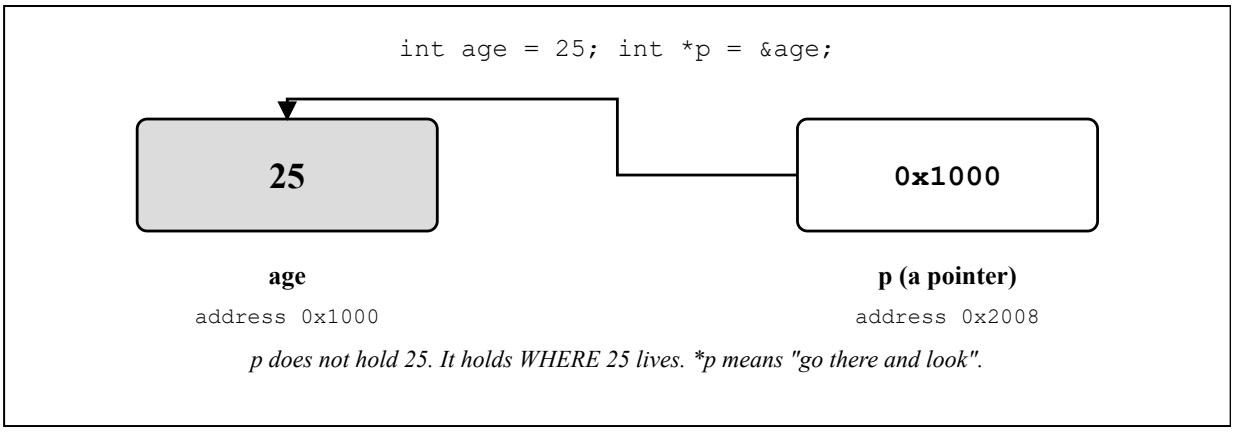


Figure 10.2. *p holds the address of age. *p means "go to that address and use what is there".*

C

```c
#include <stdio.h>

int main(void) {
    int age = 25;
    int *p = &age;                  // & = "address of"
    printf("age=%d, *p=%d\n", age, *p);   // * = "the thing p points to"
    *p = 30;                        // change age THROUGH the pointer
    printf("age is now %d\n", age);
    printf("same address? %d\n", p == &age);
    return 0;
}
```

OUTPUT

```
age=25, *p=25
age is now 30
same address? 1
```

Why bother? Because with a pointer a function can change the caller's variable:

C

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main(void) {
    int x = 1, y = 2;
    swap(&x, &y);
    printf("x=%d y=%d\n", x, y);
    return 0;
}
```

OUTPUT

```
x=2 y=1
```

Pointers and arrays are cousins

The name of an array can be used as a pointer to its first item. Adding 1 to a pointer moves it by **one item**, not one byte.

C

```
#include <stdio.h>

int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = a;
    printf("%d %d %d\n", *p, *(p + 2), p[3]);
    for (int *q = a; q < a + 4; q++) printf("%d ", *q);
    printf("\n");
    printf("distance in items: %td\n", (a + 3) - a);
    return 0;
}
```

OUTPUT

```
10 30 40
10 20 30 40
distance in items: 3
```

> **WHAT IF IT WERE DIFFERENT?** ***What if a pointer points at nothing?***
> A pointer you have not set holds random garbage. Using it (*dereferencing*) usually crashes the program with a "segmentation fault". A pointer that deliberately points nowhere is set to `NULL`. Always check `if (p != NULL)` before using a pointer that might be empty. In NOVA's debugger you can *see* a pointer's value in the Variables panel.

10.9 Memory: the stack and the heap

Your program has regions of memory with different jobs.

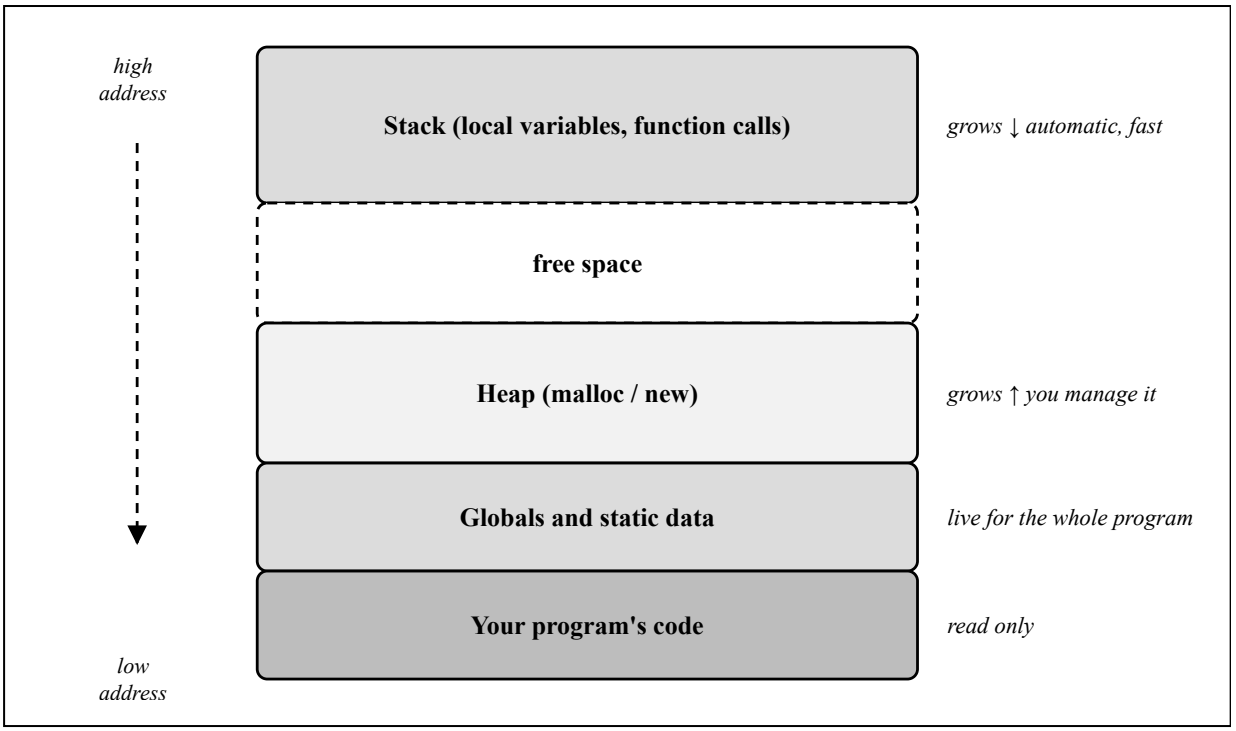


Figure 10.3. *The memory of a running program. The stack handles local variables automatically; the heap is yours to manage.*

- The **stack** holds local variables and function calls. It is automatic: variables appear when a function starts and vanish when it ends. It is fast but limited in size.
- The **heap** is a big pool you borrow from with `malloc` and give back with `free`. It lasts until *you* free it, and its size is limited only by your computer.

C

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *data = malloc(n * sizeof *data);        // borrow room for 5 ints
    if (data == NULL) { printf("out of memory\n"); return 1; }

    for (int i = 0; i < n; i++) data[i] = (i + 1) * 100;

    data = realloc(data, 8 * sizeof *data);      // grow to 8 ints
    for (int i = 5; i < 8; i++) data[i] = 0;
    for (int i = 0; i < 8; i++) printf("%d ", data[i]);
    printf("\n");

    free(data);                                  // give it back
    data = NULL;                                 // so we cannot use it by
accident
    return 0;
}
```

OUTPUT

```
100 200 300 400 500 0 0 0
```

> **WHY?** ***Why do we write malloc(n * sizeof *data) and not malloc(20)?***
> `sizeof *data` means "the size of whatever data points to". If you later change `int` to `double`, the size follows automatically. A hard-coded 20 would silently become wrong. Writing code that adapts is a mark of a careful programmer.

The three famous heap mistakes:

Mistake	Name	Result
Forgetting `free`	**Memory leak**	The program slowly eats memory
Using memory after `free`	**Dangling pointer / use-after-free**	Garbage or a crash, sometimes a security hole
Calling `free` twice on the same pointer	**Double free**	A crash

Setting the pointer to `NULL` after `free` makes the second and third mistakes much easier to catch.

10.10 Structs, enums and typedef

A **struct** groups different pieces of data under one name.

C

```
#include <stdio.h>
#include <string.h>

typedef enum { RED, GREEN, BLUE } Color;

typedef struct {
    char name[20];
    int x, y;
    Color color;
} Point;

void show(const Point *p) {                  // const: this function promises
not to change it
    static const char *names[] = {"red", "green", "blue"};
    printf("%s at (%d,%d) is %s\n", p->name, p->x, p->y, names[p->color]);
}

int main(void) {
    Point a = { "origin", 0, 0, RED };
    Point b;
    strcpy(b.name, "far");
    b.x = 10; b.y = 20; b.color = BLUE;
    show(&a);
    show(&b);
    printf("sizeof(Point) = %zu\n", sizeof(Point));
    return 0;
}
```

OUTPUT

```
origin at (0,0) is red
far at (10,20) is blue
sizeof(Point) = 32
```

- `p->x` is a shortcut for `(*p).x`: "the x of the thing p points to".
- `enum` gives names to small numbers (`RED` is 0, `GREEN` is 1, ...).
- `typedef` lets you say `Point` instead of `struct Point`.

Why is the struct bigger than its parts?

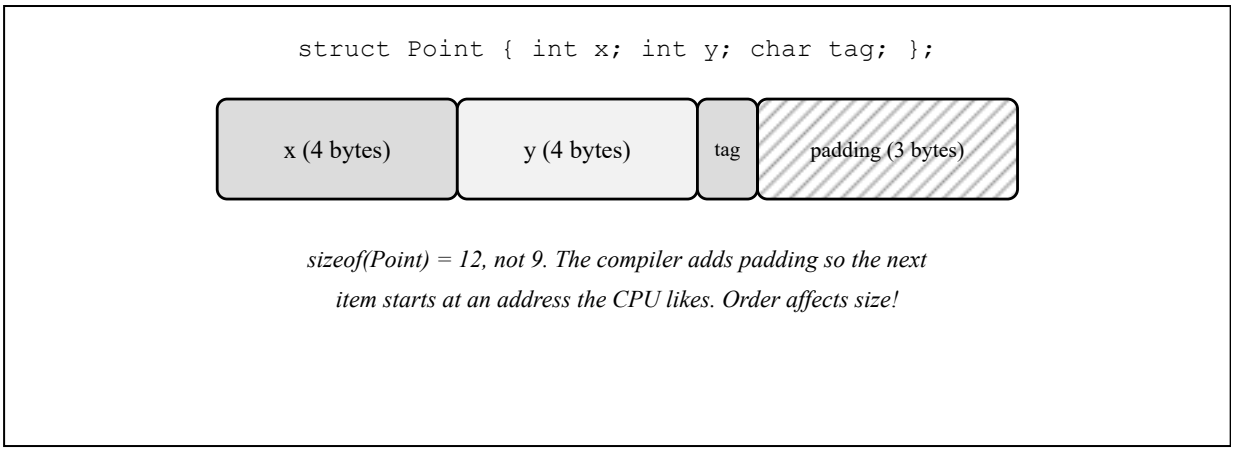


Figure 10.4. *The compiler inserts invisible padding so that every field starts at an address the CPU likes.*

C

```c
#include <stdio.h>

struct Bad  { char a; int b; char c; };     // padding between and after
struct Good { int b; char a; char c; };     // same fields, smaller

int main(void) {
    printf("Bad  = %zu bytes\n", sizeof(struct Bad));
    printf("Good = %zu bytes\n", sizeof(struct Good));
    return 0;
}
```

OUTPUT

```
Bad  = 12 bytes
Good = 8 bytes
```

WHY? ***Why does the order of fields change the size?***

CPUs read memory fastest when a 4-byte number starts at an address divisible by 4. The compiler adds padding bytes to guarantee it. Putting large fields first and small ones together wastes less. In an IDE that stores millions of tiny objects (such as syntax tokens) this saving is real.

10.11 Reading and writing files

C

```
#include <stdio.h>

int main(void) {
    FILE *f = fopen("notes.txt", "w");
    if (!f) { perror("open for writing"); return 1; }
    fprintf(f, "line one\nline two\nline three\n");
    fclose(f);

    f = fopen("notes.txt", "r");
    if (!f) { perror("open for reading"); return 1; }
    char line[100];
    int count = 0;
    while (fgets(line, sizeof line, f)) {
        count++;
        printf("%d: %s", count, line);
    }
    fclose(f);

    f = fopen("does-not-exist.txt", "r");
    if (!f) printf("could not open: file missing\n");
    remove("notes.txt");
    return 0;
}
```

OUTPUT

```
1: line one
2: line two
3: line three
could not open: file missing
```

> **WHY?** ***Why check whether fopen returned NULL?***
>
> Files can be missing, locked or unreadable. `fopen` signals failure by returning `NULL`. If you use that NULL as a file, the program crashes. Checking every operation that can fail is how professional C code stays reliable. `perror` prints a helpful reason.

10.12 Function pointers and callbacks

A function also lives at an address, so you can store it and pass it around. The standard library's `qsort` sorts *anything* because you give it a function that compares two items.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { const char *name; int lines; } File;

static int by_lines(const void *a, const void *b) {
    const File *x = a, *y = b;
    return (x->lines > y->lines) - (x->lines < y->lines);
```

```
}
static int by_name(const void *a, const void *b) {
    return strcmp(((const File *)a)->name, ((const File *)b)->name);
}

int main(void) {
    File files[] = { {"main.c", 120}, {"util.c", 45}, {"app.c", 300} };
    size_t n = sizeof files / sizeof files[0];

    qsort(files, n, sizeof files[0], by_lines);
    for (size_t i = 0; i < n; i++) printf("%s(%d) ", files[i].name,
files[i].lines);
    printf("\n");

    qsort(files, n, sizeof files[0], by_name);
    for (size_t i = 0; i < n; i++) printf("%s(%d) ", files[i].name,
files[i].lines);
    printf("\n");
    return 0;
}
```

OUTPUT

```
util.c(45) main.c(120) app.c(300)
app.c(300) main.c(120) util.c(45)
```

10.13 The preprocessor

Lines that start with `#` are handled **before** compilation.

C

```
#include <stdio.h>

#define MAX_TABS 8
#define SQUARE(x) ((x) * (x))
#define DEBUG 1

int main(void) {
    printf("%d %d\n", MAX_TABS, SQUARE(3 + 1));
#if DEBUG
    printf("debug mode on\n");
#endif
    printf("file=%s line=%d\n", "demo", __LINE__);
    return 0;
}
```

OUTPUT

```
8 16
debug mode on
file=demo line=12
```

WHAT IF IT WERE DIFFERENT? ***What if you write SQUARE(x) as x * x with no brackets?***

Then `SQUARE(3 + 1)` becomes `3 + 1 * 3 + 1`, which is 7, not 16. A macro is **text pasting**, not a function, so every bracket matters. That is why the macro above has so many. In C++ you will use `constexpr` functions and avoid most macros.

Include guards

A header file may be included from several places. The second inclusion would redefine everything and fail. **Include guards** make a header safe to include twice:

C

```
#ifndef SHAPES_H
#define SHAPES_H

double area(double radius);

#endif
```

(`#pragma once` is a shorter alternative that nearly every compiler accepts. NOVA's `Circle` class used it.)

10.14 Many files: headers and linking

A real project splits into files. The **header** (`.h`) lists *what exists*; the **source** (`.c`) holds *how it works*.

C

```
/* mathutil.h */
#ifndef MATHUTIL_H
#define MATHUTIL_H
int add(int a, int b);
#endif
```

C

```
/* mathutil.c */
#include "mathutil.h"
int add(int a, int b) { return a + b; }
```

C

```
/* main.c */
#include <stdio.h>
#include "mathutil.h"
int main(void) { printf("%d\n", add(2, 3)); return 0; }
```

Build with `gcc main.c mathutil.c -o app.exe`. Each `.c` is compiled alone into an object file, and the **linker** joins them. If you forget to list `mathutil.c` you get `undefined reference to 'add'`: the compiler saw the promise in the header but the linker found no real function.

10.15 Putting it together: a growable list

Real programs need arrays that grow. C has none built in, so we build one: a structure holding a pointer, a count and a capacity. This is the pattern behind C++'s `std::vector`.

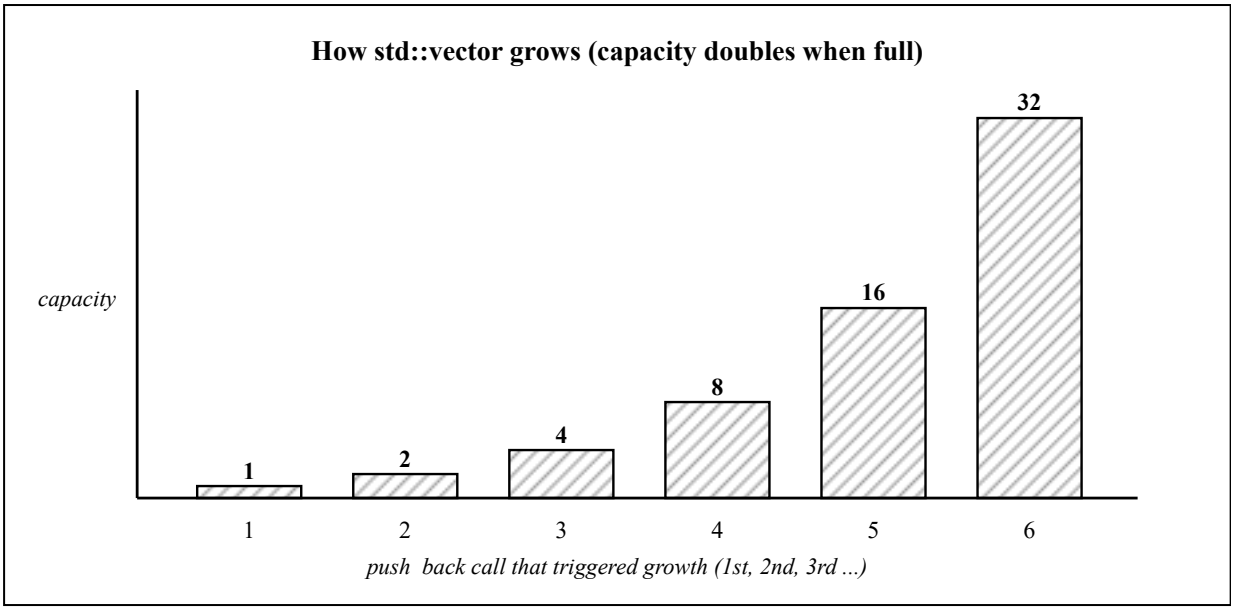


Figure 10.5. *Doubling the capacity when full means each push is cheap on average, even though a copy sometimes happens.*

C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *items;
    size_t count, capacity;
} IntList;

void list_init(IntList *l) { l->items = NULL; l->count = 0; l->capacity = 0; }

int list_push(IntList *l, int value) {
    if (l->count == l->capacity) {
        size_t cap = l->capacity ? l->capacity * 2 : 2;
        int *bigger = realloc(l->items, cap * sizeof *bigger);
        if (!bigger) return 0;                  // old memory is still valid
        l->items = bigger;
        l->capacity = cap;
    }
    l->items[l->count++] = value;
    return 1;
```

```
}

void list_free(IntList *l) { free(l->items); list_init(l); }

int main(void) {
    IntList list;
    list_init(&list);
    for (int i = 1; i <= 10; i++) {
        list_push(&list, i * i);
        printf("count=%zu capacity=%zu\n", list.count, list.capacity);
    }
    for (size_t i = 0; i < list.count; i++) printf("%d ", list.items[i]);
    printf("\n");
    list_free(&list);
    return 0;
}
```

OUTPUT

```
count=1 capacity=2
count=2 capacity=2
count=3 capacity=4
count=4 capacity=4
count=5 capacity=8
count=6 capacity=8
count=7 capacity=8
count=8 capacity=8
count=9 capacity=16
count=10 capacity=16
1 4 9 16 25 36 49 64 81 100
```

WHY? ***Why double the capacity instead of adding one?***

If you grew by one each time, ten thousand pushes would copy the data about fifty million times in total. Doubling means each item is copied only a few times on average, so pushing is fast ("amortised constant time"). This single idea is why `std::vector` and JavaScript arrays are fast.

WHAT IF IT WERE DIFFERENT? ***What if you wrote l->items = realloc(l->items, ...) directly?***

If `realloc` fails it returns `NULL`, and you would overwrite your only pointer to the old memory with `NULL`, leaking it and crashing later. That is why the code stores the result in `bigger` first and checks it. Small details like this separate careful C from risky C.

A linked list

Another classic structure: items scattered in memory, each pointing to the next.

C

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
```

```
    struct Node *next;
} Node;

Node *push_front(Node *head, int v) {
    Node *n = malloc(sizeof *n);
    n->value = v;
    n->next = head;
    return n;
}

void print_list(const Node *n) {
    for (; n; n = n->next) printf("%d -> ", n->value);
    printf("NULL\n");
}

Node *reverse(Node *head) {
    Node *prev = NULL;
    while (head) {
        Node *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

void free_list(Node *n) {
    while (n) { Node *next = n->next; free(n); n = next; }
}

int main(void) {
    Node *list = NULL;
    for (int i = 1; i <= 4; i++) list = push_front(list, i);
    print_list(list);
    list = reverse(list);
    print_list(list);
    free_list(list);
    return 0;
}
```

OUTPUT

```
4 -> 3 -> 2 -> 1 -> NULL
1 -> 2 -> 3 -> 4 -> NULL
```

10.16 Bits and bytes

C

```
#include <stdio.h>

int main(void) {
    unsigned flags = 0;
    enum { BOLD = 1u << 0, ITALIC = 1u << 1, UNDERLINE = 1u << 2 };

    flags |= BOLD | UNDERLINE;          // turn bits ON
    flags &= ~UNDERLINE;                // turn a bit OFF
    flags ^= ITALIC;                    // flip a bit
    printf("flags=%u bold=%d italic=%d underline=%d\n", flags,
           (flags & BOLD) != 0, (flags & ITALIC) != 0, (flags & UNDERLINE) !=
0);
    printf("0x%02X %d %d\n", 255, 5 >> 1, 5 << 2);
    return 0;
}
```

OUTPUT

```
flags=3 bold=1 italic=1 underline=0
0xFF 2 20
```

Bit flags let one number carry many yes/no settings in a single byte. File permissions, window styles and network headers all use them.

10.17 Debugging C in NOVA

Put a breakpoint on any line of the programs above (click the margin, or press Ctrl+F8), then press **Shift+F9**. NOVA compiles with `-g`, starts gdb and stops on your line. Use F8 to step over, F7 to step into, F9 to resume. Watch the **Variables** panel: you will *see* a pointer hold an address, a `for` loop count, and `data[i]` fill in.

TRY IT ***Try it***

Take the `IntList` program, set a breakpoint on the line `l->items[l->count++] = value;` and step through the first three calls of `list_push`. Watch `count` and `capacity` change, and notice the moment `realloc` runs.

10.18 Mini assignments

MINI ASSIGNMENT ***Assignment 10.1: Temperature table***

Print a table of Celsius 0 to 100 (step 10) and the Fahrenheit equivalent with one decimal place. Formula: F = C × 9 / 5 + 32.

MINI ASSIGNMENT ***Assignment 10.2: Prime sieve***

Use an array of 100 flags to find and print all primes below 100 using the Sieve of Eratosthenes.

MINI ASSIGNMENT ***Assignment 10.3: Safe string reverse***

Write `void reverse(char *s)` that reverses a string in place (using two pointers moving toward each other). Then write `int is_palindrome(const char *s)`.

MINI ASSIGNMENT ***Assignment 10.4: Stack with a linked list***

Write `push`, `pop`, `peek` and `is_empty` for a stack of ints using the linked-list pattern. Test that `pop` on an empty stack is handled safely (return a flag, do not crash).

MINI ASSIGNMENT ***Assignment 10.5: Find the bug***

This program has three bugs. Find them with the debugger and Problems panel: (a) off-by-one in a loop, (b) a missing `free`, (c) a missing `NULL` check after `malloc`. Write them as a short test program yourself, then fix each.

10.19 Big assignment: Library manager

BIG ASSIGNMENT ***Library manager (a complete C program)***

Goal. Build a console program that manages a collection of books and saves it to disk. This uses almost everything in the chapter.

Requirements.

1. A `Book` struct with `title` (up to 60 chars), `author` (up to 40), `year`, `pages` and `available` (0 or 1).
2. Books live in a **dynamic array** that grows by doubling (reuse the `IntList` pattern with a struct item). Free it before exit.
3. A text **menu**: add book, list books, search by title (case-insensitive substring), check out, return, sort, save, load, quit.
4. Sort by title, author or year using `qsort` with three comparison functions chosen by the user.
5. **Save** to a text file (`library.txt`) with one book per line separated by `|`; **load** parses it back with `fgets` and `strtok` or `sscanf`. Handle a missing file and malformed lines without crashing.
6. Split the program across **three files**: `book.h` / `book.c` (the struct and list functions) and `main.c` (the menu). Use include guards. Build with `gcc main.c book.c -o`

`library.exe` and also from a CMake project.

7. Validate input: never use `gets`; read lines with `fgets` and convert with `strtol`. Reject a year outside 1450 to 2100.
8. Every `malloc` and `fopen` result is checked.
9. Compile with `-Wall -Wextra` and fix **every** warning.

Stretch goals. Add an undo for the last add or remove. Write a function that exports to CSV. Replace the array with a hash table keyed by title. Use gdb in NOVA to prove, by stepping, that your list really doubles its capacity.

How to check yourself. Run the program under NOVA's debugger and look at the Variables panel while the list grows. Add 20 books, save, quit, start again, load, and confirm the list is identical.

10.20 Quiz

QUIZ

1. What does `return 0;` from main mean?
2. Why must a string end with `'\0'`?
3. What do `&x` and `*p` mean?
4. Why does `swap(int a, int b)` fail to swap, but `swap(int *a, int *b)` works?
5. What is the difference between the stack and the heap?
6. Name the three common heap mistakes.
7. Why might `sizeof(struct Bad)` be larger than the sum of its fields?
8. What is wrong with `#define SQUARE(x) x * x`?
9. What do include guards prevent?
10. What does the linker do and what error appears if it cannot find a function?
11. Why double a growable array's capacity instead of adding one?
12. Why store the result of `realloc` in a new variable first?

ANSWERS

1. The program succeeded (exit code 0).
2. The string functions find the end by looking for the zero; without it they read past the end.
3. `&x` is the address of x; `*p` is the value stored at the address p holds.
4. C passes copies. Swapping copies changes nothing. Passing addresses lets the function change the caller's variables.
5. The stack is automatic, fast and limited (local variables); the heap is large and manual (`malloc`/`free`).
6. Memory leak, use after free (dangling pointer), double free.

7. The compiler inserts padding so fields start at addresses the CPU prefers.
8. Missing brackets: `SQUARE(3 + 1)` becomes `3 + 1 * 3 + 1`. It must be `((x) * (x))`.
9. A header being included twice and its contents being defined twice.
10. It joins object files and libraries into one program. "undefined reference to ..." appears.
11. Doubling copies each element only a few times on average, so pushing is fast.
12. If `realloc` fails it returns NULL, and assigning that straight back would lose the only pointer to the old memory (a leak).

CHAPTER 11

C++: C With Superpowers

11.1 What C++ adds

C++ began as "C with classes" in 1979 and has grown into a huge language. It keeps C's speed and closeness to the machine, and adds tools to organise big programs: **classes** (your own types), **templates** (code that works for any type), a rich **standard library** (lists, maps, strings, algorithms), and **automatic resource handling**.

Nearly all C code is also valid C++. That is why you learn C first: everything in Chapter 10 still works. Now we build on top of it.

> **IN PLAIN WORDS** *In plain words*
> C gives you bricks. C++ gives you bricks **plus** ready-made walls, doors and windows, and a foreman (the destructor) who tidies up after you.

NOVA's own sample project, the one in the screenshots, is C++. Open `hello_nova` in NOVA while you read: you can run, step through and change every example.

11.2 Hello, C++

C++

```
#include <iostream>

int main() {
    std::cout << "Hello, NOVA!" << std::endl;
    int a = 6, b = 7;
    std::cout << a << " times " << b << " is " << a * b << '\n';
    return 0;
}
```

OUTPUT

```
Hello, NOVA!
6 times 7 is 42
```

- `std::cout` is the **console output stream**. The `<<` operator sends things into it, one after another.
- `std::` means "this name lives in the **standard** namespace". A **namespace** is a labelled box of names so that two libraries can both have a `sort` without clashing.
- `std::endl` ends the line and flushes the output. A plain `'\n'` ends the line and is faster.

- `main` does not need `return 0;` in C++ (it is assumed), but it is clear to write it.

> **WHY?** ***Why std:: and not using namespace std?***
> Writing `using namespace std;` saves typing, but it dumps thousands of names into your file. If you also define something called `count` or `list` you get baffling errors. Professional code uses the `std::` prefix (or imports one name at a time). NOVA's own sample code does the same.

11.3 Everyday tools

References: a nickname for a variable

C++

```
#include <iostream>

void addOne(int& n) { n += 1; }               // & means: work on the ORIGINAL
void show(const int& n) { std::cout << n << '\n'; }   // const&: read-only, no
copy

int main() {
    int x = 5;
    addOne(x);
    int& alias = x;                           // alias is another name for x
    alias = 100;
    show(x);
    return 0;
}
```

OUTPUT

```
100
```

A **reference** is a pointer wearing a friendlier face: it is automatically dereferenced and can never be null. Use `const T&` to pass big things (such as strings) without copying them.

auto and range-for

C++

```
#include <iostream>
#include <vector>
#include <string>

int main() {
    std::vector<std::string> parts = {"editor", "tree", "terminal"};
    for (const auto& p : parts) std::cout << p << ' ' << p.size() << '\n';
    auto total = 0;
    for (auto& p : parts) { p += "!"; total += static_cast<int>(p.size()); }
    std::cout << parts.back() << ' ' << total << '\n';
    return 0;
}
```

OUTPUT

```
editor 6
tree 4
terminal 8
terminal! 21
```

- `auto` lets the compiler work out the type.
- `for (const auto& p : parts)` visits every element. With `&` there is no copy; with `const` you promise not to change it. NOVA's sample project uses exactly this loop to add up circle areas.

Strings

`std::string` replaces C's fragile character arrays. It grows by itself and knows its own length.

C++

```
#include <iostream>
#include <string>

int main() {
    std::string s = "NOVA";
    s += " IDE";
    std::cout << s << ' ' << s.size() << ' ' << s.substr(0, 4) << '\n';
    std::cout << (s.find("IDE") != std::string::npos) << ' ' << s.find('V') <<
'\n';
    s.replace(0, 4, "Nova");
    std::cout << s << ' ' << (s == "Nova IDE") << ' ' << std::to_string(42) +
"!" << '\n';
    std::cout << std::stoi("123") + 1 << '\n';
    return 0;
}
```

OUTPUT

```
NOVA IDE 8 NOVA
1 2
Nova IDE 1 42!
124
```

> **WHY?** ***Why is std::string safer than char arrays?***
> There is no null terminator to forget and no buffer to overflow: when you append, the string allocates more memory itself. The whole class of bugs warned about in Chapter 10 mostly vanishes. It is a perfect example of C++'s motto: *make the safe thing the easy thing*.

11.4 The standard library containers

Container	What it is	Good at	Slow at
`std::vector<T>`	Growable array	Index access, adding at the end	Inserting in the middle

Container	What it is	Good at	Slow at
`std::array<T,N>`	Fixed-size array	Speed, no allocation	Cannot grow
`std::list<T>`	Linked list	Insert/remove anywhere	Index access
`std::map<K,V>`	Sorted key to value	Lookups in order	Slower than hash
`std::unordered_map<K,V>`	Hash table	Very fast lookups	No ordering
`std::set<T>`	Sorted unique items	"Have I seen this?"	
`std::queue`, `std::stack`	First in/first out, last in/first out		

C++

```
#include <iostream>
#include <vector>
#include <map>
#include <set>
#include <algorithm>
#include <numeric>

int main() {
    std::vector<int> v = {5, 2, 9, 1, 7};
    v.push_back(3);
    std::sort(v.begin(), v.end());
    for (int n : v) std::cout << n << ' ';
    std::cout << '\n';

    std::cout << std::accumulate(v.begin(), v.end(), 0) << ' '
              << *std::max_element(v.begin(), v.end()) << ' '
              << std::count_if(v.begin(), v.end(), [](int n) { return n % 2 ==
1; }) << '\n';

    std::map<std::string, int> lines = {{"main.cpp", 120}, {"util.cpp", 45}};
    lines["app.cpp"] = 300;
    for (const auto& [file, n] : lines) std::cout << file << "=" << n << ' ';
    std::cout << '\n';

    std::set<int> unique(v.begin(), v.end());
    std::cout << unique.size() << ' ' << unique.count(7) << ' ' <<
unique.count(8) << '\n';
    return 0;
}
```

OUTPUT

```
1 2 3 5 7 9
27 9 5
app.cpp=300 main.cpp=120 util.cpp=45
6 1 0
```

The vector growth you built in C

Remember the `IntList` from the C chapter? `std::vector` is that, perfected. It doubles its capacity when full (see the growth figure in Chapter 10). You can watch it:

C++

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    size_t last = v.capacity();
    std::cout << "cap " << last << '\n';
    for (int i = 0; i < 20; i++) {
        v.push_back(i);
        if (v.capacity() != last) { last = v.capacity(); std::cout << "size " <<
v.size() << " -> cap " << last << '\n'; }
    }
    return 0;
}
```

OUTPUT

```
cap 0
size 1 -> cap 1
size 2 -> cap 2
size 3 -> cap 4
size 5 -> cap 8
size 9 -> cap 16
size 17 -> cap 32
```

Lambdas

A **lambda** is a small unnamed function you write on the spot. `[](int n) { return n % 2 == 1; }` above is one. The square brackets choose what the lambda may use from outside:

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

int main() {
    int threshold = 3;
    std::vector<std::string> words = {"a", "abcd", "ab", "abcde", "abc"};
    std::sort(words.begin(), words.end(), [](const auto& x, const auto& y) {
return x.size() < y.size(); });
    for (auto& w : words) std::cout << w << ' ';
    std::cout << '\n';
    auto longer = std::count_if(words.begin(), words.end(), [threshold](const
std::string& w) { return (int)w.size() > threshold; });
    std::cout << longer << '\n';
    return 0;
}
```

OUTPUT

```
a ab abc abcd abcde
2
```

`[threshold]` **captures** the variable by copy; `[&threshold]` would capture a reference; `[=]` and `[&]` capture everything.

11.5 Classes: your own types

A **class** bundles data (member variables) with the functions that work on it (member functions). It is the most important feature of C++. This is the very class from NOVA's sample project, now complete.

C++

```
#include <iostream>

class Circle {
public:
    explicit Circle(double r) : radius_(r) {}          // constructor
    double area() const { return 3.14159265 * radius_ * radius_; }
    void grow(double by) { radius_ += by; }
    double radius() const { return radius_; }

private:
    double radius_;                                    // hidden from outside
};

int main() {
    Circle c(2.0);
    std::cout << c.area() << '\n';
    c.grow(1.0);
    std::cout << c.radius() << ' ' << c.area() << '\n';
    // c.radius_ = -5;      // ERROR: radius_ is private
    return 0;
}
```

OUTPUT

```
12.5664
3 28.2743
```

Reading it:

- `public:` members can be used by anyone; `private:` members only by the class itself.
- The **constructor** `Circle(double r)` runs when an object is created. `: radius_(r)` is the **initializer list**: it sets the member directly.
- `const` after `area()` is a promise: *this function will not change the object*. A `const Circle` can only call `const` functions.
- `explicit` stops accidental conversions such as `Circle c = 5;`.

> **WHY?** ***Why make radius_ private?***
> So nobody can set it to an impossible value like -5 behind the class's back. If the only way to change the radius is through `grow`, then `grow` can check the rules. This is **encapsulation**: the class guards its own data. Later, if you store the radius differently, no outside code needs to change.

WHAT IF IT WERE DIFFERENT? ***What if area() were not marked const?***
Then you could not call it on a `const Circle&`, which is what you get when you write `for (const auto& c : circles)`. The compiler would refuse with an error about discarding qualifiers. Marking everything that does not modify state as `const` is a habit worth forming from day one.

Constructors and destructors: birth and death

C++

```
#include <iostream>
#include <string>

class Noisy {
public:
    explicit Noisy(std::string n) : name_(std::move(n)) { std::cout << "  build
" << name_ << '\n'; }
    ~Noisy() { std::cout << "  destroy " << name_ << '\n'; }
private:
    std::string name_;
};

int main() {
    std::cout << "start\n";
    Noisy a("A");
    {
        Noisy b("B");
        std::cout << "inside block\n";
    }
    Noisy c("C");
    std::cout << "end\n";
    return 0;
}
```

OUTPUT

```
start
  build A
  build B
inside block
  destroy B
  build C
end
  destroy C
  destroy A
```

Run it and study the order. Objects are destroyed **in reverse order** of creation, and **automatically** at the closing brace of their scope. Nobody has to remember to call anything.

11.6 RAII: the idea that defines C++

RAII stands for *Resource Acquisition Is Initialization*. It is a clumsy name for a simple, powerful idea: **tie the life of a resource (a file, memory, a lock) to the life of an object**. The constructor takes the resource and the destructor gives it back.

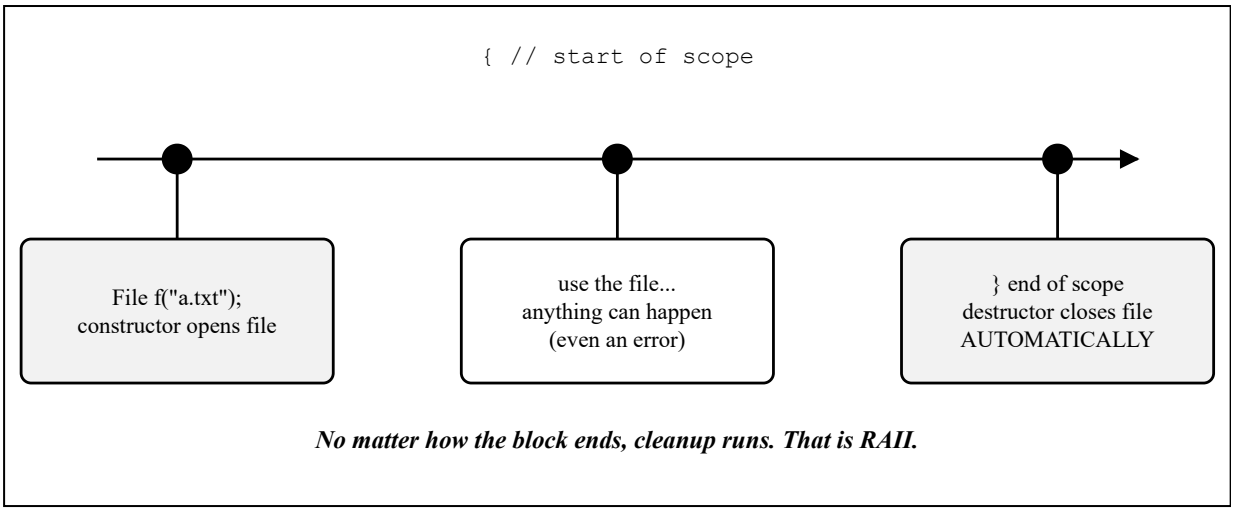


Figure 11.1. *With RAII the cleanup is guaranteed, however the block ends.*

C++

```
#include <cstdio>
#include <iostream>
#include <stdexcept>

class File {
public:
    File(const char* path, const char* mode) : f_(std::fopen(path, mode)) {
        if (!f_) throw std::runtime_error("cannot open file");
    }
    ~File() { if (f_) { std::fclose(f_); std::cout << "[file closed]\n"; } }
    File(const File&) = delete;                  // forbid copying (two owners
would double-close)
    File& operator=(const File&) = delete;
    void write(const char* text) { std::fputs(text, f_); }
private:
    std::FILE* f_;
};

int main() {
    try {
        File f("raii_demo.txt", "w");
        f.write("hello\n");
        throw std::runtime_error("something went wrong halfway");
    } catch (const std::exception& e) {
        std::cout << "caught: " << e.what() << '\n';
    }
    std::remove("raii_demo.txt");
    return 0;
}
```

OUTPUT

```
[file closed]
caught: something went wrong halfway
```

Notice that although an error was thrown, the line `[file closed]` is printed: the destructor still ran, because leaving the block, even by exception, destroys its objects.

> **WHAT IF IT WERE DIFFERENT?** ***What if you used fopen and fclose by hand?***
> You would need an `fclose` on every possible path out of the function: normal return, early return, and every exception. Forget one and you leak the file handle. In big programs this is unmanageable. RAII reduces it to *nothing to remember*.

Smart pointers

Raw `new` and `delete` are RAII's enemy: forgetting `delete` leaks. **Smart pointers** wrap them in objects that delete automatically.

C++

```
#include <iostream>
#include <memory>
#include <vector>

struct Tab {
    explicit Tab(int id) : id(id) { std::cout << "open tab " << id << '\n'; }
    ~Tab() { std::cout << "close tab " << id << '\n'; }
    int id;
};

int main() {
    auto a = std::make_unique<Tab>(1);          // exactly ONE owner
    std::cout << a->id << '\n';
    auto b = std::move(a);                      // ownership moves; a is now
empty
    std::cout << (a == nullptr) << ' ' << b->id << '\n';

    auto s1 = std::make_shared<Tab>(2);         // SHARED ownership, counted
    {
        auto s2 = s1;
        std::cout << "owners: " << s1.use_count() << '\n';
    }
    std::cout << "owners: " << s1.use_count() << '\n';

    std::vector<std::unique_ptr<Tab>> tabs;
    tabs.push_back(std::make_unique<Tab>(3));
    std::cout << "leaving main\n";
    return 0;
}
```

OUTPUT

```
open tab 1
1
1 1
open tab 2
owners: 2
owners: 1
open tab 3
leaving main
close tab 3
close tab 2
close tab 1
```

Type	Meaning	Use it when
`std::unique_ptr<T>`	One owner, deleted when it goes away	**Default choice**
`std::shared_ptr<T>`	Many owners, deleted when the last one goes away	You truly need shared ownership
`T&` or `T*` (raw)	"Just looking, not owning"	Passing things around without ownership

WHY? ***Why is unique_ptr the default?***

Single ownership is simple to reason about: the owner decides when the object dies. `shared_ptr` costs a counter and makes lifetimes harder to see. Modern C++ code almost never contains a bare `new` or `delete`.

11.7 Inheritance and polymorphism

A class can **inherit** from another: it gets all its members and adds more. With **virtual functions** a base-class pointer can run the right version for the real object. This is **polymorphism**.

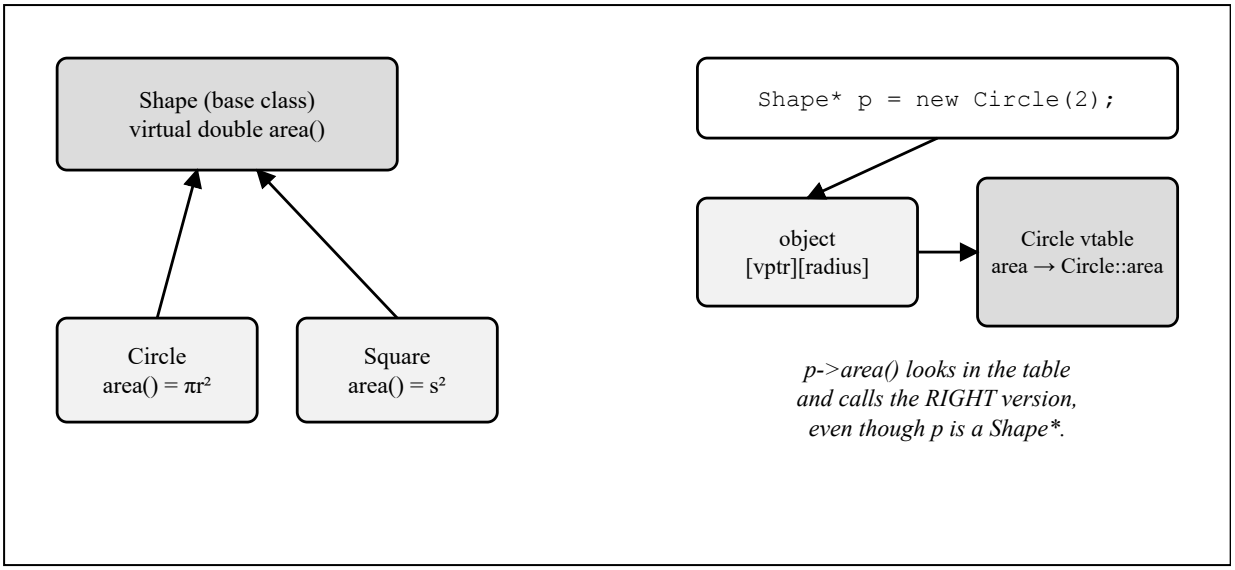


Figure 11.2. *A virtual call looks inside the object for a hidden table of functions, so p->area() runs the Circle version even through a Shape pointer.*

C++

```
#include <iostream>
#include <memory>
#include <vector>
#include <cmath>

class Shape {
public:
    virtual ~Shape() = default;                 // always make base destructors
virtual
    virtual double area() const = 0;            // "= 0": pure virtual, must be
overridden
    virtual const char* name() const = 0;
};

class Circle : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159265 * r_ * r_; }
    const char* name() const override { return "circle"; }
private:
    double r_;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w_(w), h_(h) {}
    double area() const override { return w_ * h_; }
    const char* name() const override { return "rectangle"; }
private:
    double w_, h_;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rect>(3.0, 4.0));
    shapes.push_back(std::make_unique<Circle>(2.0));

    double total = 0;
    for (const auto& s : shapes) {
        std::cout << s->name() << ": " << s->area() << '\n';
        total += s->area();
    }
    std::cout << "total: " << total << '\n';
    return 0;
}
```

OUTPUT

```
circle: 3.14159
rectangle: 12
circle: 12.5664
total: 27.708
```

WHY? ***Why virtual ~Shape()?***
When a `unique_ptr<Shape>` deletes a `Circle` through a `Shape` pointer, only a **virtual** destructor makes C++ run the `Circle` destructor. Without it the Circle part is never cleaned up properly: undefined behaviour. Rule: *a class meant to be a base needs a virtual destructor.*

WHAT IF IT WERE DIFFERENT? ***What if you write override and misspell the name?***
`override` tells the compiler "this must replace a base function". If you write `aera()` or change a `const`, the compiler stops with an error. Without `override` you would silently create a *new* function and the base one would never be replaced. It is a tiny keyword that prevents a very sneaky bug.

11.8 Operator overloading

You can teach your own types to use `+`, `==`, `<<` and so on.

C++

```
#include <iostream>

struct Vec2 {
    double x = 0, y = 0;
    Vec2 operator+(const Vec2& o) const { return {x + o.x, y + o.y}; }
    Vec2 operator*(double k) const { return {x * k, y * k}; }
    bool operator==(const Vec2& o) const { return x == o.x && y == o.y; }
};
std::ostream& operator<<(std::ostream& os, const Vec2& v) { return os << '(' <<
v.x << ", " << v.y << ')'; }

int main() {
    Vec2 a{1, 2}, b{3, 4};
    std::cout << a + b << ' ' << a * 3 << ' ' << (a == b) << '\n';
    return 0;
}
```

OUTPUT

```
(4, 6) (3, 6) 0
```

11.9 Templates: code for any type

A **template** is a recipe the compiler uses to write code for each type you need.

C++

```
#include <iostream>
#include <string>
#include <vector>

template <typename T>
T biggest(const T& a, const T& b) { return a < b ? b : a; }

template <typename T>
```

```
class Stack {
public:
    void push(const T& v) { items_.push_back(v); }
    T pop() { T v = items_.back(); items_.pop_back(); return v; }
    bool empty() const { return items_.empty(); }
    size_t size() const { return items_.size(); }
private:
    std::vector<T> items_;
};

int main() {
    std::cout << biggest(3, 9) << ' ' << biggest(2.5, 1.5) << ' ' <<
biggest(std::string("pear"), std::string("apple")) << '\n';
    Stack<std::string> history;
    history.push("main.cpp");
    history.push("util.cpp");
    std::cout << history.pop() << ' ' << history.size() << '\n';
    return 0;
}
```

OUTPUT

```
9 2.5 pear
util.cpp 1
```

WHY? ***Why templates instead of writing three versions?***

`biggest` for `int`, `double` and `string` would be three copies of the same logic, and a bug fixed in one would stay in the others. A template is written once, checked once, and compiled into exactly the versions you use, with **no speed penalty**. `std::vector`, `std::map` and all the containers are templates.

11.10 Errors: exceptions

C++

```
#include <iostream>
#include <stdexcept>
#include <string>

int parsePort(const std::string& s) {
    int n = std::stoi(s);                      // throws if it is not a number
    if (n < 1 || n > 65535) throw std::out_of_range("port out of range: " + s);
    return n;
}

int main() {
    for (std::string s : {"8080", "99999", "abc"}) {
        try {
            std::cout << "port " << parsePort(s) << '\n';
        } catch (const std::out_of_range& e) {
            std::cout << "range error: " << e.what() << '\n';
        } catch (const std::exception& e) {
            std::cout << "other error: " << e.what() << '\n';
        }
    }
```

```
    return 0;
}
```

OUTPUT

```
port 8080
port range error: port out of range: 99999
port other error: stoi
```

11.11 Move semantics, briefly

Copying a big object is slow. When the original is about to be thrown away anyway, C++ lets you **move**: steal its insides instead of copying them.

C++

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>

struct Buffer {
    std::vector<int> data;
    explicit Buffer(size_t n) : data(n, 7) { std::cout << "created " << n <<
'\n'; }
    Buffer(const Buffer& o) : data(o.data) { std::cout << "COPY\n"; }
    Buffer(Buffer&& o) noexcept : data(std::move(o.data)) { std::cout <<
"move\n"; }
};

Buffer make() { return Buffer(1000); }

int main() {
    Buffer a(1000);
    Buffer b = a;                    // copies: a is still needed
    Buffer c = std::move(a);         // moves: a gives up its insides
    std::cout << a.data.size() << ' ' << c.data.size() << '\n';
    Buffer d = make();               // returned objects are moved (or built in
place)
    std::string s = "text";
    std::string t = std::move(s);
    std::cout << t << ' ' << s.size() << '\n';
    return 0;
}
```

OUTPUT

```
created 1000
COPY
move
0 1000
created 1000
text 0
```

After a move, the old object is left empty but valid. This is why `return` of big vectors and strings from functions is cheap in modern C++.

11.12 Modern C++ goodies

C++

```
#include <iostream>
#include <optional>
#include <string>
#include <string_view>
#include <variant>
#include <map>

std::optional<int> findLine(const std::map<std::string, int>& m, const
std::string& k) {
    auto it = m.find(k);
    if (it == m.end()) return std::nullopt;     // "no value"
    return it->second;
}

constexpr int square(int x) { return x * x; }   // can run at compile time
static_assert(square(5) == 25);

int main() {
    std::map<std::string, int> lines = {{"main.cpp", 120}};
    if (auto n = findLine(lines, "main.cpp")) std::cout << "found " << *n <<
'\n';
    std::cout << findLine(lines, "none").value_or(-1) << '\n';

    std::variant<int, std::string> v = 42;      // holds ONE of several types
    v = std::string("now text");
    std::visit([](const auto& x) { std::cout << "variant: " << x << '\n'; }, v);

    std::string_view sv = "a view, no copy";    // a cheap look at existing text
    std::cout << sv.substr(2, 4) << ' ' << square(9) << '\n';

    auto [lo, hi] = std::pair{3, 8};            // structured bindings
    std::cout << lo + hi << '\n';
    return 0;
}
```

OUTPUT

```
found 120
-1
variant: now text
view 81
11
```

11.13 Threads, in one example

C++

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int main() {
    int counter = 0;
    std::mutex m;
```

```
    auto work = [&] {
        for (int i = 0; i < 1000; i++) {
            std::lock_guard<std::mutex> lock(m);     // RAII lock: released at
the closing brace
            ++counter;
        }
    };
    std::vector<std::thread> pool;
    for (int t = 0; t < 4; t++) pool.emplace_back(work);
    for (auto& t : pool) t.join();
    std::cout << "counter = " << counter << '\n';
    return 0;
}
```

OUTPUT

```
counter = 4000
```

WHAT IF IT WERE DIFFERENT? ***What if there were no mutex?***

Four threads would read, add and write `counter` at the same time and overwrite each other's work. The result would usually be less than 4000 and different every run: a **race condition**. The mutex lets only one thread in at a time. `lock_guard` is RAII again: the lock is released at the closing brace even if an exception is thrown.

11.14 Worked project: a C++ tokenizer

NOVA colours your code with Monaco. But how would you do it yourself? The first step is a **tokenizer**: cut the text into pieces and label each one. This program does exactly that, using classes, enums, vectors, sets and strings from this chapter.

C++

```
#include <cctype>
#include <iostream>
#include <map>
#include <set>
#include <string>
#include <vector>

enum class Kind { Keyword, Identifier, Number, String, Comment, Symbol };

const char* name(Kind k) {
    switch (k) {
        case Kind::Keyword: return "keyword";
        case Kind::Identifier: return "identifier";
        case Kind::Number: return "number";
        case Kind::String: return "string";
        case Kind::Comment: return "comment";
        case Kind::Symbol: return "symbol";
    }
    return "?";
}

struct Token { Kind kind; std::string text; };
```

```
std::vector<Token> tokenize(const std::string& src) {
    static const std::set<std::string> keywords = {"int", "return", "if",
"else", "for", "while", "void", "double", "class", "const", "auto"};
    std::vector<Token> out;
    size_t i = 0;
    while (i < src.size()) {
        unsigned char c = static_cast<unsigned char>(src[i]);
        if (std::isspace(c)) { ++i; continue; }
        size_t start = i;
        if (std::isalpha(c) || c == '_') {
            while (i < src.size() && (std::isalnum(static_cast<unsigned char>
(src[i])) || src[i] == '_')) ++i;
            std::string word = src.substr(start, i - start);
            out.push_back({keywords.count(word) ? Kind::Keyword :
Kind::Identifier, word});
        } else if (std::isdigit(c)) {
            while (i < src.size() && (std::isdigit(static_cast<unsigned char>
(src[i])) || src[i] == '.')) ++i;
            out.push_back({Kind::Number, src.substr(start, i - start)});
        } else if (c == '"') {
            ++i;
            while (i < src.size() && src[i] != '"') { if (src[i] == '\\') ++i;
++i; }
            ++i;
            out.push_back({Kind::String, src.substr(start, i - start)});
        } else if (c == '/' && i + 1 < src.size() && src[i + 1] == '/') {
            while (i < src.size() && src[i] != '\n') ++i;
            out.push_back({Kind::Comment, src.substr(start, i - start)});
        } else {
            ++i;
            out.push_back({Kind::Symbol, std::string(1, static_cast<char>(c))});
        }
    }
    return out;
}

int main() {
    const std::string code = "int total = 42; // the answer\nreturn
compute(total, \"hi\");";
    std::map<std::string, int> counts;
    for (const auto& t : tokenize(code)) {
        std::cout << name(t.kind) << "\t" << t.text << '\n';
        ++counts[name(t.kind)];
    }
    std::cout << "---\n";
    for (const auto& [k, n] : counts) std::cout << k << ": " << n << '\n';
    return 0;
}
```

OUTPUT

```
keyword int
identifier      total
symbol  =
number  42
symbol  ;
comment // the answer
keyword return
identifier      compute
symbol  (
identifier      total
symbol  ,
string  "hi"
symbol  )
symbol  ;
---
comment: 1
identifier: 3
keyword: 2
number: 1
string: 1
symbol: 6
```

WHY? ***Why enum class and not a plain enum?***

A plain `enum` leaks its names into the surrounding scope (`Keyword` would be visible everywhere) and converts silently to an `int`. An `enum class` keeps names inside (`Kind::Keyword`) and refuses silent conversion. Fewer clashes and fewer accidents.

11.15 Reading C++ error messages

C++ errors can be long, especially from templates. Use this recipe:

1. Read only the **first** error. Later ones are often fallout.
2. Find the file and line (NOVA underlines it and lists it in Problems).
3. Read the first sentence: it usually says exactly what is wrong (`'cnt' was not declared`, `expected ';'`).
4. For a template error, look for the line in **your** file inside the long chain, labelled `required from here`.
5. Fix one thing, rebuild, repeat.

11.16 Mini assignments

MINI ASSIGNMENT ***Assignment 11.1: Word frequency***

Read a sentence into a `std::string`, split it into words, count them with a `std::map<std::string,int>` (ignoring case), and print them sorted by count (highest first). *Hint: copy the map into a vector of pairs and `std::sort` with a lambda.*

MINI ASSIGNMENT ***Assignment 11.2: A BankAccount class***

Write `BankAccount` with a private balance, `deposit`, `withdraw` (throw `std::runtime_error` if the money is not there) and a `const` `balance()`. Test it with a try/catch.

MINI ASSIGNMENT ***Assignment 11.3: Shapes***

Add a `Triangle` to the shapes example. Add a `perimeter()` virtual function to all shapes. Print the shape with the largest area using `std::max_element` and a lambda.

MINI ASSIGNMENT ***Assignment 11.4: Generic Queue***

Write `template <typename T> class Queue` with `push`, `pop`, `front` and `size`. Use `std::deque` inside. Test it with `int` and `std::string`.

MINI ASSIGNMENT ***Assignment 11.5: Prove RAII***

Write a `Timer` class that records the time in its constructor and prints the elapsed milliseconds in its destructor. Put one at the top of a function to time it automatically.

11.17 Big assignment: A source-code outline tool

BIG ASSIGNMENT ***"Outline": a mini version of NOVA's Structure panel***

Goal. Write a command-line program that reads a C++ source file and prints an outline: the classes and functions it contains with their line numbers, plus a summary table.

Example output.

```
class Circle                    line 4
  function area()               line 7
  function grow(double)         line 8
function main()                 line 25
Summary: 1 class, 3 functions, 62 lines, 11 comments
```

Requirements.

1. Read a file name from `argv[1]`. Read the whole file with `std::ifstream` (RAII will close it).
2. Reuse the **tokenizer** from this chapter and extend it to track the **line number** of every token and to recognise multi-line `/* ... */` comments and `#` preprocessor lines.
3. Write a second stage, `outline(tokens)`, that finds `class`/`struct` names (the identifier after the keyword) and **functions** (an identifier followed by `(` at brace depth 0 or 1 where the matching `)` is followed by `{` or `const {`). Track brace depth by counting `{` and `}`.
4. Store results in a `struct Symbol { std::string kind, name; int line; int depth; }` held in a `std::vector`.
5. Print with `std::setw` for aligned columns, indented by depth.
6. Use at least: one class, one `enum class`, `std::map` for the summary counts, one lambda, `const&` parameters and range-for.
7. Handle errors (missing file, no file argument) with exceptions or clear messages and a non-zero exit code.
8. Split into `lexer.h/.cpp`, `outline.h/.cpp` and `main.cpp`, and build with a CMake project (Chapter 12). Run it in NOVA on **its own source files**.
9. No `new`/`delete` anywhere and no compiler warnings under `-Wall -Wextra`.

Stretch goals. Output JSON instead of plain text (the format NOVA's language server uses). Skip functions inside string literals and comments. Add `--sort name` to sort the outline. Compare your result to the Structure panel in NOVA for the same file and find where clangd, which really understands C++, differs from your simple version. That gap is why real tools use a proper parser.

How to check yourself. Run it on `hello_nova/src/main.cpp` and on `shapes.cpp`. The functions should match those in NOVA's Structure panel.

11.18 Quiz

QUIZ

1. Why prefer `std::cout` with the `std::` prefix over `using namespace std;`?
2. What does `const` after a member function's parameter list promise?
3. What is a constructor's initializer list for?
4. What is RAII in one sentence?
5. Why does `std::unique_ptr` make `delete` unnecessary?
6. When do you use `shared_ptr` instead of `unique_ptr`?
7. What does `virtual` do and why must a base class destructor be virtual?
8. What does `override` protect you from?
9. What is a template and why is it better than copy-pasting a function?

10. What is a race condition and how does a mutex help?
11. What does `std::move` actually do?
12. Why is `enum class` safer than plain `enum`?

ANSWERS

1. It avoids dumping thousands of names into your file, which can clash with your own names.
2. That the function does not change the object, so it can be called on const objects.
3. To initialise member variables directly when the object is created.
4. Tie a resource's lifetime to an object so that the destructor releases it automatically, however the scope ends.
5. The unique_ptr's destructor deletes the object when the pointer goes out of scope.
6. When several owners truly must share one object and it should die when the last owner is gone.
7. It chooses the function by the real object type at run time. A virtual base destructor makes deleting through a base pointer run the derived destructor too.
8. Silently creating a new function instead of replacing the base version because of a typo or mismatched signature.
9. A recipe the compiler uses to generate code for each type; one tested version replaces many copies.
10. Several threads changing the same data at once give wrong, varying results; a mutex lets one thread in at a time.
11. It casts to an rvalue so that move constructors/assignments may steal the contents; it does not move anything by itself.
12. Its names stay inside the enum's scope and it does not convert silently to integers.

CHAPTER 12

CMake: The Recipe Book

12.1 The problem CMake solves

Building a one-file program is easy: `g++ main.cpp -o hello.exe`. Now imagine a project with 200 files, three libraries, different compilers on Windows, Mac and Linux, debug and release versions, and tests. The command would fill a page and break the moment someone moved a file.

CMake solves this. You write one recipe, `CMakeLists.txt`, that says *what* to build. CMake then writes the real build instructions for whatever tool and system you have.

> **KEY WORD** ***CMake***
> A **build-system generator**. It does **not** compile anything itself. It reads your `CMakeLists.txt` and produces build files for another tool (Ninja, Make, Visual Studio), which then run the compiler.

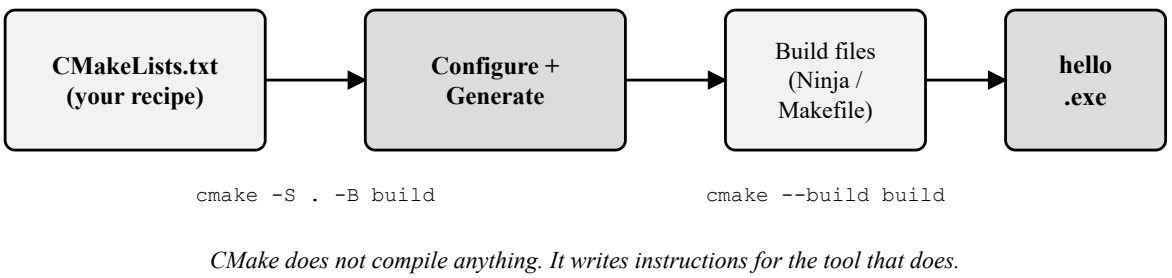


Figure 12.1. *CMake reads your recipe and writes build files. The build tool then does the compiling and linking.*

NOVA uses CMake because **CLion does** and because it is the industry standard for C and C++. Every new NOVA project starts with a `CMakeLists.txt`, and the build button runs exactly two commands: *configure* and *build*.

12.2 Your first CMakeLists.txt

This is the file NOVA's New Project wizard creates (and the sample project in the screenshots uses):

CMAKE

```
cmake_minimum_required(VERSION 3.16)
project(hello_nova LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(hello_nova src/main.cpp src/shapes.cpp)
target_include_directories(hello_nova PRIVATE include)
```

Command	What it says
`cmake_minimum_required(VERSION 3.16)`	"I need CMake 3.16 or newer", and use those rules.
`project(hello_nova LANGUAGES CXX)`	Name the project; it uses C++. (Use `C` for C, or both.)
`set(CMAKE_CXX_STANDARD 17)`	Use the C++17 language rules.
`set(CMAKE_CXX_STANDARD_REQUIRED ON)`	Fail if the compiler cannot do C++17.
`add_executable(name files...)`	Make a program called `name` from these files.
`target_include_directories(name PRIVATE include)`	Let `name` find headers inside the `include` folder.

WHY? ***Why cmake_minimum_required first?***

CMake's behaviour changes between versions. This line pins the rules, so a project written today behaves the same in five years and fails clearly if the CMake is too old, instead of misbehaving silently. Newer CMake versions even refuse very old values.

WHAT IF IT WERE DIFFERENT? ***What if you forget the CXX_STANDARD lines?***

The compiler uses its own default, which differs between compiler versions. Your code might compile on your machine and fail on a colleague's because they use a different default. Always state the standard you need.

12.3 The two commands

```
cmake -S . -B cmake-build-debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build cmake-build-debug
```

1. **Configure**: `-S .` is the **S**ource folder (where `CMakeLists.txt` lives), `-B cmake-build-debug` is the **B**uild folder (where all the generated files go), `-G Ninja` picks the generator, `-D NAME=value` sets a variable.
2. **Build**: `--build` runs the real build tool inside the build folder.

When you press Run in NOVA, its output panel shows exactly these commands. The second line you see is the compiler at work: `[1/2] Building CXX object ...` then `[2/2] Linking CXX executable hello_nova.exe`. The `[1/2]` counter comes from Ninja.

> **WHY?** ***Why a separate build folder?***
> This is called an **out-of-source build**. All the mess (object files, caches, the executable) goes in one folder that you can delete at any time, and your source folder stays clean. Add the build folder to `.gitignore`. NOVA names it `cmake-build-debug`, the same as CLion, and its New Project wizard writes that into `.gitignore` for you.

12.4 Build types: Debug and Release

Type	Flags (roughly)	Use for
`Debug`	`-g -O0`	Developing and debugging: full information, no optimisation, so stepping follows your code line by line
`Release`	`-O2 -DNDEBUG`	Shipping: fast, no debug info

NOVA always builds `Debug`, because its debugger needs the `-g` information. If you built Release and tried to debug, the debugger would jump around unpredictably, because the optimiser rearranges and removes your lines.

12.5 Libraries and linking

Real projects split into **libraries** (reusable code) and **executables** (programs). A target can **link** to a library.

CMAKE

```
cmake_minimum_required(VERSION 3.16)
project(shapes_demo LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 1. A library: code that other targets can use
add_library(geometry STATIC src/circle.cpp src/rect.cpp)
target_include_directories(geometry PUBLIC include)

# 2. A program that uses the library
add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE geometry)

# 3. A second program: tests
add_executable(tests tests/test_main.cpp)
target_link_libraries(tests PRIVATE geometry)
```

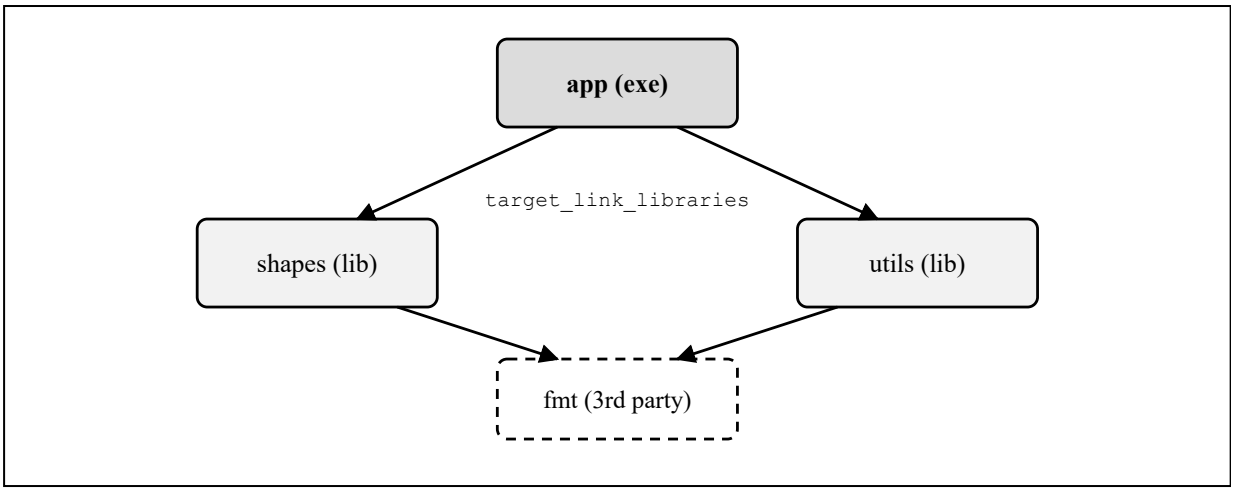


Figure 12.2. *Targets depend on each other. CMake works out the order to build them.*

Key ideas:

- `add_library(name STATIC files...)` makes a library. `STATIC` means it is copied into each program that uses it (a `.a` or `.lib` file). `SHARED` makes a DLL/`.so` loaded at run time.
- `target_link_libraries(app PRIVATE geometry)` says "app uses geometry". CMake builds `geometry` first and links it.
- `PUBLIC`, `PRIVATE`, `INTERFACE` describe who needs a setting:

Keyword	Meaning
`PRIVATE`	Only this target needs it.
`PUBLIC`	This target **and** anything that links to it needs it.
`INTERFACE`	Only the users need it, not the target itself.

WHY? ***Why does include have PUBLIC on the library?***
Anyone who uses `geometry` must also be able to `#include "circle.h"`. `PUBLIC` makes the `include` folder travel along: `app` and `tests` get it **automatically** when they link to `geometry`. If it were `PRIVATE`, `app` would fail with *"circle.h: No such file"* even though the library built perfectly. It is the most common CMake beginner error.

12.6 Variables, conditions and loops

CMAKE

```
set(SOURCES src/a.cpp src/b.cpp src/c.cpp)       # a list
add_executable(app ${SOURCES})                    # ${...} reads a variable

option(ENABLE_WARNINGS "Turn on strict warnings" ON)
if(ENABLE_WARNINGS)
  target_compile_options(app PRIVATE -Wall -Wextra)
endif()
```

```
foreach(name IN ITEMS alpha beta)
  message(STATUS "target: ${name}")
endforeach()

if(WIN32)
  target_link_libraries(app PRIVATE ws2_32)
elseif(APPLE)
  message(STATUS "building on a Mac")
endif()
```

`message(STATUS ...)` prints during configure. `message(FATAL_ERROR ...)` stops with an error. `option()` creates a switch that users flip with `-DENABLE_WARNINGS=OFF`.

12.7 Splitting a big project: add_subdirectory

```
project/
  CMakeLists.txt          <- top level: add_subdirectory(lib) and
add_subdirectory(app)
  lib/
    CMakeLists.txt        <- add_library(geometry ...)
    include/geometry/circle.h
    src/circle.cpp
  app/
    CMakeLists.txt        <- add_executable(app ...) and
target_link_libraries(app PRIVATE geometry)
    main.cpp
```

Each folder owns its own small recipe. The top file simply says `add_subdirectory(lib)` then `add_subdirectory(app)`.

> **WHY?** ***Why many small files?***
> So that each part can be understood, changed and reused alone, exactly like source code. NOVA's scanner finds *every* `CMakeLists.txt` in your project and reads each `add_executable(...)` to fill the run-configuration drop-down.

12.8 Testing with CTest

CMAKE

```
enable_testing()
add_test(NAME area_test COMMAND tests)
```

After building, run `ctest` in the build folder. A test passes if the program returns **exit code 0**, the same code you met in Chapters 9 and 10. That is why checking exit codes matters.

12.9 Fetching other people's code

CMAKE

```
include(FetchContent)
FetchContent_Declare(fmt GIT_REPOSITORY https://github.com/fmtlib/fmt GIT_TAG
10.2.1)
FetchContent_MakeAvailable(fmt)
target_link_libraries(app PRIVATE fmt::fmt)
```

CMake downloads the library and builds it with your project. `find_package(SomeLib REQUIRED)` instead looks for a library that is already installed.

12.10 compile_commands.json: how clangd learns your flags

Add this line to your configure command (NOVA does):

```
-DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

CMake then writes `compile_commands.json` in the build folder: a list that says, for every source file, the **exact compiler command** used.

JSON

```
[
  {
    "directory": "C:/proj/cmake-build-debug",
    "command": "g++ -IC:/proj/include -g -std=gnu++17 -o
CMakeFiles/app.dir/src/main.cpp.obj -c C:/proj/src/main.cpp",
    "file": "C:/proj/src/main.cpp"
  }
]
```

WHY? ***Why does the language server need it?***

To understand `#include "shapes.h"` clangd must know where the `include` folder is. To understand `std::optional` it must know you are using C++17. The compile database hands it every flag, so its idea of your code matches the compiler's. Without it clangd would guess, and show false errors on perfectly good code. NOVA runs a silent configure when you open a CMake project, just to get this file.

12.11 A real NOVA story

IN PLAIN WORDS ***The fallback database***

For folders that have **no** CMakeLists.txt (a few loose `.cpp` files) NOVA creates its own tiny `compile_commands.json` in a hidden `.nova` folder, listing each file with the C++ standard from the settings. Then clangd works there too. It is a nice example of generating a needed file when the "official" source is missing.

12.12 Common errors and fixes

Message	Likely cause	Fix
`Could not find CMAKE_CXX_COMPILER`	No compiler on PATH	Install MinGW/MSYS2 or LLVM and reopen NOVA
`No CMAKE_CXX_COMPILER could be found` after changing compilers	Stale cache	Delete the build folder and configure again
`undefined reference to 'foo'`	Source file not listed in `add_executable`	Add the `.cpp` to the target
`fatal error: x.h: No such file`	Missing include directory	`target_include_directories(... PRIVATE include)`
`CMake Error ... CMakeLists.txt:N`	Syntax mistake on that line	Check brackets and spelling at line N
Nothing rebuilds	Wrong build folder	Check the `-B` path

WATCH OUT ***Watch out***

When you add a new `.cpp` file you must add it to `CMakeLists.txt` (or use a glob) and **reconfigure**. NOVA marks the project "dirty" whenever you save a `CMakeLists.txt`, so the next Run reconfigures automatically.

12.13 Mini assignments

MINI ASSIGNMENT ***Assignment 12.1: Hello CMake***

Make a folder with `main.cpp` (print a message) and a `CMakeLists.txt`. Configure and build it from a terminal using the two commands, then run the program. Find the `.exe` in the build folder.

MINI ASSIGNMENT ***Assignment 12.2: Two files***

Move a function into `util.cpp` with `util.h`. Make it build. Then deliberately forget to list `util.cpp` and read the linker error. Fix it.

MINI ASSIGNMENT ***Assignment 12.3: A switch***

Add `option(VERBOSE "Print extra output" OFF)` and use `target_compile_definitions` so that `#ifdef VERBOSE` code in your program compiles only when you pass `-DVERBOSE=ON`.

MINI ASSIGNMENT ***Assignment 12.4: Read compile_commands.json***

Configure a project with `-DCMAKE_EXPORT_COMPILE_COMMANDS=ON`. Open the JSON file and find the exact command for one source file. Identify the include path and the language standard.

12.14 Big assignment: A multi-target project

BIG ASSIGNMENT ***"GeoKit": library, app, tests, options***

Goal. Build a small but realistic project with the structure of a professional C++ code base.

Requirements.

1. Folder layout: `lib/` (library), `app/` (program), `tests/` (tests), top-level `CMakeLists.txt`, `.gitignore`.
2. Library `geometry` (STATIC) with classes `Circle`, `Rect` and a free function `totalArea(const std::vector<std::unique_ptr<Shape>>&)`. Use the polymorphism example from Chapter 11. Public headers in `lib/include/geometry/`.
3. Program `geokit` that reads shapes from a text file (one per line, for example `circle 2.5`) and prints each area and the total.
4. A `tests` executable that checks at least six cases with a small `check(condition, message)` helper and returns a non-zero exit code if any check fails. Register it with `add_test`.
5. Options: `GEOKIT_WARNINGS` (default ON, adds `-Wall -Wextra -Wpedantic`) and `GEOKIT_SANITIZE` (default OFF, adds `-fsanitize=address,undefined` for compile **and** link).
6. Use `target_include_directories` with correct `PUBLIC`/`PRIVATE` choices, and prove they are right by removing `PUBLIC` and reading the error.
7. Export the compile commands and confirm clangd (in NOVA) shows no false errors in any file.
8. Open the folder in NOVA: the run-configuration drop-down must list `geokit` and `tests`. Run both, and debug `geokit` with a breakpoint inside `Circle::area`.

Stretch goals. Add an `install` rule. Add a `Release` build and compare run times. Pull in a testing framework (Catch2 or GoogleTest) using `FetchContent`.

How to check yourself. Delete the build folder, press Run in NOVA, and the whole project must configure, build and run with no manual steps.

12.15 Quiz

QUIZ

1. Does CMake compile code? Explain.
2. What do `-S` and `-B` mean?
3. Why use a separate build folder?
4. What is the difference between `Debug` and `Release`?
5. What do `PUBLIC` and `PRIVATE` mean in `target_include_directories`?
6. What does `option()` create?
7. Why does the language server need `compile_commands.json`?
8. What error appears if a `.cpp` is missing from `add_executable`?
9. Why does `add_test` depend on exit codes?
10. Why does NOVA build in Debug mode?

ANSWERS

1. No. It writes build files for another tool (Ninja, Make...), which runs the compiler.
2. `-S` is the source folder; `-B` is the build folder.
3. It keeps generated files out of your sources and can be deleted safely.
4. Debug has debugging information and no optimisation; Release is optimised and has no debug info.
5. `PRIVATE` is for the target only; `PUBLIC` is for the target and everything that links to it.
6. A yes/no switch that users can change with `-DNAME=ON/OFF`.
7. It lists the exact flags (include folders, language standard) for every file, so clangd understands the code the same way the compiler does.
8. A linker error: `undefined reference to ...`.
9. A test passes when its program returns exit code 0 and fails otherwise.
10. The debugger needs the `-g` information, and without optimisation, stepping follows the source lines.

CHAPTER 13

JSON and Protocols: How Programs Talk

13.1 Why this chapter exists

NOVA is a team of programs: the window, clangd, gdb, the compiler. Programs cannot read each other's minds. They must agree on **how** to send messages. An agreed way of talking is a **protocol**. In this chapter you learn the three "languages" NOVA speaks with its helpers: **JSON**, the **Language Server Protocol** (LSP) and **GDB/MI**, plus the terminal's secret code, **ANSI escape sequences**.

> **KEY WORD** ***Protocol***
> A **protocol** is a set of rules for a conversation between programs: what a message looks like, who speaks first, how to say "I did not understand", and how to know a message has ended.

13.2 JSON: the common alphabet

JSON (JavaScript Object Notation) is a simple text format for data. It is easy for people to read and easy for every language to parse. It has only six kinds of value:

Kind	Example
string	`"main.cpp"`
number	`42`, `3.14`, `-7`
boolean	`true`, `false`
null	`null`
array (a list)	`[1, 2, 3]`
object (named parts)	`{"name": "main.cpp", "line": 12}`

JSON

```
{
  "theme": "dark",
  "fontSize": 14,
  "ligatures": true,
  "recent": ["C:/projects/hello_nova", "C:/projects/geokit"],
  "debugExtConsole": null
}
```

This is, almost exactly, NOVA's settings file.

JAVASCRIPT

```
const settings = { theme: "dark", fontSize: 14, recent: ["a", "b"], extra:
undefined };
const text = JSON.stringify(settings);
console.log(text);                          // undefined fields are dropped
console.log(JSON.stringify(settings, null, 2));   // pretty, with 2-space indent

const back = JSON.parse(text);
console.log(back.fontSize + 1, back.recent[1]);

try { JSON.parse("{ 'bad': 1 }"); } catch (e) { console.log("invalid JSON:",
e.name); }
```

OUTPUT

```
{"theme":"dark","fontSize":14,"recent":["a","b"]}
{
  "theme": "dark",
  "fontSize": 14,
  "recent": [
    "a",
    "b"
  ]
}
15 b
invalid JSON: SyntaxError
```

WHY? ***Why JSON's strict rules?***

JSON requires **double quotes** and forbids trailing commas and comments. That strictness is what lets any program in any language read it without argument. NOVA wraps `JSON.parse` for its settings in `try/catch`: if a hand-edited file has a mistake, NOVA starts with defaults instead of crashing.

WATCH OUT ***Watch out***

`JSON.stringify` cannot carry functions, `undefined` or circular references, and turns a `Map` or `Set` into `{}`. Convert them first (for example `[...mySet]`).

13.3 Framing: where does one message end?

Programs talk through **pipes**, which are just streams of bytes with no message boundaries. If you write `{"a":1}` and then `{"b":2}`, the reader might receive `{"a":1}{"b` first and `":2}` later. The receiver needs a rule to split the stream into messages. This rule is called **framing**.

LSP uses a header with the byte length:

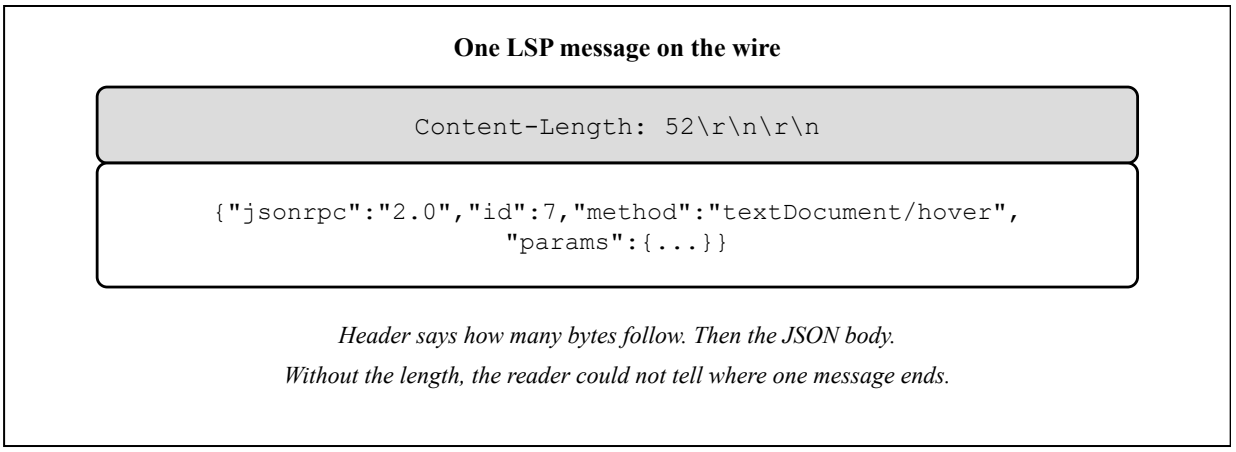


Figure 13.1. *One LSP message: a header telling how many bytes follow, then exactly that many bytes of JSON.*

Here is a working reader built the same way as the one in NOVA's `main.js`. It handles messages split at any point.

JAVASCRIPT

```
function makeReader(onMessage) {
  let buf = Buffer.alloc(0);
  return (chunk) => {
    buf = Buffer.concat([buf, chunk]);
    for (;;) {
      const headerEnd = buf.indexOf("\r\n\r\n");
      if (headerEnd < 0) return;                          // header not
complete yet
      const m = /Content-Length:\s*(\d+)/i.exec(buf.slice(0,
headerEnd).toString("ascii"));
      if (!m) { buf = buf.slice(headerEnd + 4); continue; }
      const len = Number(m[1]);
      if (buf.length < headerEnd + 4 + len) return;       // body not complete
yet
      const body = buf.slice(headerEnd + 4, headerEnd + 4 +
len).toString("utf8");
      buf = buf.slice(headerEnd + 4 + len);
      onMessage(JSON.parse(body));
    }
  };
}

const frame = (obj) => {
  const json = JSON.stringify(obj);
  return Buffer.from(`Content-Length: ${Buffer.byteLength(json,
"utf8")}\r\n\r\n${json}`, "utf8");
};

const feed = makeReader((msg) => console.log("got:", JSON.stringify(msg)));
const wire = Buffer.concat([frame({ id: 1, method: "hello" }), frame({ id: 2,
method: "naïve ✓" })]);

// Deliver the bytes in awkward pieces: 7 bytes, 5 bytes, then the rest.
feed(wire.slice(0, 7));
feed(wire.slice(7, 12));
feed(wire.slice(12));
```

OUTPUT

```
got: {"id":1,"method":"hello"}
got: {"id":2,"method":"naïve ✓"}
```

WHY? ***Why Buffer.byteLength and not text.length?***

The header counts **bytes**, not characters. The text `naïve ✓` has 7 characters but 10 bytes in UTF-8, because `ï` and `✓` take more than one byte. Counting characters would make the reader cut a message in the wrong place and every later message would be garbage. NOVA's main process uses `Buffer.byteLength` for exactly this reason.

WHAT IF IT WERE DIFFERENT? ***What if a message is incomplete and the reader parses it anyway?***

`JSON.parse` would throw on half a message, and the stream would be permanently out of step: the next chunk would begin in the middle of something. Returning and waiting for more bytes (as the code above does) keeps the reader in sync.

13.4 JSON-RPC: asking questions across the pipe

LSP messages follow **JSON-RPC 2.0**. There are three kinds:

Kind	Has `id`?	Has `method`?	Meaning
Request	yes	yes	"Please do this and answer me."
Response	yes (same id)	no (has `result` or `error`)	The answer to one request.
Notification	no	yes	"FYI", no answer expected.

JSON

```
{"jsonrpc":"2.0","id":7,"method":"textDocument/hover",
 "params":{"textDocument":{"uri":"file:///c%3A/proj/main.cpp"},"position":
{"line":10,"character":17}}}
```

Reply:

JSON

```
{"jsonrpc":"2.0","id":7,"result":{"contents":{"kind":"markdown","value":"###
function `square`..."}}}
```

The **id** is the clever part. Many requests can be in flight at once, and answers may come in any order. The id says which answer belongs to which question.

JAVASCRIPT

```
const pending = new Map();      // id -> function to call with the answer
let nextId = 0;

function request(send, method, params, timeoutMs = 200) {
  return new Promise((resolve) => {
```

```
    const id = ++nextId;
    const timer = setTimeout(() => { pending.delete(id); resolve(null); },
timeoutMs);   // give up
    pending.set(id, (result) => { clearTimeout(timer); resolve(result); });
    send({ jsonrpc: "2.0", id, method, params });
  });
}
function onResponse(msg) {
  const cb = pending.get(msg.id);
  if (cb) { pending.delete(msg.id); cb(msg.result); }
}

// A fake server that answers the SLOW request after the FAST one.
const server = (msg) => {
  const delay = msg.method === "slow" ? 60 : 10;
  setTimeout(() => onResponse({ id: msg.id, result: msg.method + " answer" }),
delay);
};
const silent = () => { /* never answers */ };

(async () => {
  const [a, b] = await Promise.all([request(server, "slow"), request(server,
"fast")]);
  console.log(a, "|", b);                          // each got its OWN answer
  console.log(await request(silent, "ignored"));  // times out -> null
})();
```

OUTPUT

```
slow answer | fast answer
null
```

This is the pattern in `lsp.js`: a `Map` of waiting requests, a timeout so a stuck server cannot freeze the editor, and the id to pair answers with questions.

> **WHY?** ***Why a timeout on every request?***
> A language server can crash, hang or be busy indexing. Without a timeout, the code that asked "what can I type here?" would wait forever, and a stuck completion popup would be the least of your problems. NOVA gives every request eight seconds (thirty for the first `initialize`), then simply returns "no answer".

The LSP conversation in order

1. **`initialize`** (request): NOVA says "my project folder is here, and I understand these features". The server answers with *its* capabilities.
2. **`initialized`** (notification): "Great, let's go."
3. **`textDocument/didOpen`** (notification): "I opened this file; here is its text."
4. **`textDocument/didChange`** (notification): "The text changed." (NOVA sends the whole text, debounced by 120 ms.)
5. **Requests** such as `completion`, `hover`, `definition`, `references`, `rename`, `formatting`, `codeAction`.

6. `textDocument/publishDiagnostics` (notification, from the server): "Here are the errors in this file." NOVA turns them into wavy lines.

> **WHAT IF IT WERE DIFFERENT?** ***What if NOVA forgot to send didChange?***
> The server would keep analysing the *old* text. Completion would suggest names that no longer exist, hover would point at wrong lines, and error underlines would sit in the wrong places. The editor and server must always agree about the file's contents.

The off-by-one trap

LSP positions are **0-based** (line 0 is the first line). Monaco's are **1-based**. NOVA converts at the border:

JAVASCRIPT

```
const toPos = (p) => ({ line: p.lineNumber - 1, character: p.column - 1 });
const fromRange = (r) => new monaco.Range(r.start.line + 1, r.start.character +
1, r.end.line + 1, r.end.character + 1);
```

Forget the conversion and every underline lands one line too high.

13.5 GDB/MI: talking to the debugger

gdb has a human interface (type `print x`). It also has a **machine interface** called **GDB/MI** designed for programs like NOVA. You start gdb with `--interpreter=mi3` and talk to it in lines of text.

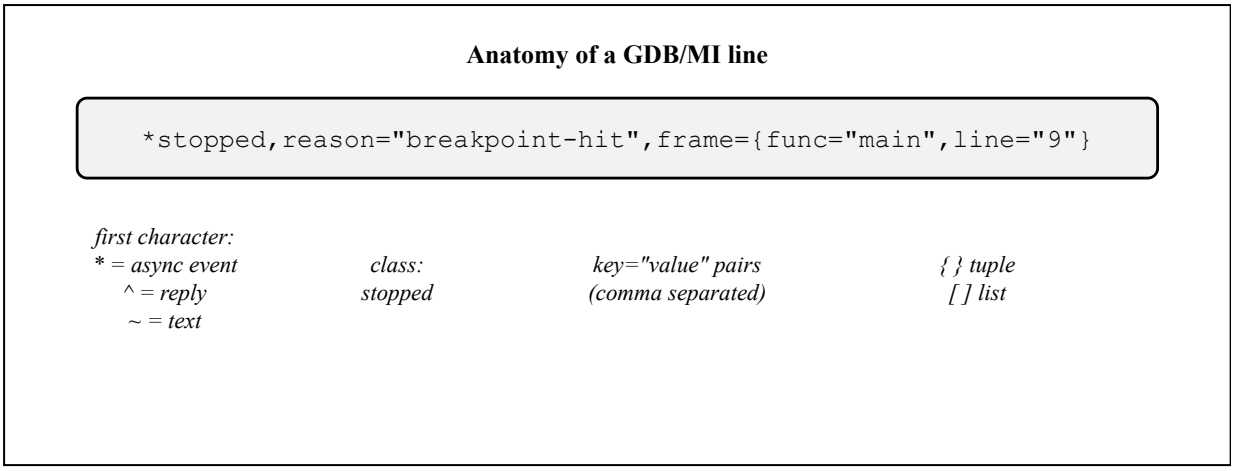


Figure 13.2. *The first character of every MI line tells you what kind of message it is.*

First char	Kind	Example
`^`	**Result** of a command	`^done,bkpt={number="1"}`
`*`	**Async** event (program state changed)	`*stopped,reason="breakpoint-hit"`

First char	Kind	Example
`=`	Notification (breakpoint added, library loaded)	`=breakpoint-modified,...`
`~`	Console output text	`~"Hello\n"`
`@`	The program's own output	`@"Total: 5\n"`
`&`	gdb's log	`&"print x\n"`

A command may begin with a **token** (a number) so you can match the result: you send `7-exec-next` and gdb answers `7^running`.

After the class name, data is a list of `key="value"` pairs, where a value can be a string, a `{tuple}` or a `[list]`. Here is a working parser that is the same logic as NOVA's `debug.js`:

JAVASCRIPT

```
function miCString(p) {
  if (p.s[p.i] !== '"') return "";
  p.i++;
  let out = "";
  while (p.i < p.s.length) {
    const c = p.s[p.i++];
    if (c === '"') break;
    if (c === "\\") {
      const n = p.s[p.i++];
      if (n === "n") out += "\n"; else if (n === "t") out += "\t"; else out +=
n;
    } else out += c;
  }
  return out;
}
function miValue(p) {
  const c = p.s[p.i];
  if (c === '"') return miCString(p);
  if (c === "{") { p.i++; return miResults(p, "}"); }
  if (c === "[") {
    p.i++;
    const arr = [];
    while (p.i < p.s.length && p.s[p.i] !== "]") {
      if (/^[\w-]+=/.test(p.s.slice(p.i))) {          // a named item such as
frame={...}
        const eq = p.s.indexOf("=", p.i);
        const k = p.s.slice(p.i, eq);
        p.i = eq + 1;
        arr.push({ [k]: miValue(p) });
      } else arr.push(miValue(p));
      if (p.s[p.i] === ",") p.i++;
    }
    p.i++;
    return arr;
  }
  return "";
}
```

```
function miResults(p, close) {
  const o = {};
  while (p.i < p.s.length && p.s[p.i] !== close) {
    const eq = p.s.indexOf("=", p.i);
    if (eq < 0) break;
    const k = p.s.slice(p.i, eq);
    p.i = eq + 1;
    o[k] = miValue(p);
    if (p.s[p.i] === ",") p.i++;
  }
  if (close && p.s[p.i] === close) p.i++;
  return o;
}
function miParseLine(line) {
  const m = /^(\d*)([\^*=~@&])(.*)$/.exec(line);
  if (!m) return { t: "raw", text: line };
  const [, tok, ch, rest] = m;
  if ("~@&".includes(ch)) return { t: ch, text: miCString({ s: rest, i: 0 }) };
  const ci = rest.indexOf(",");
  const cls = ci < 0 ? rest : rest.slice(0, ci);
  const data = ci < 0 ? {} : miResults({ s: rest.slice(ci + 1), i: 0 });
  return { t: ch, tok: tok ? +tok : null, cls, data };
}

const samples = [
  '*stopped,reason="breakpoint-hit",frame={func="main",line="9"}',
  '7^done,stack=[frame={level="0",func="square",line="4"},frame=
{level="1",func="main",line="11"}]',
  '~"Hello, \\"NOVA\\"\\n"',
  "^running",
  "just some program output",
];
for (const s of samples) console.log(JSON.stringify(miParseLine(s)));
```

OUTPUT

```
{"t":"*","tok":null,"cls":"stopped","data":{"reason":"breakpoint-hit","frame":
{"func":"main","line":"9"}}}
{"t":"^","tok":7,"cls":"done","data":{"stack":[{"frame":
{"level":"0","func":"square","line":"4"}},{"frame":
{"level":"1","func":"main","line":"11"}}]}}
{"t":"~","text":"Hello, \"NOVA\"\n"}
{"t":"^","tok":null,"cls":"running","data":{}}
{"t":"raw","text":"just some program output"}
```

Study the second output: a nested structure with a list of frames turned into plain JavaScript data. NOVA reads `data.stack`, maps each `frame` to `{func, line, file}` and draws the Frames panel.

> **WHY?** ***Why write a parser instead of using a regular expression?***
> MI values **nest**: a tuple contains a list that contains tuples. Regular expressions cannot match arbitrarily deep nesting. A parser that calls itself for each nested level (`miValue` calls `miResults`, which calls `miValue`...) handles any depth. It is the same *recursive descent* idea as the calculator in Chapter 8.

13.6 The terminal: bytes and escape codes

A terminal is a program that shows a grid of characters and sends your key presses to a shell. A shell program cannot draw colours or move the cursor with function calls, so it sends special byte sequences hidden inside its text, beginning with the character **ESC** (code 27, written `\x1b`). These are **ANSI escape sequences**.

JAVASCRIPT

```
const ESC = "\x1b";
const coloured = `${ESC}[31mred${ESC}[0m normal ${ESC}[1;32mbold green${ESC}
[0m`;
console.log(JSON.stringify(coloured));

// What NOVA's Run panel does: strip them, because it draws its own colours
const plain = coloured.replace(/\x1b\[[0-9;]*[A-Za-z]/g, "");
console.log(plain);
```

OUTPUT

```
"\u001b[31mred\u001b[0m normal \u001b[1;32mbold green\u001b[0m"
red normal bold green
```

Common ones: `ESC[31m` red text, `ESC[0m` reset all, `ESC[2J` clear screen, `ESC[10;5H` move the cursor to row 10, column 5. The **xterm.js** library inside NOVA understands them all and draws the result.

> **WHY?** ***Why does NOVA use a PTY for the terminal instead of simple pipes?***
> Programs behave differently when they know a human is watching. Many check "is my output a **terminal**?" and, if not, turn off colours, buffer their output and refuse to ask questions. A **PTY** (pseudo-terminal; on Windows, **ConPTY**) pretends to be a real terminal, so shells, git, Python and text-mode programs behave normally. With plain pipes, PowerShell would print no prompt and interactive tools would not work.

> **IN PLAIN WORDS** ***The buffered printf***
> The Run panel uses pipes, not a PTY, so a C program's `printf("Enter a number: ")` may stay hidden in the program's own buffer until it ends. C's standard library buffers output fully when it is not connected to a terminal. NOVA fixes this by quietly adding a tiny header to the build, `-include nova_unbuf.h`, which calls `setvbuf(stdout, NULL, _IONBF, 0)` to turn the buffering off. This is why prompts appear instantly in NOVA. A small trick, but it needed a good understanding of how protocols between a program and its viewer really work.

13.7 Designing your own protocol: lessons

1. **Frame your messages.** Never assume a read equals a message.

2. **Give requests ids.** Answers arrive out of order.
3. **Add timeouts.** The other side can hang or crash.
4. **Separate requests from notifications.** Not everything needs an answer.
5. **Be strict when sending, tolerant when receiving.** NOVA ignores unknown messages instead of crashing.
6. **Make states explicit.** The debugger moves between *not debugging, running, paused, ended*:

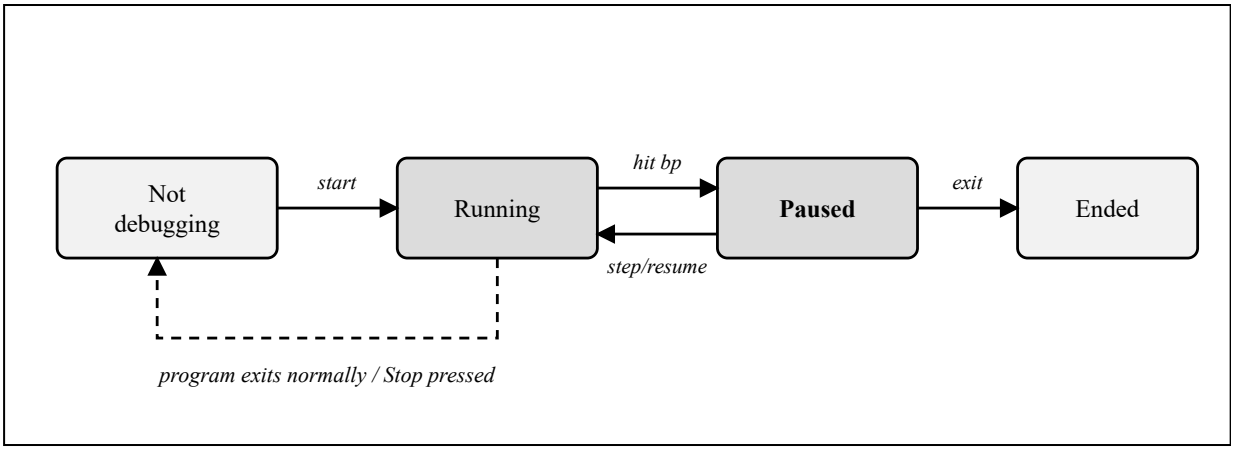


Figure 13.3. *The debugger is a small state machine. Buttons are enabled or disabled depending on the current state.*

> **WHY?** ***Why model the debugger as states?***
> You can only step when the program is *paused*. Pressing Step while it is *running* makes no sense. Writing the states down means every button simply asks "what state am I in?" instead of a tangle of true/false variables. NOVA's `dbgUi` function sets all the toolbar buttons from the two flags `D.on` and `D.running`.

13.8 Mini assignments

> **MINI ASSIGNMENT** ***Assignment 13.1: Settings round trip***
> Write a Node script that loads `settings.json` (create defaults if missing), changes the font size, and saves it back with 2-space indent. If the file contains invalid JSON, print a message and use defaults.

> **MINI ASSIGNMENT** ***Assignment 13.2: Strip and count***
> Take a string with ANSI colour codes in it. Strip the codes and count how many red (`31`) and green (`32`) sections there were.

MINI ASSIGNMENT ***Assignment 13.3: Frame stress test***

Using `makeReader`, send 100 random messages split into random-sized chunks (1 to 20 bytes). Check that all 100 arrive intact and in order.

MINI ASSIGNMENT ***Assignment 13.4: Parse more MI***

Extend the MI parser to decode `^done,variables=[{name="x",value="1"},{name="y",value="2"}]` into `{ x: "1", y: "2" }`. Add a function `framesFrom(record)` that returns an array of `{ func, line }`.

13.9 Big assignment: A fake language server

BIG ASSIGNMENT ***"EchoLS": your own tiny LSP server***

Goal. Write a real, working language server in Node that speaks LSP over stdin/stdout, then connect NOVA-style client code to it.

Requirements.

1. The server reads framed JSON-RPC messages from `process.stdin` using a reader like `makeReader`, and writes framed replies to `process.stdout`.
2. It implements `initialize` (reply with capabilities `{ hoverProvider: true, completionProvider: {} }`), `initialized`, `shutdown`, `exit`.
3. It keeps a map of open documents from `didOpen` / `didChange` / `didClose`.
4. `textDocument/hover` returns the **word under the cursor** and how many times it appears in the file.
5. `textDocument/completion` returns every distinct word in the document that starts with the word being typed.
6. After every `didOpen` and `didChange` it sends `publishDiagnostics` with a **warning on every line containing the text** `TODO` and an **error on any line longer than 100 characters**.
7. Write a second script, the **client**, that starts the server with `spawn`, performs the handshake, opens a document, asks for hover and completion at known positions, prints the answers, and shuts down cleanly. Use a `pending` map with ids and timeouts.
8. Handle unknown methods by replying with an error (`code -32601`) for requests and ignoring notifications.
9. Remember positions are **0-based**.

Stretch goals. Add `textDocument/definition` that jumps to the first place a word appears. Plug your server into a tiny Monaco page. Better still: change `lsp.js` in NOVA to start your server for `.txt` files and watch your own warnings appear as wavy lines.

How to check yourself. Pipe a hand-written framed message into your server in a terminal and read the framed answer. The headers' lengths must match the byte counts exactly.

13.10 Quiz

QUIZ

1. Name the six kinds of JSON value.
2. Why can a pipe not simply carry "one message per read"?
3. How does LSP frame a message?
4. Why must the header count bytes, not characters?
5. What are the three kinds of JSON-RPC message?
6. What is the `id` for?
7. What does the first character of a GDB/MI line tell you? Give two examples.
8. Why does a parser for MI need recursion?
9. What is an ANSI escape sequence?
10. Why does the terminal need a PTY, while the Run panel gets by with pipes?
11. Why must LSP positions be converted for Monaco?

ANSWERS

1. String, number, boolean, null, array, object.
2. Pipes are plain byte streams; a read can contain half a message or several. You need a rule (framing) to split them.
3. A `Content-Length: N` header, a blank line, then exactly N bytes of JSON.
4. Because the reader counts bytes on the wire; multi-byte characters would otherwise make it cut in the wrong place.
5. Request, response and notification.
6. To match an answer to its question, since many requests may be pending at once and answers can arrive in any order.
7. The message kind, e.g. `^` result of a command, `*` async event such as `*stopped`, `~` console text.
8. Values nest to any depth (tuples inside lists inside tuples) and only a self-calling function can follow that.
9. A hidden byte sequence starting with ESC that tells a terminal to colour text, move the cursor, clear the screen, etc.
10. Interactive programs behave properly only if they think they talk to a terminal; the Run panel only shows output from your own program.
11. LSP is 0-based and Monaco is 1-based; without conversion every position is off by one.

Part III — Building NOVA, Step by Step

CHAPTER 14

Build Guide, Part 1: Setup and the First Window

14.1 The plan

You now know the languages. In this part you will **build NOVA** from an empty folder, in twelve steps. Each step adds one feature, ends with a **CHECKPOINT** that shows what you should see, and explains the **business logic**: what the feature must do for the person using the IDE, and why.

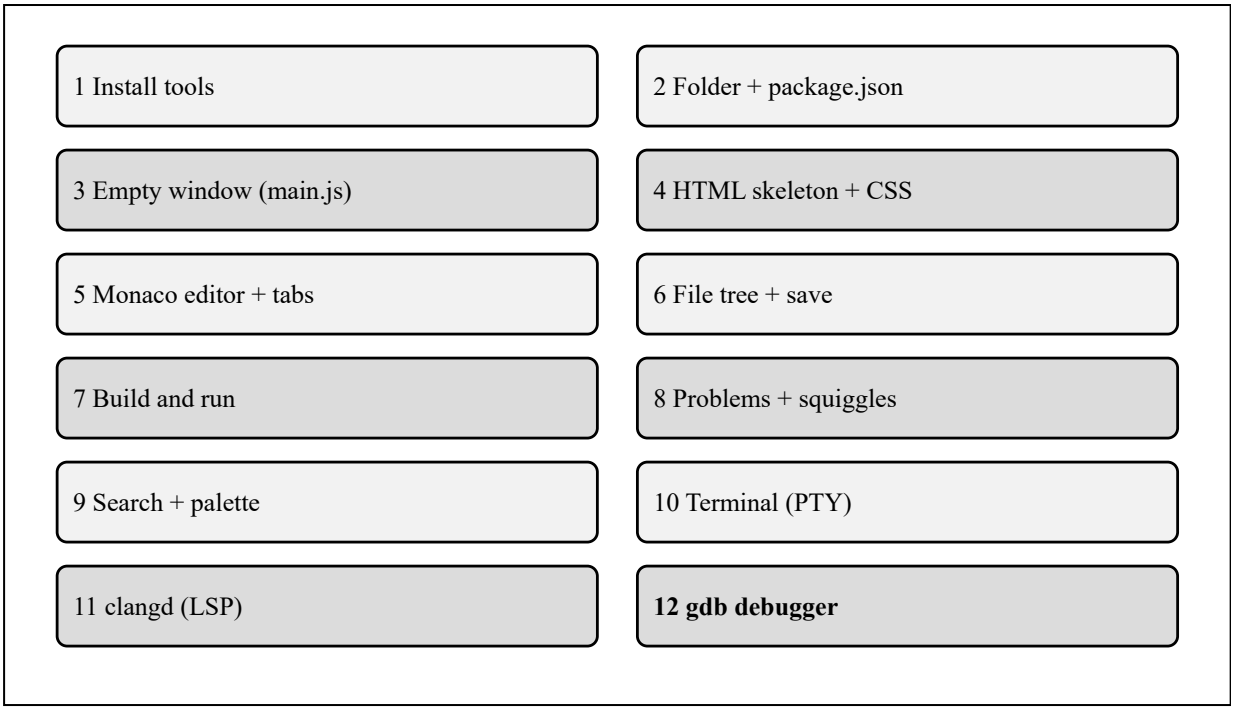


Figure 14.1. *The twelve steps of the build. Each one leaves you with a working program.*

> **KEY WORD** ***Business logic***
>
> In this book, **business logic** means *the rules about what the program must do for its user*. For example: "A tab shows a dot when its file has unsaved changes" or "When a build fails, show the errors and do not start the program". Business logic is different from *technical* code (how to read a file). Good programmers write the rules first, then the code.

How to use this part:

- Build in order. Each step uses the one before it.

- Type the code yourself. The finished project is in the folder `NOVA - C Interpreter` for comparison.
- After each step, run the program and compare your screen with the checkpoint.
- Where the book shows a short excerpt, the full version is in the matching file. This book cannot print all two thousand lines, but it explains every important one.

14.2 Step 1: Install the tools

Goal. Get a computer ready.

Business rule. NOVA does not contain a compiler. It drives tools the user already has. So the machine must have them, and NOVA must *find* them.

On **Windows** the easiest way to get a full C and C++ toolchain is **MSYS2** (msys2.org). Install it, open the *MSYS2 MINGW64* terminal, and run:

```
pacman -S --needed mingw-w64-x86_64-toolchain mingw-w64-x86_64-cmake mingw-w64-
x86_64-ninja mingw-w64-x86_64-gdb mingw-w64-x86_64-clang-tools-extra
```

This installs, in one go: g++ and gcc (compilers), CMake, Ninja (fast build tool), gdb (debugger), and clangd plus clang-format (code intelligence and formatting). Then add `C:\msys64\mingw64\bin` to your Windows **PATH** so that every program can find them.

> **KEY WORD** ***PATH***
> **PATH** is a list of folders. When you type `g++`, the system looks through those folders, in order, for a program with that name. If the folder holding g++ is not in PATH you get *"g++ is not recognised"*. NOVA searches PATH the same way (with the `where` command on Windows).

Then install **Node.js** (LTS version) from nodejs.org. It brings `npm`, the package installer.

Open a normal terminal and check everything:

```
node --version
npm --version
g++ --version
cmake --version
gdb --version
clangd --version
```

> **CHECKPOINT** ***Checkpoint 1***
> Every command prints a version number (yours may be newer than ours: Node 24, g++ 15.2, CMake 4.2, gdb 17). If one says "not recognised", fix your PATH and open a **new** terminal (old terminals do not see the change).

> **WHY?** ***Why check from a terminal first?***
> If NOVA cannot find g++, the Run button will fail and it will be hard to tell whether the bug is in NOVA or in your setup. Two minutes of checking now saves an hour of confusion later. Professionals call this *eliminating variables*.

14.3 Step 2: Create the project

Goal. An empty project with the building blocks downloaded.

Make a folder (we used `NOVA - C Interpreter`). Open a terminal inside it and run:

```
npm init -y
npm install electron monaco-editor node-pty @xterm/xterm @xterm/addon-fit
```

`npm init -y` creates `package.json`. `npm install` downloads the libraries into a folder called `node_modules` and records them in `package.json`. Edit `package.json` so it looks like this:

JSON

```
{
  "name": "nova-ide",
  "productName": "NOVA IDE",
  "version": "0.1.0",
  "description": "In-house C/C++ IDE inspired by the JetBrains workflow",
  "main": "main.js",
  "scripts": { "start": "node launch.js" },
  "license": "UNLICENSED",
  "private": true,
  "dependencies": {
    "electron": "^45.3.1",
    "monaco-editor": "^0.52.2",
    "node-pty": "^1.1.0",
    "@xterm/addon-fit": "^0.11.0",
    "@xterm/xterm": "^6.0.0"
  }
}
```

(Your version numbers will differ a little. That is fine.)

Library	Its job in NOVA
`electron`	The window, and a built-in Node.js
`monaco-editor`	The code editor (the same one VS Code uses)
`node-pty`	A pseudo-terminal, for the real terminal panel
`@xterm/xterm` and `@xterm/addon-fit`	Draws the terminal in the window and resizes it

WHY? ***Why "main": "main.js"?***

Electron needs to know which file to run first. The `main` field in `package.json` is that file. It becomes the *main process*. If it is missing or misspelled, Electron shows a default welcome window instead, and you will wonder why your code never runs.

WATCH OUT ***Watch out***

If, after `npm install`, `node_modules/electron/dist/electron.exe` is missing, the download step was skipped. Run `node node_modules/electron/install.js` once to fetch it. We hit this on the first install.

CHECKPOINT ***Checkpoint 2***

You have: `package.json`, `package-lock.json`, and a `node_modules` folder with `electron`, `monaco-editor`, `node-pty` and `@xterm` inside.

14.4 Step 3: The first window

Goal. A window opens. Nothing else yet.

Business rule. The window should look modern and professional from the first frame: dark, no flash of white, with the native minimise/maximise/close buttons kept.

Create `main.js` (this is a trimmed version of NOVA's real start):

JAVASCRIPT

```
const { app, BrowserWindow } = require('electron');
const path = require('path');

let win;

function createWindow() {
  win = new BrowserWindow({
    width: 1500, height: 920, minWidth: 900, minHeight: 600,
    backgroundColor: '#1e1f22',
    frame: false,
    titleBarStyle: 'hidden',
    titleBarOverlay: { color: '#2b2d30', symbolColor: '#dfe1e5', height: 40 },
    title: 'NOVA IDE',
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false
    }
  });
  win.setMenuBarVisibility(false);
  win.loadFile(path.join(__dirname, 'renderer', 'index.html'));
}
```

```
app.whenReady().then(createWindow);
app.on('window-all-closed', () => app.quit());
```

Create `preload.js` (empty bridge for now):

JAVASCRIPT

```
const { contextBridge } = require('electron');
contextBridge.exposeInMainWorld('nova', { ping: () => 'pong' });
```

Create `renderer/index.html` with a title and one line of text, then create `launch.js`:

JAVASCRIPT

```
// Starts NOVA IDE. Clears ELECTRON_RUN_AS_NODE, which would make Electron
behave like plain Node.
const { spawn } = require('child_process');
const electron = require('electron');
const env = { ...process.env }; delete env.ELECTRON_RUN_AS_NODE;
spawn(electron, [__dirname, ...process.argv.slice(2)], { env, stdio: 'inherit',
detached: true }).unref();
```

Run it:

```
npm start
```

WHY? ***Why a separate launch.js?***

Some tools (including editor extensions) leave `ELECTRON_RUN_AS_NODE=1` in the environment. Electron then behaves like plain Node, `require('electron')` returns nothing useful, and the program dies with an unreadable error. `launch.js` removes that variable and starts Electron cleanly. A small file that spares every future user a long headache. (`Launch NOVA.bat` does the same for people who prefer double-clicking.)

WHAT IF IT WERE DIFFERENT? ***What if you start Electron with the variable set?***

You see `TypeError: Cannot read properties of undefined (reading 'whenReady')` at the line `app.whenReady()`. `app` is undefined because `require('electron')` returned a path string instead of the API. Seeing this exact error tells you to look at the environment.

CHECKPOINT ***Checkpoint 3***

A dark window appears, 1500 by 920 pixels, with a **40-pixel dark strip at the top** that has the standard window buttons at the right. Your text from `index.html` is below. Closing the window ends the program.

14.5 Step 4: The skeleton and the skin

Goal. The layout of an IDE appears: toolbar, side strip, project panel, editor area, bottom panel, status bar. No behaviour yet.

Business rules.

1. The editor must always get **all** the leftover space.
2. The bottom panel and the sidebar can be hidden and resized.
3. The look must support a dark and a light theme.
4. Everything must be reachable by keyboard.

The HTML

NOVA's `renderer/index.html` has these major blocks (shortened):

```HTML
<div id="app">
  <header id="titlebar">…logo, project chip, run/debug/build/stop buttons…
</header>

  <div id="workspace">
    <nav class="stripe" id="stripeLeft">…icon buttons…</nav>
    <aside id="sidebar">
      <div class="panel-head"><span id="sidebarTitle">Project</span>…</div>
      <div id="tree" class="scroll"></div>
      <div id="structure" class="scroll" hidden></div>
    </aside>
    <div class="splitter v" id="splitSide"></div>
    <main id="center">
      <div id="tabs"></div>
      <div id="crumbs"></div>
      <div id="editorWrap"><div id="editor"></div><div id="welcome">…</div>
</div>
    </main>
  </div>

  <div class="splitter h" id="splitBottom" hidden></div>
  <section id="bottom" hidden>…Run, Debug, Problems, Find, Terminal panes…
</section>

  <footer id="status">…line:column, toolchain, language…</footer>
</div>
```

Compare it with the anatomy figure from Chapter 2: header (1), stripe (2), sidebar (3), tabs (4), editor (5), bottom (6), footer (7).

The CSS grid

CSS

```
#app { display: grid; grid-template-rows: 40px 1fr auto auto 24px; height:
100vh; }
#workspace { display: grid; grid-template-columns: 40px var(--side-w, 280px) 4px
1fr; min-height: 0; }
```

and the colour variables that give both themes:

CSS

```
:root {
  --bg: #1e1f22;  --bg2: #2b2d30;  --text: #dfe1e5;  --accent: #3574f0;  /* …and
more… */
}
:root[data-theme="light"] {
  --bg: #ffffff;  --bg2: #f7f8fa;  --text: #1e1f22;
}
[hidden] { display: none !important; }
```

> **WHY?** ***Why is the editor column 1fr and everything else fixed?***
> Because when the user resizes the window, the *editor* should absorb the change and the toolbars should keep their size. If you wrote percentages everywhere, the stripe would grow wide on a big monitor. `1fr` states the rule exactly.

> **WHY?** ***Why a "splitter" element and not a CSS resize handle?***
> Each splitter is a 4-pixel strip. Dragging it runs a few lines of JavaScript that change `--side-w` (or `--bottom-h`). The grid recalculates by itself. It is a tiny example of *CSS and JavaScript cooperating through a variable*.

The icons

`renderer/icons.js` defines each icon as one SVG path string and a helper:

JAVASCRIPT

```
window.ICONS = { play: 'M4.5 3l8 5-8 5z', stop: 'M4 4h8v8H4z', search: 'M7 12a5
5 0 100-10 5 5 0 000 10zM14 14l-3.4-3.4', /* … */ };
window.icon = (n) => `<svg viewBox="0 0 16 16" fill="none" stroke="currentColor"
stroke-width="1.35"
  stroke-linecap="round" stroke-linejoin="round"><path d="${ICONS[n] || ''}"/>
</svg>`;
window.hydrateIcons = (root = document) =>
  root.querySelectorAll('i[data-i]').forEach(el => { if (!el.firstChild)
el.innerHTML = icon(el.dataset.i); });
```

In the HTML you write `<i data-i="play"></i>`, and `hydrateIcons()` fills it with the SVG at start-up.

WHY? ***Why generate icons from a table?***

One short list holds all thirty icons, so the whole HTML stays tiny and readable, and changing the style of every icon (thicker lines, say) is a one-line edit.

CHECKPOINT ***Checkpoint 4***

Your window shows the full IDE frame: the toolbar with buttons, the stripe with icons, a "No project open" area, a welcome card in the middle and a status bar at the bottom. The buttons do nothing yet. Clicking anywhere should not cause errors in the DevTools console (press **F12** if you added it, or run Electron with DevTools).



Figure 14.2. *What the finished welcome screen looks like. Your Step 4 version will have the same layout but no recent projects and no working buttons yet.*

14.6 Troubleshooting Steps 1 to 4

Symptom	Cause	Fix
White window, no layout	CSS path wrong in `index.html`	Check `<link href="styles.css">` and the file name
"Cannot read properties of undefined (reading 'whenReady')"	`ELECTRON_RUN_AS_NODE` is set	Start with `launch.js` / the `.bat`

Symptom	Cause	Fix
Window opens but no buttons or icons	`icons.js` not loaded or `hydrateIcons` not called	Check the `<script>` tags and call `hydrateIcons()` after load
Bottom bar hangs off the screen	Missing `min-height: 0` on grid children	Add it to `#workspace` and the panels
Panel stays visible with `hidden`	Missing `[hidden] {display:none!important}`	Add that line

14.7 Mini assignments

MINI ASSIGNMENT ***Assignment 14.1: Verify your toolchain***

Write a tiny Node script `doctor.js` that checks for `g++`, `gcc`, `cmake`, `ninja`, `gdb`, `clangd` and `git` using `child_process.spawnSync(where, [name])` and prints a tick or cross for each. (This is almost what NOVA's `tools:detect` does.)

MINI ASSIGNMENT ***Assignment 14.2: Your own colours***

Change `--accent` to a different colour and `--bg` to a deep blue. Relaunch. Make a third theme named `midnight` selected by `data-theme="midnight"`.

MINI ASSIGNMENT ***Assignment 14.3: A custom title***

Change the window title, the logo SVG colours and the product name. Which of these changes need a restart and which need only a reload (Ctrl+R in DevTools)?

14.8 Quiz

QUIZ

1. Why must the machine have g++ installed before NOVA works?
2. What is PATH?
3. What is the job of the `main` field in `package.json`?
4. Why does `launch.js` delete `ELECTRON_RUN_AS_NODE`?
5. Why use `backgroundColor` on the window?
6. What does `1fr` do in the layout grid?
7. Why are icons stored as SVG path strings?
8. What do `contextIsolation: true` and `nodeIntegration: false` protect?

ANSWERS

1. NOVA does not contain a compiler; it starts the one on the machine.
2. A list of folders the system searches for programs.
3. It tells Electron which file to run first (the main process).
4. With that variable set Electron acts like plain Node and `require('electron')` returns nothing useful.
5. It is shown for the instant before the page paints, avoiding a white flash on a dark app.
6. It gives that row/column all the remaining free space.
7. They stay sharp at any size, take the text colour, and a single table keeps the HTML small.
8. They stop web content in the window from using Node's powers, such as reading and deleting files.

CHAPTER 15

Build Guide, Part 2: Editor, Files, Build and Problems

15.1 Step 5: Bring in the code editor

Goal. A real code editor appears with colours, line numbers and a minimap.

Business rules.

1. Writing code must feel as good as in VS Code: instant response, smart brackets, undo that works.
2. Dark and light themes must both look good.
3. The editor must use the user's font and size settings.

Loading Monaco

Monaco is delivered as an old-style module system called **AMD**, started by a file named `loader.js`. In `index.html` you add:

HTML

```
<script src="../node_modules/monaco-editor/min/vs/loader.js"></script>
```

Then, in JavaScript (this is NOVA's real `boot` function, trimmed):

JAVASCRIPT

```
async function boot() {
  hydrateIcons();
  S.settings = { ...DEFAULTS, ...(await nova.loadSettings()) };
  applyTheme();
  S.tools = await nova.detectTools();
  updateToolchain();

  const b = new URL('../node_modules/monaco-editor/min/', location.href).href;
  window.MonacoEnvironment = {
    getWorkerUrl: () => 'data:text/javascript;charset=utf-8,' +
encodeURIComponent(
      `self.MonacoEnvironment=
{baseUrl:'${b}'};importScripts('${b}vs/base/worker/workerMain.js');`)
  };
  require.config({ paths: { vs: b + 'vs' } });
  require(['vs/editor/editor.main'], () => { initMonaco(); wireUI();
renderRecent(); renderWelcomeState(); });
}
```

Walk through it:

- `{ ...DEFAULTS, ...(await nova.loadSettings()) }` merges two objects. Saved settings win; anything missing falls back to the defaults. (The spread operator from Chapter 8.)
- `new URL('../node_modules/...', location.href).href` builds the **full address** of the Monaco folder, relative to where `index.html` lives.
- `MonacoEnvironment.getWorkerUrl` tells Monaco where its **workers** are. A **web worker** is a background thread. Monaco uses workers so that heavy jobs (like checking JSON) never freeze typing.
- `require(['vs/editor/editor.main'], callback)` loads Monaco, then runs our `callback` when it is ready.

> **WHY?** ***Why the strange data: URL for the worker?***
> Workers must normally be loaded from a normal web address, but our page is loaded from a **file** (`file:///...`), where browsers refuse to start a worker straight from another file. The trick: create a tiny script *as text* inside a `data:` URL that simply says "import the real worker file". The browser accepts that. It is a known workaround and the single most puzzling line in the whole project. Without it the editor still appears, but silently loses features and the console fills with worker errors.

> **WHAT IF IT WERE DIFFERENT?** ***What if initMonaco ran before Monaco finished loading?***
> `monaco` would be `undefined` and the first line that used it would crash with *"monaco is not defined"*. That is why everything that needs Monaco is inside the callback. This is the same lesson as the `async/await` section: **wait for the thing you depend on**.

Creating the editor

JAVASCRIPT

```
S.editor = monaco.editor.create($('#editor'), {
  model: null, theme: 'nova-dark', automaticLayout: true, glyphMargin: true,
  fontSize: S.settings.fontSize, fontFamily: S.settings.fontFamily,
fontLigatures: S.settings.ligatures,
  minimap: { enabled: S.settings.minimap, renderCharacters: false },
  smoothScrolling: true, cursorSmoothCaretAnimation: 'on', cursorBlinking:
'smooth',
  bracketPairColorization: { enabled: true }, guides: { bracketPairs: true },
  renderLineHighlight: 'all', padding: { top: 8 }, scrollBeyondLastLine: false,
  mouseWheelZoom: true, fixedOverflowWidgets: true, tabSize: 4, insertSpaces:
true,
  stickyScroll: { enabled: true }
});
```

Key options explained:

Option	Why it is there
`model: null`	Start with no file; a *model* is the text of one file
`automaticLayout: true`	The editor resizes itself when its container changes (the splitters!)
`glyphMargin: true`	Leaves a column for breakpoint dots and the debugger arrow
`bracketPairColorization`	Matching brackets share a colour; nesting mistakes are visible
`stickyScroll`	The current function's name stays pinned at the top while scrolling
`fixedOverflowWidgets`	Popups (completion, hover) can escape the editor box instead of being clipped

Our own colours

A **theme** maps *token names* to colours. Monaco's C++ grammar already labels each word (`keyword`, `string`, `number`, `comment`...). NOVA only decides the colours:

JAVASCRIPT

```
monaco.editor.defineTheme('nova-dark', {
  base: 'vs-dark', inherit: true,
  colors: { 'editor.background': '#1e1f22', 'editor.foreground': '#dfe1e5', /* …
*/ },
  rules: [
    { token: 'comment', foreground: '7a7e85', fontStyle: 'italic' },
    { token: 'keyword', foreground: 'cf8e6d' },
    { token: 'string',  foreground: '6aab73' },
    { token: 'number',  foreground: '2aacb8' },
    { token: 'metatag', foreground: 'b3ae60' }   // #include lines
  ]
});
```

> **WHY?** ***Why "inherit: true"?***
>
> It means "start from Monaco's ready-made dark theme and change only what I list". Without it, every colour you forgot would turn black and the editor would be unreadable. Always inherit and override.

Snippets

A **snippet** is a template that expands when you pick it from the completion list. NOVA registers them for C and C++:

JAVASCRIPT

```
const snip = (label, body, doc) => ({ label, insertText: body, kind:
monaco.languages.CompletionItemKind.Snippet,
  insertTextRules:
monaco.languages.CompletionItemInsertTextRule.InsertAsSnippet, documentation:
doc });
snip('for', 'for (int ${1:i} = 0; ${1:i} < ${2:n}; ++${1:i}) {\n\t$0\n}', 'for
loop')
```

`${1:i}` is a **tab stop**: your cursor lands there first, you type a new name, and every other `${1:i}` changes with it. `$0` is where the cursor ends.

> **CHECKPOINT** ***Checkpoint 5***
> Open the DevTools console and run `S.editor.setModel(monaco.editor.createModel('int main() { return 0; }', 'cpp'))`. A coloured C++ line appears with line numbers, a minimap and bracket colours. Press Ctrl+Space after typing `for` and choose the `for` snippet. The cursor should jump between tab stops.

15.2 Step 6: The file tree, tabs and saving

Goal. Open a folder, see its files, click to open them in tabs, edit, and save.

What the main process offers

The renderer cannot touch the disk, so `main.js` offers a small set of **handlers** (you studied the pattern in Chapter 9):

Channel	Does
`fs:readDir`	List a folder (folders first, sorted)
`fs:read`	Read a text file (binary files are refused)
`fs:write`	Save text
`fs:create`	Make a file or folder
`fs:rename`, `fs:delete`	Rename or delete
`dialog:openFolder`	The system folder picker
`fs:listAll`	Every file under a folder (used by Search Everywhere)

> **WHY?** ***Why such a short, boring list?***
> Because it is the **entire power** the window has over your disk. If something goes wrong in the window, the damage is limited to those operations. Short lists are easy to audit. (This is called the *principle of least privilege*.)

The tree, loaded lazily

The tree must feel instant even for a project with 20,000 files. NOVA only reads a folder **when you open it**:

JAVASCRIPT

```
const setOpen = async (open) => {
  row.classList.toggle('open', open); kids.hidden = !open;
  if (open) S.expanded.add(ent.path); else S.expanded.delete(ent.path);
  if (open && !loaded) {
    loaded = true;
    try { for (const c of await nova.readDir(ent.path)) kids.appendChild(await
nodeEl(c, depth + 1)); } catch { }
  }
};
```

- `loaded` makes sure each folder is read **once**.
- `S.expanded` is a `Set` remembering which folders are open, so after a refresh the tree reopens the same folders.
- `nodeEl` calls itself for each child: a **recursive** function that draws the whole tree.

> **WHAT IF IT WERE DIFFERENT?** ***What if the tree read every folder at start-up?***
> Opening a project with `node_modules` (often tens of thousands of files) would take many seconds and use hundreds of megabytes. Lazy loading plus the ignore list in `main.js` (`.git`, `node_modules`, `.idea`...) keeps it instant.

Tabs and models

When you click a file, NOVA does the steps in this figure:

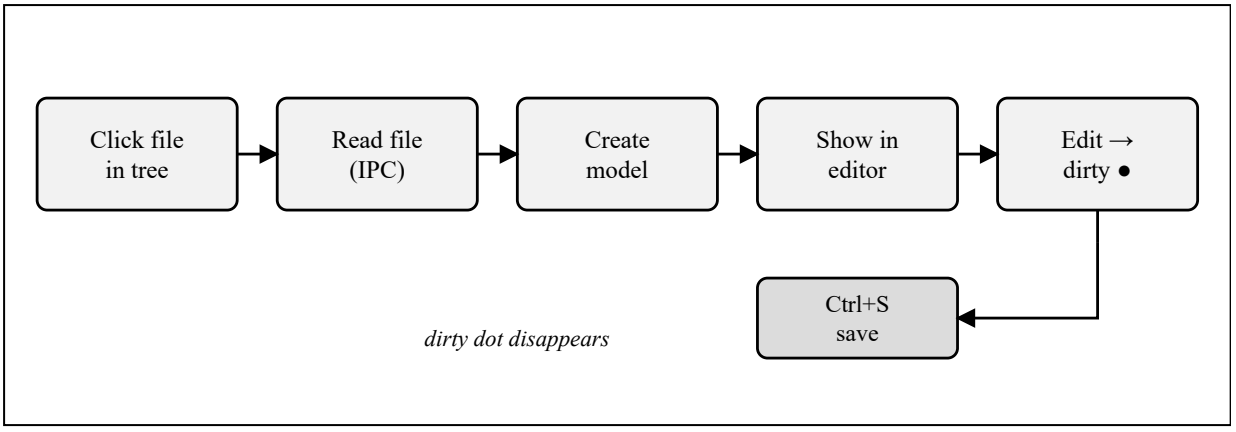


Figure 15.1. *From a click in the tree to a dirty dot on the tab.*

JAVASCRIPT

```
async function openFile(path, pos) {
  let tab = S.tabs.find(t => key(t.path) === key(path));
  if (!tab) {
    let r; try { r = await nova.read(path); } catch (e) { return toast('Cannot
open: ' + e.message, 'error'); }
    if (r.binary) return toast('Binary file – not opened in the editor.');
    const model = monaco.editor.createModel(r.text, langOf(path),
monaco.Uri.file(path));
    tab = { path, name: base(path), model, dirty: false, saved:
model.getAlternativeVersionId(), view: null };
    model.onDidChangeContent(() => {
      const d = model.getAlternativeVersionId() !== tab.saved;
      if (d !== tab.dirty) { tab.dirty = d; renderTabs(); }
    });
    S.tabs.push(tab);
  }
  activate(tab);
}
```

The business rules are visible in the code: *opening a file that is already open just switches to it*; *binary files are refused politely*; *a broken file shows a message instead of crashing*.

> **WHY?** ***Why getAlternativeVersionId and not a simple "changed" flag?***
>
> Every edit gives the model a new version number, and **undo** goes back to an *old* number. By storing the version at save time and comparing, NOVA is correct even when you type a letter and then undo it: the numbers match again and the dirty dot disappears. A simple flag would stay "dirty" forever after the first keystroke. It is the same trick editors like VS Code use.

Remembering where you were

JAVASCRIPT

```
function activate(tab) {
  if (S.active && S.active !== tab) S.active.view = S.editor.saveViewState();
  S.active = tab; S.editor.setModel(tab.model);
  if (tab.view) S.editor.restoreViewState(tab.view);
  renderTabs(); renderCrumbs(); renderWelcomeState(); /* … */
}
```

`saveViewState` stores the scroll position, cursor and selection of the tab you are leaving; `restoreViewState` puts them back. When you switch tabs you return to *exactly* where you left.

Saving

JAVASCRIPT

```
async function saveTab(tab) {
  await nova.write(tab.path, tab.model.getValue());
  tab.saved = tab.model.getAlternativeVersionId(); tab.dirty = false;
renderTabs();
  if (base(tab.path) === 'CMakeLists.txt') { S.cmakeDirty = true; scanTargets();
}
}
```

Notice the **side effect rule**: saving a `CMakeLists.txt` marks the CMake project "dirty" so the next build reconfigures, and rescans for new targets. NOVA also **auto-saves** all changed files when the window loses focus and before every build, so you never compile stale code.

> **WHY?** ***Why auto-save before build?***
> A classic beginner trap: you edit, press Run, and see the *old* behaviour because you forgot to save. By saving automatically before building, the rule "what you see is what is built" is always true. (A setting turns it off for people who dislike it.)

Context menu, rename, delete

Right-click a node and `showMenu` builds a small menu from a list of `{ label, run }` objects:

JAVASCRIPT

```
showMenu(e.clientX, e.clientY, [
  { label: 'New File…', run: () => newEntry(parent, false) },
  { label: 'Rename…',   key: 'F2', run: () => renameEntry(ent) },
  { label: 'Reveal in Explorer', run: () => nova.reveal(ent.path) },
  '-',
  { label: 'Delete', danger: true, run: () => deleteEntry(ent) }
]);
```

Deleting first asks for confirmation (`choiceModal`), because it cannot be undone. Renaming also renames any open tab so the file stays linked.

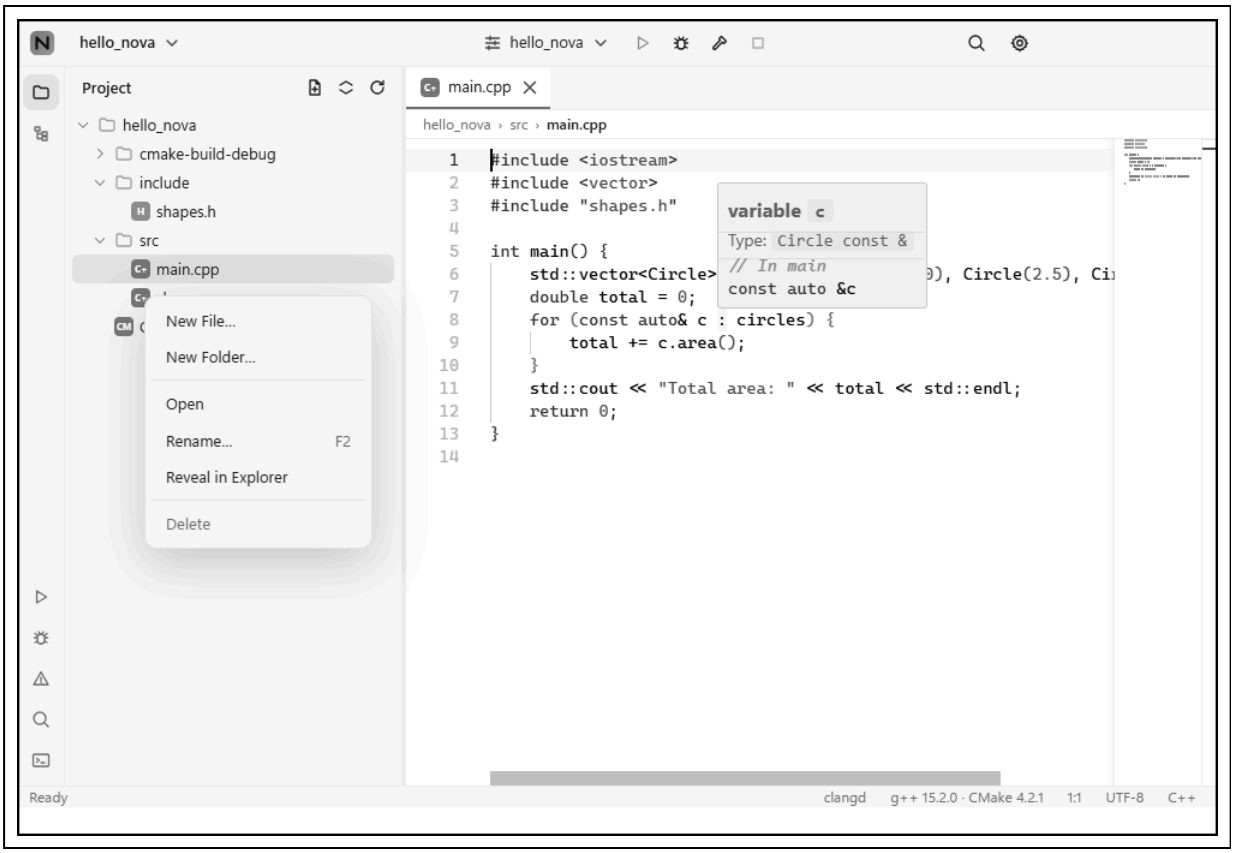


Figure 15.2. *The right-click menu on a file in the tree.*

> **CHECKPOINT** ***Checkpoint 6***
>
> Click *Open Folder* and choose a project. The tree shows folders first, then files with coloured icons. Click a file: it opens in a tab. Type a letter: a dot appears on the tab. Press **Ctrl+S**: the dot disappears. Undo your typing before saving: the dot vanishes. Right-click a file: the menu appears. Close the tab with the middle mouse button.

15.3 Step 7: Build and run

Goal. One click compiles the code and runs the program; its output appears in a panel; a Stop button works.

Business rules.

1. Pressing Run when nothing is ready must say *why*, not fail silently.
2. If the build fails, **do not run** the old program.
3. The user may press Stop at any time.
4. Output must appear live, not all at once at the end.
5. The user can type input to a running program.

Finding the tools

`main.js` asks the system where each tool is:

JAVASCRIPT

```
function which(cmd) {
  const r = spawnSync(process.platform === 'win32' ? 'where' : 'which', [cmd], {
encoding: 'utf8' });
  return r.status === 0 ? r.stdout.split(/\r?\n/)[0].trim() : null;
}
ipcMain.handle('tools:detect', () => {
  const tools = {};
  for (const t of ['g++', 'gcc', 'clang++', 'clang', 'cmake', 'ninja', 'make',
'mingw32-make', 'gdb', 'lldb', 'clang-format', 'clangd', 'git']) {
    const p = which(t); if (p) tools[t] = { path: p, version: ver(t) };
  }
  return tools;
});
```

The window gets an object such as `{ 'g++': {path, version}, cmake: {...} }` and shows "g++ 15.2.0 · CMake 4.2.1" in the status bar. Later code asks `if (!S.tools.cmake)` and shows a clear message.

Starting a process and streaming its output

In the main process (compare with the Chapter 9 runner):

JAVASCRIPT

```
function startProc(cmd, args, cwd, opts = {}) {
  const id = ++procSeq;
  const child = spawn(cmd, args, { cwd, shell: !!opts.shell, windowsHide: true,
                                   env: { ...process.env, ...(opts.env || {}) }
});
  procs.set(id, child);
  const send = (stream, data) => win && !win.isDestroyed() &&
        win.webContents.send('proc:data', { id, stream, data: data.toString()
});
  child.stdout.on('data', d => send('out', d));
  child.stderr.on('data', d => send('err', d));
  child.on('error', e => { send('err', `Failed to start: ${e.message}\n`);
                           win.webContents.send('proc:exit', { id, code: -1 });
procs.delete(id); });
  child.on('close', code => { win.webContents.send('proc:exit', { id, code });
procs.delete(id); });
  return { id, child };
}
```

Every process gets an **id** and lives in a `Map`, so many processes (a build, a program, git) can run at once and the window can tell their output apart.

Stopping is more careful than you might expect:

JAVASCRIPT

```
ipcMain.handle('proc:kill', (_e, id) => {
  const p = procs.get(id); if (!p) return;
  if (process.platform === 'win32') spawnSync('taskkill', ['/pid',
String(p.pid), '/T', '/F']);
  else p.kill('SIGKILL');
});
```

WHY? ***Why taskkill with /T?***

On Windows `child.kill()` ends only the *one* process you started. A build tool starts a compiler which starts other programs; killing only the top one leaves orphans running invisibly. `taskkill /T` ends the whole **tree**, and `/F` forces it. Without this, pressing Stop would leave a program running in the background, still holding your files open.

The renderer's side: a race condition and its cure

The window starts a process, gets an `id`, and *then* registers a listener for that id. But output could arrive in the tiny gap between. NOVA solves it with a holding area:

JAVASCRIPT

```
const sinks = {}, early = {};
nova.onData(m => { const s = sinks[m.id]; if (s) s.data(m); else (early[m.id] =
early[m.id] || []).push({ t: 'd', m }); });
nova.onExit(m => { const s = sinks[m.id]; if (s) s.exit(m.code); else
(early[m.id] = early[m.id] || []).push({ t: 'x', m }); });

async function spawnProc(cmd, args, cwd, sink, opts) {
  const id = await nova.start(cmd, args, cwd, opts);
  const wrapped = { data: sink.data, exit: (code) => { delete sinks[id];
sink.exit(code); } };
  sinks[id] = wrapped;
  const pend = early[id] || []; delete early[id];
  for (const ev of pend) ev.t === 'd' ? wrapped.data(ev.m) :
wrapped.exit(ev.m.code);
  return id;
}
```

WHAT IF IT WERE DIFFERENT? ***What if early output were just dropped?***

A program that prints and exits instantly (like "Hello, NOVA!") could finish *before* the window registered its listener. Its output and exit code would vanish, and the Run panel would stay empty. This kind of bug appears only sometimes, which makes it extremely hard to find. A **race condition** is two things racing to happen first. The `early` buffer removes the race.

The pipeline: build, check, run

JAVASCRIPT

```
async function runPipeline(steps, label) {
  if (S.running) { await stopRun(); }
  $('#runOut').innerHTML = ''; S.problems = []; forceBottom('run');
setBusy(true, label);
  const t0 = performance.now(); let code = 0;
  for (const st of steps) {
    code = await execStep(st, { parseProblems: !st.run });
    if (code !== 0) break;                  // stop at the first failure
  }
  const secs = ((performance.now() - t0) / 1000).toFixed(1);
  out('#runOut', `\nProcess finished with exit code ${code}  (${secs}s)\n`, code
=== 0 ? 'ok' : 'err');
  setBusy(false, code === 0 ? 'Ready' : `Failed (exit ${code})`);
  renderProblems(); applyMarkers();
  return code;
}
```

`steps` is a list such as `[ buildStep, runStep ]`. The loop is the heart of the business rule *"if the build fails, do not run"*: the `break` on a non-zero exit code means the run step never starts.

Single file: g++ directly

JAVASCRIPT

```
async function planFile() {
  const tab = S.active;
  const cc = compilerFor(tab.path);
  const outDir = join(S.root || dirname(tab.path), '.nova', 'out');
  const name = base(tab.path).replace(/\.[^.]+$/, ''), exe = join(outDir, name +
(IS_WIN ? '.exe' : ''));
  const std = ext(tab.path) === 'c' ? '-std=' + S.settings.cStd : '-std=' +
S.settings.cppStd;
  return {
    build: { cmd: cc, args: ['-g', '-O0', std, '-Wall', '-Wextra', ...(await
unbufFlag()), tab.path, '-o', exe], cwd: dirname(tab.path) },
    run:   { cmd: exe, args: [], cwd: dirname(tab.path), run: true }
  };
}
```

The flags mean: `-g` keep debug information, `-O0` do not optimise (so stepping follows your lines), `-std=c++17` the language version, `-Wall -Wextra` helpful warnings.

CMake projects

```javascript
function cmakeSteps(target) {
  const dir = join(S.root, 'cmake-build-debug'), steps = [];
  if (S.cmakeDirty || !S.cmakeCfgOk) {
    const a = ['-S', S.root, '-B', dir, '-DCMAKE_BUILD_TYPE=Debug', '-
DCMAKE_EXPORT_COMPILE_COMMANDS=ON'];
    if (generator()) a.push('-G', generator());
    if (S.tools.gcc) a.push('-DCMAKE_C_COMPILER=gcc');
    if (S.tools['g++']) a.push('-DCMAKE_CXX_COMPILER=g++');
    steps.push({ cmd: 'cmake', args: a, cwd: S.root, cfg: true });
  }
  const b = ['--build', dir]; if (target) b.push('--target', target);
  b.push('-j', String(navigator.hardwareConcurrency || 4));
  steps.push({ cmd: 'cmake', args: b, cwd: S.root });
  return { dir, steps };
}
```

- **Configure only when needed.** If nothing in `CMakeLists.txt` changed, skip straight to the build; that makes the second Run much faster.
- The compilers are named explicitly so CMake does not pick a different one (such as Visual Studio's) that you did not mean.
- `-j N` builds N files in parallel, using all your processor cores.

To fill the drop-down of run targets, `scanTargets` reads every `CMakeLists.txt` and pulls out the names with a regular expression:

```javascript
for (const m of txt.matchAll(/add_executable\s*\(\s*([^\s)]+)/gi)) { /* collect
m[1] */ }
```

Programs that read input

The Run panel has a text box at the bottom. When you press Enter, its text plus a newline is written to the program's **stdin**:

```javascript
$('#runIn').onkeydown = (e) => {
  if (e.key === 'Enter' && S.running) {
    const v = e.target.value; nova.stdin(S.running, v + '\n');
    out('#runOut', v + '\n', 'cmd'); e.target.value = '';
  }
};
```

And the quiet fix for hidden prompts (Chapter 13): NOVA writes `nova_unbuf.h` and adds `-include` of it to every build, so `printf("Enter: ")` shows up before the program waits for you.

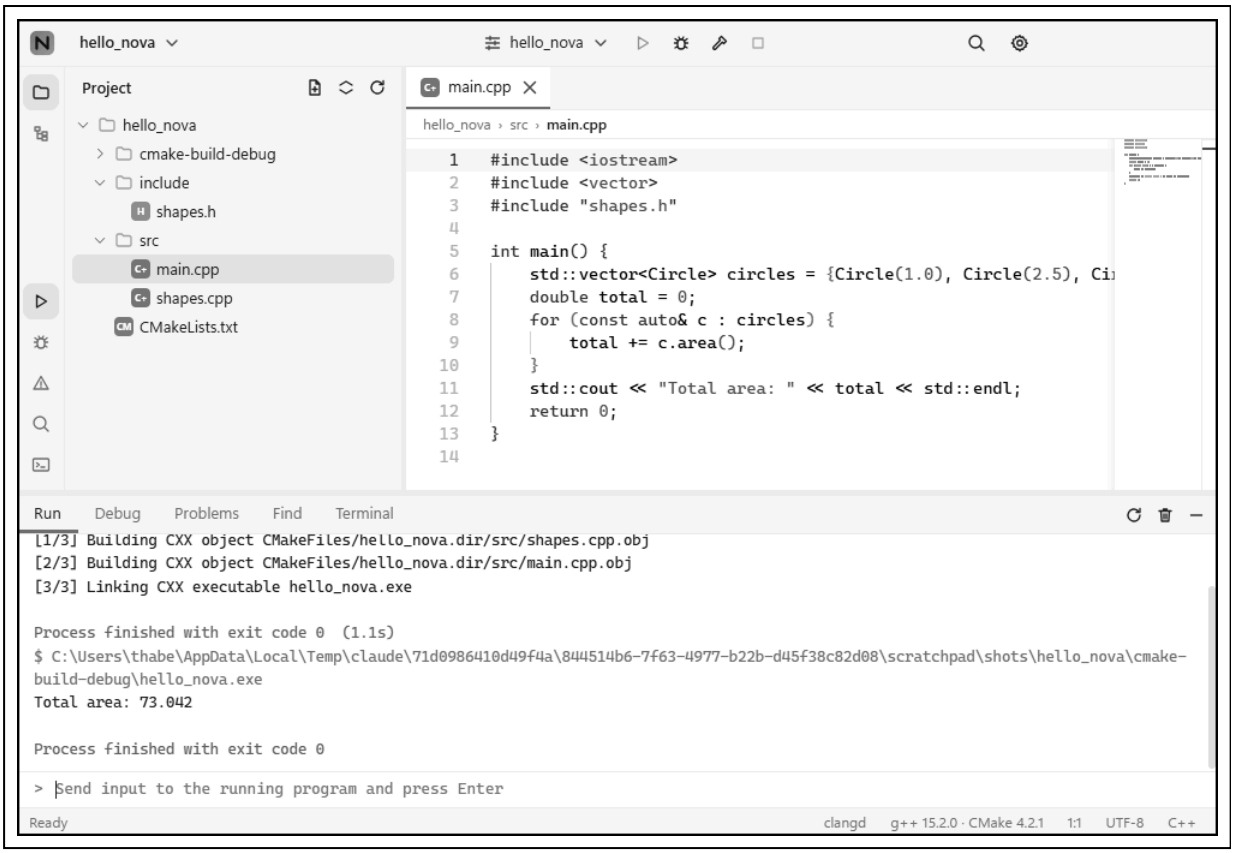


Figure 15.3. *CHECKPOINT 7: a CMake project built and run. The Run panel shows the two build steps, the exit code with timing, the program's output "Total area: 73.062", and the input box at the bottom.*

> **CHECKPOINT** ***Checkpoint 7***
>
> Open (or create) a project with a `CMakeLists.txt` and a `main.cpp` that prints something. The toolbar drop-down shows your target name. Press the green button or **Shift+F10**. The bottom panel opens and shows the `cmake` commands, `[1/2] Building CXX object ...`, `[2/2] Linking CXX executable ...`, your program's output and *"Process finished with exit code 0"*. Press it again: the configure step is skipped and it is faster. Add `while(true){}` to your program and press the red Stop button: it ends.

15.4 Step 8: Problems, squiggles and links

Goal. Compiler errors become wavy underlines, a list in the Problems panel, and clickable links.

Business rules.

1. Show errors where the user's eyes already are: **on the code**.
2. Clicking an error jumps to its place.
3. Errors from different sources (compiler, language server) appear together, **without duplicates**.

4. A clean build clears the list.

Parsing the compiler's text

You met the pattern in Chapter 8. Here it is in context:

JAVASCRIPT

```
const DIAG_RE = /^((?:[A-Za-z]:)?[^:\n]+?):(\d+):(?:(\d+):)? (fatal
error|error|warning|note): (.*)$/;

function parseDiag(line, cwd) {
  const m = line.match(DIAG_RE); if (!m) return;
  S.problems.push({ file: resolveFile(m[1], cwd), line: +m[2], col: +(m[3] ||
1),
                    sev: m[4] === 'note' ? 'note' : m[4] === 'warning' ?
'warning' : 'error', msg: m[5] });
}
```

`resolveFile` turns the compiler's (often relative) path into a full one, because g++ prints paths relative to where it ran.

Squiggles

JAVASCRIPT

```
function applyMarkers() {
  for (const t of S.tabs) {
    const ms = S.problems.filter(p => key(p.file) === key(t.path)).map(p => {
      const w = t.model.getWordAtPosition({ lineNumber: Math.min(p.line,
t.model.getLineCount()), column: p.col });
      const ln = Math.min(p.line, t.model.getLineCount());
      return { startLineNumber: ln, startColumn: p.col, endLineNumber: ln,
endColumn: w ? w.endColumn : p.col + 1,
               message: p.msg, severity: p.sev === 'error' ?
monaco.MarkerSeverity.Error : /* … */ monaco.MarkerSeverity.Warning,
               source: 'compiler' };
    });
    monaco.editor.setModelMarkers(t.model, 'compiler', ms);
  }
}
```

A **marker** is Monaco's word for "draw a squiggle here with this message". We take the compiler's line and column, find the **word** at that spot (so the squiggle covers `cnt`, not just one letter), and hand the list to Monaco under the owner name `'compiler'`.

> **WHY?** ***Why an "owner" name like 'compiler'?***
> Markers are grouped by owner. Later, clangd's diagnostics use the owner `'clangd'`. Setting one owner's markers never erases the other's. If both used the same owner, each update would wipe the other's squiggles.

The Problems panel, merged and de-duplicated

JAVASCRIPT

```
const seen = new Set(), all = [];
for (const p of S.problems.concat(lspProblems())) {
  const k = key(p.file) + ':' + p.line + ':' + p.msg;
  if (!seen.has(k)) { seen.add(k); all.push(p); }
}
```

A `Set` remembers each *file:line:message* already shown. Identical messages from the compiler and clangd appear once. The list is then **grouped by file** and sorted with errors first.

> **WHAT IF IT WERE DIFFERENT?** ***What if grouping used the raw file string?***
> This bug really happened. The compiler wrote the path as `src\bad.cpp` while clangd wrote `c:/.../src/bad.cpp`: the same file with different spellings. The panel split one file into two groups. The fix: group by `key(p.file)`, which lowercases and unifies slashes on Windows. The lesson: **normalise before you compare**.

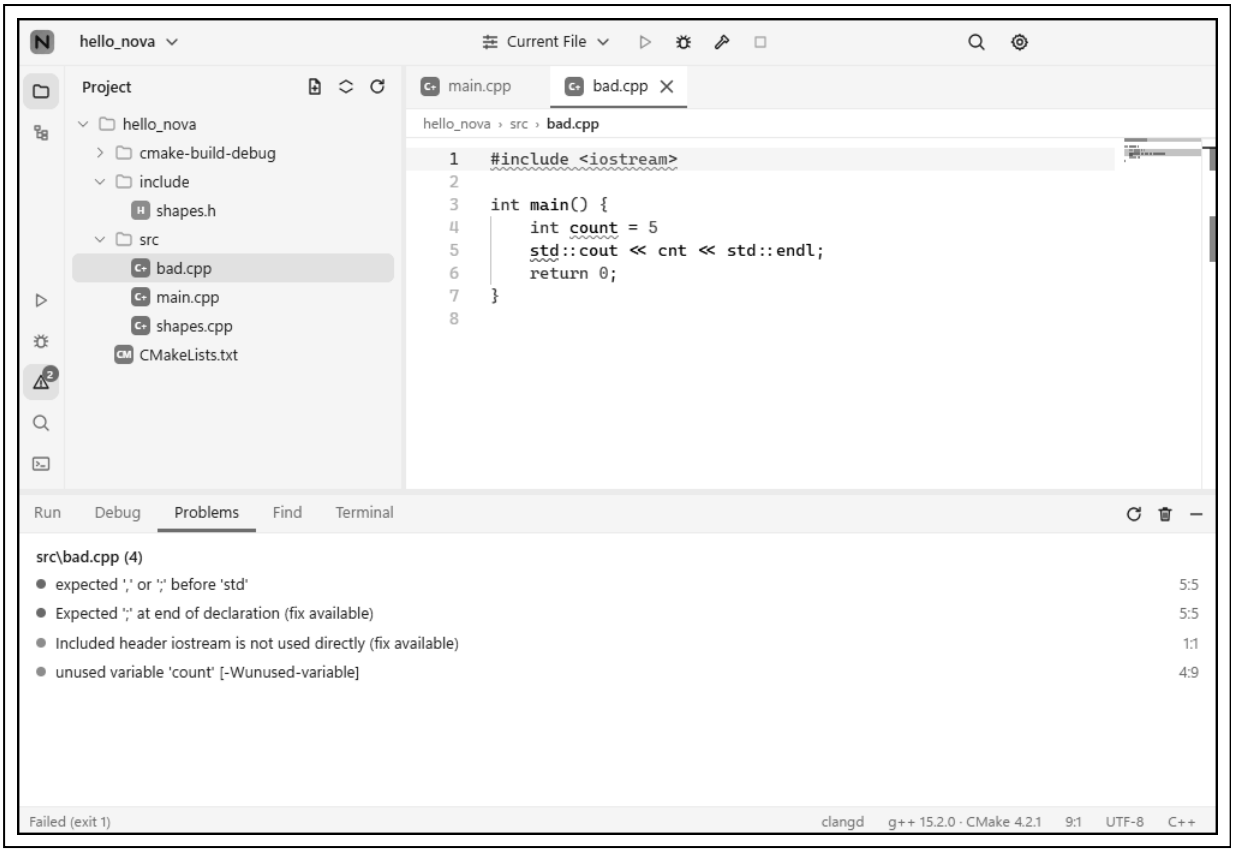


Figure 15.4. *CHECKPOINT 8: errors in action. Wavy red lines under the faulty code, a red badge on the Problems icon, a yellow warning on line 4, and four entries in the panel. Clicking one jumps to it.*

CHECKPOINT ***Checkpoint 8***

Create `bad.cpp` with a missing semicolon and a misspelled variable. Press Run. The build fails, the program does **not** start, red wavy lines appear under the mistakes, a red badge shows on the Problems icon, and the list shows each message with its line and column. Click one: the cursor jumps there. Fix the file and run again: the squiggles and the list clear.

15.5 Mini assignments

MINI ASSIGNMENT ***Assignment 15.1: Ignore list***

Add `.vscode` and `build` to the ignore list in `main.js`. Reload and confirm those folders no longer appear in the tree.

MINI ASSIGNMENT ***Assignment 15.2: Status counts***

Show "Ln 12, Col 5, 3 lines selected" in the status bar by listening to `S.editor.onDidChangeCursorSelection`.

MINI ASSIGNMENT ***Assignment 15.3: Build timer***

Make the status bar show the build time ("Built in 2.4s") after each successful build, and keep it until the next action.

MINI ASSIGNMENT ***Assignment 15.4: A clean-build button***

Add *Clean* to the Actions list. It should delete the `cmake-build-debug` folder (with confirmation) and mark the project dirty so the next Run reconfigures.

15.6 Quiz

QUIZ

1. What is a Monaco *model*?
2. Why does NOVA compare version ids instead of using a "changed" flag?
3. Why is the file tree loaded lazily?
4. Why does Stop use `taskkill /T`?
5. What is a race condition? Which one did `spawnProc` have to solve?
6. Why does `runPipeline` stop at the first non-zero exit code?
7. Why does NOVA skip the CMake configure step on the second Run?
8. Why do problem markers use an owner name?

9. Why must paths be normalised before being compared?

ANSWERS

1. The text (and its undo history) of one file.
2. Undoing back to the saved text should clear the dirty state; version ids make that automatic.
3. Reading every folder at start-up would be slow for big projects; reading on demand keeps the UI instant.
4. Windows only kills the one process by default; `/T` kills the whole tree of child processes so nothing is left running.
5. Two things racing to happen first. Output or the exit event could arrive before the window registered its listener; the `early` buffer replays them.
6. Because a failed build must not run the old or half-built program.
7. Nothing in CMakeLists.txt changed, so the configuration is still valid; this makes builds faster.
8. So that different sources (compiler, clangd) can set and clear their own markers without wiping each other's.
9. The same file can be spelled differently (slashes, upper/lower case); without normalising, equal paths look different.

CHAPTER 16

Build Guide, Part 3: Search, Terminal, Intelligence and Debugger

16.1 Step 9: Search, Search Everywhere and the command palette

Goal. Find text across the whole project, jump to any file or symbol by typing a few letters, and run any action by name.

Business rules.

1. Finding a file must take about **two seconds** no matter how big the project is.
2. Typing "shp" should find `shapes.cpp`: the letters must match **in order, not necessarily next to each other** (this is called *fuzzy matching*).
3. Results that match at the start of a word, or in a row, should rank higher.
4. The shortcut must be fast to reach: **double-tap Shift**, like JetBrains.

Find in Files (done by the main process)

The window sends the query; the main process walks the folders and reads the files; it sends back a list of matches. Doing the heavy work in the main process means the window never freezes.

JAVASCRIPT

```
ipcMain.handle('fs:grep', (_e, root, query, opts = {}) => {
  const files = []; walk(root, files);
  const results = [];
  const src = opts.regex ? query : query.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
  const re = new RegExp(opts.word ? `\\b${src}\\b` : src, opts.case ? '' : 'i');
  for (const f of files) {
    const buf = fs.readFileSync(f); if (buf.includes(0)) continue;      // skip
binaries
    buf.toString('utf8').split(/\r?\n/).forEach((ln, i) => {
      if (re.test(ln) && results.length < 2000) results.push({ file: f, line: i
+ 1, text: ln.trim().slice(0, 300) });
    });
  }
  return results;
});
```

Three protective details, each a business rule in disguise:

- The long `replace` **escapes** special characters, so searching for `a.b` or `(x)` means the literal text, not a pattern. (Unless the user ticks the regex box.)
- Files over 2 MB and binary files are skipped, so one giant log cannot freeze the search.
- Results stop at 2000. Nobody reads more, and a million-item list would crash the display.

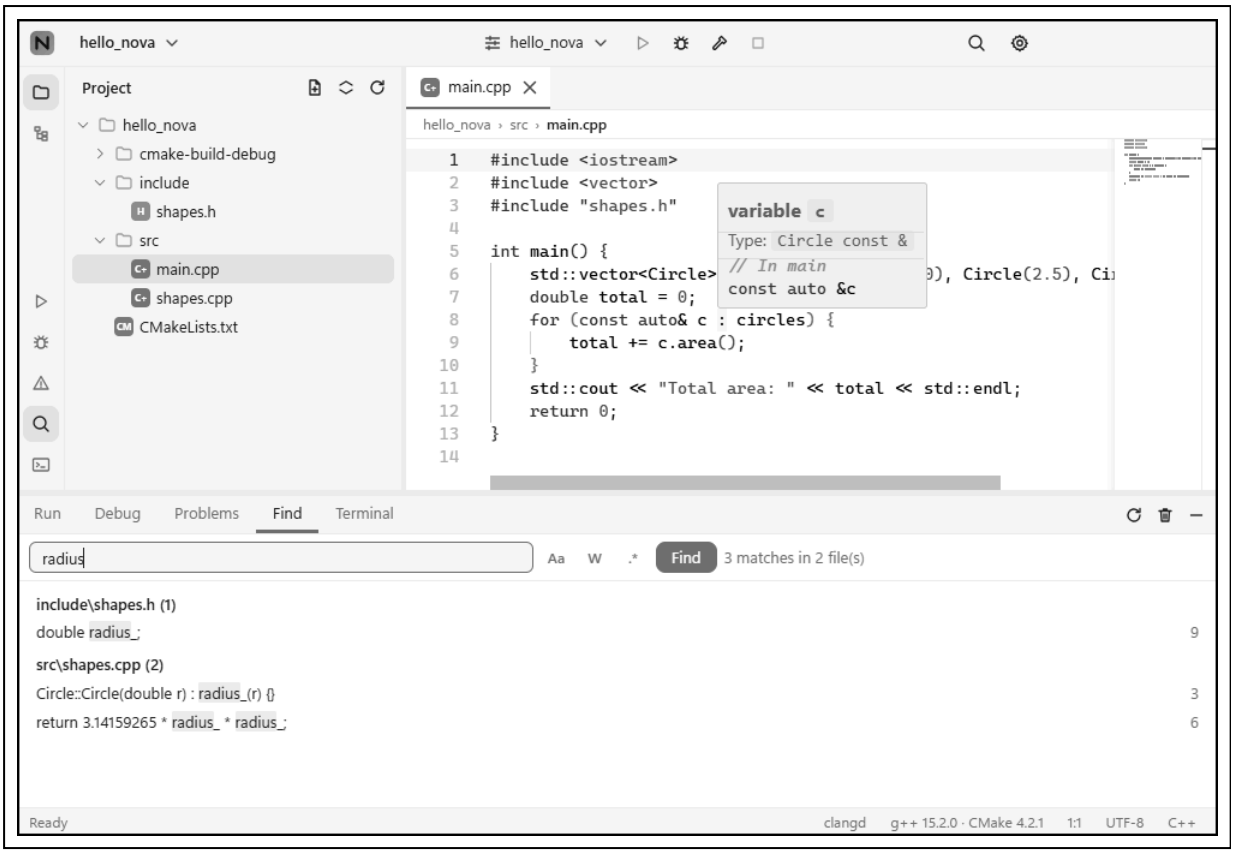


Figure 16.1. *CHECKPOINT 9a: Find in Files for "radius". Results are grouped by file with counts; each line is clickable.*

Fuzzy matching

This is the real scoring function from NOVA's `app.js`. You can run it.

JAVASCRIPT

```
function fuzzy(q, s) {
  q = q.toLowerCase(); const t = s.toLowerCase(); let i = 0, score = 0, last =
-2;
  for (let j = 0; j < t.length && i < q.length; j++) {
    if (t[j] === q[i]) {
      score += (j === last + 1 ? 5 : 1) + (j === 0 || /[\/\\._\- ]/.test(t[j -
1]) ? 4 : 0);
      last = j; i++;
    }
  }
  return i === q.length ? score - t.length * 0.01 : -1;
}

const files = ["shapes.cpp", "shapes.h", "CMakeLists.txt", "main.cpp",
"sh_utils.cpp", "dispatch.cpp"];
for (const q of ["sha", "shp", "mc", "cpp", "zzz"]) {
  const ranked = files.map(f => [f, fuzzy(q, f)]).filter(x => x[1] >=
0).sort((a, b) => b[1] - a[1]).map(x => x[0]);
  console.log(q.padEnd(4), "->", ranked.join(", ") || "(nothing)");
}
```

OUTPUT

```
sha  -> shapes.h, shapes.cpp
shp  -> shapes.h, shapes.cpp, sh_utils.cpp, dispatch.cpp
mc   -> main.cpp
cpp  -> main.cpp, shapes.cpp, sh_utils.cpp, dispatch.cpp
zzz  -> (nothing)
```

How it scores:

1. Walk through the name; every time the next wanted letter is found, give a point.
2. **+5 bonus** if it directly follows the previous matched letter (a run like "sha").
3. **+4 bonus** if it begins the name or follows a separator (`/ \ . _ - space`): word starts matter.
4. A tiny **penalty for length**, so shorter names rank above longer ones with the same match.
5. If not all the letters were found, return `-1`: no match.

> **WHY?** ***Why such a simple formula?***
>
> Because it runs on **thousands** of names on every keystroke and must feel instant. Real IDEs use cleverer scoring, but this 8-line version captures what users expect: starts of words and unbroken runs win. Simple, fast and good enough is often the right engineering answer.

Double Shift

JAVASCRIPT

```
let lastShift = 0, shiftClean = false;
addEventListener('keydown', (e) => { if (e.key === 'Shift') { if (!e.repeat)
shiftClean = true; } else shiftClean = false; }, true);
addEventListener('keyup', (e) => {
  if (e.key === 'Shift' && shiftClean) {
    const n = Date.now();
    if (n - lastShift < 380) { lastShift = 0; openPalette('all'); } else
lastShift = n;
  }
}, true);
```

`shiftClean` is true only if Shift was pressed **alone** (no other key in between), so typing a capital letter with Shift never triggers it. Two clean releases within 380 milliseconds open the palette.

> **WHAT IF IT WERE DIFFERENT?** ***What if there were no shiftClean?***
> Every time you typed "Hello" (Shift+H), the release of Shift would count as a tap, and typing two capital letters fast would pop up the search window in the middle of your sentence. The extra flag prevents that annoyance.

One list for files, symbols and actions

Search Everywhere merges three sources: **files** (from `fs:listAll`, cached), **symbols** (from the Structure panel) and **actions** (a plain array of `{ name, key, run }`). Pressing Tab cycles *All, Files, Symbols, Actions*. Pressing Enter runs the highlighted item. The action list is how you reach everything without a mouse, for example *Restart Language Server (clangd)*.

> **CHECKPOINT** ***Checkpoint 9***
> Press **Shift twice**. A window appears. Type `sha`: `shapes.cpp` and `shapes.h` are first. Press Enter: the file opens. Press **Ctrl+Shift+F**, type a word, press Enter: matches are listed by file and clicking one jumps to it. Press **Ctrl+Shift+A** and type `term`: the terminal action appears.

16.2 Step 10: A real terminal

Goal. A genuine command line inside NOVA.

Business rules.

1. Everything must behave as in a normal terminal: colours, history, Ctrl+C, programs that ask questions.
2. It starts in the **project folder**.

3. When the shell exits, the user can restart it with a keypress.
4. It matches the IDE theme.

The shell behind the glass

In the main process:

JAVASCRIPT

```
ipcMain.handle('pty:spawn', (_e, cols, rows, cwd) => {
  const shell = process.platform === 'win32' ? 'powershell.exe' :
(process.env.SHELL || 'bash');
  const env = { ...process.env }; delete env.ELECTRON_RUN_AS_NODE;
  const id = ++ptySeq;
  const t = nodePty.spawn(shell, ['-NoLogo'], { name: 'xterm-256color', cols:
cols || 80, rows: rows || 24, cwd: cwd || os.homedir(), env });
  ptys.set(id, t);
  t.onData(d => win.webContents.send('pty:data', { id, data: d }));
  t.onExit(() => { ptys.delete(id); win.webContents.send('pty:exit', { id });
});
  return { id };
});
```

`node-pty` starts the shell attached to a **pseudo-terminal**, so the shell believes a real person sits at a real terminal.

The face: xterm.js

In the window, `terminal.js` creates the drawing surface and wires the two ends together:

JAVASCRIPT

```
T.term = new Terminal({ fontFamily: S.settings.fontFamily, fontSize:
Math.max(11, S.settings.fontSize - 1),
                        cursorBlink: true, theme: termTheme(), scrollback: 5000
});
T.fit = new FitAddon.FitAddon(); T.term.loadAddon(T.fit); T.term.open(host);
T.fit.fit();
T.term.onData(d => T.id && nova.ptyWrite(T.id, d));          // keys you press
-> shell
T.term.onResize(({ cols, rows }) => T.id && nova.ptyResize(T.id, cols, rows));
nova.onPtyData(({ id, data }) => { if (id === T.id && T.term)
T.term.write(data); });   // shell output -> screen
```

Three pipes in a loop: **keyboard → xterm → IPC → PTY → shell**, and back **shell → PTY → IPC → xterm → screen**.

> **WHY?** ***Why the "fit" add-on and the resize message?***
> The shell must know how many columns and rows the window has, so that long lines wrap in the right place and programs such as `nano` or `top` draw correctly. When you drag the splitter, the add-on measures the new size and NOVA tells the PTY. Without it, a resized terminal would wrap text in the wrong place and full-screen programs would look garbled.

WHAT IF IT WERE DIFFERENT? ***What if the terminal started lazily or not at all?***
NOVA starts the shell the **first time** you open the Terminal tab (`termEnsure`). If it started at launch, every startup would pay for a PowerShell process nobody may use. Lazy start makes the IDE open faster.

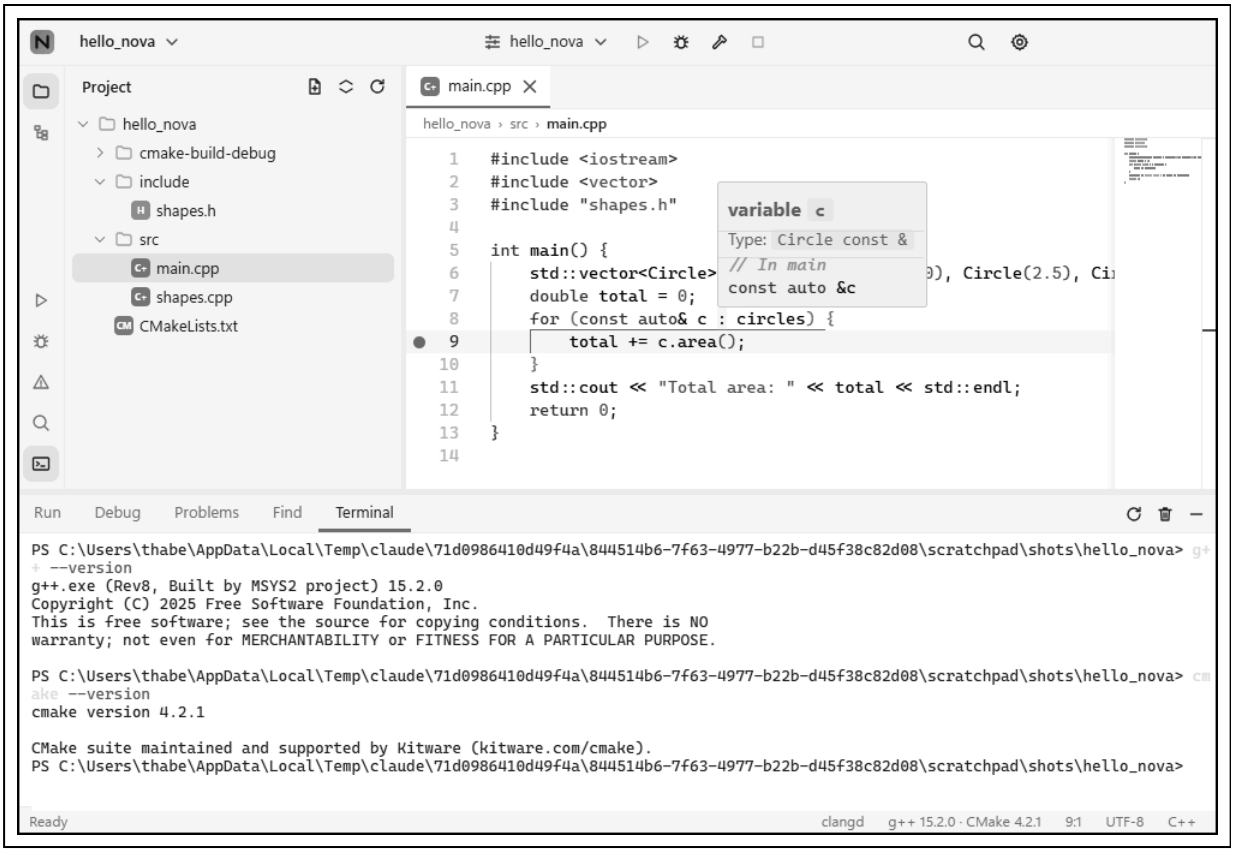


Figure 16.2. *CHECKPOINT 10: the integrated terminal running two commands. This is a real PowerShell, with its own prompt, started in the project folder.*

CHECKPOINT ***Checkpoint 10***
Press **Alt+F12**. A prompt appears in your project folder. Type `g++ --version` and `dir`. Type `exit`: the message "process exited, press any key to restart" appears; press a key and a fresh shell starts. Drag the splitter above the panel: the text re-wraps. Switch the theme in Settings: the terminal changes colours.

16.3 Step 11: Code intelligence with clangd

Goal. Completion, hover documentation, go to definition, rename, quick fixes and live error underlines, from the real C++ compiler front-end.

Business rules.

1. clangd must understand the project **exactly** as the real compiler does.

2. The editor must never freeze waiting for it.
3. If clangd is missing or crashes, the editor still works.
4. The server and editor must always agree on the text of every open file.

Giving clangd the facts

JAVASCRIPT

```
async function lspPrepareCompileDb() {
  if (S.isCmake) {
    const dir = join(S.root, 'cmake-build-debug');
    if (await nova.exists(join(dir, 'compile_commands.json'))) return dir;
    if (S.tools.cmake) {                        // silent configure so clangd
gets real flags
      setLspStatus('configuring CMake…');
      /* … runs the cmake configure step …*/
      if (await nova.exists(join(dir, 'compile_commands.json'))) return dir;
    }
  }
  /* fallback: write our own compile_commands.json for loose files into .nova/
*/
}
```

Order of preference: (1) CMake's real database; (2) run CMake quietly to make one; (3) invent one for loose files. In every case clangd gets *some* answer.

Starting the server

JAVASCRIPT

```
ipcMain.handle('lsp:start', (_e, root, compileDir) => {
  const exe = which('clangd'); if (!exe) return { error: 'clangd not found' };
  const args = ['--background-index', '--clang-tidy=false', '--header-
insertion=never',
                '--completion-style=detailed', '--query-
driver=**/g++*,**/gcc*,**/clang*', '--log=error'];
  if (compileDir) args.push('--compile-commands-dir=' + compileDir);
  const child = spawn(exe, args, { cwd: root, windowsHide: true });
  /* …read framed messages from child.stdout and forward them to the window… */
});
```

Option	Why
`--background-index`	Index the whole project quietly so "find references" and "go to definition" work across files
`--header-insertion=never`	Do not edit the user's `#include` lines behind their back
`--completion-style=detailed`	Show full signatures in the list
`--query-driver=...`	Ask **your** g++ where its standard headers are, so `<vector>` is found
`--compile-commands-dir`	Where to find `compile_commands.json`

> **WHY?** ***Why --query-driver?***
> clangd is built on LLVM but your project uses MinGW's g++ and its own copy of the standard library. Without this option clangd cannot find `<iostream>` and underlines every standard include in red. It is a perfect example of *small flags with huge effects*.

Talking in requests

The JSON-RPC client is tiny:

JAVASCRIPT

```
function lspRequest(method, params, timeout = 8000) {
  if (!LSP.on) return Promise.resolve(null);
  return new Promise((res) => {
    const id = ++LSP.id;
    const t = setTimeout(() => { LSP.pending.delete(id); res(null); }, timeout);
    LSP.pending.set(id, (r) => { clearTimeout(t); res(r); });
    lspSend({ id, method, params });
  });
}
```

It is the `request`/`pending` code from Chapter 13, with one extra line: if clangd is not running (`LSP.on` false) the answer is instantly `null`, so the editor carries on as normal. That is rule 3 in one line.

Keeping the server in sync

JAVASCRIPT

```
function lspDidChange(tab) {
  if (!LSP.initialized || !isCFamily(tab) || !LSP.versions.has(tab.path))
return;
  clearTimeout(LSP.timers.get(tab.path));
  LSP.timers.set(tab.path, setTimeout(() => lspFlush(tab), 120));       //
debounce
}
```

The 120 ms debounce (Chapter 8) means a burst of typing causes **one** message. And before any request such as completion, NOVA calls `lspFlush(tab)` to make sure the server has the very latest text. Rule 4 in practice.

Plugging into Monaco

Monaco has "provider" hooks. You register a function for each feature; Monaco calls it at the right moment. Here is hover:

JAVASCRIPT

```
monaco.languages.registerHoverProvider(lang, {
  async provideHover(model, pos) {
    const t = live(model); if (!t) return null; sync(t);
    const r = await lspRequest('textDocument/hover', { textDocument: { uri:
lspUri(t.path) }, position: toPos(pos) });
    if (!r || !r.contents) return null;
    const c = r.contents; const arr = Array.isArray(c) ? c : [c];
    return { range: r.range ? fromRange(r.range) : undefined,
             contents: arr.map(x => typeof x === 'string' ? { value: x } : {
value: x.value }) };
  }
});
```

Everything that the user sees comes from that tiny bridge: **Monaco asks → NOVA translates positions → clangd thinks → NOVA translates the reply → Monaco draws.** The same shape repeats for completion, signature help, definition, references, outline, formatting, rename and quick fixes (all in `lsp.js`).

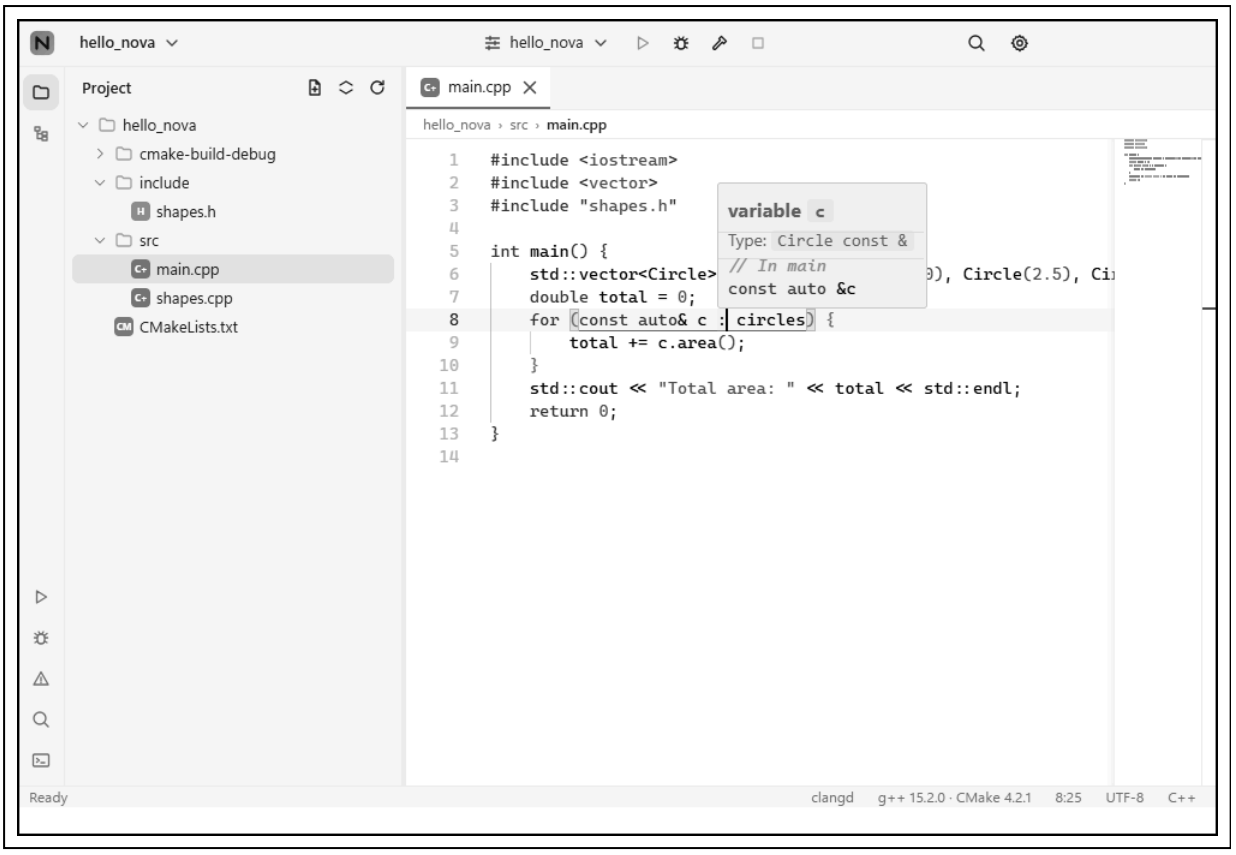


Figure 16.3. *CHECKPOINT 11a: hover. Pointing at a variable shows clangd's description: its kind, its type and where it was declared.*

WHAT IF IT WERE DIFFERENT? ***What if the diagnostics used the compiler only?***

You would see errors only when you press Run. With clangd, a squiggle appears about a second after you stop typing. The compiler is the final judge; clangd is the quick friend that spots slips as you go.

The outline from the server

The Structure panel used to be built from a regular expression. With clangd it asks the server for *document symbols* and indents children under their classes:

JAVASCRIPT

```
async function lspStructure(tab) {
  const r = await lspRequest('textDocument/documentSymbol', { textDocument: {
uri: lspUri(tab.path) } }, 5000);
  if (!r) return null;                          // no answer: fall back to the
regex version
  /* …flatten the tree into rows with a depth… */
}
```

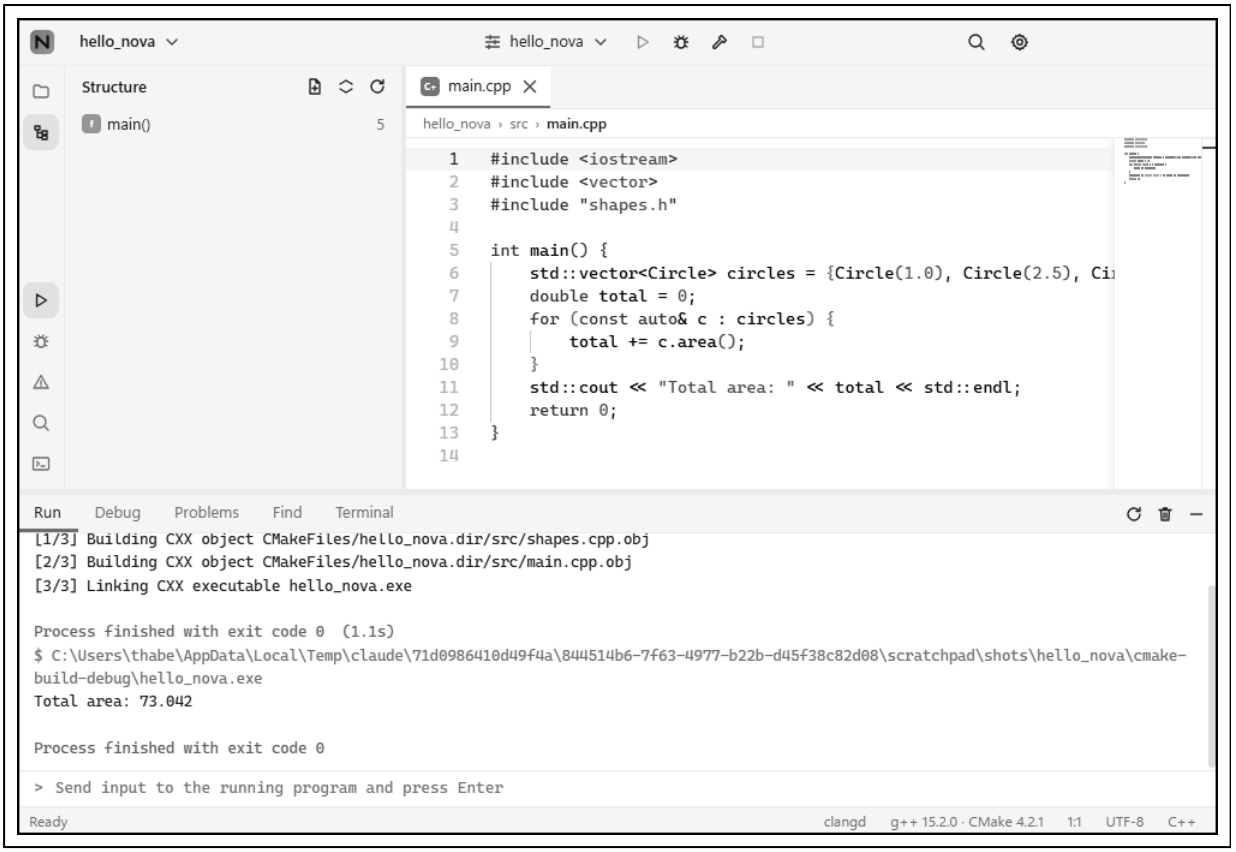


Figure 16.4. *CHECKPOINT 11b: the Structure panel, filled by clangd. Clicking a row jumps to that line.*

CHECKPOINT ***Checkpoint 11***

Open a C++ project. After a few seconds the status bar says **clangd**. Type `std::vec` and pause: a list appears with `vector`. Hover over a function: documentation appears. **Ctrl+click** a function call: you jump to its definition. Press **F2** on a variable: rename it everywhere. Type a mistake: a red squiggle appears with no build. Open *Actions → Restart Language Server* and see it recover.

16.4 Step 12: The debugger

Goal. Stop a running program at breakpoints, look at its variables, and step through it line by line.

Business rules.

1. Breakpoints are set by clicking the margin, and can be added or removed **while** the program runs.
2. The debugger builds first (with `-g`); if the build fails it must not start.
3. When the program stops, the editor must show **exactly where**: highlighted line, arrow, and the right file.
4. Buttons must only be enabled when they make sense.
5. Crashes (like a null pointer) must be reported in plain words.

Starting gdb and talking MI

JAVASCRIPT

```
async function dbgStart(exe, cwd) {
  const r = await nova.gdbStart(exe, cwd, []);
  if (r.error) return toast('Could not start gdb: ' + r.error, 'error');
  Object.assign(D, { on: true, running: false, frames: [], vars: [], sel: 0,
bpNum: new Map(), exe, prevVars: new Map() });
  await dbgCmd('-gdb-set breakpoint pending on');
  await dbgCmd('-gdb-set confirm off');
  await dbgCmd('-gdb-set new-console ' + (S.settings.debugExtConsole === false ?
'off' : 'on'));
  await dbgCmd('-environment-cd ' + miq(fwd(cwd)));
  for (const [k, set] of Object.entries(S.breaks)) { /* … */ for (const line of
set) await dbgInsert(path, line); }
  dbgCmd('-exec-run');
}
```

Each `await dbgCmd(...)` sends one MI command and waits for its numbered answer (the token matching you saw in Chapter 13). The setup commands are rules in disguise:

- `breakpoint pending on`: a breakpoint in code that is not loaded yet is accepted instead of refused.
- `confirm off`: gdb never stops to ask "are you sure?", which would freeze our machine-driven conversation.
- `new-console on`: your program gets its own window, so it can read the keyboard.

Reacting to a stop

JAVASCRIPT

```
async function dbgStopped(d) {
  D.running = false;
  const reason = d.reason || '';
  if (reason.startsWith('exited')) { /* print the exit code and end the session
*/ return; }
  if (reason === 'signal-received' || reason === 'exited-signalled') {
    const msg = `Program received signal ${d['signal-name']} (${d['signal-
meaning']})`;
    dcon('\n' + msg + '\n', 'err'); toast(esc(msg), 'error', 6000);
  }
  D.sel = 0; await dbgRefresh(true);
}
```

`dbgRefresh` asks gdb for the stack (`-stack-list-frames`) and variables (`-stack-list-variables --all-values`), draws the panels, opens the file and highlights the line:

JAVASCRIPT

```
function dbgDecorate() {
  if (D.on && !D.running && D.last && S.active && key(S.active.path) ===
key(D.last.path))
    D.deco.set([{ range: new monaco.Range(D.last.line, 1, D.last.line, 1),
                  options: { isWholeLine: true, className: 'dbg-line',
glyphMarginClassName: 'dbg-arrow' } }]);
  else D.deco.set([]);
}
```

A **decoration** is Monaco's word for "style this range of text". The yellow arrow and blue band are just CSS classes (`dbg-arrow`, `dbg-line`) applied to the current line.

Live breakpoints

JAVASCRIPT

```
function dbgSyncBreak(path, line, added) {
  if (!D.on) return;
  const k = key(path) + ':' + line;
  if (added) dbgInsert(path, line);
  else if (D.bpNum.has(k)) { dbgCmd('-break-delete ' + D.bpNum.get(k));
D.bpNum.delete(k); }
}
```

gdb gives each breakpoint a **number** when it is created; NOVA remembers it in a `Map` keyed by *file:line* so it can delete it later. That is rule 1.

The buttons obey the state machine

JAVASCRIPT

```
function dbgUi() {
  const on = D.on, stopped = on && !D.running;
  $$('#dbgBar [data-d]').forEach(b => { b.disabled = b.dataset.d === 'stop' ?
!on : !stopped; });
  $('#dbgState').textContent = !on ? 'Not debugging' : D.running ? 'Running…' :
'Paused';
}
```

Two booleans, four states (the figure in Chapter 13), and one function that sets every button. Rule 4.

> **WHY?** ***Why does the Variables panel highlight changed values?***
> You press Step and ask yourself "what did that line do?" NOVA remembers the previous values (`D.prevVars`) and draws any that changed in a warm colour. Your eye jumps straight to the effect of the line you just ran. It is a tiny feature that makes debugging noticeably faster.

> **WHAT IF IT WERE DIFFERENT?** ***What if you stepped while the program was running?***
> gdb would reject the command or behave unpredictably. With buttons disabled during *Running*, the user cannot even try. Preventing mistakes in the interface is better than reporting them afterwards.

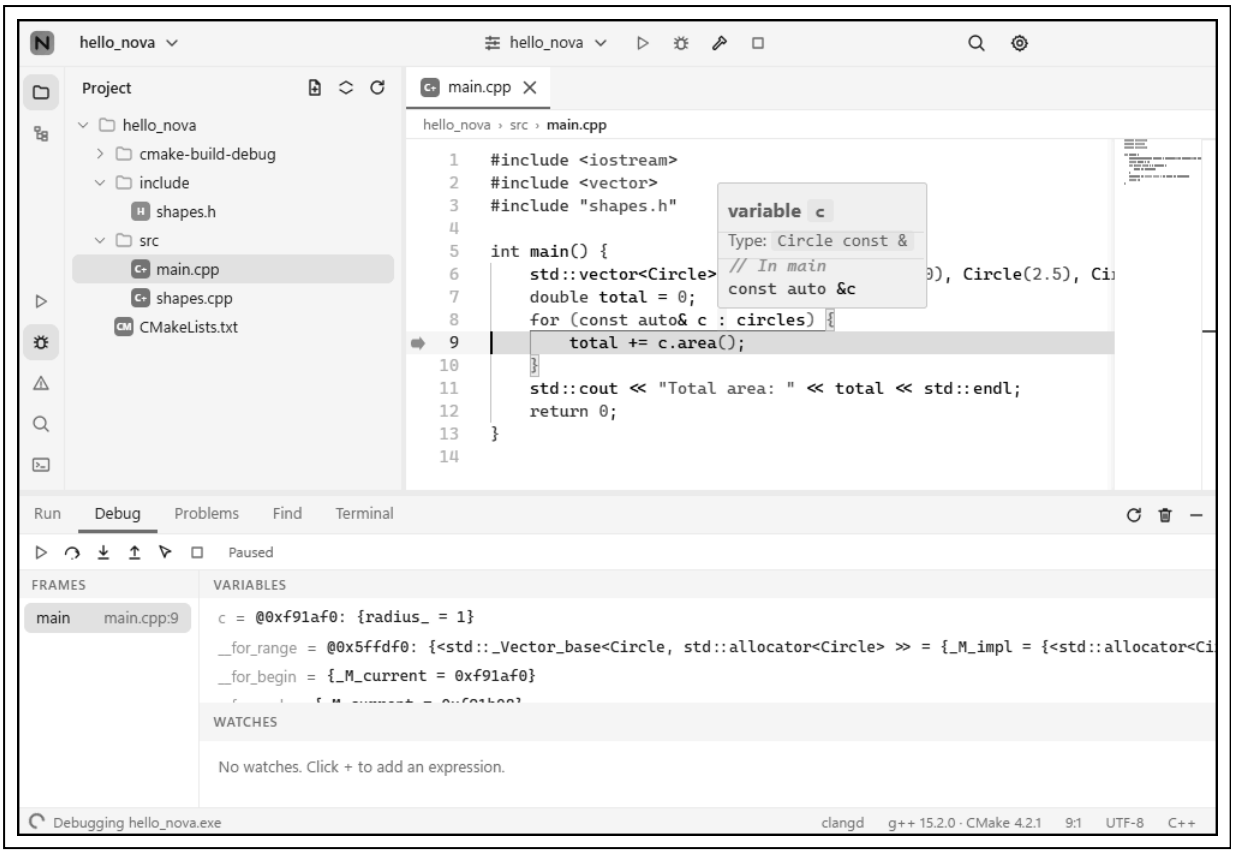


Figure 16.5. *CHECKPOINT 12: the debugger paused on line 9. The arrow and band mark the current line; the red dot would be the breakpoint; Frames, Variables, Watches and the gdb console fill the panel. The floating box is the hover evaluation.*

> **CHECKPOINT** ***Checkpoint 12***
>
> Open a project and click in the margin next to a line inside a loop (a red dot appears). Press **Shift+F9**. After the build, the program stops there: the line is highlighted with an arrow, Variables show the current values, Frames show the call stack. Press **F8** to step over: values that changed turn orange. Press **F9** to resume: it stops at the next hit. Add `v * 10` as a watch. Hover a variable: its value pops up. Press the red Stop button: the session ends cleanly.

16.5 Finishing touches

Settings

The Settings window edits the `S.settings` object and saves it (as JSON) to the user's data folder. Changes apply **live**: toggling the theme calls `applyTheme()`, which sets `data-theme`, updates Monaco's theme and the terminal's colours.

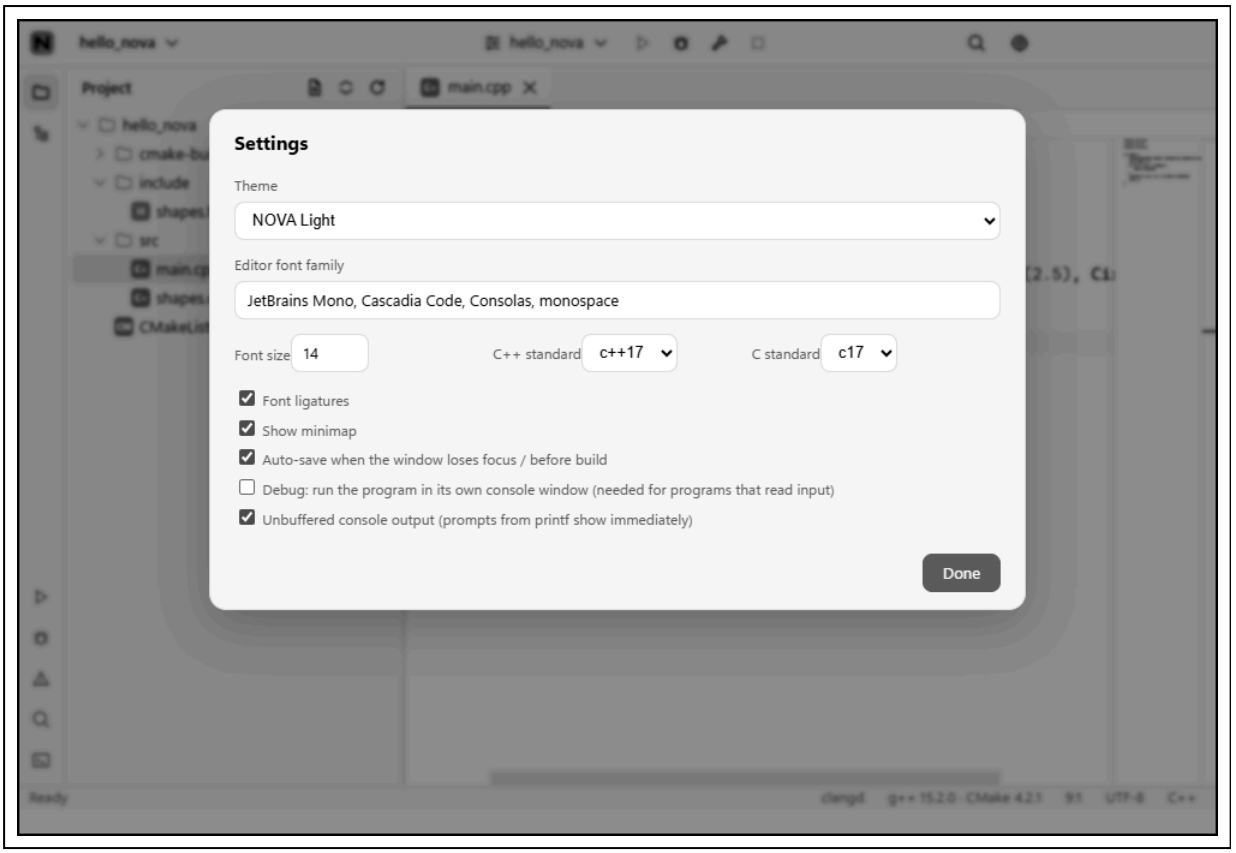


Figure 16.6. *The Settings window. Everything changes the moment you pick it.*

The new project wizard

It checks the name with a regular expression (letters, digits, `_`, `-`), refuses a folder that already exists, then writes `CMakeLists.txt`, `main.cpp` (or `main.c`) and `.gitignore`, and opens the new project.

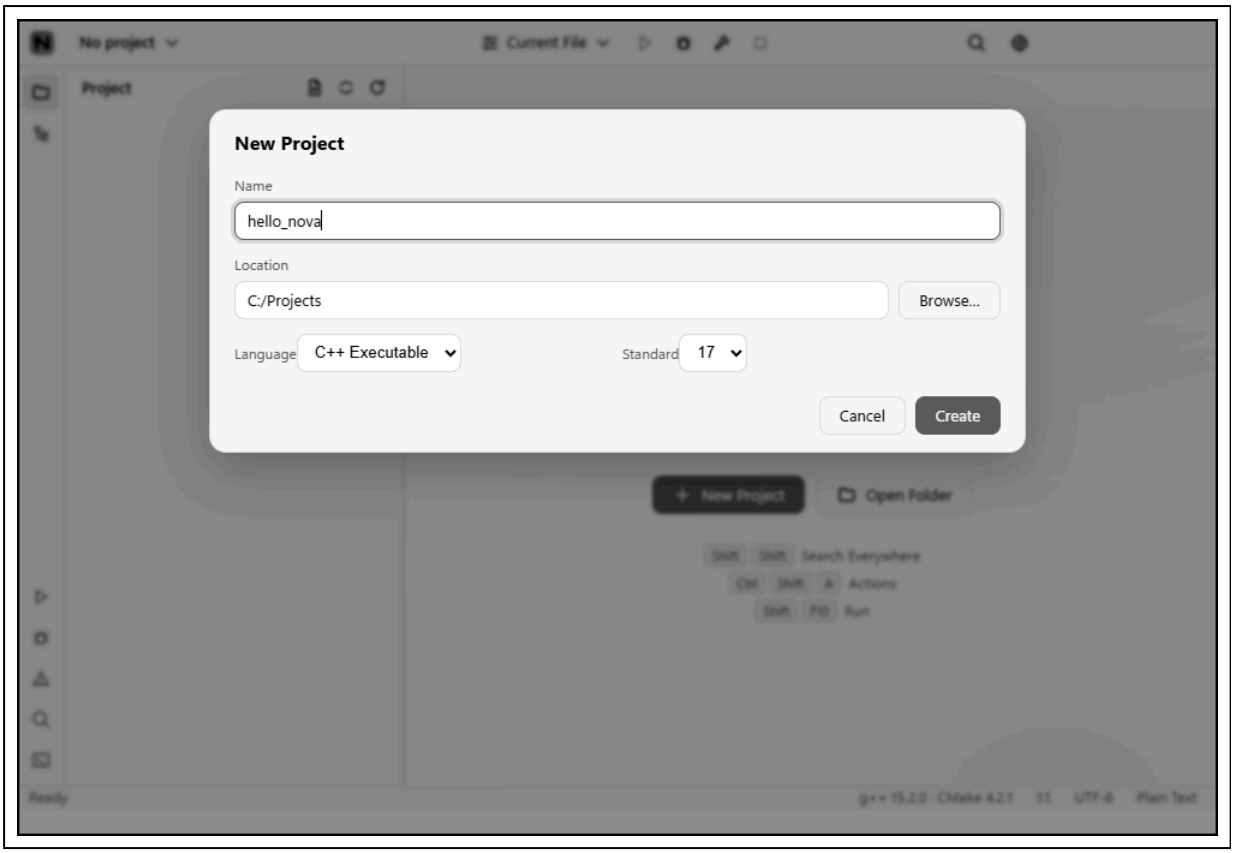


Figure 16.7. *The New Project wizard. Choose C or C++, the standard, and a location.*

The dark theme

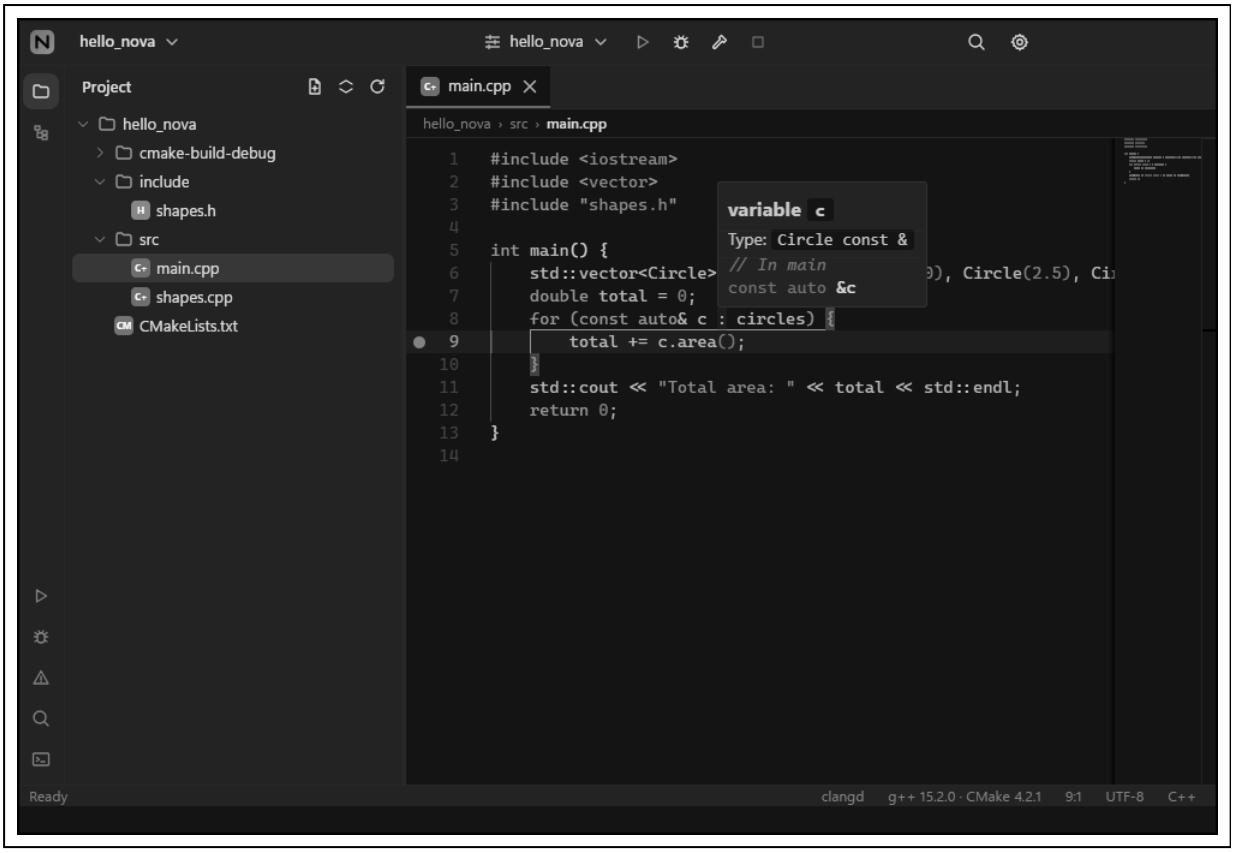


Figure 16.8. *The default Darcula-style dark theme. All the screenshots in this book use the light theme for printing, but this is how NOVA looks by default.*

The test harness

How do you test a window full of buttons without clicking? NOVA has a hidden **self-test mode**. If the environment variable `NOVA_SELFTEST` points to a JavaScript file, NOVA starts, runs that file *inside the window*, saves a screenshot and the console log, and quits:

JAVASCRIPT

```
win.webContents.on('did-finish-load', async () => {
  await new Promise(r => setTimeout(r, 2500));
  const script = fs.readFileSync(process.env.NOVA_SELFTEST, 'utf8');
  const r = await win.webContents.executeJavaScript(script);      // run the
test script
  const img = await win.webContents.capturePage();                // take a
picture
  fs.writeFileSync(process.env.NOVA_SHOT, img.toPNG());
  app.quit();
});
```

A test script can simply call NOVA's own functions: `await openProject(...)`, `await doRun()`, `toggleBreak(...)`, `await startDebugSession()`, and then return data to check. **Every screenshot in this book was captured this way**, and so were the checks that proved the debugger, language server and terminal worked.

> **WHY?** ***Why build testing into the program?***
> UI code is notoriously hard to test, so many teams skip it, and bugs creep in. A 15-line harness that can "press the buttons" and take pictures makes it possible to check the whole pipeline in under a minute after every change. It also produced this book's pictures, which would otherwise have had to be taken by hand.

16.6 The file-by-file reference

Keep this table next to you when you read the source code.

The project root

File or folder	What it is
`package.json`	Name, version, entry point (`main`), start script, libraries
`main.js`	The **main process**: window, disk, processes, PTY, clangd, gdb, settings
`preload.js`	The safe bridge: defines `window.nova` for the window
`launch.js` and `Launch NOVA.bat`	Start Electron without `ELECTRON_RUN_AS_NODE`
`README.md`	Short instructions for the team
`node_modules/`	Downloaded libraries (never edit; safe to delete and reinstall)
`renderer/`	Everything that runs inside the window
`docs/`	This book, its Markdown source, diagram generator and build script

main.js (about 250 lines), in the order they appear

Section	What it does
`createWindow`	The window options, preload, and loading `index.html`
file system handlers	`fs:readDir`, `fs:read`, `fs:write`, `fs:create`, `fs:rename`, `fs:delete`, `fs:reveal`, `fs:listAll`, `fs:grep`
dialogs	`dialog:openFolder`, `dialog:openFile`
`which`, `tools:detect`	Finds compilers and tools on the PATH
`startProc` and `proc:*`	Start, feed (`stdin`) and stop programs; streams output as events
PTY handlers	`pty:spawn`, `pty:write`, `pty:resize`, `pty:kill`
clangd handlers	`lsp:start`, `lsp:send`, `lsp:stop` with the Content-Length reader
gdb handlers	`gdb:start`, `gdb:send`, `gdb:stop`, `gdb:interrupt`

Section	What it does
`app:unbuf`	Writes the header that un-buffers `printf`
settings and window handlers	`settings:load`, `settings:save`, `win:*`, and the self-test hooks

renderer files

File	What it contains
`index.html`	The skeleton: title bar, stripe, sidebar, editor area, bottom panes, status bar
`styles.css`	Variables for both themes, grid layout, tabs, tree, panels, modals, debugger and terminal styling
`icons.js`	The SVG icon table and the `icon()` / `hydrateIcons()` helpers
`app.js`	The main logic: utilities (toast, menu, modals), file icons, `boot`, theme, Monaco setup, settings, project open/new, file tree, tabs and saving, breakpoints, structure view, panels and splitters, process running (`runPipeline`, `planFile`, `cmakeSteps`, `scanTargets`), problems and markers, Find in Files, Search Everywhere and actions, and the keyboard shortcuts
`lsp.js`	The clangd client: start-up and compile database, message routing, document sync, diagnostics, all the Monaco providers, rename edits and the outline
`debug.js`	The gdb client: the MI parser, command/answer matching, session start and stop, stack, variables and watches, stepping controls, decorations and hover evaluation
`terminal.js`	xterm.js set-up, PTY connection, resize, restart and theme

The shared object `S`

Field	Meaning
`S.root`	The open project folder
`S.tabs`, `S.active`	Open files and the one on screen
`S.settings`	User settings (also saved to disk)
`S.tools`	Which compilers and tools were found
`S.config`, `S.targets`	The selected run target and the targets found in CMake
`S.problems`	Diagnostics from the compiler for the last build
`S.breaks`	Breakpoints by file
`S.expanded`	Which folders are open in the tree
`S.running`	The id of the program that is currently running

CHECKPOINT ***Final checkpoint***

Start NOVA with `Launch NOVA.bat`. You can: create a project; see colours and wavy errors as you type; build and run with one key; read the output; debug with breakpoints, stepping and watches; use the terminal; search by double-tapping Shift; change themes. If all of that works, you have built a real IDE. Congratulations.

16.7 Mini assignments

MINI ASSIGNMENT ***Assignment 16.1: A new action***

Add an action *Copy File Path* to the Actions list that copies the active tab's path to the clipboard (`navigator.clipboard.writeText`). Show a toast when done.

MINI ASSIGNMENT ***Assignment 16.2: Recent files***

Remember the last ten opened files in `S.settings` and add a *Recent Files* list to Search Everywhere that shows them first when the search box is empty.

MINI ASSIGNMENT ***Assignment 16.3: Colour the problems badge***

Make the Problems badge show the number of **errors** in red, or, when there are none, the number of **warnings** in yellow. (The code for this is in `renderProblems`; find it and read it first.)

MINI ASSIGNMENT ***Assignment 16.4: A log pane for clangd***

Write every LSP message to a hidden array and add a debug action *Show LSP Log* that opens it in a tab. It teaches you how the protocol really behaves.

MINI ASSIGNMENT ***Assignment 16.5: Break on condition***

Extend breakpoints so that Shift+click makes a **conditional** breakpoint. Use MI's `-break-insert -c "condition"` form.

16.8 Quiz

QUIZ

1. Why is Find in Files done in the main process?
2. In the fuzzy function, which two things earn bonuses?
3. What problem does the `shiftClean` flag prevent?

4. Why does the terminal use a PTY?
5. What does `--query-driver` solve for clangd?
6. Why does NOVA debounce `didChange`, and why also flush before a request?
7. What does `-gdb-set confirm off` do and why is it needed?
8. How does NOVA know which breakpoint number to delete?
9. How is the current debug line drawn in the editor?
10. What is the NOVA self-test mode and what is it good for?

ANSWERS

1. It reads many files; doing that in the window would freeze the interface. The main process can walk the disk and return only the results.
2. A match directly after the previous matched letter, and a match at the start of a name or after a separator.
3. Capital letters (Shift pressed together with another key) triggering Search Everywhere.
4. Shells and tools only behave interactively (colours, prompts, full-screen programs) when they think they are attached to a terminal.
5. It lets clangd ask your g++ where its standard headers are, so standard includes are found.
6. Debounce sends one message per burst of typing; flushing before a request makes sure the server has the very latest text.
7. It stops gdb from asking "are you sure?" prompts which would block the machine-driven conversation.
8. gdb returns a number on creation; NOVA stores it in a Map keyed by file and line.
9. A Monaco decoration with CSS classes (`dbg-line`, `dbg-arrow`) on the current line.
10. An environment variable runs a script inside the window, saves a screenshot and log, and quits. It allows automatic testing and taking screenshots.

Part IV — Going Further

CHAPTER 17

Beyond NOVA: Bigger Projects to Build Next

17.1 You are further along than you think

If you reached this page after building NOVA, you have used six languages, three protocols and about a dozen big ideas: processes, pipes, events, promises, state, parsing, memory, builds and debugging. Most working programmers never touch half of that.

This chapter gives you **projects to grow into**. For each one you get: what it is, why it is worth doing, what you will learn, the basic first steps, and what to read next. They are ordered roughly from easiest to hardest.

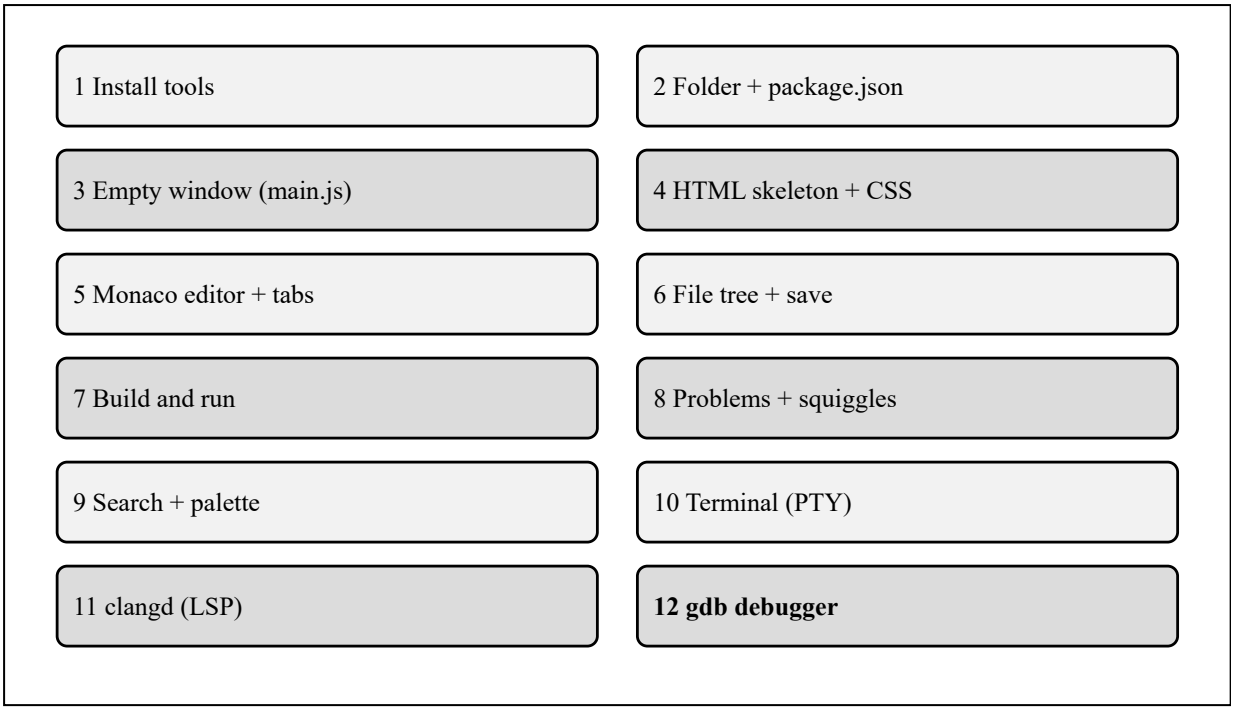


Figure 17.1. *The same ladder you climbed in Part III. The projects below are the next rungs.*

17.2 The menu of projects

#	Project	Difficulty	You will learn	Builds on
1	Git panel (status, diff, commit)	Easy	Running tools, parsing text, UI lists	Ch. 9, 14 to 16

#	Project	Difficulty	You will learn	Builds on
2	Expandable variable tree	Easy	GDB/MI variable objects, recursive UI	Ch. 13, Step 12
3	Test runner with gutter icons	Medium	CTest, parsing results, Monaco decorations	Ch. 12
4	Package NOVA as an installer	Medium	electron-builder, signing, auto-update	Ch. 9
5	Plug-in system	Medium	API design, security, loading code	Ch. 8, 9
6	Sanitizer and memory-leak viewer	Medium	AddressSanitizer output, markers	Ch. 10, 11
7	Profiler and coverage view	Medium/Hard	gcov, perf data, charts	Ch. 11, 12
8	Your own language server	Hard	LSP in depth, parsing	Ch. 13
9	A toy programming language	Hard	Lexer, parser, interpreter, compiler	Ch. 3, 8
10	Syntax colouring with a real parser	Hard	Grammars, incremental parsing	Ch. 8, 13
11	Remote development over SSH	Hard	Networking, security	Ch. 9
12	Move NOVA to TypeScript and a bundler	Medium	Types, build tools	Ch. 8

17.3 Project 1: A Git panel

What. A new bottom tab that shows which files changed, a diff, and a box to commit.

Why. Almost every project lives in Git. Having it inside your editor removes a constant window switch.

First steps.

1. Run `git status --porcelain=v1` with the existing `execCapture(cmd, args, cwd)` helper. The output is stable and easy to parse: two status letters, a space, then the path.
2. Parse each line into `{ status, path }`. Show them in a list with icons for *modified*, *added*, *untracked*, *deleted*.
3. Clicking a file runs `git diff -- path` and shows the text in the panel, colouring lines that start with `+` green and `-` red (use the same `out()` helper with a class).
4. Add a message box and a *Commit* button that runs `git add -A` then `git commit -m "message"`.
5. Re-run the status after every save and commit.

6. Bonus: show a coloured stripe in the editor margin on changed lines with Monaco decorations, using `git diff -U0` to learn the changed line numbers.

> **WHY?** ***Why start here?***
> It uses only things you already built: a process runner, text parsing, a panel and a list. It is a "same ingredients, new dish" project, perfect for confidence.

17.4 Project 2: An expandable variable tree

What. In the debugger, click an arrow next to a struct or array to see its members, to any depth.

Why. Real programs are full of structs, vectors and pointers. A single line of text such as `{ _M_impl = { _M_start = 0x1f40...` is hard to read.

First steps.

1. Use GDB's **variable objects**: `-var-create - * myvar` creates one for the expression; `-var-list-children name` returns its children; `-var-update *` tells you what changed.
2. Replace `-stack-list-variables --all-values` with `--simple-values`, which gives `name` and `type` and a `value` only for simple things.
3. For each variable create a var object (`-var-create`), keep its name, and show a triangle if `numchild` is greater than zero.
4. When the user clicks the triangle, call `-var-list-children` and insert child rows below, indented. Cache them.
5. After each step, call `-var-update` to refresh values and highlight those that changed.
6. Delete the var objects (`-var-delete`) when the frame changes.

17.5 Project 3: A test runner

What. Little green and red marks beside each test; a panel with results; one key to run the failing tests again.

First steps.

1. Use CMake's `enable_testing()` and `add_test()` (Chapter 12). Run `ctest --output-on-failure` in the build folder.
2. Add `--output-junit results.xml` (or `-T Test` parsing) and read the result. Start simply: parse the lines `Test #1: area_test ... Passed`.
3. Show the list in a new panel with a tick or cross.

4. For GoogleTest or Catch2 projects, learn their `--gtest_list_tests` option to list the tests, then run one with `--gtest_filter=Name`.
5. Use Monaco decorations to place a green or red icon in the margin next to each `TEST(...)` line.

17.6 Project 4: Package NOVA as an installer

What. A single `NOVA-Setup.exe` anyone can double-click, without installing Node.

First steps.

1. `npm install --save-dev electron-builder`.
2. Add a `"build"` block to `package.json` with `appId`, `productName`, the files to include and the Windows target (`nsis`).
3. Run `npx electron-builder --win`. The installer appears in a `dist` folder.
4. Remember that `node-pty` is a native module: electron-builder rebuilds it for the right Electron version automatically (check that the log says so).
5. Add an application icon (`.ico`), then explore code signing (to avoid Windows warnings) and `electron-updater` for automatic updates.

> **WATCH OUT** ***Watch out***
> The tools NOVA drives (g++, cmake, gdb, clangd) are **not** inside the installer. Show a friendly *Toolchain Check* screen at first start (you already detect them) and link to install help.

17.7 Project 5: A plug-in system

What. Let anyone add a feature by dropping a folder with a script into `plugins/`.

First steps.

1. Define the **API** you will allow: a small object with functions such as `registerAction(name, fn)`, `getActiveFile()`, `showMessage(text)` and `onSave(callback)`. Fewer functions are safer and easier to keep stable.
2. At start-up, list the `plugins` folder from the main process, read each `plugin.json` (name, version, entry file) and send the script text to the window.
3. Run the script inside a function that receives only your API object: `new Function('nova', code)(api)`.
4. Catch every exception from plug-in code (`try/catch`) so a bad plug-in cannot crash NOVA.
5. Add a *Plugins* list in Settings with an on/off switch for each.

> **WHY?** ***Why is this safe only in small steps?***
> A plug-in is code from someone else running inside your program. Start by trusting only plug-ins that *you* wrote, give them the smallest API, and only then think about sandboxing (for example running them in a hidden window or a Worker with limited access). Security is a ladder, not a switch.

17.8 Project 6: A sanitizer viewer

What. Compile with `-fsanitize=address,undefined` and show bugs (use-after-free, buffer overflow, leaks) as clickable problems.

First steps.

1. Add a **Run with Sanitizers** action that adds `-fsanitize=address,undefined -fno-omit-frame-pointer -g` to the compile line (and to linking).
2. Run the program; capture the **stderr** text. AddressSanitizer prints reports starting with `==ERROR: AddressSanitizer:` followed by a stack of lines like `#0 0x... in main main.cpp:12`.
3. Reuse the file-and-line link regex from Chapter 8 to make each frame clickable.
4. Turn the first stack frame in your code into a **marker** in the editor, as you do for compiler errors.

17.9 Project 7: Profiler and coverage

What. See which lines run most (profiling) and which never run (coverage).

First steps.

1. Coverage with GCC: compile with `--coverage`, run the program, then run `gcov main.cpp`. It writes `main.cpp.gcov`, a copy of your source with a count at the start of each line.
2. Parse the counts: `-` means "no code", `#####` means "never run". Show those lines with a red margin mark and executed lines with green (decorations again).
3. For a profile, use `gprof` (compile with `-pg`) or a sampling tool, and draw a bar chart of time per function using SVG (Chapter 6).

17.10 Project 8: Your own language server

You already designed it in the Chapter 13 big assignment. To grow it:

1. Write a real **lexer** and **parser** for a small language (maybe a configuration language or your toy language from Project 9).
2. Keep the AST for each open document; when `didChange` arrives, re-parse.
3. Report **syntax errors** with accurate ranges as diagnostics.
4. Implement `textDocument/definition` using a **symbol table** (a Map from names to where they were declared).
5. Connect it to NOVA by changing `lspBoot` to start your server for your file extension.

17.11 Project 9: A toy programming language

This is the project the folder name hinted at, "C Interpreter". Writing a language is the best way to understand every language.

First steps.

1. Start with the calculator interpreter from Chapter 8.
2. Add **variables** (`x = 5`) with an environment (a Map).
3. Add **statements**: `print`, `if`, `while`, blocks.
4. Add **functions** and **recursion**, with a call stack of environments.
5. Add **strings** and a few built-in functions.
6. Then rewrite the evaluator as a **compiler to bytecode** plus a small virtual machine. Measure how much faster it is.
7. Finally, give your language **syntax colouring in NOVA** (a Monaco *Monarch* tokenizer, like the one NOVA already has for CMake) and a **Run** action that calls your interpreter.

> **IN PLAIN WORDS** ***A path of real books***
> *Crafting Interpreters* by Robert Nystrom (free to read online) walks through exactly this project, step by step, first as a tree-walking interpreter, then as a bytecode virtual machine in C. After building NOVA you are ready for it.

17.12 Project 10: A real parser for colouring

What. Replace regular-expression-style colouring with a true grammar, as modern editors do with **Tree-sitter**.

First steps. Learn what a **grammar** is; run Tree-sitter's C++ grammar over a file from a small Node script and print its syntax tree; then use the tree to produce Monaco's **semantic tokens** so that, for instance, a function name and a variable name get different colours even though both are identifiers. clangd can also provide semantic tokens over LSP.

17.13 Project 11: Remote development

What. Edit and build code that lives on another computer.

First steps. Use SSH to run commands remotely (the `ssh` program, or Node's `ssh2` library), replace `fs:read` and `fs:write` with versions that go over SSH or SFTP, and start `clangd` and `gdbserver` on the remote machine. Study how VS Code Remote and JetBrains Gateway divide work between a thin local UI and a remote "backend". It is a big project: security and network failure are the real topics.

17.14 Project 12: Grow up: TypeScript and a bundler

NOVA is plain JavaScript on purpose: nothing to install, nothing between you and the code. As a project grows to ten thousand lines, **TypeScript** (JavaScript with types) catches whole families of mistakes before you run the program, and a **bundler** (such as esbuild) combines files and enables real `import` statements.

First steps. Rename `lsp.js` to `lsp.ts`. Add `npm install --save-dev typescript`. Describe the shapes you use (`interface Tab { path: string; dirty: boolean; ... }`). Let the compiler list every place where the shape is misused. Then add esbuild to produce the files the window loads.

17.15 How to learn from here

- **Read other people's code.** Open the source of a small, well-loved tool and trace one feature from the click to the result, as you did with Run in this book.
- **Rewrite something small three times.** The first version teaches you the problem. The second shows you what you misunderstood. The third is good.
- **Keep a "bugs I have met" file.** In this book you saw several real ones: the `define` clash, the `hidden` attribute, the path-casing duplicate, the TDZ race. Each bug you remember is a shield for life.
- **Teach it.** Explain a chapter to a friend. Where you stumble is what you have not understood yet.

Recommended reading

Topic	Title	Why
C	*The C Programming Language* (Kernighan and Ritchie)	The classic; short and precise
C++	*A Tour of C++* (Bjarne Stroustrup)	The creator's concise guide to modern C++
C++	*Effective Modern C++* (Scott Meyers)	The "why" behind good habits

Topic	Title	Why
JavaScript	*Eloquent JavaScript* (Marijn Haverbeke), free online	Friendly and deep
Web	MDN Web Docs (developer.mozilla.org)	The best reference for HTML, CSS and JS
C++ reference	cppreference.com	Every function, with examples
CMake	*Professional CMake* (Craig Scott)	The modern way to write CMake
Interpreters	*Crafting Interpreters* (Robert Nystrom), free online	Build a language from scratch
Debugging	The GDB manual and the GDB/MI chapter	Every command we used
Protocols	The LSP specification (microsoft.github.io/language-server-protocol)	The full list of messages

QUIZ

1. Why is the Git panel a good first extension project?
2. Which GDB/MI commands create and fetch the children of a variable object?
3. Why should a plug-in system start with a tiny API?
4. What flags make GCC produce coverage data?
5. Why is the toy language project a good way to understand every language?
6. Name two benefits of moving a big JavaScript project to TypeScript.

ANSWERS

1. It reuses only skills already built: running a tool, parsing its text, and showing a list in a panel.
2. `-var-create` to make it and `-var-list-children` to get its children.
3. A plug-in is someone else's code running in your program; a small API limits what can go wrong and is easier to keep stable.
4. `--coverage` (then run the program and use `gcov`).
5. You must implement lexing, parsing, scope, calls and memory yourself, which shows how every language works inside.
6. Types catch mistakes before running and make large code bases easier to change safely; also better editor help.

Appendices

APPENDIX A

Nomenclature (Words and Functions)

This is your dictionary. Each entry says **what it is**, **what it means in plain words**, and **why it matters**. Words are grouped by topic. Use the Contents page to jump here whenever a word is new.

A.1 How to read this list

Every table has three columns: **Term** (the word or function), **What it is** (a plain-English meaning), and **Why it matters** (the reason you should care). When a word is a function, you will see brackets after its name.

A.2 General IDE and programming words

Term	What it is	Why it matters
IDE	Integrated Development Environment: one program with editor, build, debug and more	It is the workshop where you spend your working day
Source code	The human-readable text of a program	It is what you write; everything else is made from it
Compiler	Translates the whole program into machine code before it runs	Makes C/C++ programs fast; reports your mistakes
Interpreter	Runs a program line by line as it reads it	Starts instantly; used by JavaScript and Python
Linker	Joins compiled pieces and libraries into one program	Explains "undefined reference" errors
Library	Ready-made code you can use in your program	Saves you rewriting common things
Build	The whole process of turning source into a runnable program	The thing the Run and Build buttons start
Executable (.exe)	A finished program file	The result of a successful build
Debug build	A build with extra information for debuggers	Needed for stepping and breakpoints
Release build	A build optimised for speed, without debug information	What you ship to users
Syntax	The grammar rules of a language	A syntax error means you broke the grammar

Term	What it is	Why it matters
Syntax highlighting	Colouring words by their role	Lets your eyes see structure
Token	A small meaningful piece of text (a number, a name, a symbol)	The first thing any language tool produces
Lexer	Cuts text into tokens	Step one of an interpreter or compiler
Parser	Arranges tokens into a tree by the grammar	Gives meaning to code
AST	Abstract Syntax Tree: the tree a parser builds	Used by compilers, language servers and refactoring
Refactoring	Changing code structure without changing behaviour	IDEs do it safely (rename, extract)
Snippet	A short template that expands into code	Saves typing common patterns
Breakpoint	A mark telling the debugger to pause on a line	The basic debugging tool
Stack frame	One function call's information on the call stack	The debugger's Frames list is a list of these
Call stack	The chain of functions that led to the current line	Shows how you got here
Watch	An expression the debugger re-evaluates at every stop	Track a value without hunting for it
Step over / into / out	Run one line / enter the function / leave it	The core debugger moves (F8 / F7 / Shift+F8)
Diagnostic	A message from a tool about a problem in your code	The red and yellow squiggles
Warning	A message about code that compiles but looks risky	Fixing warnings prevents bugs
Exit code	The number a program returns when it ends; 0 = success	Build tools and tests rely on it
Stdin / stdout / stderr	Input stream, normal output stream, error output stream	How programs talk to the console and to NOVA
Pipe	A one-way channel of bytes between two programs	How NOVA reads output from the compiler
Process	A running program with its own memory	NOVA runs many: main, window, g++, gdb...
Environment variable	A named setting inherited by programs (like PATH)	Explains many "works on my machine" problems
PATH	The list of folders searched for programs	If a tool is not on PATH, NOVA cannot find it
Terminal / shell	A text window / the program that reads your commands	Where command-line tools run

Term	What it is	Why it matters
PTY / ConPTY	A pseudo-terminal that makes a program think a person is watching	Needed for a real integrated terminal
ANSI escape sequence	Hidden codes in text for colours and cursor moves	How terminals draw colours
Git	A tool that records the history of your files	Almost every project uses it
Race condition	A bug that depends on which of two things happens first	Hard to find; needs careful design
Undefined behaviour	Code the language does not define: anything may happen	The root of many C/C++ bugs
Memory leak	Memory that is taken and never given back	Makes programs slowly eat RAM
Dangling pointer	A pointer to memory that was freed	Crashes or security holes

A.3 Web words: HTML, CSS and the browser

Term	What it is	Why it matters
HTML	Markup describing what is on a page	The skeleton of NOVA's window
Tag / element	A labelled piece such as `<p>...</p>`	The building block of HTML
Attribute	Extra information inside a tag (`id`, `class`, `href`)	Lets CSS and JavaScript find and configure elements
DOM	The tree of elements the browser builds	JavaScript changes the DOM to change the screen
CSS	Rules for how elements look	Colours, sizes, spacing, themes
Selector	The part of a CSS rule that chooses elements	Decides what a rule affects
Specificity	Which rule wins when two clash	Explains why "my style does nothing"
Box model	Content, padding, border, margin	Every layout problem starts here
Flexbox	CSS layout for rows or columns	Toolbars, tab bars
Grid	CSS layout for rows and columns together	The whole IDE frame
`fr` unit	A fraction of free space in a grid	Lets the editor take the leftover room
CSS variable	A named value such as `--bg`	Themes in one place
Pseudo-element	Generated content like `::before`	Spinner and decorations without extra HTML
SVG	Vector graphics described in text	Sharp icons and diagrams
Viewport	The visible area of the page	Layouts are measured against it
Event	Something that happens (click, key, resize)	How the UI reacts to the user

Term	What it is	Why it matters
Event delegation	One listener on a parent handling many children	Efficient and works for new elements
Web worker	A background thread in the browser	Keeps heavy jobs from freezing the editor
Sandbox	A restricted area where web code runs	Why a web page cannot read your files

A.4 JavaScript words

Term	What it is	Why it matters
`let` / `const`	Variable that can change / cannot change	Prefer const to avoid surprises
`===`	Strict equality (value and type)	Avoids hidden conversions
Arrow function	Short function syntax `(a) => a * 2`	Used everywhere in modern JS
Closure	A function that remembers its surrounding variables	Debounce, event handlers, private data
Callback	A function passed to be called later	The old style of asynchronous code
Promise	An object for a future value or error	The base of async code
`async` / `await`	Write promise code like normal code	Keeps the UI alive while waiting
Event loop	The mechanism that runs queued callbacks one at a time	Why JS never does two things at once
Template string	A backtick string with `${...}`	Readable text building
Destructuring	Pulling parts out of an object or array	Cleaner code
Spread `...`	Copy or expand arrays/objects	Safe copying and merging
Optional chaining `?.`	Stop safely if something is missing	Avoids "cannot read property of undefined"
Regex	A pattern for matching text	Parsing compiler output
JSON	A text format for data	Settings and protocols
Debounce	Wait until activity stops before acting	Fewer messages to the language server
Module	A bundle of code loaded with `require` or `import`	How large programs are split
Scope	The region where a name is visible	Source of many bugs when misunderstood

A.5 Node.js and Electron words

Term	What it is	Why it matters
Node.js	JavaScript outside the browser	The engine of NOVA's main process

Term	What it is	Why it matters
npm	Node's package installer	Downloads Electron, Monaco, etc.
`package.json`	Project description and dependency list	Tells npm and Electron what to do
Electron	Toolkit for desktop apps with web tech	Gives NOVA its window
Main process	The Node side of an Electron app	The only side that may touch the system
Renderer process	The web page side	Draws the UI, limited power
Preload script	Runs before the page; builds the safe bridge	Controls what the page can do
`contextBridge`	Electron's way to expose chosen functions to the page	The safe door
IPC	Messages between main and renderer	How the window asks for files
`ipcMain.handle`	Register an answer for a channel	The main process side of a question
`ipcRenderer.invoke`	Ask the main process and await the answer	The window side of a question
`webContents.send`	Main pushes a message to the window	Streaming output events
`BrowserWindow`	An application window	NOVA's main frame
Native module	A Node add-on written in C/C++	`node-pty` is one; must match Electron's version
Monaco	The editor component of VS Code	NOVA's editor
xterm.js	A terminal emulator for the web	NOVA's terminal display

A.6 C and C++ words

Term	What it is	Why it matters
Header file (`.h`)	A file listing what exists	Lets many files share declarations
`#include`	Paste another file's text here	How code is shared
Include guard	`#ifndef`/`#pragma once` protection	Stops double inclusion
Macro	Text substitution by the preprocessor	Powerful but dangerous
Pointer	A variable holding a memory address	The core of C
Reference	An alias for another variable (C++)	Pass without copying
Stack	Automatic memory for locals and calls	Fast, but small
Heap	Manual memory from `malloc`/`new`	Big, but you must free it

Term	What it is	Why it matters
`NULL` / `nullptr`	A pointer pointing at nothing	Check before using a pointer
Struct	A group of variables under one name	Your own data type
Padding	Hidden bytes inserted for alignment	Explains sizeof surprises
Class	A type bundling data and functions (C++)	The core of C++
Constructor / destructor	Runs when an object is created / destroyed	Set-up and tidy-up
RAII	Tie a resource's life to an object	The key C++ safety idea
Smart pointer	An object that owns and frees a pointer	Removes most manual `delete`
Inheritance	A class extending another	Reuse and polymorphism
Virtual function	Chosen at run time by the real type	Polymorphism
Template	Code generated for any type	`std::vector`, generic code
Namespace	A named box of names	Avoids name clashes
STL	The C++ standard library of containers and algorithms	Ready-made, fast, tested
Lambda	An inline unnamed function	Used with algorithms
Move semantics	Steal an object's insides instead of copying	Makes returning big objects cheap
`const`	A promise not to change something	Safer code, better optimisation
`constexpr`	Computed at compile time	Faster programs
UB sanitizer / ASan	Compiler tools that detect memory bugs at run time	Find bugs you would miss

A.7 Build and tool words

Term	What it is	Why it matters
CMake	A build-system generator	The standard for C/C++ projects
`CMakeLists.txt`	CMake's recipe file	Describes targets and options
Target	A thing to build (program or library)	The unit CMake thinks in
Generator	The tool CMake writes files for (Ninja, Make)	Chosen with `-G`
Ninja	A very fast build tool	Quick incremental builds
Configure / generate	CMake reads the recipe and writes build files	The first of the two steps
Build folder	Where all generated files go	Keeps sources clean

Term	What it is	Why it matters
`compile_commands.json`	The exact compiler command for each file	How clangd understands your code
LSP	Language Server Protocol	Standard messages between editor and language server
clangd	A C/C++ language server	NOVA's completion, hover, diagnostics
clang-format	A code formatter	One consistent style
gdb	The GNU debugger	NOVA's debugger
GDB/MI	gdb's machine-readable interface	How NOVA controls gdb
Token (MI)	A number sent before an MI command	Matches an answer to its command
Sanitizer	A tool that checks for bugs while the program runs	Finds leaks and overflows
Coverage	A report of which lines ran	Shows what tests miss
Profiler	A tool that measures where time is spent	Finds slow code

A.8 Most-used functions and commands

JavaScript and the browser

Function	What it does	Why it matters
`console.log(x)`	Prints x	Your first debugging tool
`document.querySelector(css)`	Finds the first matching element	How code reaches the page
`element.addEventListener(type, fn)`	Runs fn on an event	Makes the UI interactive
`element.classList.toggle(name)`	Adds or removes a class	Switch looks and states
`element.dataset.name`	Reads a `data-name` attribute	Per-element information
`array.map(fn)`	New array with fn applied to each item	Transform lists
`array.filter(fn)`	Items for which fn is true	Pick from lists
`array.find(fn)`	First item matching	Look up an item
`array.reduce(fn, start)`	Combine to one value	Sums, counts, groups

Function	What it does	Why it matters
`setTimeout(fn, ms)`	Run fn later	Delays, debounce
`JSON.parse(text)` / `JSON.stringify(obj)`	Text to data / data to text	Settings and messages
`Promise.all([...])`	Wait for several promises	Parallel jobs
`str.match(regex)` / `str.replace(re, x)`	Find / change text by pattern	Text parsing
`Object.keys(obj)`	List of property names	Loop over objects

Node.js and Electron

Function	What it does	Why it matters
`require('fs')`	Loads the file module	Access to the disk
`fs.promises.readFile(path, 'utf8')`	Reads a text file without blocking	Open files
`fs.promises.writeFile(path, text)`	Writes a file	Save files
`fs.promises.readdir(dir, {withFileTypes:true})`	Lists a folder	The file tree
`path.join(a, b)`	Joins parts with the right separator	Portable paths
`child_process.spawn(cmd, args)`	Starts a program with streams	Compilers, debuggers
`spawnSync('where', [name])`	Starts and waits	Detect tools
`process.env` / `process.platform`	Environment / OS name	Platform-specific code
`app.whenReady()`	Resolves when Electron is ready	Start of every app
`new BrowserWindow(options)`	Creates a window	The UI frame
`ipcMain.handle(channel, fn)`	Answers window questions	Main-process services
`ipcRenderer.invoke(channel, ...args)`	Asks the main process	Window-side calls
`contextBridge.exposeInMainWorld(name, api)`	Offers functions to the page	The safe bridge
`dialog.showOpenDialog(win, opts)`	Native file/folder picker	Open Folder

C

Function	What it does	Why it matters
`printf(fmt, ...)`	Prints formatted text	Output and debugging
`scanf` / `fgets(buf, n, stdin)`	Reads input (prefer `fgets`)	Safe input
`malloc(n)` / `free(p)`	Borrow / return heap memory	Dynamic data
`realloc(p, n)`	Resize heap memory	Growing arrays
`strlen(s)`	Length of a string	Needed everywhere
`strcpy`, `strcat`, `strcmp`	Copy, join, compare strings	Beware buffer sizes
`snprintf(buf, n, fmt, ...)`	Safe formatted write	Prevents overflow
`fopen`, `fgets`, `fprintf`, `fclose`	File input/output	Read and write files
`qsort(base, n, size, cmp)`	Sort anything	Callbacks in action
`memcpy`, `memset`	Copy / fill bytes	Fast block work
`exit(code)`	End the program	Exit codes

C++

Function / type	What it does	Why it matters
`std::cout << x`	Prints x	Output
`std::string`	A safe string	No buffer overflows
`std::vector<T>`	A growable array	The default container
`v.push_back(x)` / `v.size()`	Add at the end / count	Everyday use
`std::map<K,V>` / `std::unordered_map<K,V>`	Key to value	Lookups
`std::sort(b, e)`	Sorts a range	Algorithms beat hand loops
`std::find`, `std::count_if`, `std::accumulate`	Search, count, sum	Common tasks
`std::make_unique<T>(args)`	Creates a uniquely owned object	Safe ownership
`std::make_shared<T>(args)`	Creates a shared object	Shared ownership
`std::move(x)`	Allow x to be moved from	Avoids copies
`std::optional<T>`	A value that may be missing	Safer than magic values
`std::thread`, `std::mutex`, `std::lock_guard`	Threads and locks	Concurrency

Function / type	What it does	Why it matters
`std::ifstream` / `std::ofstream`	File streams (RAII)	Files that close themselves

CMake commands

Command	What it does	Why it matters
`cmake_minimum_required(VERSION x)`	Pins rules	Stable behaviour
`project(name LANGUAGES CXX)`	Names the project	Always required
`add_executable(t files...)`	Defines a program	The usual target
`add_library(t STATIC files...)`	Defines a library	Reusable code
`target_link_libraries(t PRIVATE lib)`	t uses lib	Links targets
`target_include_directories(t PUBLIC dir)`	Header search folders	Fixes "no such file"
`target_compile_options(t PRIVATE -Wall)`	Compiler flags for t	Per-target options
`set(var value)` / `option(NAME "doc" ON)`	Variable / switch	Configuration
`add_subdirectory(dir)`	Include another folder's recipe	Large projects
`enable_testing()` / `add_test()`	Testing with CTest	Automated checks

gdb commands (typed in NOVA's debug console)

Command	What it does	Why it matters
`break file.cpp:12`	Set a breakpoint	Stop where you choose
`run` / `continue`	Start / resume	Move forward
`next` / `step` / `finish`	Over / into / out	Line by line
`print expr`	Show a value	Inspect state
`bt`	Backtrace (call stack)	See how you got here
`info locals`	All local variables	Quick overview
`watch var`	Stop when var changes	Find who changes it
`x/8xw &var`	Examine memory	See raw bytes

Most-used CSS properties

Property	What it does	Why it matters
`display: flex / grid / none`	Layout mode or hide	Foundation of layout
`gap`, `padding`, `margin`	Spacing	Breathing room

Property	What it does	Why it matters
`background`, `color`, `border`	Paint	Visual identity
`width`, `height`, `min-*`, `max-*`	Size limits	Resizable panels
`overflow: auto`	Scrollbars when needed	Long lists
`position: absolute / fixed`	Place freely	Popups, overlays
`transition`, `animation`	Motion	Polish
`var(--name)`	Use a variable	Themes

APPENDIX B

Cheat Sheets and Troubleshooting

B.1 NOVA keyboard shortcuts

Keys	Action
Shift + F10	Run
Shift + F9	Debug
Ctrl + F9	Build
Ctrl + F2	Stop
F8 / F7 / Shift + F8	Step over / into / out (while debugging)
F9	Resume (while debugging)
Alt + F9	Run to cursor
Ctrl + F8	Toggle breakpoint
Shift, Shift	Search Everywhere
Ctrl + Shift + A	Actions palette
Ctrl + P or **Ctrl + Shift + N**	Go to file
Ctrl + F12	Go to symbol in this file
Ctrl + Shift + F	Find in files
F2	Rename (symbol in editor, or file in the tree)
Ctrl + click	Go to definition
Shift + F12	Find references
Ctrl + .	Quick fix
Ctrl + Alt + L	Reformat code
Ctrl + S / Ctrl + Shift + S	Save / save all
Ctrl + W	Close tab
Ctrl + Tab	Next tab
Alt + 1 / 7	Project / Structure panel
Alt + 4 / 5 / 6 / F12	Run / Debug / Problems / Terminal
Ctrl + Alt + S	Settings
F12	Developer tools (for debugging NOVA itself)

B.2 Command-line cheat sheet

```
gcc  main.c   -o app.exe          build a C file
g++  main.cpp -o app.exe          build a C++ file
g++ -g -O0 -Wall -Wextra -std=c++17 main.cpp -o app.exe     the flags NOVA uses
cmake -S . -B cmake-build-debug -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build cmake-build-debug
ctest --test-dir cmake-build-debug
gdb app.exe                       start the debugger by hand
clang-format -i main.cpp          format a file
node file.js                      run JavaScript
npm install                       install the libraries in package.json
npm start                         start NOVA
```

B.3 Troubleshooting table

Symptom	Likely cause	What to do
NOVA window does not appear	`ELECTRON_RUN_AS_NODE` set	Start with `Launch NOVA.bat` or `npm start` (uses `launch.js`)
"g++ not found" / no compiler in the status bar	Not on PATH	Add `C:\msys64\mingw64\bin` to PATH, reopen NOVA
Run does nothing, no message	Active file is not `.c` / `.cpp` and no CMake target	Open a C/C++ file or a project with `CMakeLists.txt`
Output of `printf` appears only at the end	Buffered output	Keep *Unbuffered console output* ticked in Settings
Squiggles under every `#include <...>`	clangd cannot find the standard headers	Ensure g++ is on PATH; check `--query-driver`; restart clangd
Squiggles appear on correct code	Wrong compile flags	Reconfigure CMake; delete `cmake-build-debug`; Actions → Reload CMake Project
"undefined reference"	A `.cpp` is not in `add_executable`	Add the file and reconfigure
Debug button says "gdb not found"	gdb not installed	`pacman -S mingw-w64-x86_64-gdb`, restart NOVA
Debugger skips lines or shows `??`	Optimised build or missing `-g`	Use Debug builds (NOVA does)
Program cannot read input while debugging	Inline console mode	Tick *Debug: run in its own console window*
Terminal shows "unavailable"	`node-pty` did not load	Run `npm install` again; ensure the Electron version matches

Symptom	Likely cause	What to do
Blank editor, console errors about workers	Monaco worker URL problem	Check the `getWorkerUrl` code and the `node_modules` path
`Terminal is not defined`	xterm script loaded after Monaco's loader	Load xterm scripts **before** `loader.js`
Panel will not hide	Missing `[hidden]` rule	Add `[hidden] { display: none !important; }`
`npm start` opens Electron's default window	`main` missing in `package.json`	Set `"main": "main.js"`
`Cannot find module 'node-pty'`	Not installed	`npm install node-pty`

TIP ***How to debug NOVA itself***

Press **F12** to open Chromium's developer tools for the window. The **Console** tab shows errors from the renderer files. Type `S` to see NOVA's whole state. For the main process, look at the terminal where you ran `npm start`: `console.log` from `main.js` appears there.

APPENDIX C

Roadmaps for the Big Assignments

These are *roadmaps*, not solutions. Each lists milestones in a sensible order. Finish one milestone, test it, then move on. Never write 300 lines before running anything.

C.1 MiniIDE: HTML (Chapter 6)

1. Write the outer shell: header, nav, aside, main, section, footer, with no content.
2. Fill the header (logo SVG, three buttons).
3. Build the nested `ul` tree. Check that it indents.
4. Add the tabs and the code block.
5. Add the four bottom panels, with only one visible.
6. Add `id`/`data-` attributes and `alt`/`aria-label` text.
7. Validate: open the file; read it top to bottom; compare with `renderer/index.html`.

C.2 MiniIDE: CSS (Chapter 7)

1. Add the variables and the `[hidden]` and `box-sizing` rules.
2. Make `body` fill the window and create the outer grid.
3. Make the middle area its own grid.
4. Style the tree, then the tabs, then the buttons.
5. Add the second theme and a temporary toggle.
6. Add the spinner animation.
7. Shrink the window to 700px and fix overlaps.

C.3 MiniIDE: JavaScript (Chapter 8)

1. Create the `S` state object and the fake project.
2. Draw the tree from the object.
3. Open tabs; show the file in the textarea.
4. Add dirty tracking and Ctrl+S.
5. Add the Run button with the `wait()`-based fake build, then Stop.
6. Add find in files (with debounce) and the Problems panel.

7. Add theme and settings with `localStorage`.
8. Add shortcuts. Finally, check the console for errors.

C.4 Folder Peek: Node and Electron (Chapter 9)

1. Hello Electron window with the three files.
2. Add the preload and expose one function; prove `require` is blocked.
3. Add *Open Folder* and `readDir` (folders first).
4. Draw the tree; load children lazily.
5. Read and show files; add save.
6. Add Run with `spawn`: stdout, stderr, exit code, Stop.
7. Handle errors (missing program, binary file). Write the README.

C.5 Library manager: C (Chapter 10)

1. Define `Book` and the growable array; test with hard-coded books.
2. Write `add` and `list`.
3. Write the menu loop with safe input reading.
4. Add search (case-insensitive), check out and return.
5. Add `qsort` with three comparers.
6. Add save and load; test with a missing and a damaged file.
7. Split into `book.h/.c` and `main.c`; add include guards; build with CMake.
8. Compile with `-Wall -Wextra`; run under the debugger; check for leaks by reasoning about every `malloc`.

C.6 Outline: C++ (Chapter 11)

1. Take the tokenizer from the chapter and add line numbers.
2. Add multi-line comments and preprocessor lines.
3. Print all tokens with their lines to prove the lexer.
4. Track brace depth.
5. Find classes, then functions; store `Symbol` objects.
6. Print the aligned outline and the summary map.
7. Split into files, add error handling, build with CMake, run on your own sources.

C.7 GeoKit: CMake (Chapter 12)

1. Create the folders and an empty top-level recipe.
2. Add the library with two classes; build it alone.
3. Add the executable and link; get it to print one area.
4. Add file reading.
5. Add the tests executable and `add_test`; make one test fail on purpose to see ctest report it.
6. Add the options and test each with `-D...`.
7. Open in NOVA; check the run configurations; debug into the library.

C.8 EchoLS: Protocols (Chapter 13)

1. Write the framing reader and writer; test with a hand-made frame.
2. Handle `initialize` and `shutdown`; test from a small client.
3. Track documents (`didOpen`/`didChange`).
4. Implement hover.
5. Implement completion.
6. Publish diagnostics.
7. Write the client with the `pending` map and timeouts; test every feature.
8. Add error replies for unknown methods.

> **TIP** ***The golden rule of every roadmap***
> Always keep something working. After every milestone, run the program. If you write for an hour without running it and it fails, you will not know where the problem is. Small steps, run often, fix at once.